Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

Senior Project Final Report

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor Dr. Omaia Al-Omari

Semester & Academic Year Senior Project I & II – Academic Year 2026–2027

Submission Date _____

Supervisor's Signature _____

Approval Date _____

Acknowledgments

We would like to express our sincere gratitude to our supervisor, **Dr. Omaia Al-Omari**, for the guidance, patience and constructive feedback provided throughout this project.

We thank the faculty of the Information Systems Department and the College of Computer and Information Sciences at Prince Sultan University for the knowledge and support that made this work possible, and the IS499 course coordinator for organising the senior project. We are also grateful to the open-source communities whose work Mirsad builds on, including the developers of the Model Context Protocol, FastAPI, SQLAlchemy, scikit-learn, React and Ollama, and to the researchers who published the prompt-injection benchmarks and datasets that informed our design.

Finally, we thank our families for their encouragement and patience during the long evenings this project required.

Abstract

Large language model (LLM) agents now read e-mail, browse the web and call business tools through the Model Context Protocol (MCP) and OpenAI-style function calling. Any content an agent reads can carry hidden instructions (indirect prompt injection) that turn an authorised agent against its own organisation, and published defences are almost entirely English-centric. This project built Mirsad (مرصاد), a runtime security gateway that sits between an unmodified agent and its tools. Its objectives were to enforce action-level policy with a median overhead of at most 200 ms per call, to detect injected instructions in Arabic and English with an F1 score of at least 0.85, and to keep humans in control while reducing attack success by at least 80% and wrongly blocking no more than 5% of benign actions. Mirsad combines a declarative YAML policy engine, taint tracking of untrusted and confidential data, an Arabic/English injection detector, human approval through an installable bilingual mobile web app, and an HMAC hash-chained audit log with decision replay. On 48 benign English and Arabic tasks, full protection automatically denied 0.0% of benign calls, escalated 17.6% to a human, and 272 of 272 audited decisions replayed identically. Median added latency was 2.7 ms per call and 166 ms under 50 concurrent sessions. Detector F1 and attack success rate remain pending until the team's attack-labelled evaluation data is complete. Mirsad gives organisations an auditable control point that supports obligations under Saudi Arabia's Essential Cybersecurity Controls and Personal Data Protection Law.

الملخص

أصبح وكلاء النماذج اللغوية الكبيرة يقرؤون البريد الإلكتروني ويتصفحون الويب ويستدعون أدوات الأعمال عبر بروتوكول سياق النموذج (MCP) واستدعاء الدوال على نمط OpenAI. وقد يتضمن أي محتوى يقرؤه الوكيل تعليمات مخفية تحوّل الوكيل المصرَّح له إلى أداة ضد مؤسسته، وهو ما يُعرف بحقن الأوامر غير المباشر، في حين تكاد الدفاعات المنشورة تقتصر على اللغة الإنجليزية. طوّر هذا المشروع «مرصاد» (Mirsad)، وهو بوابة أمنية تعمل أثناء التشغيل بين الوكيل وأدواته دون الحاجة إلى تعديل الوكيل. وتمثلت أهدافه في فرض السياسات على مستوى الإجراءات على ألا يتجاوز وسيط الزمن الإضافي 200 ملّي ثانية لكل استدعاء، وكشف التعليمات المحقونة بالعربية والإنجليزية بمقياس F1 لا يقل عن 0.85، وإبقاء القرار بيد الإنسان مع خفض معدل نجاح الهجمات بنسبة 80% على الأقل، على ألا يتجاوز الحجب الخاطئ للإجراءات السليمة 5%. ويجمع مرصاد بين محرك سياسات تصريحي بصيغة YAML، وتتبّع مصدر البيانات غير الموثوقة والسرية، وكاشف حقن للعربية والإنجليزية، وموافقة بشرية عبر تطبيق ويب ثنائي اللغة قابل للتثبيت على الهاتف، وسجل تدقيق مترابط بسلسلة تجزئة HMAC يتيح إعادة تشغيل القرارات. وفي 48 مهمة سليمة بالعربية والإنجليزية، بلغت نسبة الاستدعاءات السليمة التي رفضتها الحماية الكاملة آليًا 0.0%، وأحالت 17.6% منها إلى مراجع بشري، وطابقت إعادةُ التشغيل القرارَ الأصلي في 272 من أصل 272 قرارًا مسجّلًا. وبلغ وسيط الزمن الإضافي 2.7 ملّي ثانية لكل استدعاء، و166 ملّي ثانية مع 50 جلسة متزامنة. أما مقياس F1 للكاشف ومعدل نجاح الهجمات فما زالا معلّقين إلى حين اكتمال بيانات الهجمات المصنّفة. ويقدّم مرصاد للمؤسسات نقطة تحكم قابلة للتدقيق تدعم وفاءها بضوابط الهيئة الوطنية للأمن السيبراني ونظام حماية البيانات الشخصية.

Table of Contents

Acknowledgments i
Abstract ii
الملخص iii
Table of Contents iv
List of Figures x
List of Tables xiv
List of Acronyms xvii
Chapter 1 – Introduction 1
1.1 Background 1
1.2 Problem Statement 2
1.3 Project Objectives 2
1.4 Scope and Constraints 3
1.5 Stakeholders and User Personas 4
1.5.1 Stakeholder Personas 4
1.6 Industry Relevance & IS Application 5
1.7 Project Timeline (Gantt Chart) 6
Chapter 2 – Literature Review 8
2.1 Overview of Previous Studies 8
2.1.1 LLM agents and tool use 8
2.1.2 Prompt injection: origins and taxonomy 9
2.1.3 Benchmarks for agent security 11
2.1.4 Defences at the model, prompt, runtime and architecture levels 12
2.1.5 Multilingual and Arabic LLM safety 14
2.1.6 Standards and Saudi regulation 15
2.1.7 Foundations used by Mirsad 16
2.2 Comparative Analysis 17
2.3 Research Gap and Positioning 21
Chapter 3 – Methodology 23
3.1 Research Design / Project Approach 23

3.2 Data Collection / Requirements Gathering 24
3.2.1 Elicitation methods 24
3.2.2 Threat model and assumptions 25
3.2.3 Use-case model 25
3.2.4 Requirements specification 29
3.3 System Design & Architecture 32
3.3.1 Architectural style and system context 33
3.3.2 Components and responsibilities 35
3.3.3 Core processes 39
3.3.4 Policy language design 44
3.3.5 Taint-tracking design 47
3.3.6 Injection detector design 49
3.3.7 Database design 50
3.3.8 Security design and threat model 53
3.3.9 Interface design 55
3.3.10 User-interface design 57
3.3.11 Deployment architecture and technology stack 61
3.4 Implementation Strategy 63
3.4.1 Development methodology 63
3.4.2 Mapping to the IS499 deliverables 64
3.4.3 Team roles and responsibilities 65
3.4.4 Development environment, tools and version control 65
3.4.5 Changes from the proposal and their justification 66
3.4.6 Budget 67
3.5 Testing & Validation 67
3.5.1 Test strategy 68
3.5.2 Test environment, tools and responsibilities 70
3.5.3 Entry and exit criteria 71
3.5.4 Continuous integration and the evidence pipeline 72
3.5.5 Metrics 73
3.5.6 Test inventory 73

3.5.7 User acceptance testing 75
3.6 Ethical, Legal & Security Considerations 76
3.6.1 Data privacy and the PDPL 76
3.6.2 Regulatory and standards alignment 76
3.6.3 Security of the gateway itself 76
3.6.4 Responsible handling of attack material 77
3.6.5 Social and professional considerations 77
3.6.6 Licences, third-party components and AI assistance 77
Chapter 4 – Implementation and Evaluation 79
4.1 Implementation 79
4.1.1 Development environment and repository structure 79
4.1.2 Gateway pipeline 80
4.1.3 Policy engine 81
4.1.4 Taint tracker 82
4.1.5 Injection detector 83
4.1.6 Audit log and approvals 83
4.1.7 Protocol adapters and SDK 84
4.1.8 REST API and command line 85
4.1.9 Web dashboard 86
4.1.10 Approver PWA 87
4.1.11 Demonstration workspace and agent 87
4.1.12 Mirsad-Bench 88
4.1.13 Integration issues found and fixed 89
4.2 Evaluation 90
4.2.1 Evaluation setup 90
4.2.2 Detector evaluation 92
4.2.3 Attack resistance on Mirsad-Bench 92
4.2.4 Benign behaviour 93
4.2.5 Performance 96
4.2.6 Security testing of the gateway 98
4.2.7 Audit integrity and reproducibility 99

4.2.8 Usability: approval decision time 99
4.2.9 Success criteria 101
4.2.10 Limitations and threats to validity 102
4.3 Individual and Group Contributions 103
Chapter 5 – Conclusion and Future Work 106
5.1 Summary of Outcomes 106
5.2 Contributions 107
5.3 Lessons Learned 108
5.4 Limitations 108
5.5 Future Work 109
5.6 Concluding Remarks 109
References 111
Appendix A – Standards & Constraints Matrix 120
A.1 Standards, Regulations and Guidelines 120
A.2 Constraints Matrix 123
Appendix B – Test Reports & Validation Results 125
B.1 Test environment and evidence 125
B.2 Requirements traceability matrix 126
B.3 Test case specification 133
B.3.1 Policy language and engine 134
B.3.2 Gateway and protocol adapters 135
B.3.3 Taint tracking 138
B.3.4 Detector and result handling 139
B.3.5 Approvals and audit 141
B.3.6 Administration, user interface, demonstration, benchmark and storage 144
B.4 Security test cases 146
B.5 Test results 148
B.5.1 Execution summary 148
B.5.2 Code coverage 149
B.5.3 Static analysis and dependency scanning 149
B.5.4 Performance 150

B.5.5 Benchmark results on benign tasks 152
B.5.6 Audit verification and decision replay 154
B.5.7 LLM smoke run 154
B.5.8 Results pending team-supplied data 155
B.6 Known limitations and open items 155
Appendix C – System Screenshots / Interface Designs 158
C.1 Administration Dashboard (English) 158
C.2 Dashboard in Arabic and in the Dark Theme 166
C.3 Approver App (English) 168
C.4 Approver App (Arabic) 172
Appendix D – Poster / Presentation Slides 175
Appendix E – Software User Guide 180
E.1 Overview and System Requirements 180
E.2 Installation 181
E.2.1 Python virtual environment 181
E.2.2 Docker Compose with PostgreSQL 181
E.3 Configuration 182
E.4 First Run and Demonstration 183
E.4.1 Initialising and starting the server 183
E.4.2 Local model for the demo agent 183
E.4.3 Running a demonstration 183
E.5 Administration Dashboard 184
E.5.1 Signing in and navigation 184
E.5.2 Overview 185
E.5.3 Activity and audit log 185
E.5.4 Approvals 186
E.5.5 Policies, agents and users 187
E.5.6 Tools, detector playground, demo console and settings 188
E.6 Approver App 190
E.6.1 Installing on Android and iOS 190
E.6.2 Reviewing and deciding 191

E.6.3 Notifications 192
E.6.4 Arabic interface 193
E.7 Writing and Activating Policies 193
E.7.1 Policy structure and an example 193
E.7.2 Validating 194
E.7.3 Activating, monitor mode and rollback 195
E.8 Connecting Agents 195
E.8.1 Before connecting 195
E.8.2 MCP hosts: stdio proxy and HTTP 196
E.8.3 OpenAI-compatible agents 196
E.8.4 Python SDK 196
E.9 Command-Line Reference 197
E.10 Audit Verification, Replay and Export 197
E.11 Training the Detector and Running the Benchmark 198
E.11.1 Detector 198
E.11.2 Benchmark 199
E.12 Troubleshooting and FAQ 199
E.13 Security Recommendations for Operators 200
Appendix F – Stakeholder Questionnaire 202

List of Figures

Figure 1.1: Project schedule with task owners (abbreviations as in the text) and milestones M1–M7 7

Figure 2.1: A three-axis taxonomy of prompt-injection attacks: the channel that carries the text, the technique used to obtain compliance and the attacker's goal (synthesised from the sources reviewed in Section 2.1.2)..... 10

Figure 3.1: Use-case diagram with misuse cases (black) and the security use cases that mitigate them 26

Figure 3.2: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools..... 33

Figure 3.3: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services..... 34

Figure 3.4: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules..... 36

Figure 3.5: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract)..... 38

Figure 3.6: Sequence of a tool call through the gateway: decision, execution and inspection of the result..... 40

Figure 3.7: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step..... 41

Figure 3.8: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call..... 42

Figure 3.9: State machine of an approval request; every terminal state other than Approved blocks the call 43

Figure 3.10: Context-level (level 0) data-flow diagram of Mirsad..... 44

Figure 3.11: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4 . 44

Figure 3.12: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value..... 51

Figure 3.13: Threat model with trust boundaries A to E and the STRIDE categories of each data flow 53

Figure 3.14: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event 59

Figure 3.15: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right 59

Figure 3.16: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary 60

Figure 3.17: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons 61

Figure 3.18: Deployment architecture: the Mirsad container, the database, optional components and the clients 62

Figure 4.1: Benign false-positive rate, approval friction and utility per defence configuration (scripted agent, denying approver) 95

Figure 4.2: Approval friction on benign calls by language and defence configuration 95

Figure 4.3: Distribution of added latency per call, sequential and with 50 concurrent sessions (server in a separate process, logarithmic scale) 97

Figure 4.4: Gateway latency per benign call by defence configuration (decision plus result inspection, excluding human waiting) 97

Figure B.1: Cumulative distribution of added latency over 1,000 benign calls, server in a separate process (log scale) 151

Figure B.2: Cumulative distribution of added latency over 1,000 benign calls, server in the same process as the client (log scale) 151

Figure B.3: Benign false-positive rate, approval friction and utility per defence configuration with the denying approver (run `scripted-benign-deny`) 153

Figure B.4: Gateway time per benign call (decision plus result inspection, human wait excluded), p50 and p95, run `scripted-benign-deny` 154

Figure C.1: Dashboard sign-in page. The language switch at the top right changes the interface to Arabic before signing in, and the second button selects the colour theme .. 158

Figure C.2: Dashboard Overview after the seeded activity 159

Figure C.3: Activity and audit log 160

Figure C.4: Detail drawer of an audit event (a denied `transfer_funds` request) 161

Figure C.5: Approvals page. The live pending queue shows each request's risk level, plain-language summary and countdown to expiry with Approve and Deny buttons; the history below records who decided each request and their note 161

Figure C.6: Policy editor with the active YAML policy of the demonstration organisation and its version hash. Changes can be compared with the active version, reverted, validated and activated without restarting the gateway 162

Figure C.7: Agents page: the demonstration agent, the policy agent it is mapped to, its key prefix (the full key is shown only once, when it is created or rotated), last-seen time, and rotate, disable and delete actions 162

Figure C.8: Users page with the three roles and what each may do: administrators manage everything, approvers decide on paused actions, and auditors have read-only access to the audit log, policies and tools 163

Figure C.9: Tools page: the tools discovered from the upstream workspace server and how the active policy profiles each one for the selected agent (allowed, trust of its results, sensitivity, where it sends data, risk and approval mode) 163

Figure C.10: Detector playground scanning a benign mixed Arabic–English supplier message 164

Figure C.11: Demo console: protection switch, scenarios, conversation, tool-call timeline and attacker inbox 165

Figure C.12: Settings page: theme (system, light or dark), language, background notifications for new approvals, account details and server status 165

Figure C.13: Overview page in Arabic. Labels, key figures and time-range buttons are translated and arranged for right-to-left reading 166

Figure C.14: Activity and audit log in Arabic. The navigation moves to the right and the filters and table columns are mirrored, while tool names, agent identifiers and dates keep their left-to-right form 166

Figure C.15: Policy editor in Arabic. The surrounding interface is right-to-left, while the YAML policy stays left-to-right so that it can be edited and reviewed as code 167

Figure C.16: Overview page in the dark theme, which can follow the operating-system setting or be selected in Settings 167

Figure C.17: Approver app sign-in on a phone-sized screen. The app is served by the gateway at `/approve` and can be installed to the home screen 168

Figure C.18: Pending requests in the approver app: risk level, tool, requesting agent, the start of the plain-language summary, time since the request and countdown, a prompt to turn on push alerts, and Pending and History tabs 169

Figure C.19: Detail of a critical-risk request to delete a customer record: what the agent wants to do, why Mirsad paused it with the layer behind each reason, and Deny and Approve buttons fixed at the bottom of the screen 170

Figure C.20: Full-length capture of the same request, including the section on where the data came from (here no outside data was linked) and the collapsible list of arguments. The fixed Deny and Approve bar appears in the middle only because of the full-page capture method 171

Figure C.21: Pending requests in the Arabic interface. The layout is mirrored, summaries are written in Arabic, and tool names and e-mail addresses keep their original form.... 172

Figure C.22: Detail of the same deletion request in Arabic, with the summary, the reasons for the pause, and the Approve and Deny buttons in right-to-left order..... 173

Figure C.23: Full-length capture of the Arabic request detail, including the data-provenance section and the collapsible arguments; as in the English capture, the fixed button bar appears mid-page because of the capture method 174

Figure D.1: CCIS EXPO poster (preview rendering) 176

Figure D.2: Final presentation, slides 1–6 177

Figure D.3: Final presentation, slides 7–12 178

Figure D.4: Final presentation, slides 13–18 179

Figure E.1: Overview page of the dashboard..... 185

Figure E.2: Activity and audit log with the detail drawer of an approval decision 186

Figure E.3: Approvals page with two pending requests and the decision history 187

Figure E.4: Policies page with the YAML editor 188

Figure E.5: Agents page with the demo agent created by mirsad init --demo..... 188

Figure E.6: Detector playground scanning an Arabic business e-mail..... 189

Figure E.7: Demo console with the protection switch, scenario selector and attacker inbox 190

Figure E.8: Approver app: pending requests 191

Figure E.9: Approver app: request detail with the Approve and Deny buttons..... 192

Figure E.10: Approver app: request detail in Arabic..... 193

List of Tables

Table 1: Acronyms used in this report xvii
Table 1.1: Project scope 3
Table 1.2: Stakeholders and their interests 4
Table 1.3: Project phases, durations and responsible members (from the D2 work breakdown) 6
Table 1.4: Project milestones 6
Table 2.1: Key studies on agent-security evaluation and their implications for Mirsad 11
Table 2.2: Feature comparison of runtime guardrails, MCP gateways, research systems and Mirsad (N/D = not documented in the sources reviewed) 18
Table 3.1: Use-case catalogue 26
Table 3.2: Use-case specification UC-01 – Invoke tool via gateway 27
Table 3.3: Use-case specification UC-05 – Approve / deny action 28
Table 3.4: Use-case specification UC-08 – Manage policies 28
Table 3.5: Use-case specification UC-14 – Verify audit integrity 28
Table 3.6: Functional requirements 29
Table 3.7: Non-functional requirements 31
Table 3.8: Main design decisions 35
Table 3.9: Components, responsibilities and interfaces 36
Table 3.10: Structure of a policy document (schema version 1) 45
Table 3.11: Reason codes of the policy, detector, approval and system layers 46
Table 3.12: Taint rules evaluated for each tool call 48
Table 3.13: Data dictionary of the database tables 52
Table 3.14: STRIDE analysis of the gateway 53
Table 3.15: Agent-facing interfaces 56
Table 3.16: REST API overview 56
Table 3.17: Sprint plan 63
Table 3.18: IS499 deliverables and where they are addressed 64
Table 3.19: Workstream ownership 65
Table 3.20: Technology stack and justification 65

Table 3.21: Original and revised budget (SAR) 67
Table 3.22: Test levels and the artefacts they verify (V-model)..... 69
Table 3.23: Exit criteria and their status for the reported version 71
Table 3.24: Continuous-integration jobs..... 72
Table 3.25: Automated tests by area..... 74
Table 3.26: Planned user acceptance scenarios 75
Table 4.1: Top-level folders of the Mirsad repository..... 79
Table 4.2: Issues found and fixed during integration 89
Table 4.3: Defence configurations and approver modes 91
Table 4.4: Evaluation metrics 91
Table 4.5: Detector results on held-out splits (ensemble, threshold 0.5)..... 92
Table 4.6: Attack success rate (ASR) by defence configuration 93
Table 4.7: Benign results of the scripted agent (48 tasks, 68 calls per configuration) 93
Table 4.8: Full protection by language (scripted agent) 95
Table 4.9: Added latency per tool call (1,000 benign calls) 96
Table 4.10: Security tests of the gateway by STRIDE category..... 98
Table 4.11: Keystroke-Level Model estimate of the approval decision time (seconds). 100
Table 4.12: Success criteria, targets and measured results 101
Table 4.13: Project Tasks & Task Assignments 103
Table 5.1: Outcomes against the SMART objectives..... 106
Table A.1: Mapping of standards, regulations and guidelines to Mirsad features and evidence 120
Table A.2: Project constraints and how they were handled..... 123
Table B.1: Evidence files behind this appendix..... 125
Table B.2: Requirements traceability matrix 126
Table B.3: Verification status summary 133
Table B.4: Test cases TC-01 to TC-08, policy language and engine 134
Table B.5: Test cases TC-09 to TC-17, gateway and protocol adapters..... 135
Table B.6: Test cases TC-18 to TC-23, taint tracking 138
Table B.7: Test cases TC-24 to TC-30, detector and result handling..... 139
Table B.8: Test cases TC-31 to TC-39, approvals and audit 141

Table B.9: Test cases TC-40 to TC-46, administration, UI, demonstration, benchmark and storage 144
Table B.10: Security test cases ST-01 to ST-23 146
Table B.11: Security tests by STRIDE category 148
Table B.12: Test execution summary 148
Table B.13: Statement coverage by package 149
Table B.14: Static analysis and dependency scan results 149
Table B.15: Added latency per tool call (ms) 150
Table B.16: Benign benchmark results per defence and approver 152
Table B.17: Audit verification and replay results 154
Table B.18: Results that need team-supplied attack data 155
Table E.1: Reading guide by audience 180
Table E.2: System requirements 180
Table E.3: Environment variables and defaults 182
Table E.4: Dashboard pages and roles 184
Table E.5: Ways to connect an agent 195
Table E.6: Commands of the mirsad command line 197
Table E.7: Headline results and how they are produced 199
Table E.8: Common problems and solutions 199
Table F.1: Section A – Respondent profile 202
Table F.2: Section B – Importance of capabilities (1 = not important, 5 = critical) 202
Table F.3: Section C – Operational preferences 203
Table F.4: Section D – After a demonstration (optional, 1 = strongly disagree, 5 = strongly agree) 203

List of Acronyms

Table 1: Acronyms used in this report

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
ASR	Attack Success Rate
ATLAS	Adversarial Threat Landscape for Artificial-Intelligence Systems (MITRE)
AUC	Area Under the (ROC) Curve
CEL	Common Expression Language
CI	Continuous Integration
CISO	Chief Information Security Officer
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CPU / GPU	Central / Graphics Processing Unit
CRM	Customer Relationship Management
CSP	Content Security Policy
CSV	Comma-Separated Values
DFD	Data-Flow Diagram
ECC	Essential Cybersecurity Controls (NCA)
ERD	Entity-Relationship Diagram
F1	Harmonic mean of precision and recall
FPR	False-Positive Rate
FR / NFR	Functional / Non-Functional Requirement
HMAC	Hash-based Message Authentication Code
HS256	HMAC with SHA-256 (JSON Web Token signature algorithm)
HTML	HyperText Markup Language
HTTP(S)	Hypertext Transfer Protocol (Secure)
IBAN	International Bank Account Number
IP	Internet Protocol
ISO / IEC	International Organization for Standardization / International Electrotechnical Commission
JSON	JavaScript Object Notation
JWT	JSON Web Token
KLM	Keystroke-Level Model

Acronym	Meaning
LLM	Large Language Model
MCP	Model Context Protocol
MoSCoW	Must, Should, Could, Won't prioritisation
MSA	Modern Standard Arabic
NCA	National Cybersecurity Authority (Saudi Arabia)
NFKC	Unicode Normalization Form KC (compatibility composition)
NIST	National Institute of Standards and Technology
OPA	Open Policy Agent
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PDPL	Personal Data Protection Law (Saudi Arabia)
PEP	Policy Enforcement Point
PWA	Progressive Web App
RBAC	Role-Based Access Control
ReDoS	Regular-Expression Denial of Service
REST	Representational State Transfer
RMF	Risk Management Framework (NIST AI RMF)
ROC	Receiver Operating Characteristic
RTL	Right-to-Left
SAR	Saudi Riyal
SAST	Static Application Security Testing
SDAIA	Saudi Data and AI Authority
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SPAP	Senior Project Agreement and Proposal
SQL	Structured Query Language
SRS	Software Requirements Specification
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security
UAT	User Acceptance Testing
UC	Use Case

Acronym	Meaning
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
VAPID	Voluntary Application Server Identification (Web Push)
VAT	Value-Added Tax
VM	Virtual Machine
WCAG	Web Content Accessibility Guidelines
XACML	eXtensible Access Control Markup Language
YAML	YAML Ain't Markup Language

Chapter 1 – Introduction

1.1 Background

Large language models (LLMs) have moved quickly from conversational assistants to *agents*: software that receives a goal in natural language, plans a sequence of steps, and carries those steps out by calling tools. A modern agent can read a mailbox, search a customer relationship management (CRM) system, open web pages and documents, call internal APIs, send messages and even initiate payments. Two technical developments made this practical. The first is native *function calling*, in which the model emits a structured request (a tool name plus JSON arguments) that the host application executes and whose result is fed back to the model, following the reason-and-act loop popularised by ReAct [1] and tool-learning work such as Toolformer [2]. The second is the Model Context Protocol (MCP) [3], an open protocol that standardises how agents discover and invoke tools hosted by independent servers, so the same agent can be connected to many enterprise systems without bespoke integration code.

This capability creates a new security problem. An agent treats everything placed in its context window as potential instructions, including the *content returned by its tools*. If an attacker can influence any of that content, for example by sending an e-mail to the organisation, editing a public web page, uploading a CV, or writing a product review, the attacker can embed instructions that the agent may follow as if they came from its user. This class of attack, *indirect prompt injection*, was first demonstrated against real LLM-integrated applications by Greshake et al. [4] and is ranked first (LLM01) in the OWASP Top 10 for LLM Applications [5]. Standardised benchmarks such as AgentDojo [6] and InjecAgent [7] have since shown that tool-using agents built on leading models remain vulnerable, and that the risk is greatest when three ingredients are present together: access to private data, exposure to untrusted content, and the ability to communicate externally [8].

The Kingdom of Saudi Arabia is investing heavily in artificial intelligence under Vision 2030 [9], with the Saudi Data and AI Authority (SDAIA) issuing AI ethics principles [10] and the National Cybersecurity Authority (NCA) mandating the Essential Cybersecurity Controls (ECC) for national entities [11]. Banks, government service providers and customer-support centres in the Kingdom serve their customers largely in Arabic, so the agents they adopt will need to read and write Arabic. Two facts make this context distinctive. First, the Personal Data Protection Law (PDPL) [12] makes unauthorised disclosure of personal data, exactly the outcome of a successful exfiltration attack, a regulatory violation. Second, almost all published prompt-injection defences and benchmarks are English-centric, while research on multilingual jailbreaks shows that

safety measures trained mainly on English degrade on other languages, scripts and transliterations [13], [14], [15].

Mirsad (Arabic مرصاد, "watchtower" or "observation post") is our response: a runtime security gateway that sits between an agent and its tools, inspects every tool call and every tool result, and enforces the organisation's security policy at the point where the agent acts on the world.

1.2 Problem Statement

Organisations that connect LLM agents to business systems currently have no practical, language-aware control point that can stop a hijacked agent from *acting* on injected instructions. The problem has four parts.

1. **Conventional controls cannot see the attack.** The malicious instruction travels as ordinary data (an e-mail body, a web page) and the harmful action is issued by an authorised agent using valid credentials. Web application firewalls, identity and access management and data-loss prevention tools observe legitimate traffic and therefore do not intervene.
2. **Most defences classify text instead of governing actions.** Prompt-injection classifiers and guardrail frameworks operate at the model boundary and produce a probability that a text is malicious. A classifier that misses a novel phrasing provides no further protection, and none of these tools expresses rules such as "this agent may read the CRM but may never export more than 50 records" or "never e-mail data that came from the CRM to an external domain".
3. **Architectural defences are hard to adopt.** Research designs that separate trusted control flow from untrusted data, such as CaMeL [16] and the design patterns of Beurer-Kellner et al. [17], are strong but require the agent to be rebuilt around a specific architecture, which is unrealistic for organisations that buy or reuse agents.
4. **Arabic is largely untested.** Detectors and benchmarks are trained and evaluated almost exclusively on English, leaving Modern Standard Arabic, Gulf dialect, Arabizi and mixed Arabic-English injections without measured protection.

The project therefore asks: *can a drop-in gateway, placed at the standard tool-calling boundary and requiring no change to the agent, measurably reduce the success of indirect prompt-injection attacks in Arabic and English while keeping legitimate work flowing, adding little latency, and leaving an auditable record of every decision?*

1.3 Project Objectives

The proposal (SPAP form) listed six objectives, O1–O6. Following the report template, they are consolidated below into three SMART objectives; the original objectives each one covers are shown in brackets, and the measurable targets are the success criteria agreed in the proposal.

1. **Enforce action-level security at the tool boundary (O1, O2, O3).** By milestone M5 (March 2027), deliver a model-agnostic gateway that intercepts 100% of the tool calls and tool results of agents that use MCP (stdio and HTTP) or OpenAI-style function calling. The gateway enforces a declarative YAML policy with per-tool and per-parameter constraints and rate limits, uses taint tracking to block or escalate flows of untrusted or confidential data to external destinations, and adds a median latency of no more than 200 ms per tool call.
2. **Detect injected instructions in Arabic and English (O4).** By milestone M4 (February 2027), build a bilingual (Modern Standard Arabic, Gulf dialect, Arabizi, mixed Arabic-English and English) prompt-injection detector, together with a documented, labelled dataset, that reaches an F1 score of at least 0.85 on held-out Arabic and English test sets that were written separately from the training data.
3. **Keep humans in control and prove the effect (O5, O6).** By milestone M6 (April 2027), provide a mobile approval application and a web dashboard with a tamper-evident audit log in which every blocked or approved action can be replayed. On an Arabic-extended agent benchmark (Mirsad-Bench), show that the gateway reduces attack success by at least 80% relative to an unprotected baseline while wrongly blocking no more than 5% of benign actions.

1.4 Scope and Constraints

Table 1.1 summarises what is in and out of scope.

Table 1.1: Project scope

	Items
In scope	Gateway core and adapters (MCP proxy, OpenAI-compatible proxy, Guard API and Python SDK); declarative policy language and engine; taint tracking; Arabic/English injection detector and its dataset; human-approval workflow with an installable mobile web application; administration dashboard; hash-chained audit log with verification and replay; a simulated enterprise workspace and demonstration agent; the Mirsad-Bench benchmark; documentation and an open-source release.
Out of scope	Protecting the LLM itself from direct jailbreaks typed by its own user; network- and host-level security; securing model training data or weights; production-grade high availability, horizontal scaling and multi-tenant isolation; integration with real customer data.

Technical constraints. The project is developed by five students on personal laptops without GPUs. The demonstration agent and an LLM smoke run therefore use an open-weight model (Qwen2.5-7B) run locally through Ollama, the main benchmark uses a deterministic scripted agent, and the detector must run on a CPU within the latency budget. MCP is a young, evolving specification, so protocol handling is isolated behind an adapter layer. The demonstration environment uses simulated data only.

Organisational constraints. Work is limited to two academic semesters with bi-weekly supervisor meetings and a modest budget (Section 3.4.6 and Appendix A.2). Each workstream has a single owner, so cross-training and code review are needed to avoid single points of failure.

Ethical and legal constraints. No real personal data is used; all customers, e-mails and accounts in the demonstration workspace and benchmark are fictional and use reserved `.example` domains. Attack payloads are used only against the project's own simulated environment. The gateway must not become a privacy risk itself, so secrets are never logged and personal identifiers in the audit log are masked in line with the PDPL [12]. AI-assisted work is disclosed (Section 3.6).

1.5 Stakeholders and User Personas

Table 1.2 lists the stakeholders, their interests and how Mirsad serves them.

Table 1.2: Stakeholders and their interests

Stakeholder	Interest in Mirsad	How Mirsad serves them
Chief Information Security Officer (CISO) / security team	Allow AI agents without creating an uncontrolled data-exfiltration channel; evidence for NCA ECC compliance	Central policies, blocked-attack statistics, tamper-evident audit evidence
Approvers (line managers, security analysts)	Decide quickly on risky actions without reading logs	Push alert with a plain-language Arabic/English summary and one-tap approve/deny
Agent developers / integrators	Secure an existing agent without rewriting it	Drop-in MCP proxy, OpenAI-compatible endpoint, Python SDK
Business users (employees using the agent)	Agent keeps working for legitimate tasks	Low false-positive rate; only high-risk actions are paused
Compliance officers / internal auditors	Reconstruct who allowed what and why	Searchable, exportable, verifiable audit log with decision replay
Data protection officer	PDPL compliance, no leakage of personal data	Taint rules for confidential data, masking of identifiers in logs
IT operations	Simple deployment and low overhead	Single-service deployment (one gateway container plus a database), health endpoint, median added latency target of at most 200 ms
Customers of the organisation (indirect)	Their personal data is not leaked or misused	Exfiltration and destructive actions blocked or escalated

1.5.1 Stakeholder Personas

Persona 1 – Noura Al-Qahtani, Security Operations Analyst (primary user of the approval app). Noura, 29, works in the security operations centre of a Riyadh bank that has deployed an AI assistant for its customer-support team. She is comfortable with

security tools but cannot read every log line and is often away from her desk. She needs to know immediately when the assistant attempts something risky, to understand in a few seconds *what* the agent wants to do and *why* it was paused, and to decide from her phone. She reads Arabic and English and prefers Arabic on mobile. *Goals:* fast, confident decisions; no surprises in the audit. *Frustrations:* alerts without context; false alarms. *Mirsad features:* push notification, bilingual plain-language summary, "where the data came from" section, decision in under 30 seconds.

Persona 2 – Faisal Al-Harbi, AI Platform Engineer (agent developer). Faisal, 33, integrates LLM agents with the bank's CRM and e-mail through MCP servers and does not want to rewrite agents each time security requirements change. *Goals:* put a control point in front of existing agents by changing a URL; express rules as configuration; test policies safely. *Frustrations:* security reviews that block releases; tools that only work with one LLM vendor. *Mirsad features:* MCP and OpenAI-compatible proxies, YAML policies with validation and monitor mode, detector playground.

Persona 3 – Huda Al-Shehri, Compliance and Internal Audit Officer. Huda, 41, prepares evidence for NCA ECC and PDPL audits. *Goals:* prove that controls work, reconstruct incidents, export evidence. *Frustrations:* logs that can be edited silently; decisions that cannot be explained. *Mirsad features:* HMAC hash-chained audit log, integrity verification, decision replay, CSV/JSON export.

1.6 Industry Relevance & IS Application

Mirsad is an information-systems solution to an organisational problem as much as a technical one. It places a *policy enforcement point* in the information flow between a new class of actor (the AI agent) and existing business systems, so that the organisation's rules, rather than the model's judgement, decide what data may leave and which actions need a human. This mirrors established IS governance practice: segregation of duties (the agent proposes, a human approves high-risk actions), least privilege (per-tool and per-parameter permissions), and accountability (a verifiable audit trail).

The work is directly relevant to Saudi organisations adopting AI under Vision 2030 [9]. The audit and approval features support NCA ECC requirements for event logging, monitoring and access control [11]; taint tracking and identifier masking support PDPL obligations [12]; and human oversight aligns with SDAIA's AI ethics principles of accountability and human-centricity [10] and with the "Govern" and "Manage" functions of the NIST AI Risk Management Framework [18]. Because Mirsad is model-agnostic and deployable as a single service, it suits banks, government service portals, telecom and e-commerce customer support, sectors in which agents would handle Arabic-language customer data.

1.7 Project Timeline (Gantt Chart)

The project follows the two-semester plan in the approved proposal: Senior Project I (Fall 2026) covers investigation, requirements, design and a working prototype; Senior Project II (Spring 2027) covers the full implementation, testing, evaluation and final delivery. Table 1.3 lists the phases with their durations and responsible members, Table 1.4 lists the milestones and the objectives they evidence, and Figure 1.1 shows the schedule, task owners and milestones. Members are abbreviated as BS (Bader Alshuraie), AO (Abdullah Naif Abdullah Al-Orabi), BY (Bader Abdullah Abbod Alyamini), MA (Mohammed Yasser Alanssari) and AB (Abdulmalik bin Saedan); ALL means the whole team. The weekly breakdown against the IS499 deliverables D1–D6 is given in Section 3.4.

Table 1.3: Project phases, durations and responsible members (from the D2 work breakdown)

Phase	Planned dates	Duration	Responsible
Project management (whole project)	1 Sep 2026 – 20 May 2027	37 weeks	AB
1 Investigation and requirements	1 Sep – 25 Oct 2026	8 weeks	ALL (threat model BY, AB; benchmark and dataset design AB, AO)
2 Design	18 Oct – 15 Nov 2026	4 weeks	BS, BY, MA
3 Prototype (Senior Project I)	15 Nov – 24 Dec 2026	6 weeks	BS, BY, AO, ALL
4 Implementation I	10 Jan – 20 Feb 2027	6 weeks	AO, BY, BS
5 Implementation II	1 Feb – 20 Mar 2027	7 weeks	MA, AB, BS, BY
6 Testing and evaluation	10 Mar – 20 Apr 2027	6 weeks	ALL (benchmark AB, detector evaluation AO, penetration test AB, BY, MA)
7 Finalization	10 Apr – 20 May 2027	6 weeks	MA, BS, ALL

The dates are the baseline of the project plan. Actual progress ran ahead of it: on the evidence date of this report (2026-10-14T07:58:56+00:00), before the M1 baseline date, a working and tested prototype of the components planned for phases 3 to 5 already existed (Chapter 4), while the results that depend on team-authored attack data, which M4 and M6 require, were still open. D2 (Section 3.5) records progress against the baseline.

Table 1.4: Project milestones

ID	Milestone	Target date	Objectives
M1	Proposal approved; threat model and SRS agreed	20 Oct 2026	1, 2, 3 (scoping)
M2	Architecture and policy language approved	15 Nov 2026	1
M3	Prototype intercepts MCP traffic with basic policies	20 Dec 2026	1

ID	Milestone	Target date	Objectives
	(Senior Project I demo)		
M4	Taint tracking and Arabic/English detector integrated	20 Feb 2027	1, 2
M5	Approval app, dashboard and audit log complete	20 Mar 2027	1, 3
M6	Benchmark results and security testing complete	20 Apr 2027	2, 3
M7	Final report, poster, presentation and open-source release	20 May 2027	all

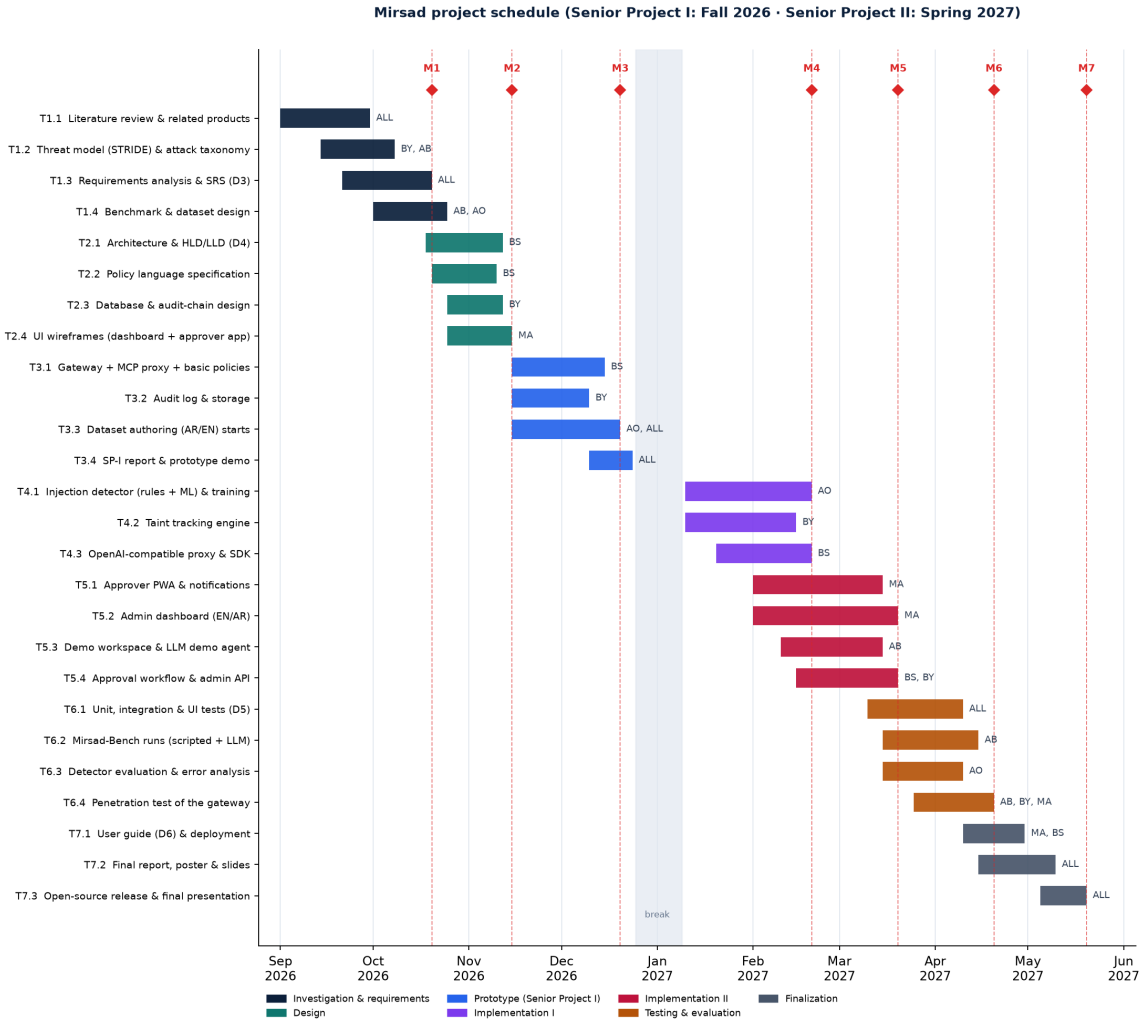


Figure 1.1: Project schedule with task owners (abbreviations as in the text) and milestones M1–M7

Chapter 2 – Literature Review

This chapter reviews the work that Mirsad builds on. Section 2.1 surveys previous studies by theme, Section 2.2 compares existing products and research systems with Mirsad, and Section 2.3 states the gap that Mirsad addresses and the limits of its contribution.

The sources come from six research tracks compiled by the team and fact-checked against primary records (publisher pages, arXiv, standards bodies, official repositories and vendor documentation) on 11 October 2026. Numbers quoted from papers are the values their authors report; the team has not reproduced them, and they are tied to model versions that age quickly. Products are described only as their documentation describes them. Preprints and practitioner blogs are cited where they are the original or most current record of an influential result.

2.1 Overview of Previous Studies

2.1.1 LLM agents and tool use

An LLM agent couples a language model with tools that it can invoke. ReAct [1] established the pattern that most agent frameworks follow: the model alternates between free-text reasoning and actions, and the observation returned by each action is fed into the next reasoning step. On two decision-making benchmarks, ReAct with one or two in-context examples beat imitation- and reinforcement-learning baselines by 34 and 10 absolute points of success rate. Toolformer [2] showed that a model can learn, with little human annotation, when to call an external API, which arguments to pass and how to use the result. Both papers treat tool use as a capability problem, and neither has a threat model for tool outputs: ReAct assumes that observations are honest, and Toolformer's tools are read-only. Nothing in the reason-act-observe loop separates data returned by a tool from instructions, which is the root cause of indirect prompt injection.

Two interfaces carry this loop in practice. OpenAI-style function calling lets the model emit a tool name and JSON arguments that the host application executes. The Model Context Protocol (MCP) [3] standardises discovery and invocation across independent servers: a host uses clients to connect over JSON-RPC 2.0 to servers that expose tools, resources and prompts. The current revision (2026-07-28) is stateless, so each request carries its own protocol version and capabilities. The specification is direct about security. Clients must treat tool annotations as untrusted unless they come from a trusted server, there should always be a human able to deny a tool call, and clients should show tool inputs to the user, validate tool results before passing them to the model and log tool usage for audit. It also says that the protocol cannot enforce these principles and leaves them to implementers. Mirsad can be read as an implementation of these client-side recommendations, placed outside the host so that it works for any agent.

Tool use also does not work equally well in every language. In what its authors describe as the first benchmark of Arabic tool calling, accuracy fell by roughly 5–10% on average when the user wrote in Arabic, whether the tool descriptions were in Arabic or English [19]. Lower accuracy means more malformed or unexpected tool arguments, which per-parameter checks catch whatever their cause.

2.1.2 Prompt injection: origins and taxonomy

Willison named the attack class in September 2022 [20]. He compared it with SQL injection: applications build prompts by concatenating trusted instructions with untrusted input, and the model cannot tell where one ends and the other begins. He suggested that a real fix might resemble parameterised queries but doubted that it was feasible with current architectures. Perez and Ribeiro [21] gave the first systematic study, using their PromptInject framework against GPT-3, and defined two goals that later work adopted: *goal hijacking*, in which the model abandons its task for one chosen by the attacker, and *prompt leaking*, in which it reveals its hidden instructions.

The decisive step for agents was *indirect* prompt injection. Greshake et al. [4] showed that an attacker who never interacts with the model can plant instructions in content that an LLM-integrated application retrieves later, and that the retrieved text can then steer which external APIs the application calls. They built a taxonomy of consequences that includes data theft and self-propagating injections, and demonstrated attacks on Bing's GPT-4-based chat and on code-completion tools. Liu et al. [22] formalised an attack as turning the data of a *target task* into compromised data that carries an *injected task*, showed that earlier techniques are special cases of this framework, and combined them into a stronger attack. Mirsad-Bench mirrors this framing: each attack episode pairs a benign user task with an injection task. OWASP ranks prompt injection first (LLM01) among risks to LLM applications [5], and NIST's taxonomy of adversarial machine learning treats indirect injection as an attack made possible by control over resources that the model reads [23].

Across these sources, attacks can be classified on three largely independent axes, summarised in Figure 2.1. The *channel* is how the malicious text reaches the model: typed by the user (direct injection), planted in data that the agent reads (indirect injection), or hidden in tool metadata. The *technique* is how the attacker obtains compliance. The *goal* is what the attacker wants the agent to do. For a tool-using agent, the goals that matter most are data exfiltration, which needs a flow from a private source to a sink that the attacker can reach, and tool misuse, in which legitimate tools are called with harmful arguments. Indirect injection is the channel Mirsad is built for, because it lets an outsider borrow the user's own privileges, data and tools.

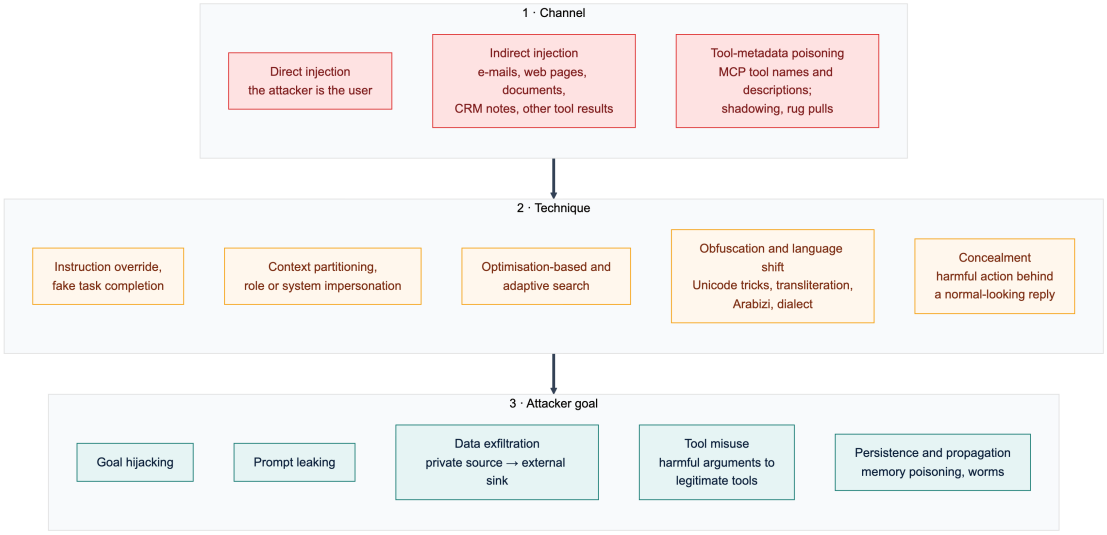


Figure 2.1: A three-axis taxonomy of prompt-injection attacks: the channel that carries the text, the technique used to obtain compliance and the attacker's goal (synthesised from the sources reviewed in Section 2.1.2)

MCP has added channels of its own. Invariant Labs disclosed *tool poisoning* [24], in which an MCP server hides instructions in a tool description that the model reads in full but the user usually does not see. The same disclosure described *shadowing*, where one server's description changes how the agent uses another server's trusted tool, and *rug pulls*, where a description changes after the user has approved it. A later case study [25] described a "toxic agent flow" through the official GitHub MCP server: an issue in a public repository led an agent to copy information from the owner's private repositories into a public pull request. No tool was compromised; the harm came from combining trusted tools with untrusted input. The authors also observed that users often switch tool confirmations to "Always Allow", which argues for asking a human only about risky calls.

Willison's "lethal trifecta" [8] condenses the exfiltration risk: an agent that has access to private data, is exposed to content an attacker controls and can communicate externally can be told to read secrets and send them out, and MCP makes it easy for users to assemble all three without realising it. The risk has already appeared in production. EchoLeak [26] was a zero-click flaw in Microsoft 365 Copilot in which one crafted e-mail caused Copilot to leak internal data. The exploit evaded Microsoft's cross-prompt-injection classifier, bypassed link redaction with reference-style Markdown and used images that the client fetched automatically. Two lessons from EchoLeak shaped Mirsad: a dedicated injection classifier was bypassed in a deployed product, and the fix the authors propose, provenance-based access control, is in effect taint tracking.

2.1.3 Benchmarks for agent security

Benchmarks have moved from text tasks, where success is judged on the model's output, to agents whose success is judged on changes to an environment. Table 2.1 summarises the studies that most influenced Mirsad's evaluation.

BIPIA [27], presented as the first benchmark for indirect injection, found all 25 models tested vulnerable and traced the weakness to models that cannot distinguish informational context from instructions. InjecAgent [7] moved to tool-using agents. It separates *direct harm* through a tool from *data stealing*, a two-step pattern in which private data is first read and then transmitted; the second pattern is exactly the source-to-sink flow that taint tracking is designed to break. AgentDojo [6] replaced fixed test cases with a stateful environment in which success is checked by code against the environment's state, and reports utility, utility under attack and targeted attack success rate (ASR) together. Two of its baseline results are central to Mirsad's design. A *tool filter*, which restricts the agent to the tools a task needs, cut targeted ASR for GPT-4o to about 7% while keeping benign utility (73%, against 69% with no defence). A *prompt-injection detector* reached similar security (about 8%) but cut benign utility from 69% to 41.5%, and utility under attack from 50% to 21.1%, because of false positives. The authors also note that a tool filter fails when the tools a task needs are enough for the attack, as in 17% of their cases. Later benchmarks extended the attack surface to agent memory and planning [28], web agents [29] and MCP tool metadata [30], [31]. In MCPTox the harm is carried out through *legitimate* tools, which is why constraining the parameters of every tool matters more than blocking a suspicious one.

Large red-teaming exercises show that static benchmarks overstate robustness. NIST's Center for AI Standards and Innovation extended AgentDojo with remote-code-execution, database-exfiltration and phishing tasks. Red-teaming with the UK AI Security Institute raised ASR from 11% for the strongest baseline attack to 81%, and running each attack 25 times instead of once raised average success from 57% to 80% [32]. A public competition on *concealed* attacks, in which a harmful action is taken while the final reply looks normal, produced 8,648 successful attacks from about 272,000 attempts against 13 frontier models; every model was vulnerable [33]. When an attack is concealed, the user cannot see it in the chat transcript, so an independent record of tool calls becomes necessary.

Table 2.1: Key studies on agent-security evaluation and their implications for Mirsad

Study	Setting	Main reported finding	Implication for Mirsad
Liu et al. [22]	5 attacks, 10 defences, 10 LLMs, text tasks	No defence sufficient; detectors miss attacks or raise many false alarms	A detector cannot be the only gate
BIPIA [27]	Indirect injection in 5 scenarios, 25 LLMs	All models vulnerable	Enforce outside the model

Study	Setting	Main reported finding	Implication for Mirsad
InjecAgent [7]	1,054 cases; 17 user and 62 attacker tools	ReAct GPT-4 agent compromised in 24% of cases (47% with a fixed prefix)	Data stealing is a source-to-sink flow that can be tracked
AgentDojo [6]	4 stateful suites, 97 user tasks, 629 security cases	Tool filter: about 7% ASR, utility kept. Detector: about 8% ASR, utility 69% to 41.5%	Least privilege first; tune the detector for few false positives
ASB [28]	10 scenarios, 13 model backbones	Highest average ASR 84.30%; memory and planning also attacked	Treat memory writes as sinks
WASP [29]	Web agents in GitLab and Reddit sandboxes	Up to 86% start following injections; at most about 17% complete them	Stop the first unsafe tool call
MCPTox [30]	45 live MCP servers, 353 tools, 20 agents	Highest ASR 72.8%; harm done through legitimate tools	Parameter rules on legitimate tools
MSB [31]	12 MCP attack types, 2,000 instances	MCP-specific attacks missing from earlier benchmarks	Evaluate at the MCP boundary
NIST CAISI [32]	AgentDojo with new high-risk tasks	Red-teaming 11% to 81%; 25 attempts 57% to 80%	Report per task and over repeated attempts
Dziemian et al. [33]	Public competition, 13 frontier models	Every model vulnerable to concealed attacks	Keep an independent audit trail of tool calls

Mirsad-Bench follows the norms these studies set: success is checked on side effects in the environment, utility is reported beside security, false positives on benign tasks are measured, and results are broken down by task and language. It reuses AgentDojo's combination of user tasks and injection tasks on a simulated Saudi company. As far as the team could determine, every benchmark reviewed here is in English; no Arabic benchmark for agent security was found.

2.1.4 Defences at the model, prompt, runtime and architecture levels

Published defences work at four levels, which differ in what an organisation deploying an agent can actually adopt.

Model level. These defences train the model to give less weight to instructions that arrive in data. The instruction hierarchy [34] defines a priority order (system above user above tool output) and trains the model to ignore conflicting lower-priority instructions. SecAlign [35] treats the problem as preference optimisation and reports ASR below 10% even against optimisation-based attacks, at a cost of two to three points on a general-knowledge benchmark. Such defences raise the baseline for everyone who uses the model, but they need the provider's cooperation or open weights and GPU fine-tuning, and they give no guarantee. An organisation that connects a hosted model to MCP servers cannot apply them.

Prompt level. Spotlighting [36] marks untrusted text before it enters the context, by delimiting it, interleaving a marker character through it or encoding it, and tells the model to treat the marked text as data. With GPT-3.5 and GPT-4 it reduced ASR from over 50% to under 2% with little loss of task performance, and Microsoft offers the encoding variant as a preview option of Prompt Shields [37]. Its authors did not test adaptive attackers, and its effect on Arabic script, where marker characters interact with word shape and diacritics, has not been studied.

Runtime detectors. Classifiers screen user input, tool outputs or tool calls. Meta's Llama Prompt Guard 2 [38] is an 86M-parameter classifier with a 512-token window whose model card lists eight supported languages, none of them Arabic, and names vulnerability to adaptive attacks as a limitation. Protect AI's DeBERTa classifier [39] handles English only and is no longer maintained. PIGuard [40] addressed *over-defence*: on 339 benign sentences full of words common in injections, leading guard models fell to roughly 60% accuracy. Among commercial services, Lakera Guard (now Check Point AI Guardrails) screens tool responses and tool descriptions and states coverage of more than 100 languages, Arabic among them [41], while Microsoft states that Azure Prompt Shields was tested with English only [42]. The only open Arabic model found, Ara-Prompt-Guard [43], was trained on 33,000 machine-translated examples, supports Modern Standard Arabic (MSA) only and has no paper or independent evaluation.

The evaluation literature is consistent about detectors. Hackett et al. [44] evaded six deployed detectors, among them Azure Prompt Shield and Protect AI's models, with character-level tricks such as zero-width characters, homoglyphs, bidirectional text and diacritics; one technique reached 100% evasion against every guardrail on each threat type it was tested on. Nasr et al. [45] scaled up gradient, reinforcement-learning, search-based and human red-teaming attacks against twelve recent defences and bypassed most with over 90% success, including Spotlighting (above 95%) and Meta-SecAlign, a successor of SecAlign (96%). PIGuard was the most resistant detector at 71%. Stacking several detectors did not stop adaptive attacks, and many successful triggers did not look malicious, so they could not be caught without also flagging benign inputs.

Runtime frameworks and policy enforcement. NeMo Guardrails [46] wraps an LLM application in programmable rails written in Colang. Its authors say it should not be used as a standalone safety solution, and its documentation warns that tool results are trusted by default [47]. LlamaFirewall [48] combines Prompt Guard 2, an experimental auditor of the agent's reasoning and a scanner for generated code; on AgentDojo it reduced ASR from 17.6% to 1.75%, while utility fell from 47.7% to 42.7%. Two runtime systems enforce rules instead of classifying text. AgentSpec [49] provides a rule language of trigger, predicate and enforcement, with a pause for human inspection as one enforcement action, and prevented unsafe executions in over 90% of code-agent cases with millisecond-level overhead. Progent [50] applies least privilege through default-

deny rules over tool names and arguments, can run as a proxy in front of LLM and MCP endpoints, and sends requests to widen privileges to an approver; it reports AgentDojo ASR falling from 39.9% to 1.0%. Neither system tracks data flow, so neither can tell whether an allowed outbound call carries content supplied by an attacker.

Architecture level. These designs restructure the agent so that untrusted data cannot change its control flow. CaMeL [16] has a privileged LLM write a restricted program from the trusted request alone, a quarantined LLM parse untrusted data without tool access, and an interpreter attach capabilities (provenance and allowed readers) to every value and check policies before each tool call. It completes 77% of AgentDojo tasks with provable security, against 84% for an undefended agent. FIDES [51] labels data for integrity and confidentiality, propagates the labels conservatively through the planner and enforces two generic policies: consequential tools run only from a trusted context, and egress tools send data only to recipients allowed to read it. When a policy is violated, FIDES aborts. Beurer-Kellner et al. [17] generalised these ideas into six design patterns, such as plan-then-execute, while warning that human confirmation is often burdensome and prone to fatigue. A 2026 position paper [52] argues that system-level defences should form the skeleton of agentic systems, with learned models confined to constrained roles and human interaction reserved for ambiguous cases.

For Mirsad, the review leads to the following conclusions. First, detectors are useful signals but weak boundaries: every classifier tested against adaptive attackers was bypassed, and over-defence becomes alert fatigue when a flag triggers a human approval. Second, the strongest results since 2024 come from designs that ask whether an action is allowed given where its inputs came from, rather than whether a text looks malicious. Third, those designs are costly to adopt: CaMeL and FIDES require the agent to be rebuilt. Among the research designs reviewed, only Progent's proxy mode brings deterministic tool-call enforcement to unmodified agents; Section 2.2 shows that some gateway products now do the same for rules on tools and arguments.

2.1.5 Multilingual and Arabic LLM safety

Safety behaviour learned mainly from English data does not transfer evenly to other languages, scripts and dialects. Yong et al. [13] machine-translated harmful instructions into twelve languages and found that low-resource languages, combined, bypassed GPT-4's safeguards about 79% of the time, whereas MSA had a bypass rate of only 3.65%. Deng et al. [14] built MultiJail, unsafe prompts translated by native speakers into nine languages with Arabic among them, and showed that deliberately combining a harmful instruction with a multilingual prompt raised unsafe output to 80.92% for ChatGPT and 40.71% for GPT-4. The most directly relevant result comes from Al Ghanim et al. [15]. GPT-4 and Claude 3 Sonnet mostly refused harmful prompts written in standard Arabic script but produced unsafe output when the same prompts were written in Latin transliteration or Arabizi, the chat style that uses digits such as 3 and 7 for Arabic letters.

The authors attribute this to refusal behaviour tied to particular written forms of words. MSA alone is therefore not the hard case; the less standard forms that Saudi users actually type are.

Arabic safety benchmarks matured quickly in 2025 and 2026, and they point the same way. SalamahBench [53] wrote 8,270 harmful prompts in MSA and five regional varieties, including Saudi Arabic, and found robustness across varieties to depend strongly on the model. ArabicDialectSafety [54] showed that a fine-tuned MARBERTv2 encoder (macro-F1 0.95 on binary safety classification) outperformed prompted frontier LLMs, although its dialects do not include Gulf Arabic. ASAS [55] found obfuscation-based attacks the most effective against Arabic-capable models, and poor agreement between an LLM judge and human annotators. All of these benchmarks measure harmful *requests* to a model; none contains instructions hidden in tool outputs that try to hijack an agent.

Work on Arabic prompt *injection* is sparse. In the only indirect-injection study found that includes Arabic, instructions hidden in about 500 academic papers changed LLM review outcomes when written in English, Japanese or Chinese but had little or no effect in Arabic [56]. The authors do not explain the result, which may reflect weaker instruction-following in Arabic rather than stronger defences; as Arabic-capable models improve, Arabic injections should be expected to become more effective. MIPIAD [57] is the closest published design to a multilingual detector of indirect injection, combining a small Qwen2.5 classifier with lexical features (best F1 0.9205), but it was evaluated only on synthetic English and Bangla data.

For the detector itself, the CAMeLBERT study [58] found that how closely an Arabic encoder's pre-training variety matches the task data matters more than the amount of pre-training data, which is a concern for Gulf dialect and Arabizi. Mirsad's first detector therefore uses character n-grams with a linear model, combined with weighted Arabic and English rules, including rules for Gulf dialect and Arabizi. Before scoring, a normalisation step removes diacritics, elongation and invisible or bidirectional control characters, and folds letter variants, digits and look-alike characters, in response to the evasion techniques above. The choice favours CPU latency and inspectable features. The model is implemented but has not yet been trained on the team's labelled data, so the packaged detector currently runs its rules only; fine-tuning an Arabic encoder is planned as a second stage (Chapter 5).

2.1.6 Standards and Saudi regulation

International frameworks now agree on the controls that agents need. Under LLM01, OWASP's Top 10 for LLM Applications [5] recommends constraining model behaviour, validating outputs with deterministic code, enforcing least privilege, requiring human approval for high-risk actions and segregating untrusted content, and notes that it is

unclear whether fool-proof prevention exists. OWASP's Top 10 for Agentic Applications for 2026 [59] adds risks specific to agents, including agent goal hijack (ASI01) and tool misuse and exploitation (ASI02). NIST's AI Risk Management Framework [18] organises AI risk work into the Govern, Map, Measure and Manage functions, and NIST AI 100-2e2025 [23] recommends designing systems on the assumption that injection will happen, for example by giving access to untrusted data only through well-defined interfaces. With the MCP specification, these documents agree on least privilege, complete mediation, separation of untrusted content, human approval and logging. None of them enforces anything.

Saudi regulation adds concrete obligations. The NCA's Essential Cybersecurity Controls (ECC-2:2024) [11] apply to government entities and operators of critical national infrastructure. Control 2-2-3 requires least privilege, need-to-know and segregation of duties; 2-12-3 requires event logging, continuous monitoring and log retention of at least 12 months; and 4-1 covers third-party cybersecurity, under which third-party MCP servers fall. The Personal Data Protection Law (PDPL) [12], in force since September 2023, requires technical measures to protect personal data (Article 19), requires breaches to be notified (Article 20) and restricts transfers outside the Kingdom (Article 29); its Implementing Regulation [60] sets a 72-hour window for notifying SDAIA of a breach that could cause harm. SDAIA's AI Ethics Principles [10] include privacy and security, reliability and safety, fairness, and accountability, and its Generative AI Guidelines for Government [61] require human review of generative-AI output and keep data classified as restricted or higher out of these tools. These obligations sit within the digital-government goals of Vision 2030 [9]. None of the Saudi documents mentions AI agents or tool calls, so an organisation that deploys agents has to translate them into technical controls itself. Appendix A maps each relevant provision to the Mirsad feature that supports it.

2.1.7 Foundations used by Mirsad

Information-flow control and taint tracking. Denning's lattice model [62] binds every object to a security class and allows a flow only if it respects the ordering of the classes. Labels are combined with a join, so data tends to become more restricted over time, a problem known as label creep. Sabelfeld and Myers [63] surveyed language-based enforcement of noninterference and discussed declassification, the controlled release of information to a lower class. Newsome and Song [64] introduced dynamic taint analysis of unmodified software, with the source, propagation and sink vocabulary that later systems use. All three assume a program whose flows can be analysed. An LLM is opaque and may mix any input into any output, so Mirsad cannot claim noninterference and applies taint at the boundary between tool results and tool calls instead. A session becomes tainted once it has read untrusted content, which is sound but coarse, as in FIDES. Recognisable values such as e-mail addresses, IBANs and record identifiers are

tracked back to the result they came from, which is precise but incomplete. Human approval acts as an explicit, logged declassification step.

Reference monitors. Saltzer and Schroeder's design principles [65] justify the gateway architecture: complete mediation (every tool call passes through the gateway), fail-safe defaults (unlisted tools are denied), least privilege (rules per tool and parameter) and separation of privilege (a human approves high-risk calls).

Tamper-evident logging and policy languages. Schneier and Kelsey [66] combined hash chains with message authentication codes for forensic audit logs and stated the limits of any such scheme: entries written after a compromise cannot be protected, and cryptography can detect deletion but not prevent it. Mirsad's audit log chains events with HMAC-SHA-256 [67]. Because HMAC is symmetric, whoever holds the key could rewrite the chain, and truncation of the newest entries can only be detected against a copy of the head hash kept outside the gateway. For policy, XACML 3.0 [68] separates the policy enforcement point from the policy decision point and the attribute sources that inform it, resolves conflicts with combining algorithms, and lets a decision carry obligations. In Mirsad the gateway is the enforcement point and the YAML policy engine the decision point; the taint tracker and detector supply attributes, and "require approval" behaves like an obligation.

Threat modelling and human-AI interaction. Shostack's method of data-flow diagrams with trust boundaries and STRIDE threat categories [69] structures Mirsad's threat model (Section 3.3.8). For the approval workflow, Amershi et al. [70] give 18 validated guidelines for human-AI interaction, among them making clear why the system acted and supporting efficient correction and dismissal. Parasuraman et al. [71] show that automation can be set to different levels for analysis, decision and action, and warn that supervising automation brings complacency and workload problems. Mirsad therefore automates analysis and decision for most calls and asks a human only about calls that its policy marks as high-risk.

2.2 Comparative Analysis

Table 2.2 compares Mirsad with the commercial products, open-source projects and research systems closest to it. The criteria follow from Section 2.1: where the control sits; whether it enforces action-level rules on tools and their parameters; whether it tracks where data came from; whether its injection detection covers Arabic; whether it offers human approval; whether its log is tamper-evident; and whether it is open source. Each cell records only what the cited sources state as of 11 October 2026. "N/D" (not documented) means that nothing was found in the pages reviewed, not that the capability is absent, and vendor detection claims were not tested by the team. The Mirsad row describes the implemented prototype.

Table 2.2: Feature comparison of runtime guardrails, MCP gateways, research systems and Mirsad (N/D = not documented in the sources reviewed)

System	Deployment point	Action-level policy (tool / parameter)	Taint / data flow	Arabic injection detection	Human approval	Tamper-evident audit	Open source
Lakera Guard / Check Point AI Guardrails [41], [72]	Screening API for model and tool messages	Tool allow-list; parameter rules N/D	N/D	Yes (vendor claim)	N/D	N/D	No
NVIDIA NeMo Guardrails [46], [47]	Library or service around the LLM	Custom rails written in code	No; tool results trusted by default	N/D	N/D	N/D	Yes
Meta LlamaFirewall with Prompt Guard 2 [48], [38]	Library in the agent loop	N/D	No (audits reasoning)	Not listed (8 languages)	N/D	N/D	Yes
Azure Prompt Shields with Foundry agents [37], [73], [42]	Model I/O; agent tool responses (preview)	Tool allow-list; parameter rules N/D	No (Spotlighting marks only)	Tested in English only	Approval primitive; interface left to developer	N/D	No
AWS Bedrock Guardrails with AgentCore Policy [74], [75], [76], [77]	AgentCore Gateway (tool layer)	Yes: Cedar rules, default deny	Partial: session history, no provenance labels	Yes (Standard tier)	Policy pattern; no approval app documented	Partial: opt-in signed log digests	No
Google Model Armor [78]	Model I/O screening API	No per-tool or parameter rules	No (stateless)	Not among tested languages	N/D	N/D	No
Protect AI LLM Guard [79], [39]	Library on prompts and outputs	No	No	No (English only)	No	No	Yes (archived)
Invariant Guardrails and Gateway (Snyk) [80], [81]	LLM and MCP proxy	Yes: tool and argument rules	Partial: call-sequence rules	N/D	N/D	N/D	Yes (maintenance unclear)
Docker MCP Gateway [82]	MCP gateway with	Tool on/off; parameters	No	No detector	N/D	Call logs; integrity	Yes

System	Deployment point	Action-level policy (tool / parameter)	Taint / data flow	Arabic injection detection	Human approval	Tamper-evident audit	Open source
	containerised servers	via custom interceptors				N/D	
agentgateway [83]	MCP and agent proxy	Yes: CEL rules including arguments	No	N/D	N/D	N/D	Yes
IBM ContextForge [84]	MCP federation gateway with plugins	Via OPA plugin on arguments	N/D	English-only plugin	N/D	N/D	Yes
Progent (research) [50]	Library or LLM/MCP proxy	Yes: default-deny argument rules	No	Not applicable	For privilege expansion	N/D	N/D
AgentSpec (research) [49]	Hooks in the agent framework	Yes: trigger-predicate rules	No	Not applicable	Yes (user inspection)	N/D	N/D
CaMeL (research) [16], [85]	Inside the agent (custom interpreter)	Yes: policies on arguments and capabilities	Yes: data-flow graph	Not applicable	Proposed for deployment	No	Yes (unmaintained)
FIDES (research) [51]	Inside the agent (planner)	Trusted-action and permitted-flow policies	Yes: integrity and confidentiality labels	Not applicable	No (aborts)	N/D	N/D
Mirsad	Tool-layer gateway: MCP, OpenAI-compatible proxy, Guard API	Yes: YAML rules, default deny, rate limits	Yes: session and value level (coarser than CaMeL)	Yes: AR/EN rules now, model awaiting training data; F1 pending	Yes: installable bilingual PWA	Yes: HMAC chain per decision, with replay	Yes (Apache-2.0)

The systems in the table fall into five groups, each with characteristic strengths and weaknesses.

Screening services and classifiers (Lakera, Azure Prompt Shields, Google Model Armor, Bedrock Guardrails, Prompt Guard 2, LLM Guard) are easy to adopt, and two of

them (Lakera and the Standard tier of Bedrock Guardrails) are the only products in the sample that document Arabic coverage. However, they return a verdict and leave enforcement to the application, they are bypassed by adaptive attackers (Section 2.1.4), and most were tested only in English. Hosted services also receive every prompt and tool output, which matters under the PDPL's rules on transfers outside the Kingdom.

Guardrail frameworks (NeMo Guardrails, LlamaFirewall) are open source and layered, but security at the tool layer has to be programmed by hand, NeMo trusts tool results by default, components that call an LLM add latency and can themselves be injected, and neither documents an approval workflow or a durable audit store.

Cloud agent platforms (AWS AgentCore Policy, Azure Foundry). AgentCore Policy is the closest commercial analogue to Mirsad: deterministic, default-deny Cedar policies on tool names and input parameters outside the agent's code [74], temporal conditions on session history such as requiring an earlier approval action [75], and CloudTrail log files that can be validated with chained, signed digests [77]. However, it enforces only on traffic routed through an AgentCore Gateway in AWS, documents no provenance labels, models approval as a policy pattern rather than an application for approvers, and offers only opt-in, file-level log integrity. Azure Foundry offers per-tool allow-lists and an approval gate for MCP tools but leaves the approver's interface and the audit trail to the developer [73].

MCP gateways (Docker MCP Gateway, agentgateway, IBM ContextForge) are vendor-neutral, open-source infrastructure with container isolation, tool authorisation and, in agentgateway and ContextForge, rules on tool arguments. None documents taint tracking, an approval channel or a tamper-evident log, and the only documented injection filter wraps the archived English-only LLM Guard model. They are possible hosts for a component like Mirsad rather than complete alternatives to it.

Research policy and information-flow systems (Progent, AgentSpec, Invariant Guardrails, CaMeL, FIDES) give the strongest guarantees. CaMeL and FIDES track data flow precisely, and AgentSpec shows that deterministic rule enforcement, including a pause for human inspection, adds only millisecond-level overhead. On the other hand, CaMeL and FIDES require the agent to be rebuilt, at a cost in utility; Progent, AgentSpec and Invariant's sequence rules do not track where a value came from; research code is often unmaintained; and none of them evaluates the approval interface or documents a tamper-evident record.

Model-level defences such as SecAlign are absent from the table because they are properties of a model rather than components an organisation can deploy. They complement a gateway: a hardened model lowers the rate of attacks that reach the tool boundary, and the gateway contains the ones that do.

2.3 Research Gap and Positioning

The review supports six observations. Detectors and model-level defences are bypassed by adaptive attackers [45], so a classifier cannot be the security boundary. The strongest system-level guarantees require the agent to be rebuilt [16], [51], while deployable tool-layer controls enforce rules on tools and arguments but do not track provenance [50], [74]. Arabic is barely covered, and no Arabic benchmark of indirect injection against tool-using agents was found. Human approval is offered as a primitive rather than a product, and its interface is not evaluated [17]. Audit integrity is an afterthought: only AWS documents cryptographic log validation, at the level of log files. Finally, the MCP specification and Saudi regulations state obligations but leave their enforcement for agents to implementers [3], [11], [12].

Gap statement. As of October 2026, none of the products, open-source projects or research prototypes reviewed documents, in one vendor-neutral and self-hostable component, all five of the following: (1) declarative per-tool and per-parameter policy enforced at the MCP and function-calling boundary; (2) taint labels that follow untrusted tool output to externally visible sinks; (3) an injection detector designed for Arabic as well as English; (4) a ready-to-use human approval channel outside the agent; and (5) a tamper-evident audit trail of every decision. AWS AgentCore Policy covers the first well and the third and fifth partly, but only inside AWS. agentgateway and Invariant cover the first in open source, and Invariant also covers part of the second. CaMeL covers the second most rigorously, but only as a research agent architecture.

Positioning. Mirsad's contribution is integration and localisation rather than a new security primitive. It combines techniques that the literature has validated separately: default-deny rules on tools and parameters in the style of Progent, Invariant and Cedar; taint rules adapted from FIDES's trusted-action and permitted-flow policies; spotlighting as an optional extra layer; Arabic-aware normalisation and Arabizi rules that respond to the documented evasion techniques; human approval treated as a logged declassification step; and an HMAC-chained audit log with decision replay. It places them in one open-source gateway that needs no change to the agent, and evaluates the result on Arabic and English scenarios with a method adapted from AgentDojo. Mirsad does not claim to be the only tool-layer firewall, the only system with per-parameter policy or the only Arabic-capable detector, because AgentCore Policy, agentgateway, Invariant, Lakera and Bedrock Guardrails each contradict one of those claims. Its taint tracking is also weaker than CaMeL's, because a proxy cannot see the data flow inside the model.

Limits. Several limits follow from this position and from the literature reviewed.

- *Coarse taint causes approval friction.* Session-level taint over-approximates, and label creep turns into approval requests; Section 4.2.4 measures this friction on Mirsad-Bench. Whether approvers can handle it without fatigue has not yet been tested with users.

- *Value matching is incomplete.* An agent that paraphrases, translates or re-encodes data defeats value-level matching, and only the session-level rules then apply.
- *The detector is a signal, not a boundary.* Attackers who can query it will find inputs that evade it, as they have for every detector in Section 2.1.4.
- *Detection accuracy and attack success are not yet measured.* Both depend on attack-labelled data written by the team (Section 3.6.4); Sections 4.2.2 and 4.2.3 report them once that data exists.
- *Scope.* Mirsad mediates tool calls only. Backdoors planted during training, supply-chain compromise of MCP servers, tools that bypass the gateway, and harm that occurs only in text, such as a biased summary, are outside its reach.
- *Audit trust.* The HMAC chain reveals tampering only by parties who do not hold the key, and truncation of the newest events needs an external anchor.
- *A moving landscape.* The products compared change month to month, and several of the open-source projects reviewed have been archived or have lost their hosted services. The comparison reflects the documentation available on 11 October 2026.

Chapter 3 – Methodology

This chapter explains how the project was approached, how requirements were obtained, how the system is designed, how it was built and tested, and which ethical, legal and security considerations shaped it.

3.1 Research Design / Project Approach

Mirsad is primarily an **implementation project with a quantitative evaluation**, i.e. a design-science approach [86]: an artefact (the gateway) is built to address an identified organisational problem and is then evaluated against criteria fixed in advance. A purely qualitative design would not answer the central question of Section 1.2, which is about measurable reductions in attack success, false positives and latency. A purely experimental study without a working artefact would not satisfy the IS499 learning outcomes, which require designing, implementing and evaluating an information system.

The evaluation is quantitative and comparative:

- **Controlled experiments on Mirsad-Bench.** The same user tasks and attacks are run against an unprotected agent (baseline) and against the agent behind Mirsad, under several configurations (full protection and ablations with single layers enabled). Two agent types are used: a deterministic *worst-case compliant* agent that always obeys any injected instruction that reaches it, which isolates the effect of the gateway from the model's own robustness, and a real LLM agent (Qwen2.5-7B), so far exercised only in a smoke run of 4 tasks.
- **Classifier evaluation.** The injection detector is evaluated with precision, recall, F1, false-positive rate and ROC-AUC on held-out test sets written separately from the training data, plus, once the team adds it, an external public English dataset for out-of-distribution evidence.
- **Performance measurement.** Added latency is measured over HTTP against a direct tool call.
- **Verification.** Automated unit, integration and API tests, security tests of the gateway itself, and audit-replay checks confirm that the functional and non-functional requirements are met.

The success criteria in Section 1.3 were set in the proposal *before* implementation, which protects the evaluation from being tuned to the results.

3.2 Data Collection / Requirements Gathering

3.2.1 Elicitation methods

Mirsad addresses an emerging threat for which there is no installed user base to observe, so requirements were gathered from several complementary sources rather than from a single existing system:

1. **Document and standards analysis.** The approved proposal (SPAP), the OWASP Top 10 for LLM Applications [5], the MCP specification [3], the NCA Essential Cybersecurity Controls [11], the PDPL [12] and the NIST AI RMF [18] were analysed for obligations that a control point must satisfy (logging, access control, human oversight, personal-data protection).
2. **Literature and benchmark analysis.** Attack taxonomies and threat models from Greshake et al. [4], AgentDojo [6] and InjecAgent [7] were used to derive the attacker goals the gateway must defeat (exfiltration, destructive actions, fraud, phishing, persistence) and the success metrics.
3. **Analysis of existing solutions.** The documentation of commercial and open-source guardrails was reviewed (Section 2.2) to identify missing capabilities, which became the differentiating requirements (action-level policy, taint tracking, Arabic detection, approval, verifiable audit).
4. **Threat modelling.** A STRIDE analysis of the data-flow diagram (Section 3.3.8) produced the security requirements of the gateway itself (authentication, integrity of the audit log, fail-safe approvals, input limits).
5. **Personas and scenarios.** The personas of Section 1.5.1 and end-to-end usage scenarios of the simulated workspace were used to write use cases and to set usability targets such as deciding on an approval in under 30 seconds.
6. **Prototyping and supervisor reviews.** Under the agreed process, requirements are refined iteratively: working increments, starting with the Senior Project I prototype at milestone M3, are demonstrated in the bi-weekly supervisor meetings, and the resulting changes are recorded as backlog items.

Selection criteria. Sources were chosen by three criteria. The first was authority over the target setting: the standards and regulations that a Saudi organisation deploying agents must answer to or is likely to adopt (OWASP Top 10 for LLM Applications, NCA ECC, PDPL, NIST AI RMF and the MCP specification itself). The second was relevance to tool-using agents and *indirect* injection rather than chat-only jailbreaks. The third, for benchmarks, was an evaluation method that judges attack success on the side effects of tool calls and reports utility beside security, as AgentDojo and InjecAgent do, so that the method could be adapted to Arabic scenarios.

Tools. Research notes and the product comparison were kept as Markdown files under version control (`docs/research/`); the use-case, data-flow and threat-model diagrams were drawn in Mermaid (`docs/diagrams/`); and the requirements, use cases and acceptance criteria were written as tables in the report sources, so that the SRS (D3) and this chapter share one copy.

Primary stakeholder data. No interviews have been held and the stakeholder questionnaire in Appendix F has not been administered yet, so the requirements rest on the documentary sources above rather than on practitioner input. The team plans to administer the questionnaire, in particular its Section D, together with the user study described in Section 4.2.8, and to report aggregate results with the number of respondents.

3.2.2 Threat model and assumptions

The gateway protects against an **external attacker who can place content in data sources that the agent reads** (send an e-mail, publish or edit a web page, upload a document, write a review or support ticket, or poison a CRM note through a public form), which is the indirect prompt-injection model of [4]. The attacker cannot modify the agent's code, its system prompt, the gateway or its configuration, and cannot read the gateway's secrets. The attacker succeeds if the agent performs an action that serves the attacker: sending data to an attacker-controlled destination, deleting or corrupting records, transferring money, sending phishing messages, or planting instructions for later. The LLM itself is treated as *untrusted after it has read untrusted content*: Mirsad does not try to make the model robust, it constrains what the model's tool calls can achieve. Insider attacks on the audit log are considered up to the point of an attacker who does not hold the HMAC key.

3.2.3 Use-case model

Figure 3.1 shows the actors and use cases. Besides the four human roles (approver, administrator, auditor and agent developer) the AI agent is modelled as a system actor, and the attacker is modelled as a *misuse actor* following the misuse-case technique: each misuse case is linked to the use case it threatens and to the security use case that mitigates it.

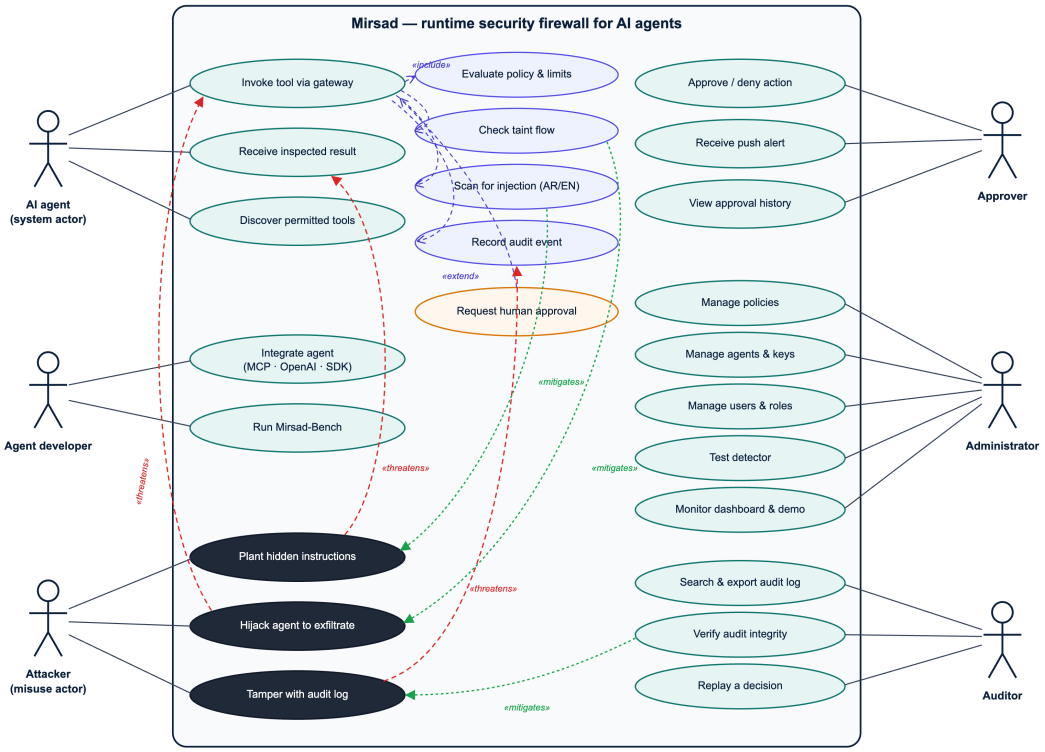


Figure 3.1: Use-case diagram with misuse cases (black) and the security use cases that mitigate them

Table 3.1: Use-case catalogue

ID	Use case	Primary actor	Summary
UC-01	Invoke tool via gateway	AI agent	The agent calls a tool; Mirsad authenticates the agent, evaluates policy, rate limits and taint, possibly asks for approval, executes the call and returns the inspected result.
UC-02	Receive inspected result	AI agent	Mirsad scans the tool result, sanitises or blocks injected instructions, labels trust and sensitivity, and returns spotlighted content.
UC-03	Discover permitted tools	AI agent	The agent lists tools and sees only those allowed for it.
UC-04	Integrate agent	Agent developer	Connect an agent through the MCP proxy, the OpenAI-compatible endpoint or the SDK.
UC-05	Approve / deny action	Approver	Review a pending high-risk action on the phone and decide.
UC-06	Receive push alert	Approver	Get a real-time notification when an action is paused.
UC-07	View approval history	Approver	Review past decisions and notes.
UC-08	Manage policies	Administrator	Edit, validate, activate or roll back the YAML policy.
UC-09	Manage agents and keys	Administrator	Register agents, issue, rotate and revoke API keys.

ID	Use case	Primary actor	Summary
UC-10	Manage users and roles	Administrator	Create approvers, auditors and administrators.
UC-11	Test detector	Administrator	Paste Arabic or English text and see the detector's score and highlighted spans.
UC-12	Monitor dashboard and demo	Administrator	Watch live statistics; run the demonstration agent in protected and unprotected modes.
UC-13	Search and export audit log	Auditor	Filter events and export evidence.
UC-14	Verify audit integrity	Auditor	Recompute the HMAC hash chain and locate any tampering.
UC-15	Replay a decision	Auditor	Re-evaluate a stored decision with its stored policy version and inputs.
UC-16	Run Mirsad-Bench	Agent developer	Reproduce the evaluation with scripted or LLM agents and ablations.
MC-01	Plant hidden instructions	Attacker	Place instructions in an e-mail, web page, document or CRM note the agent will read (mitigated by UC-02).
MC-02	Hijack agent to exfiltrate	Attacker	Make the agent send confidential data or money to the attacker (mitigated by UC-01 policy, taint and approval).
MC-03	Tamper with audit log	Attacker / insider	Modify or delete audit records to hide an incident (mitigated by UC-14).

Table 3.2, Table 3.3, Table 3.4 and Table 3.5 specify the four core use cases.

Table 3.2: Use-case specification UC-01 – Invoke tool via gateway

Field	Description
Actors	AI agent (primary); Approver (secondary, only when escalated)
Preconditions	The agent is registered and holds a valid API key; an active policy exists; the upstream tool server is reachable.
Trigger	The agent's model emits a tool call (MCP `tools/call`, an OpenAI `tool_calls` entry, or a Guard API request).
Main flow	1. Adapter authenticates the agent key and builds a `ToolCall`. 2. Policy engine resolves the tool profile, checks that the tool is allowed, evaluates parameter constraints, rate limits and global rules, and resolves the sink. 3. Taint tracker evaluates the call against the session's provenance state. 4. Gateway combines reasons (deny-overrides) and computes the risk score. 5. The decision is appended to the audit log. 6. The call is forwarded to the upstream tool. 7. UC-02 inspects the result. 8. The inspected result is returned to the agent.
Alternative flows	A1 (deny): at step 4 the action is DENY; the agent receives a bilingual explanation and the call never reaches the tool. A2 (approval): at step 4 approval is required; UC-05 is triggered and the call waits; approved → continue at 6, denied or expired → A1. A3 (monitor mode): the would-be decision is logged but the call is always forwarded.
Exceptions	E1 invalid key → HTTP 401, audited as authentication failure; E2 upstream error → error result returned to the agent and audited; E3 request larger than 1 MB → HTTP 413.
Postconditions	Exactly one `tool_call.decision` event (and, if executed, one `tool_result.inspection` event) exists in the hash chain for the call.

Field	Description
Requirements	FR1.1–FR1.3, FR2.1–FR2.4, FR3.3, FR5.1, FR6.1, NFR1, NFR3

Table 3.3: Use-case specification UC-05 – Approve / deny action

Field	Description
Actors	Approver (primary)
Preconditions	The approver is logged in to the approval app (role approver or admin); a request is pending.
Trigger	The gateway escalates a call (REQUIRE_APPROVAL) and publishes `approval.requested`.
Main flow	1. The app shows a notification and adds the request to the pending list with a risk badge and a countdown. 2. The approver opens it and reads the plain-language summary ("what", "why paused", "where the data came from"). 3. The approver optionally expands the raw arguments. 4. The approver taps Approve or Deny, optionally adds a note, and confirms. 5. The server records the decision, resumes the waiting call and audits `approval.decided`.
Alternative flows	A1: the request expires before step 4 → status EXPIRED, the call is denied. A2: another approver decides first → the app refreshes and shows the outcome.
Postconditions	The approval record stores the decision, the approver's username, the time and the note.
Requirements	FR5.1–FR5.3, NFR4, NFR6, NFR7

Table 3.4: Use-case specification UC-08 – Manage policies

Field	Description
Actors	Administrator
Preconditions	Logged in with role admin.
Main flow	1. Open the Policies page; the active YAML and version history are shown. 2. Edit the YAML. 3. Press Validate; errors are listed with line numbers. 4. Review the diff against the active version. 5. Press Activate and confirm. 6. The new version is stored, the gateway hot-swaps its policy and `policy.activated` is audited.
Alternative flows	A1: validation fails → activation disabled. A2: roll back by activating an earlier version. A3: switch to `mode: monitor` to trial a policy without enforcement.
Requirements	FR2.1, FR2.5, FR7.1

Table 3.5: Use-case specification UC-14 – Verify audit integrity

Field	Description
Actors	Auditor or administrator
Main flow	1. Press "Verify chain" (or run `mirsad audit verify`). 2. The server recomputes the HMAC of every event in sequence order and compares it with the stored hash and the next event's `prev_hash`. 3. The result shows the number of events checked, the head hash, and either "intact" or the first sequence number where the chain breaks.
Postconditions	An `audit.verified` event records the check.

Field	Description
Requirements	FR6.1, FR6.3, NFR3, NFR7

3.2.4 Requirements specification

Functional requirements

Functional requirements are grouped by the eight capabilities in the proposal (FR1–FR8) and refined into testable sub-requirements. Priorities use the MoSCoW scale (M = must, S = should, C = could). Each requirement has an acceptance criterion; the test cases that verify it are listed in the requirements traceability matrix (Appendix B of the final report; deliverable D5, Section B.2).

Table 3.6: Functional requirements

ID	Requirement	Pri.	Acceptance criterion
FR1.1	The gateway shall act as an MCP server towards agents and an MCP client towards upstream MCP servers over stdio and streamable HTTP, forwarding `tools/list` and `tools/call`.	M	An MCP client connected through Mirsad lists and calls upstream tools; results are identical to a direct connection when the policy allows.
FR1.2	The gateway shall expose an OpenAI-compatible `/v1/chat/completions` endpoint that inspects `tool` messages before forwarding and checks returned `tool_calls`.	M	A denied tool call is removed from the response and replaced by an explanation; tool messages are sanitised.
FR1.3	The gateway shall expose a Guard API (`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools/{tool}/invoke`) and a Python SDK for frameworks that execute tools themselves.	M	SDK `guard` decorator blocks a denied call by raising `MirsadBlocked`.
FR1.4	Each agent shall only discover the tools its policy allows (least-privilege discovery).	S	`tools/list` and `GET /v1/tools` omit tools with `allow: false`.
FR2.1	The system shall load policies written in a declarative YAML language with per-agent tool entries, glob patterns, defaults, global rules and a monitor mode.	M	All shipped policies validate; invalid documents are rejected with line-numbered messages.
FR2.2	The policy engine shall evaluate per-parameter constraints (`max`, `min`, `max_length`, `enum`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `type`, `required`), each with a configurable violation action (deny or require approval).	M	Each operator has passing and violating test cases.
FR2.3	The policy engine shall enforce sliding-window rate limits per agent and tool.	M	The (n+1)-th call inside the window is denied with code `RATE_LIMIT`.
FR2.4	Reasons from all layers shall be combined with a deny-overrides algorithm, and every decision shall carry machine-readable reason codes and bilingual explanations.	M	A call with one deny reason is denied regardless of other reasons.
FR2.5	Administrators shall be able to validate, activate and roll back policy versions at run time without restarting	S	Activation hot-reloads the policy and creates a `policy.activated`

ID	Requirement	Pri.	Acceptance criterion
	the gateway.		audit event.
FR3.1	Every tool result shall be labelled with a trust level (trusted/untrusted) and a sensitivity level (public/internal/confidential) from its tool profile.	M	The audit event for each inspection records both labels.
FR3.2	The taint tracker shall record identifying values (e-mail addresses, URLs, domains, IBANs, phone numbers, identifiers and text n-grams) from results, with their source tool, call and field.	M	Values embedded in JSON and in free text are both extracted.
FR3.3	The gateway shall escalate or block calls to external sinks after untrusted content was ingested, calls that carry confidential values to external sinks, and calls whose destination value originated in untrusted content.	M	The three taint rules fire on attack fixtures and do not fire when replying to a sender or e-mailing a CRM contact.
FR3.4	After an injection has been detected in a session, high-risk tools shall require approval.	S	`TAINT_AFTER_INJECTION` fires for high/critical tools in that session only.
FR4.1	Every untrusted tool result, and every result of a tool marked `scan` (for example CRM records), shall be scanned for embedded instructions in English, Modern Standard Arabic, Gulf dialect, Arabizi and mixed text, producing a score, label, language and highlighted spans.	M	Detector F1 $\geq$ 0.85 on the held-out Arabic and English test sets.
FR4.2	Text shall be normalised before detection (Unicode NFKC, invisible and bidirectional control characters, Arabic diacritics and letter variants, homoglyphs), and obfuscation shall itself be reported as a signal.	M	Zero-width, homoglyph and diacritic-obfuscated payloads are detected.
FR4.3	Depending on policy, detected content shall be sanitised (offending spans removed), blocked (content withheld) or flagged; untrusted content shall be spotlighted (data-marked) before reaching the model.	M	Sanitised output no longer contains the payload; spotlighting wrapper present.
FR5.1	High-risk actions (payments, deletions, bulk exports, outbound messages under taint) shall be suspended and an approval request with full context created.	M	The call blocks until a decision or timeout.
FR5.2	Approvers shall receive a real-time notification and a plain-language Arabic/English summary that explains what the agent wants to do, why it was paused and where the data came from.	M	Summary mentions the tool, destination and source of tainted data.
FR5.3	Approvers shall approve or deny from a mobile application with an optional note; requests not decided before the timeout shall expire and be denied (fail-safe).	M	Expired requests resolve to DENY; double decisions are rejected.
FR6.1	Every intercepted event shall be appended to an HMAC-SHA-256 hash-chained audit log with secrets and personal identifiers masked.	M	Verification detects modification, deletion and reordering of events.
FR6.2	Users shall search, filter, view and export (CSV/JSON) audit events in the dashboard.	M	Filters by agent, tool, action, type, session, time and text return correct pages.
FR6.3	Any stored decision shall be replayable: the system re-evaluates it with the stored policy version and inputs	M	100% of benchmark decisions replay to the same action.

ID	Requirement	Pri.	Acceptance criterion
	and reports whether the outcome matches.		
FR7.1	Administrators shall manage agents and their API keys (create, rotate, disable), users and roles (admin, approver, auditor), policies and tool registrations.	M	API keys are shown only once and stored hashed.
FR7.2	The dashboard shall show live statistics (calls, blocked, approvals, detections, latency) and a detector playground.	S	Overview reflects new events without reload (SSE).
FR8.1	The system shall ship a simulated enterprise workspace (e-mail, CRM, web, documents, payments) exposed as an MCP server and a demonstration LLM agent.	M	Demo scenarios run in protected and unprotected modes.
FR8.2	The system shall ship Mirsad-Bench, a reproducible benchmark harness with English and Arabic scenarios, ablation configurations and metric reports.	M	One command (`scripts/run_bench.sh`) reproduces the benchmark result tables of the evaluation (final report, Chapter 4; deliverable D5, Section B.5).

Non-functional requirements

Table 3.7: Non-functional requirements

ID	Quality attribute	Requirement	Measure / verification
NFR1	Performance	Median added latency ≤ 200 ms per tool call, excluding time waiting for a human.	Latency benchmark over 1,000 HTTP calls (p50, p95, p99).
NFR2	Interoperability / portability	Model-agnostic: no dependency on a specific LLM provider; standard protocols only (MCP, OpenAI-style JSON).	Same gateway used by an MCP client and by an OpenAI-compatible agent backed by a local model.
NFR3	Security	Agents authenticate with API keys (stored as SHA-256 hashes) and humans with JWTs and role-based access; secrets are never logged; audit integrity is verifiable; security headers, body-size limits and login throttling are applied.	Security test suite (Section 4.2.6 of the final report), redaction tests, SAST and dependency scans.
NFR4	Usability	An approver can understand and decide on a request in under 30 seconds on a phone; UI available in Arabic (RTL) and English.	Keystroke-Level Model estimate and task walkthrough.
NFR5	Deployability	Deployable with a single `docker compose up` (or one script without Docker).	Deployment guide (Appendix E); local run script verified.
NFR6	Reliability / fail-safe	If the approver does not respond, the call is denied; internal errors in a tool never crash the gateway and are reported to the agent safely.	Expiry and executor-failure tests.
NFR7	Accountability	Every allowed, denied or approved action is attributable to an agent, session, policy version and (for approvals) a named human.	Audit event schema; replay test.

ID	Quality attribute	Requirement	Measure / verification
NFR8	Privacy	Personal identifiers (national ID, IBAN, card numbers) are masked in logs; demo data is synthetic.	Redaction unit tests.
NFR9	Maintainability	Modular layers with typed interfaces, ≥ 80% statement coverage of the core package, linted code, documented APIs.	Coverage report, Ruff, OpenAPI docs.
NFR10	Accessibility	Dashboard and approval app follow WCAG 2.2 AA basics: keyboard navigation, labels, contrast ≥ 4.5:1, no colour-only signals, reduced motion.	UI review checklist.
NFR11	Scalability (bounded)	Single node handles ≥ 50 concurrent agent sessions; state is in a relational database (SQLite in development, PostgreSQL in deployment).	Concurrency test in the latency benchmark.

Business rules and constraints

- BR1: Unknown agents and tools not mentioned in the policy are denied (default-deny).
- BR2: Payments and record deletions always require human approval, whatever the risk score.
- BR3: No more than 50 customer records may be exported or e-mailed in one call.
- BR4: E-mail to domains outside the organisation requires approval; data originating from untrusted content may never choose its own destination.
- BR5: An approval request expires after 120 seconds (configurable) and expiry means denial.
- BR6: Audit events are append-only; no user role can edit or delete them through the application.

3.3 System Design & Architecture

This section defines the design of Mirsad as built: its architecture and context, components, core processes, policy language, taint tracking, injection detector, database, security design, interfaces, user interface and deployment. The design follows the engineering contract `docs/dev/BUILD_SPEC.md` (Section 3.4.1), and every diagram was drawn from, or checked against, the code in `src/mirsad/`; the Mermaid sources are kept in `docs/diagrams/`.

Four protection principles [65] guided the design:

- **Complete mediation.** Every tool call and every tool result passes through one enforcement point.

- **Fail-safe defaults.** Unknown agents and undeclared tools are denied, and an internal error, an unavailable audit log or an unanswered approval request ends in denial.
- **Least privilege.** Permissions are granted per agent, per tool and per argument, and an agent only discovers the tools its policy allows.
- **Separation of privilege.** High-risk actions need a human approver, and the auditor role can review the audit log without being able to change policies.

3.3.1 Architectural style and system context

Mirsad is a **policy enforcement point (PEP)** in the sense of the XACML reference architecture [68]. The gateway (`mirsad.gateway.Gateway`) is the PEP and the `PolicyEngine` is the policy decision point; the taint tracker and the injection detector act as attribute providers that supply the provenance of the session's data and evidence about tool results; human approval is an obligation that must be discharged before an escalated call proceeds; and the audit log records every decision. Unlike a conventional PEP, Mirsad mediates in two directions. The **call path** checks what the agent wants to do before the tool runs, and the **result path** inspects what the tool returned before the model reads it, because indirect prompt injection arrives inside tool results [4].

Placing the enforcement point at the tool-calling boundary, rather than inside the model or the agent's code, was the central architectural decision. It makes Mirsad independent of the model (NFR2), requires no change to the agent apart from its tool endpoint, and governs actions rather than text. Designs that restructure the agent, such as CaMeL [16] and the patterns of Beurer-Kellner et al. [17], give stronger guarantees but cannot be added to an existing agent. Privilege-control systems such as Progent [50] are closest to Mirsad's policy layer; Mirsad adds data-flow tracking, Arabic-aware detection, human approval and a verifiable audit trail around such a layer.

Figure 3.2 shows the system context. The agent calls enterprise tools through Mirsad on behalf of a business user; the attacker can only place content in systems the agent reads, such as e-mail, web pages and documents; approvers decide on their phones; and administrators and auditors work in the dashboard.

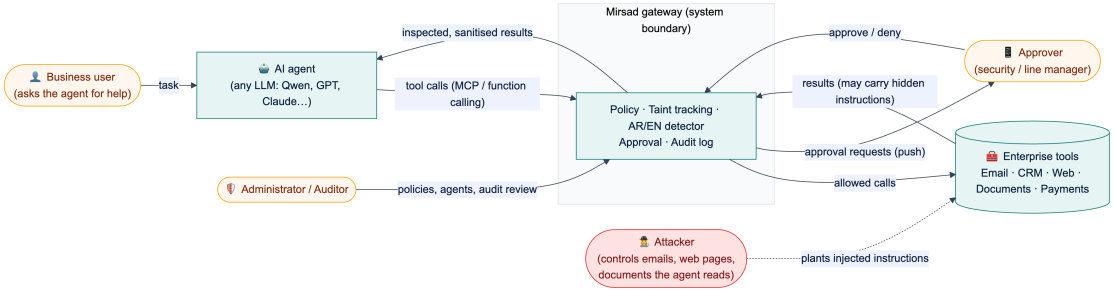


Figure 3.2: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools

Figure 3.3 shows the internal architecture, which combines four styles:

1. **Layers.** Protocol adapters (MCP proxy, OpenAI-compatible proxy, Guard API) translate external protocols into one internal model (`ToolCall`, `ToolResult`); the gateway core orchestrates the defence layers; storage sits underneath. A new agent protocol only touches the adapter layer.
2. **Pipes and filters.** On the call path, the policy engine, rate limiter and taint tracker each examine the call and emit `Reason` objects, which a combining step reduces to one decision. On the result path, detection, sanitising or spotlighting, and taint registration run in sequence.
3. **Publish–subscribe.** Approval events and new audit events are published on an in-process notifier and delivered over Server-Sent Events (SSE) and, optionally, Web Push.
4. **Modular monolith.** All components run in one Python process behind one FastAPI application. For a five-person team and a 200 ms latency budget (NFR1), in-process calls were preferred over separate services; typed interfaces and constructor injection (Section 3.3.2) keep the module boundaries, so a layer can later be moved out of process.

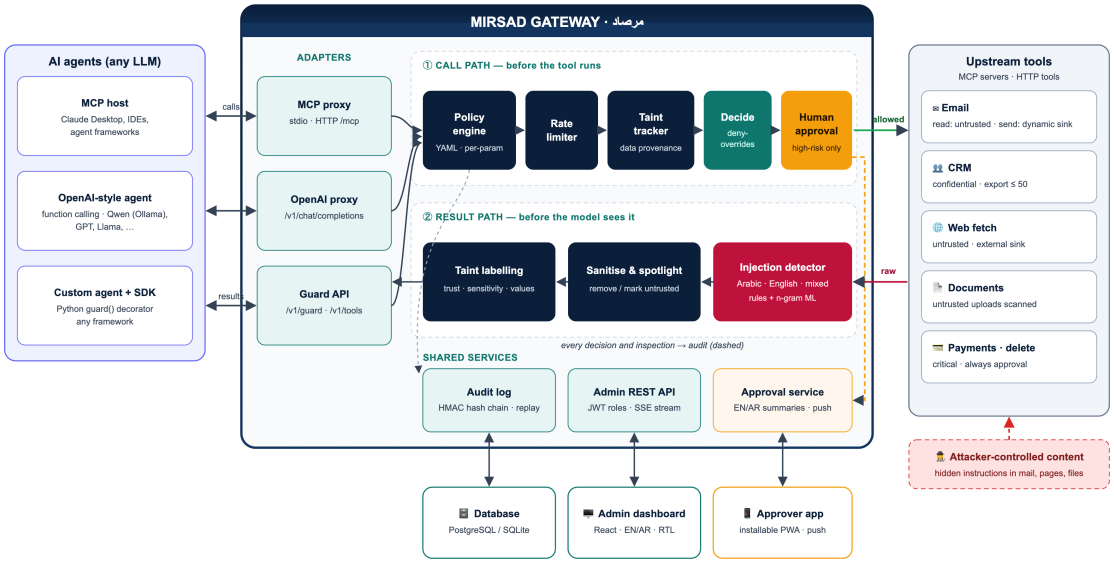


Figure 3.3: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services

Table 3.8 records the main design decisions, the alternatives considered and the reasons for each choice.

Table 3.8: Main design decisions

Decision	Alternatives considered	Rationale
Enforce at the tool boundary as a proxy	Guardrail at the model boundary; rebuild the agent around a secure architecture	Works with any LLM and agent; governs actions; inline adapters give complete mediation
Deterministic policy and taint rules as the primary control, the detector as a supporting signal	Rely mainly on an injection classifier	Published detectors are bypassed by adaptive attacks [45]; rules on actions and data flows still hold when text detection misses
Fixed deny-overrides combination with three outcomes	Configurable combining algorithms; first-applicable	Adding a layer or a rule can only make a decision stricter, so independent layers compose safely
Fail closed in every component	Fail open to preserve availability	A control that silently passes calls when it breaks gives false assurance (NFR6)
Template-based bilingual approver summaries	Summaries written by an LLM	A summary built from reason codes cannot be prompt-injected and is reproducible
HMAC hash chain inside the relational database, with optional external anchors	Separate log service; Merkle-tree log; distributed ledger	No extra infrastructure; verifiable offline from an export; a Merkle tree [87] remains an upgrade path
Store each decision with its inputs (policy version, minimal taint snapshot, rate counters)	Store only the outcome	Any decision can be replayed deterministically (FR6.3)
One process with constructor injection; SQLite or PostgreSQL	Microservices; Redis for shared state	Fits the team size and latency budget; injection lets tests and the benchmark swap layers

3.3.2 Components and responsibilities

Figure 3.4 shows the Python packages in `src/mirsad/`, the clients outside the gateway and their dependencies, derived from the import graph of the code; an arrow points from a package to the package it uses. Two dependency rules hold. First, the defence layers (`policy`, `taint`, `detector`) depend only on the shared data model (`mirsad.models`), the text utilities (`mirsad.textutil`) and, for `taint`, the policy's settings types, so each can be tested and benchmarked on its own. Second, the gateway does not construct its collaborators: at start-up `api/components.py` builds the object graph and passes the layers to the `Gateway` constructor, which imports them for type checking only (dashed "injected" arrows). Any component can be replaced through `create_app(gateway_overrides=...)`, which the test suite and the Mirsad-Bench ablations (policy only, taint only, detector only) rely on.

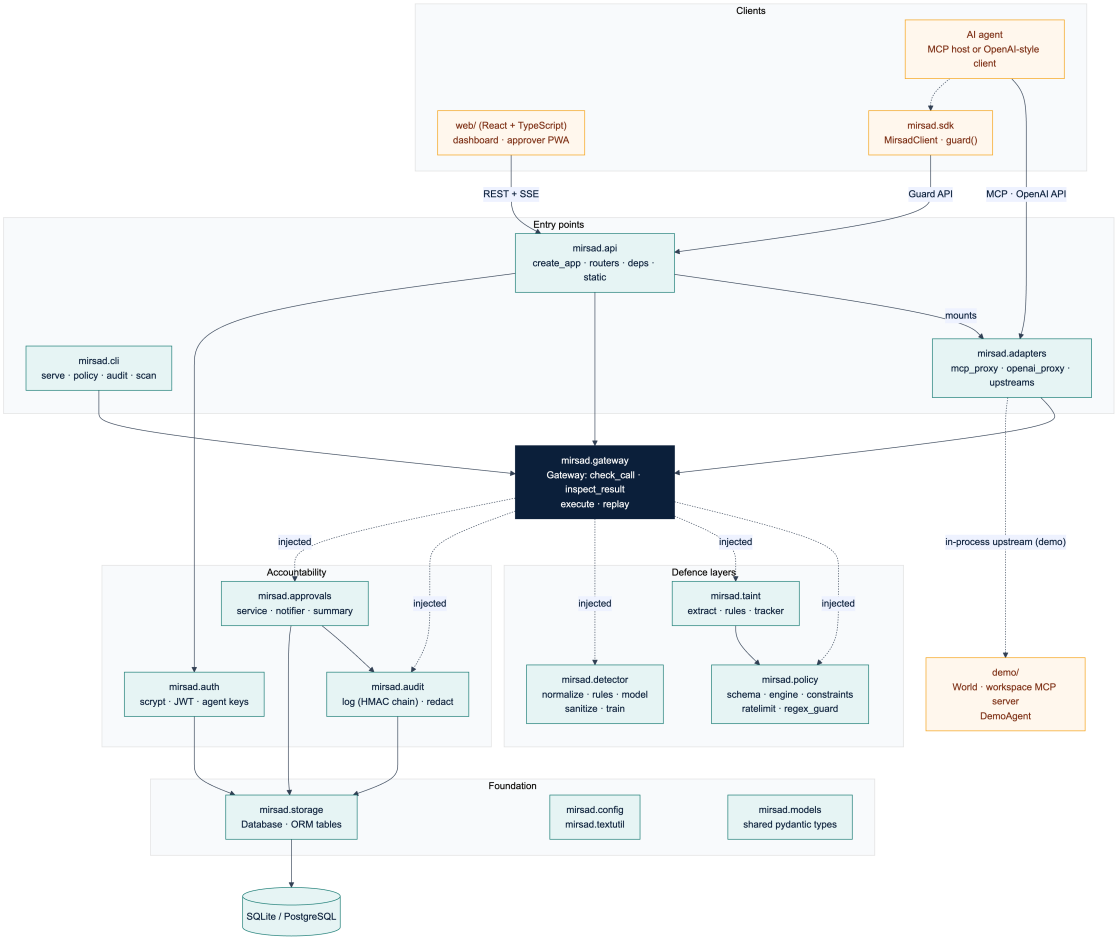


Figure 3.4: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules

Table 3.9 lists each component's responsibility, main interface and owner (BS = Bader Alshuraie, AO = Abdullah Naif Abdullah Al-Orabi, BY = Bader Abdullah Abbod Alyamini, MA = Mohammed Yasser Alanssari, AB = Abdulmalik bin Saedan).

Table 3.9: Components, responsibilities and interfaces

Component	Responsibility	Main interface	Owner
`adapters`	Translate MCP (stdio, streamable HTTP) and OpenAI chat completions into the internal model; connect to upstream MCP servers and Python tools	`build_mcp_server`, `build_openai_router`, `UpstreamManager`	BS
`gateway`	Decision pipeline: check calls, inspect results, execute allowed calls, replay stored decisions; fail closed	`Gateway.check_call`, `inspect_result`, `execute`, `replay`	BS
`policy`	Parse and validate YAML policies; resolve tool profiles and sinks; evaluate constraints, rate limits and rules	`load_policy`, `validate_policy`, `PolicyEngine`, `RateLimiter`	BS

Component	Responsibility	Main interface	Owner
`taint`	Label results with trust and sensitivity, extract values with their provenance, evaluate the four data-flow rules	`TaintTracker`, `extract_values`	BY
`detector`	Normalise Arabic and English text, score it with rules and a linear model, sanitise and spotlight	`InjectionDetector.scan`, `sanitize`, `spotlight`	AO
`audit`	Redact, chain, store, verify, query and export audit events	`AuditLog`, `verify_events`, `redact`	BY
`approvals`	Approval workflow, bilingual summaries, SSE publish–subscribe and Web Push	`ApprovalService`, `Notifier`, `summarize`	BS, BY (UX: MA)
`auth`	Password hashing, JWTs, agent API keys, user and agent management	`hash_password`, `create_token`, `authenticate_agent_key`	BY
`storage`	Database engine, sessions and ORM tables	`Database`, table classes	BY
`api`	Application factory, routers, authentication dependencies, security middleware, static hosting	`create_app`, `require_user`, `require_agent`	BS
`sdk`	Python client and `guard()` decorator for frameworks that execute tools themselves	`MirsadClient`, `AsyncMirsadClient`	BS
`cli`	Serve, initialise, manage policies, verify, export and replay the audit log, scan text, create agents and users, stdio MCP proxy	`mirsad ...` commands	BS
`web/`	Administration dashboard and approver PWA	REST and SSE client	MA
`demo/`, `bench/`	Simulated Najd Trading Co. workspace and demonstration agent; Mirsad-Bench harness	`World`, `DemoAgent`, `bench.run`	AB

Figure 3.5 shows the main classes and the shared data model. `Gateway` aggregates five runtime services; `PolicyEngine` owns its `RateLimiter`; `ApprovalService` publishes through a `Notifier`; and `UpstreamManager.call` is passed to `Gateway.execute` as the executor of allowed calls, so the gateway never needs to know how a tool is reached. The data classes (stereotype «model») are Pydantic models defined once in `mirsad/models.py`. `ToolCall` and `ToolResult` are the protocol-independent forms of a call and its result. `ToolProfile` is the security profile of one tool for one agent (allowed, trust, sensitivity, sink, risk, approval mode). A `Reason` is one piece of evidence from one layer: a machine-readable code, English and Arabic messages, a proposed `Action`, a risk score and structured evidence. A `Decision` combines the reasons with the risk score, sink, policy version, approval identifier and audit sequence number, and an `InspectionOutcome` is the verdict on a result (pass, flag, sanitize or block) with the content the model will see. `Action` is ordered (allow < require_approval < deny), and `Risk` maps a score to a level (medium from 0.3, high from 0.6, critical from 0.85). Because every layer reports in the same `Reason` type, evidence from different layers can be combined uniformly.

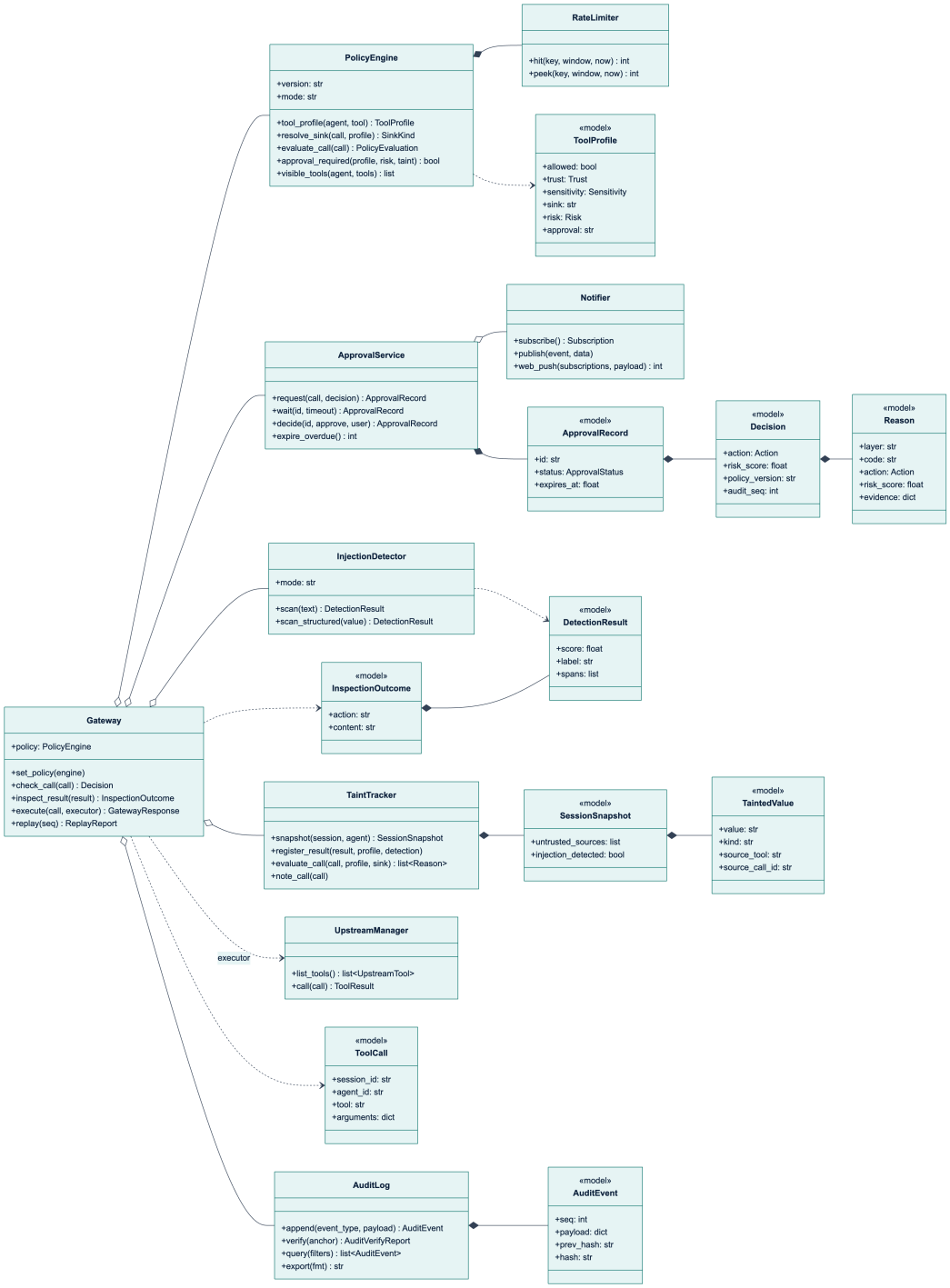


Figure 3.5: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract)

3.3.3 Core processes

P1: deciding on a tool call. Figure 3.6 shows the sequence for a call arriving through any adapter, and Figure 3.7 shows the decision logic. `Gateway.check_call` performs these steps:

1. Read the session's taint snapshot and call `PolicyEngine.evaluate_call`, which resolves the tool profile and sink and returns the policy and rate-limit reasons (Section 3.3.4).
2. Unless the agent is unknown, call `TaintTracker.evaluate_call` with the resolved sink (Section 3.3.5).
3. Combine all reasons with deny-overrides: the most severe action wins and the risk score is the highest score.
4. If the result is ALLOW but the tool's approval mode requires a human at this risk level, add `APPROVAL_POLICY` and escalate to REQUIRE_APPROVAL.
5. Append a `tool_call.decision` audit event with the call, profile, sink, all reasons, the part of the taint snapshot the rules consulted, the rate-limit counters and the risk score.
6. If approval is required, request it and wait (process P3); the final action is ALLOW only if the request is approved.
7. Record the call in the session and return the `Decision`, whose `latency_ms` excludes time spent waiting for a human.

Steps 1 to 5 run in a worker thread under one re-entrant lock, so snapshot, evaluation and audit append are atomic with respect to concurrent calls while the event loop stays free. Three fail-closed rules apply: an exception in any layer gives DENY with `INTERNAL_ERROR`; a failed audit append gives DENY with `AUDIT_UNAVAILABLE`, because no action may happen without a record; and a failed approval service gives DENY with `APPROVAL_UNAVAILABLE`. In monitor mode the would-be decision is computed and audited with `enforced: false`, but ALLOW is returned and results are not altered, so a policy can be tuned in shadow deployment before it is enforced.

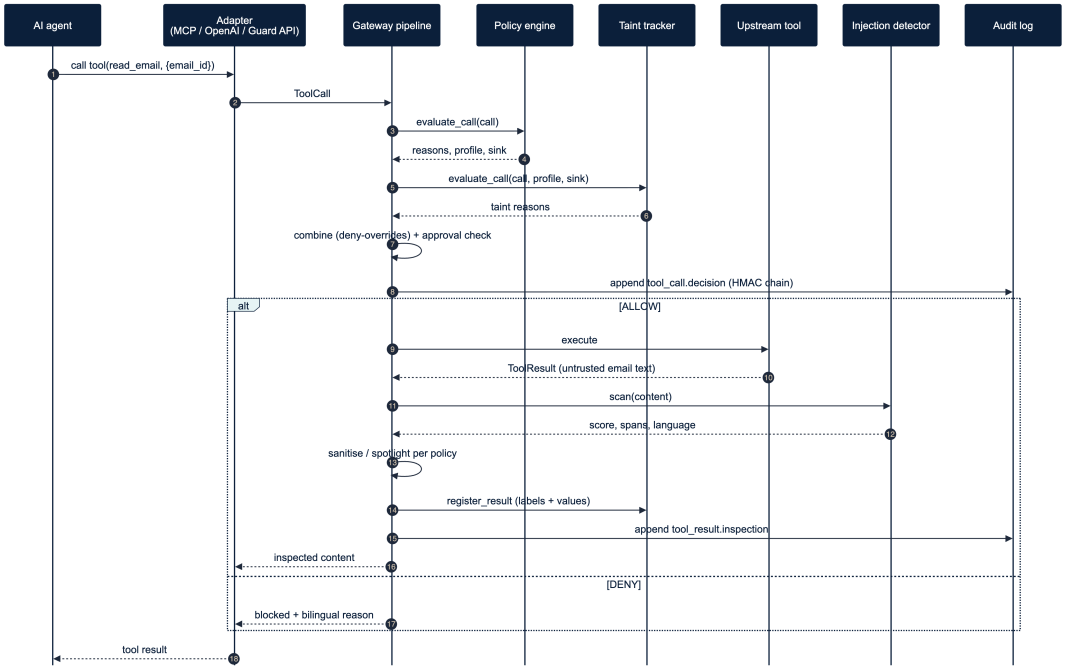


Figure 3.6: Sequence of a tool call through the gateway: decision, execution and inspection of the result

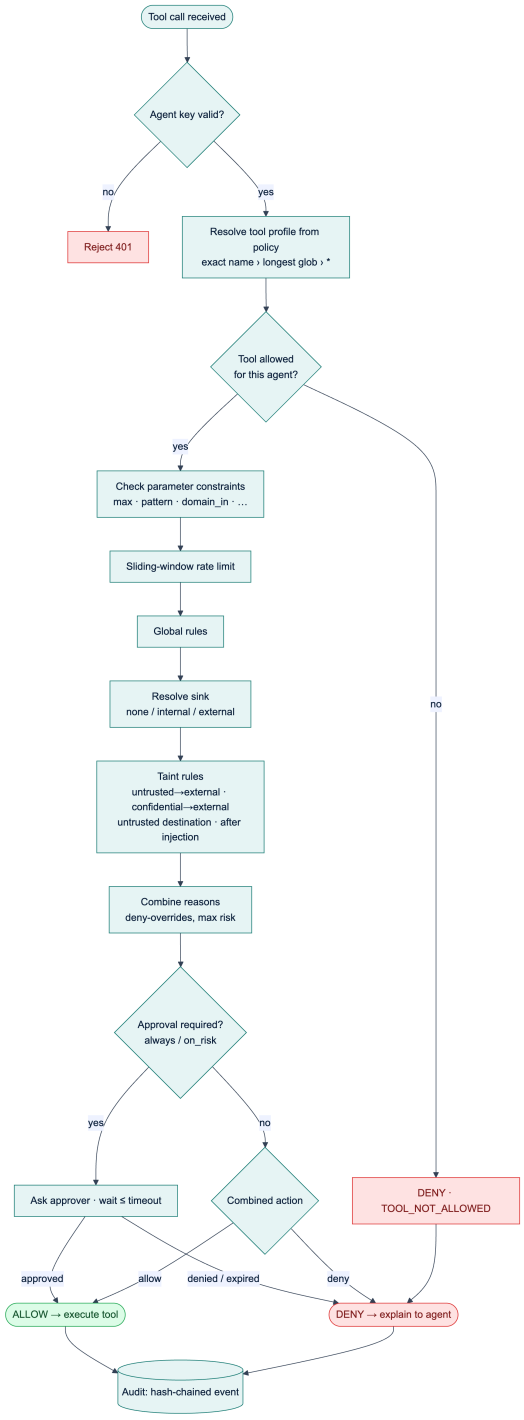


Figure 3.7: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step

P2: inspecting a tool result. `Gateway.inspect_result` scans the content when the tool is marked `scan` or its results are untrusted (results of tools the policy does not allow are always untrusted), maps the score with the tool's thresholds to pass, flag, sanitize or

block, spotlights untrusted content that is not blocked, registers the result with the taint tracker and appends a `tool_result.inspection` event. A sanitiser that cannot locate the suspicious text escalates to block, and a detector failure withholds the result. `Gateway.execute` composes P1 and P2: it runs the tool (with a 120 s default timeout) only for allowed calls and re-binds the result to the original call and session, so that an upstream cannot attribute its output to another call.

P3: human approval. Figure 3.8 and Figure 3.9 show the approval process and its state machine. `ApprovalService.request` stores a pending record with the redacted arguments, the decision, English and Arabic summaries and an expiry time (120 s by default), publishes `approval.requested` over SSE and, when configured, Web Push. The waiting call blocks on an event object, with a database re-check every 5 s for decisions made by another process. Every transition is a conditional `UPDATE ... WHERE status = 'pending'`, so exactly one decision wins when two approvers act at once, and a second decision is rejected with HTTP 409. At the deadline the request becomes EXPIRED, which the gateway treats as DENY; this also prevents a late approval of an action the agent was already told had been refused.

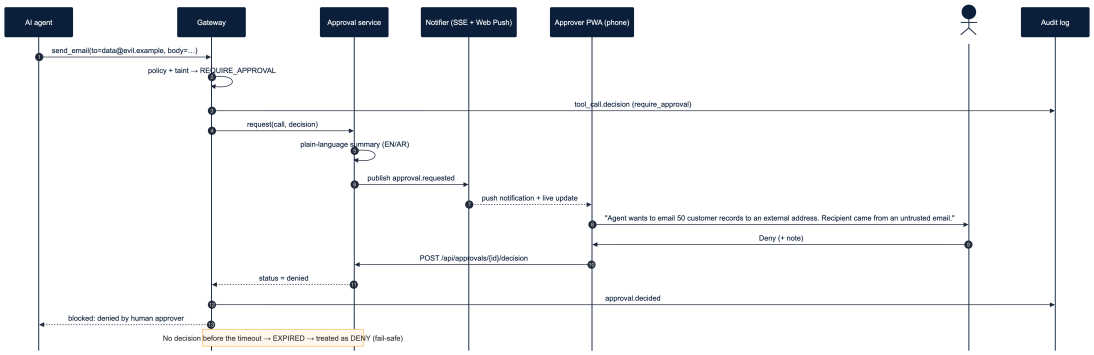


Figure 3.8: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call

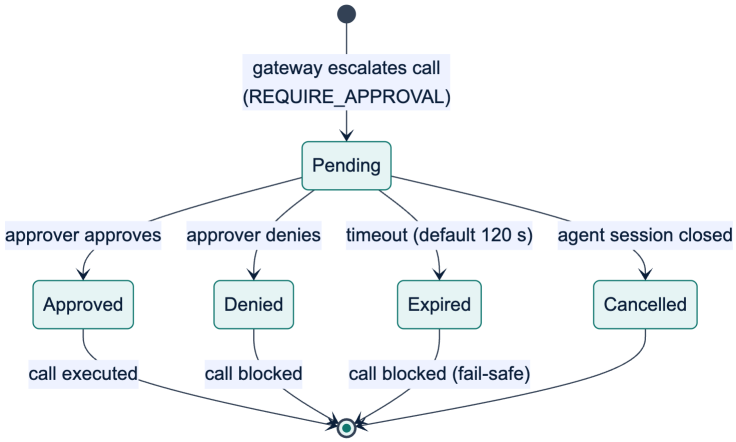


Figure 3.9: State machine of an approval request; every terminal state other than Approved blocks the call

The approver's summary is built by `approvals/summary.py` from the reason codes and their evidence, not by a language model, so attacker-controlled text cannot rewrite it; values copied from arguments are length-limited and stripped of line breaks. The summaries below were produced by this code for a benign case, in which an agent that has read an external supplier's e-mail replies to that supplier.

> The Email & CRM Assistant wants to send an email to accounts@red-sea-logistics.example (outside Najd Trading Co.) using the send_email tool. Mirsad paused it because the agent read an untrusted email (read_email call_r1) earlier in this session, and this action sends data outside the organisation, and the recipient address is outside the allowed domains (najd-trading.example). Risk level: high.

يريد الوكيل «Email & CRM Assistant» إرسال بريد إلكتروني إلى accounts@red-sea-logistics.example (خارج شركة نجد للتجارة) عبر الأداة send_email. أوقف مرصاد الطلب مؤقتًا لأن الوكيل اطّلع في هذه الجلسة على بريد إلكتروني غير موثوق (read_email call_r1)، وهذا الإجراء يرسل بيانات إلى خارج المؤسسة، ولأن عنوان المستلم خارج النطاقات المسموح بها (najd-trading.example). مستوى الخطورة: مرتفع.

The example shows a deliberate trade-off. The recipient came from the e-mail's `from` field, which the policy lists as trusted, so the rule that denies destinations chosen by untrusted text does not fire; the session-level taint rule and the domain constraint still ask a human to confirm. This is safe but adds friction to legitimate work.

P4: audit verification and replay. Verification recomputes the hash chain (Section 3.3.7). `Gateway.replay(seq)` loads the policy version recorded in a decision event, rebuilds the call and the stored taint snapshot, re-runs the policy with the stored rate counters (leaving the live limiter untouched) and the taint rules on the snapshot, and compares the action and reason codes; inspection events are rescanned. Because the taint

rules are a pure function of the call and the snapshot, replay is exact (Section 4.2.7 reports the replay results).

Data-flow view. Figure 3.10 is the context-level data-flow diagram, and Figure 3.11 decomposes the gateway into six processes: 1.0 authenticate and adapt, 2.0 evaluate the policy, 3.0 check taint, 4.0 obtain approval, 5.0 inspect the result and 6.0 audit. Data stores D1 (policies), D2 (audit events) and D3 (approvals) are database tables. D4 (session taint state) is kept in memory per process, bounded in size and expired after six idle hours; the part of D4 a decision depends on is copied into its audit event, so the audit log, not memory, is the durable record.

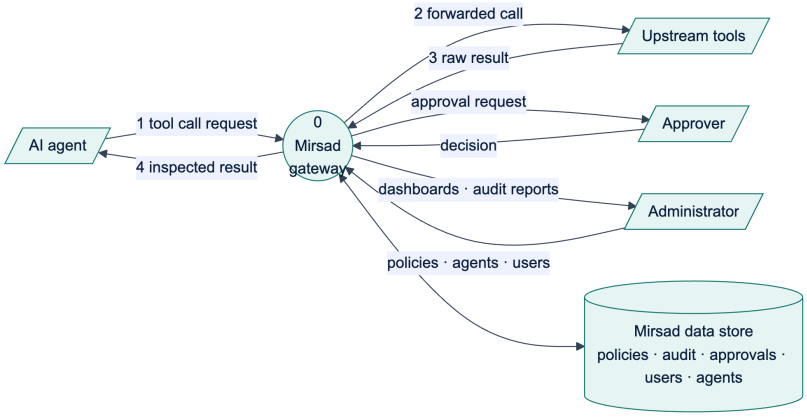


Figure 3.10: Context-level (level 0) data-flow diagram of Mirsad

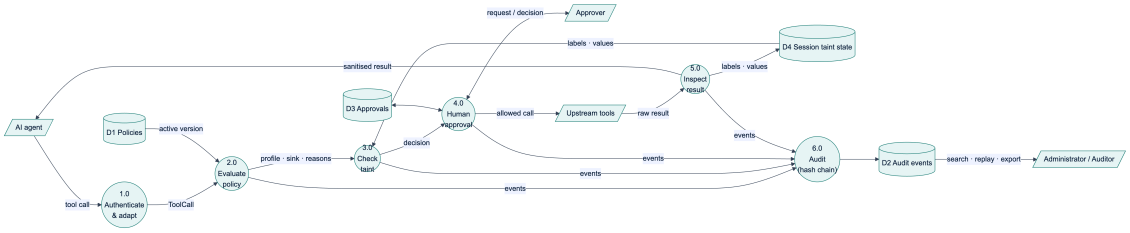


Figure 3.11: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4

3.3.4 Policy language design

The policy language had four goals: a policy must be reviewable by a security analyst in a pull request; permissions must default to deny and reach individual arguments; mistakes must be caught with precise messages before activation; and evaluation must be deterministic and versioned so that decisions can be replayed. General-purpose languages such as Rego [88] or XACML [68] are more expressive, but a domain-specific YAML format with a strict Pydantic schema fits a prompt-injection policy on one page and is harder to get dangerously wrong. Table 3.10 summarises the structure of a policy document.

Table 3.10: Structure of a policy document (schema version 1)

Element	Keys	Meaning
Document	`version`, `mode` (enforce or monitor), `organization` (`name`, `internal_domains`)	Schema version; shadow mode; domains inside the organisation
`defaults`	`decision` (deny, allow or require_approval), `approval_timeout_s`	Treatment of unmatched tools (deny by default); time a human has to decide
`detector`	`flag_threshold`, `block_threshold`, `on_detect` (sanitize, block or flag), `spotlight`	Global detector settings; agents and tools can override them
`taint`	`enabled`, `destination_params`, `trusted_fields`, one action per taint rule	Data-flow settings (Section 3.3.5); agents can override them
`agents[]`	`id`, `name`, `description`, `tools` (tool key → tool entry)	One entry per agent identity; `id` equals the agent's `policy_agent_id`
Tool entry	`allow`, `trust`, `sensitivity`, `sink` (none, internal, external or dynamic), `sink_params`, `risk`, `approval` (never, on_risk or always), `scan`, `constraints`, `rate_limit`, `detector`	Security profile of one tool for one agent
Constraint	`max`, `min`, `max_length`, `min_length`, `enum`/`in`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `required`, `type`, `on_violation`	Checks on one argument, combined with AND; a violation denies or requires approval
`rules[]`	`id`, `when` (`tool`, `agent`, `arg_matches`, `sink`, `session_tainted`, `min_risk`), `action`	Global rules evaluated after the tool entries

The excerpt below, from the tool entries of `configs/policies/default.yaml`, shows how the business rules of Section 3.2 are expressed: e-mail outside the organisation needs approval (BR4), exports are capped at 50 records (BR3), payments always need approval (BR2), and unlisted tools are denied (BR1).

```
send_email:
  allow: true
  sink: dynamic
  sink_params: [to, cc, bcc]
  risk: medium
  constraints:
    to:      {domain_in: [najd-trading.example], on_violation: require_approval}
    body:    {max_length: 20000}
  rate_limit: {max_calls: 20, per_seconds: 3600}
get_customer:     {allow: true, sensitivity: confidential, scan: true}
export_customers:
  allow: true
  sensitivity: confidential
  scan: true
  risk: high
  approval: on_risk
  constraints:
    count: {max: 50, on_violation: deny}
  rate_limit: {max_calls: 3, per_seconds: 3600}
transfer_funds:   {allow: true, risk: critical, approval: always, sink: external,
                   constraints: {amount_sar: {max: 10000, on_violation: deny}}}
"*":              {allow: false}
```

CRM results are trusted and confidential, but they are scanned (`scan: true` on `search_customers`, `get_customer` and `export_customers`), because their free-text notes can contain text that came from web forms or customer e-mails; a detection relabels the result as untrusted (Section 3.3.5).

Resolution. For a tool name the engine selects the exact key, otherwise the longest matching glob, otherwise the catch-all `"*"`, otherwise `defaults.decision`; equal-length globs that can match the same name are rejected as ambiguous. A `dynamic` sink is EXTERNAL if any recipient or URL in the destination arguments has a domain outside `internal_domains` or no recognisable domain, INTERNAL if all destinations are internal, and NONE if there is none.

Evaluation and combining algorithm. `PolicyEngine.evaluate_call` collects reasons in a fixed order: (1) an unknown agent is denied and evaluation stops; (2) a tool that is not allowed is denied and evaluation stops; (3) an informational `TOOL_RISK` reason carries the tool's risk score (0.1, 0.3, 0.6 or 0.9 for low to critical); then (4) every argument constraint, (5) the sliding-window rate limit, (6) every matching global rule and (7) `approval: always`. After step 3 nothing short-circuits, so the audit log and the approver see every objection. The gateway adds the taint reasons and applies **deny-overrides**: for the reasons r1, …, rn of a call, the final action is the most severe among them (deny > require_approval > allow) and the risk score is the maximum of their scores. The approval mode is applied last: `always` escalates every call; `on_risk` escalates when the combined risk is at least 0.6, or when the tool's own risk is high or critical and a taint rule objected; `never` does not escalate, although a deny still denies. Deny-overrides is monotonic, so adding a layer, constraint or rule can only make a decision stricter. The accepted limitation is that an `allow` rule cannot create an exception to a deny; exceptions are written as narrower deny rules. Table 3.11 lists the reason codes outside the taint layer, whose codes are given in Section 3.3.5.

Table 3.11: Reason codes of the policy, detector, approval and system layers

Code	Layer	Action	Risk score
`UNKNOWN_AGENT`, `TOOL_NOT_ALLOWED`	policy	deny	0.5
`TOOL_RISK`	policy	allow (informational)	0.1 / 0.3 / 0.6 / 0.9
`CONSTRAINT_VIOLATION`	policy	the constraint's `on_violation`	0.5
`RATE_LIMIT`	ratelimit	deny	0.5
`RULE_MATCH`	policy	the rule's `action`	rule's score, else 0.5 (0 for allow)
`APPROVAL_ALWAYS`	policy	require approval	the tool's risk score
`APPROVAL_POLICY`	approval	require approval	combined score
`APPROVED_BY_HUMAN`,	approval	allow, deny, deny	unchanged

Code	Layer	Action	Risk score
`DENIED_BY_HUMAN`, `APPROVAL_EXPIRED`			
`INTERNAL_ERROR`, `AUDIT_UNAVAILABLE`, `APPROVAL_UNAVAILABLE`	system, approval	deny (fail closed)	1.0
`MONITOR_MODE`	system	allow (records the would-be action)	0
`INJECTION_SUSPECTED`, `INJECTION_DETECTED`	detector	deny when the result is blocked; otherwise allow, with the result flagged or sanitised	detector score
`SCAN_FAILED`	detector	deny (result withheld, fail closed)	1.0
`RESULT_TRUNCATED`	system	allow (informational)	0

Safe validation. The schema is strict: unknown keys, operators and duplicate YAML keys are errors, with "did you mean" suggestions and the YAML line and column. Because policy regular expressions run against arguments written by a possibly hijacked model, the validator rejects patterns with nested ambiguous quantifiers, the classic cause of catastrophic backtracking (ReDoS); at run time at most 50,000 characters reach a policy regex, and a truncated input counts as a rule match, so the check fails closed.

Versioning and hot reload. A policy's version is the SHA-256 hash of the canonical JSON of the parsed document, so formatting does not change it but any semantic change does. Every version is stored, and the version in force is recorded in every audit event. Engines are immutable: activation swaps in a new engine atomically, and an evaluation in progress keeps the engine it started with.

3.3.5 Taint-tracking design

An LLM can mix any input into any output, so taint cannot be propagated through the model, and the guarantees of language-based information-flow control are out of reach [63]. Mirsad therefore tracks data at the boundary it controls, in the style of dynamic taint analysis [64]: tool results are **sources**, tool calls are **sinks**, and the rules look for the combination of private data, untrusted content and external communication that makes an agent exploitable [8]. The approach follows information-flow control for agents [51], [89], except that a violation can be routed to a human approver instead of always aborting.

Labels. Every result receives two labels from its tool profile, as in Denning's lattice model [62]: integrity (trusted or untrusted, where untrusted means a third party can control the content) and confidentiality (public < internal < confidential). Detector evidence overrides declared trust: a result labelled as an injection is registered as

untrusted even if its tool is declared trusted. Each recorded value also carries a `trusted_field` flag for fields vouched for by the producing system rather than the content's author, such as the sender, identifier and date of a received e-mail. This separates "reply to the sender", which is normal, from "send to an address written in the body", the typical exfiltration pattern.

Two granularities. At **session level**, a session that has read untrusted content stays tainted for the rest of its life, a conservative but sound assumption. At **value level**, recognisable values are extracted from every result with their source tool, call and field: e-mail addresses, URLs, domains, IBANs, phone numbers, numbers of at least six digits, record identifiers and, for confidential results only, hashed word 8-grams of long text. Results and arguments are canonicalised the same way (Unicode NFKC, invisible characters removed, Arabic-Indic digits converted, case folded, Saudi mobile numbers unified), so a value matches whatever its spelling. N-grams are hashed with BLAKE2b, which keeps them stable across processes for replay and avoids storing the text. Table 3.12 lists the four rules evaluated before every call.

Table 3.12: Taint rules evaluated for each tool call

Rule (reason code)	Fires when	Default action	Risk
`TAINT_UNTRUSTED_TO_EXTERNAL`	The session has read untrusted content and the call's sink is external	require approval	0.6
`TAINT_CONFIDENTIAL_TO_EXTERNAL`	An argument contains a value recorded from a confidential result and the sink is external (sending customers only their own record is exempt)	require approval	0.7
`TAINT_UNTRUSTED_DESTINATION`	A destination argument (recipient, URL, IBAN, webhook) or its domain first appeared in untrusted content outside a trusted field, and never in a trusted result; the organisation's own domains are excluded	deny	0.9
`TAINT_AFTER_INJECTION`	An injection was detected earlier in the session and the tool's risk is high or critical	require approval	0.8

Every taint reason names the source tool, call and field of the matching value, so the approver sees where the data came from. State is bounded (at most 2,000 values per session; sessions dropped after six idle hours), and the rules are one pure function of the call, profile, sink, settings and a read-only view of the session, used by both live evaluation and replay.

The coarse session-level rule is the price of soundness; Section 4.2.4 measures the approval friction it causes. Value-level matching, in turn, can be defeated by paraphrasing or re-encoding data, which is why the session-level rule and human approval remain as a backstop.

3.3.6 Injection detector design

The detector reads untrusted tool results and the results of tools marked `scan: true`, such as CRM records, whose free-text notes can contain text that came from outside the organisation. It is designed as a **signal, not a security boundary**, because adaptive attacks have bypassed published detectors with high success rates [45]; the policy and taint layers are built to block harmful actions even when the detector misses. The detector catches common cases cheaply, drives the result-path actions, arms `TAINT_AFTER_INJECTION` and gives approvers and auditors readable evidence. It works in six stages:

1. **Normalisation.** HTML entities are decoded; zero-width and bidirectional control characters removed; text converted with NFKC and lower-cased; Arabic diacritics (tashkeel) and elongation (tatweel) removed; alef, ya and ta marbuta variants and Persian look-alikes unified; digits converted to ASCII; Cyrillic, Greek and Armenian look-alikes in Latin words folded; and letter-spaced words joined. Obfuscation is reported as *signals*, and an offset map takes each span back to the original text, so highlights and sanitising point at the characters the user and the model see.
2. **Language identification** (English, Arabic, mixed or other, including Arabizi), for reporting and per-language evaluation only.
3. **Rules.** 158 weighted regular-expression rules in YAML: 76 English, 51 Arabic (Modern Standard Arabic and Gulf, with some Egyptian and Levantine forms), 7 Arabizi and 24 language-independent rules for structural tricks such as chat-template tokens, hidden HTML, markdown-image exfiltration and encoded blobs. The rule score combines the best weight of each distinct rule by noisy-OR, and evasion signals raise it only when a rule has fired, because ordinary Arabic text produces some of them.
4. **Model.** Hashed character 2- to 5-grams (2^20 features) and word 1- to 2-grams (2^18 features) feed a class-balanced logistic regression. Texts are scored over windows of 600 characters with a stride of 300 and the highest window probability is kept, so one injected sentence in a long e-mail is not diluted.
5. **Ensemble.** The score is a noisy-OR of model score m and rule score r, $1 - (1 - m)^{\alpha} (1 - r)^{\beta}$, calibrated so that the operating point chosen on the validation split (highest F1 with a false-positive rate of at most 5%) maps to 0.5; the policy defaults (flag at 0.5, block at 0.8) keep their meaning after retraining.
6. **Persistence without pickle.** The model is stored as plain arrays and JSON metadata, loaded with `allow_pickle=False` and checked for shape and finite values; if it is missing or invalid, the detector runs in rules-only mode instead of failing.

A linear model over hashed n-grams was chosen over a fine-tuned transformer because the detector runs inline on CPU, the team's dataset is small, and character n-grams

tolerate Arabic morphology, spelling variation and Arabizi (Section 3.4.5). On the result path, scores below the flag threshold pass, scores between the thresholds are flagged and audited, and higher scores trigger the tool's `on_detect` action. Sanitising widens each span to its sentence or line and replaces it with a bilingual marker; a positive detection without a span fails closed and the content is withheld. Untrusted content that reaches the model is spotlighted [36]: it is wrapped in delimiters whose boundary is derived from a SHA-256 hash of the call identifier, source and content, so the content cannot contain its own closing marker, and a bilingual note tells the model to treat it as data.

The packaged detector currently contains no trained model and runs in rules-only mode, which is also the mode recorded by the latency benchmark (`rules_only`). The F1 scores on the held-out English and Arabic test sets (target 0.85) are produced by running `make detector` after the team has added its labelled data as described in `docs/detector.md`, and Section 4.2.2 reports them. The attack-labelled data, like the attack files of Mirsad-Bench (`bench/scenarios/attacks/README.md`), is written by the team (Section 3.6.4).

3.3.7 Database design

Persistence uses SQLAlchemy 2.0 with typed declarative mappings and only portable column types, so the same schema runs on SQLite in development and PostgreSQL 16 in deployment. SQLite connections enable foreign keys and write-ahead logging, so dashboard readers never block the audit writer. Timestamps are UNIX epoch seconds stored as floats, matching the Pydantic models. Figure 3.12 shows the six tables.

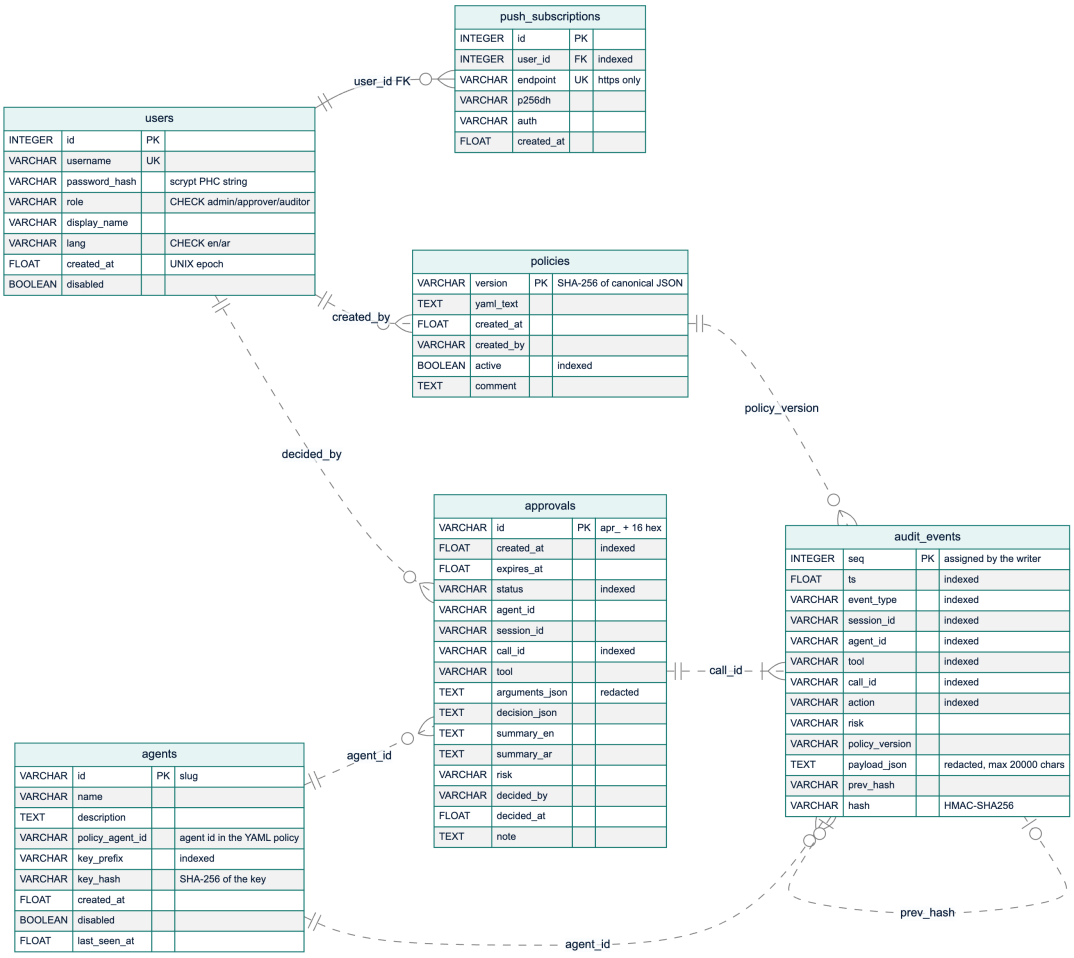


Figure 3.12: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value

Only `push_subscriptions.user_id` is an enforced foreign key (to `users.id`, with cascading delete). The other relationships are logical references by value: `audit_events.agent_id` and `approvals.agent_id` hold an agent's policy identifier; `approvals.decided_by` and `policies.created_by` hold usernames; `audit_events.policy_version` holds a `policies.version`; an approval and the audit events of its call share the `call_id`; and each event's `prev_hash` equals the previous event's hash. These references are deliberately not enforced: the audit log is append-only evidence that must outlive the deletion or renaming of what it mentions, and its hashes cover values exactly as written, so a cascading delete or a blocked insert would destroy or prevent evidence.

The management tables are in third normal form. Three columns hold JSON documents by design (`audit_events.payload_json`, `approvals.decision_json`, `approvals.arguments_json`): their shape depends on the event type and they must be stored verbatim to be hashed and replayed, while frequently filtered fields are promoted

to indexed columns. An approval copies the agent, tool and arguments of its call, so the record shows exactly what the approver saw. Table 3.13 is the table dictionary.

Table 3.13: Data dictionary of the database tables

Table	Purpose	Key and constraints	Written by / read by
`users`	People who sign in	`id` primary key; `username` unique; role restricted to admin, approver, auditor; language to en, ar	Administrators / authentication, user management
`push_subscriptions`	Browser push endpoints per user	`id` primary key; `endpoint` unique (HTTPS only); `user_id` foreign key with cascade	Approver PWA / Web Push delivery
`agents`	Agent identities and key hashes	`id` (slug) primary key; `key_prefix` indexed; only the SHA-256 hash of the key is stored	Administrators / agent authentication
`policies`	Every policy version stored	`version` (SHA-256) primary key; `active` indexed	Policy activation / gateway start-up, replay, administrators, auditors
`audit_events`	Hash-chained log of decisions, inspections, approvals, administrative changes and logins	`seq` primary key assigned by the writer (no auto-increment); seven indexes	`AuditLog.append` only / administrators, auditors, statistics, replay
`approvals`	Approval requests and outcomes	`id` primary key; indexes on `status`, `created_at`, `call_id`	Approval service / approvers, administrators, waiting calls

Hash-chain design. Each event is a link in a chain built from HMAC-SHA-256 [67], following the hash linking of Haber and Stornetta [90] and the secure audit log of Schneier and Kelsey [66]:

```
hash(n)      = HMAC-SHA256(K_audit, prev_hash(n) + "|" + canonical_json(event(n) without
"hash"))
prev_hash(n) = hash(n-1),   prev_hash(1) = "0" * 64
K_audit      = HMAC-SHA256(secret_key, "mirsad/audit-hmac/v1")
```

Canonical JSON uses sorted keys, no whitespace and UTF-8, so Arabic text is hashed as written. The hashed body contains every stored column, including `seq` and the indexed columns, so editing any field breaks the chain, and the audit key is separate from the JWT signing key although both derive from the server secret. Before hashing, `redact()` removes credentials and masks Saudi national ID and Iqama numbers, IBANs and card numbers to their last four characters (Section 3.6.1), so secrets never enter the chain; payloads are limited to 20,000 characters while keeping their structure. There is one writer per process: under a lock it reads the head and inserts `seq = head + 1` in the same transaction, and if another process takes that number first, the primary key rejects the insert and the writer retries.

Verification walks the table in sequence order and reports the first broken event: a gap in `seq` reveals a deletion, a `prev_hash` that does not link reveals an insertion or reordering, and an HMAC mismatch reveals a modification or a hash forged without the key. The same function verifies an unfiltered JSON export offline. A rewrite by someone holding the key, and truncation of the newest events, cannot be detected from the chain alone; for both, `AuditLog.anchor()` returns the current head to be stored outside the database, and verifying against an earlier anchor detects them. Because enforced calls are refused when the append fails, no action can happen without an event.

3.3.8 Security design and threat model

A firewall concentrates trust, so the gateway itself is a target. The threat model of Section 3.2 was analysed by applying STRIDE to each element of the data-flow diagram [69], [91]. Figure 3.13 shows the five trust boundaries: untrusted internet content (A), the agent runtime, whose model may be hijacked (B), the gateway (C), the enterprise systems (D) and the human approvers (E). Table 3.14 lists the main threats, the mitigation in the design and the test that verifies it. Tests ST-01 to ST-23 are in `tests/security/test_gateway_security.py`; other entries name the test modules of the component concerned.

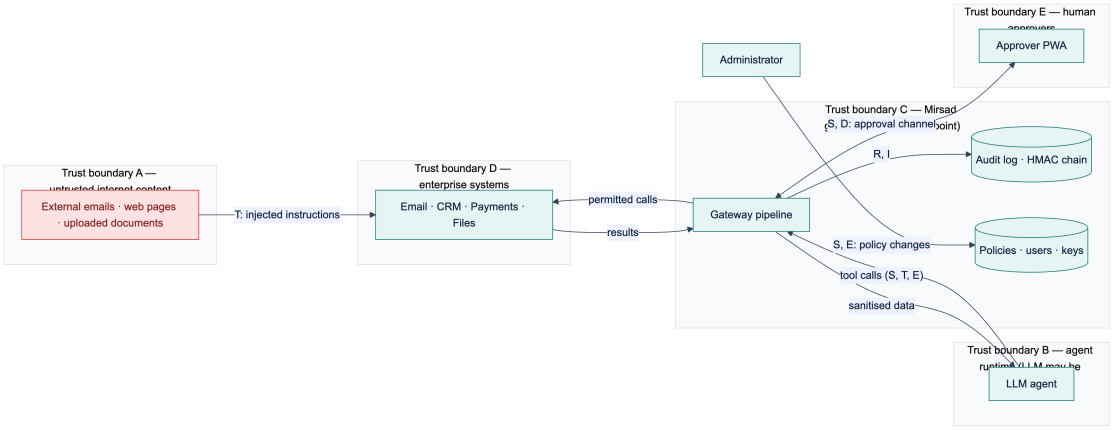


Figure 3.13: Threat model with trust boundaries A to E and the STRIDE categories of each data flow

Table 3.14: STRIDE analysis of the gateway

	Threat	Element	Mitigation	Verified by
S	Tools called with a guessed, stolen or revoked agent key	Agent APIs, `/mcp`	Per-agent keys with 256-bit secrets, stored as SHA-256 hashes, compared in constant time; rotation revokes the old key	ST-04, ST-05
S	Forged, unsigned or expired session	Human APIs	HS256 JWT with pinned algorithm,	ST-01, ST-02

	Threat	Element	Mitigation	Verified by
	token		expiry, issuer and role checks; user re-loaded on every request	
S	Online password guessing	Login	scrypt hashes; 5 failures per username and IP (30 per IP) in 5 minutes return HTTP 429	ST-19
T	Audit records edited or deleted to hide an incident	`audit_events`	HMAC hash chain, verification, external anchors	ST-06, ST-07
T	Injected instructions in a tool result steer the agent	Result path	Detector with sanitise, flag or block; spotlighting; detected injection marks the session	`tests/api/test_gateway.py`, `tests/detector/`
T	A policy that hangs the engine (ReDoS) is activated	Policy engine	Admin-only activation; validator rejects nested ambiguous quantifiers; input bound	ST-08
R	An approver denies having approved an action	Approvals	Decider, time and note stored; first decision final; `approval.decided` audited	ST-09
R	Login attempts leave no trace	Authentication	Failed logins audited with the username, never the password	ST-10
I	A hijacked agent sends customer data or money outside the organisation	Call path	Domain constraints; taint rules on untrusted and confidential flows; approval for payments	`tests/taint/`, `tests/policy/`; attack success rate pending (Chapter 4)
I	A URL tool is used to reach internal services (SSRF)	`fetch_url` policy	URL pattern constraints in the default policy	ST-23
I	Secrets in tool arguments leak into the audit log or exports	Audit log	`redact()` before hashing and storage	ST-11
I	Hashes, keys, internal errors or files leak through the API	API	No hashes in listings; generic 500 responses; static files confined to the web root	ST-12, ST-13, ST-14
I	The dashboard is framed or called from another site	Web front end	Same-origin CSP, frame denial, CORS allow-list, no tokens in URLs	ST-15, ST-16
D	Oversized	API, tools	1 MB body limit;	ST-18, `tests/policy/`

	Threat	Element	Mitigation	Verified by
	requests or call flooding		sliding-window rate limits per agent and tool	
D	No approver answers and the call hangs	Approvals	Expiry after the timeout, treated as denial	`tests/audit/test_approval_service.py`
E	A non-admin role performs administrative actions or alters its role claim	Role checks	Role-based access on every route; signed claim; role re-read from the database	ST-03, ST-20
E	An agent key is used on human APIs, or a user token on agent APIs	Authentication	Separate credential types per route group	ST-21
E	A hijacked or unknown agent calls tools outside its role	Policy engine	Default deny; least-privilege tool discovery	ST-22

Further controls apply across components: every component fails closed (Section 3.3.3); searches use parameterised ORM queries, which ST-17 checks with injection-style input; responses carry security headers (content security policy, `nosniff`, frame denial, referrer policy, HSTS over HTTPS); and the server secret is read from the environment or generated into an owner-only file. The security suite contains 23 security tests (45 test cases); Bandit reported 0 issues over 13,023 lines, and dependency audits found 0 vulnerable Python packages and 0 npm vulnerabilities (Section 4.2.6).

These tests verify the gateway's own defences. Whether the design stops hijacked agents is measured by the reduction in attack success rate on Mirsad-Bench relative to an unprotected agent (target at least 80%; Section 4.2.3), once the team adds its attack files to `bench/scenarios/attacks/` (Section 3.6.4).

3.3.9 Interface design

Mirsad has three kinds of external interface: agent interfaces, a REST and streaming interface for people, and the command line. The agent interfaces differ in who executes the tool, which determines the strength of enforcement (Table 3.15). In **inline** interfaces Mirsad executes the tool itself, so a denied call cannot run. In **advisory** interfaces the agent runtime executes tools and Mirsad removes or refuses denied calls; this protects against a manipulated model but assumes the agent's own code is honest, so inline interfaces are recommended when the runtime is not trusted.

Table 3.15: Agent-facing interfaces

Interface	Transport	Tool executed by	Enforcement
MCP proxy	stdio (`mirsad mcp-proxy`) or streamable HTTP at `/mcp`	Mirsad	Inline
Tool invocation	`POST /v1/tools/{tool}/invoke`, `GET /v1/tools`	Mirsad	Inline
OpenAI-compatible proxy	`POST /v1/chat/completions`	Agent runtime	Advisory: denied calls removed from the model's response
Guard API and Python SDK	`POST /v1/guard/tool-call`, `POST /v1/guard/tool-result`; `MirsadClient.guard()`	Agent runtime	Advisory: the SDK raises `MirsadBlocked` before the tool runs

MCP proxy. One MCP server is built per agent. `tools/list` returns only the tools the agent's policy allows, and every `tools/call` goes through `Gateway.execute`. Any refusal or internal failure returns an MCP error result with a bilingual explanation whose reason texts are clipped, so a block message cannot re-inject a long attacker-controlled value into the model's context. Only inspected text is returned; MCP structured content is never forwarded, because the sanitiser does not rewrite it.

OpenAI-compatible proxy. For each request the proxy (1) replaces every `role: tool` message with its inspected version, (2) removes tool definitions the policy does not allow, (3) forwards the request to the configured LLM endpoint without the agent's key, and (4) checks every returned tool call, removing denied calls, appending a bilingual explanation and setting `finish_reason` to `stop` when no call remains. Streaming is rejected, because a tool call must be complete before it can be checked.

REST and streaming API. People authenticate with a JWT from `POST /api/auth/login`; agents use their API key in the `Authorization` or `X-Mirsad-Key` header. Agent keys are rejected on `/api/*` and user tokens on `/v1/*`. Table 3.16 groups the endpoints. Bodies are JSON, errors have the form `{"detail": ...}`, status codes follow HTTP semantics (401, 403, 404, 409 for a conflicting decision or duplicate, 413 for bodies over 1 MB, 422, 429), and an OpenAPI description is generated from the typed models. Live updates use SSE on `GET /api/stream`, read with `fetch` and an `Authorization` header so that tokens never appear in URLs. The `mirsad` command line covers operations that should not depend on the web interface, such as policy activation, audit verification, export and replay, and agent and user creation.

Table 3.16: REST API overview

Group	Endpoints	Principal	Purpose
Health and authentication	`GET /healthz`; `POST /api/auth/login`; `GET /api/auth/me`	none; any user	Liveness and version; sign-in;

Group	Endpoints	Principal	Purpose
			current user
Agent gateway	`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools`, `/v1/tools/{tool}/invoke`, `/v1/chat/completions`, `/mcp`	agent key	Check, inspect, discover and execute tool calls
Approvals	`GET /api/approvals`, `GET /api/approvals/{id}`, `POST /api/approvals/{id}/decision`	admin, approver	Pending queue, history and decisions
Live updates	`GET /api/stream` (SSE); `GET /api/push/vapid`, `POST /api/push/subscribe`, `POST /api/push/unsubscribe`	any user	Approval and audit events; Web Push subscriptions
Audit	`GET /api/audit/events`, `GET /api/audit/events/{seq}`, `POST /api/audit/verify`, `POST /api/audit/events/{seq}/replay`, `GET /api/audit/export`	admin, auditor	Search, verify, replay, export (JSON or CSV)
Policies	`GET /api/policies`, `GET /api/policies/{version}`, `POST /api/policies/validate`, `PUT /api/policies`, `POST /api/policies/{version}/activate`	admin; auditor read-only	Versions, validation, activation, rollback
Agents and users	`/api/agents` (list, create, update, delete, rotate key); `/api/users` (list, create, update, delete)	admin	Identity management; keys shown only once
Tools, statistics, detector	`GET /api/tools`; `GET /api/stats/overview`, `GET /api/stats/timeseries`; `POST /api/detector/scan`	admin, auditor; any user	Tool profiles; dashboard figures; detector playground
Demonstration	`/api/demo/chat`, `/state`, `/reset`, `/scenarios`, `/scenarios/{id}/apply`	any user, demo mode only	Simulated workspace and demo agent

3.3.10 User-interface design

Mirsad has two front ends built from one React and TypeScript code base: the **administration dashboard** at `/` and the installable **approver PWA** at `/approve`. They share the API client, live-stream reader, translation dictionaries, explanation components and design tokens, and the gateway serves both from its own origin, so a strict content security policy that allows only same-origin scripts holds. The interface follows these principles:

1. **Plain language first, evidence on demand.** A request opens with three short sections, *What the agent wants to do*, *Why Mirsad paused it* and *Where the data came from*, followed by the raw arguments, collapsed by default. This applies the human–AI interaction guidelines of explaining why the system acted and supporting

efficient correction [70]. Only escalated calls reach a human, to limit the alert fatigue and complacency that automation research warns about [71].

2. **Bilingual and right-to-left by design.** Every string exists in English and Modern Standard Arabic with the same keys, which tests enforce. Choosing Arabic sets the document's language and direction, and CSS logical properties mirror the whole layout without separate stylesheets. Identifiers such as tool names, addresses, hashes and YAML stay left-to-right inside isolation elements, and server messages are bilingual at the source (`message_ar`, `summary_ar`).
3. **Accessibility to WCAG 2.2 level AA** [92]. Pages use semantic landmarks; every control has a label; everything works from the keyboard; dialogs trap and restore focus; live regions announce new requests; colour is never the only signal; contrast is checked by a test; reduced-motion preferences are respected; and touch targets in the approver app are at least 44 × 44 pixels.
4. **Safe, deliberate actions.** Approve and Deny open a confirmation sheet that restates the consequence and takes an optional note; the buttons are disabled when a request has expired or the phone is offline; API keys and generated passwords are shown once; and text from agents and tools is always rendered as text, never as HTML.
5. **Speed for the approver.** The target is a decision in under 30 seconds on a phone (NFR4): pending requests are sorted by soonest expiry with a countdown, and a push notification opens the request directly.

The dashboard has ten pages (Overview, Activity and audit, Approvals, Policies, Agents, Users, Tools, Detector playground, Demo console, Settings); the sidebar shows only the pages the signed-in role may use, and the API rejects any request outside that role. Figure 3.14 shows the audit-event drawer with its hash-chain fields and replay and verification actions, and Figure 3.15 shows the policy editor in Arabic with the mirrored layout. Figure 3.16 and Figure 3.17 show the approver app's pending list in Arabic and a request in English. The screenshots were taken from the running system with the simulated workspace; Appendix C contains the complete set.

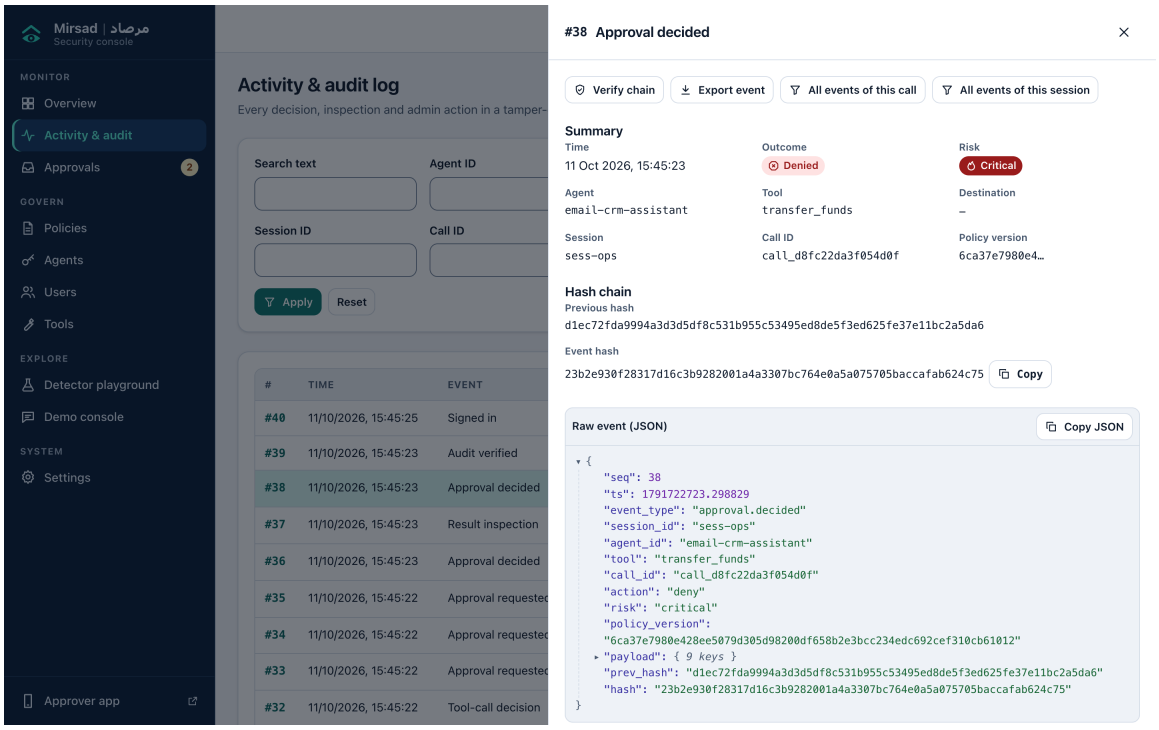


Figure 3.14: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event

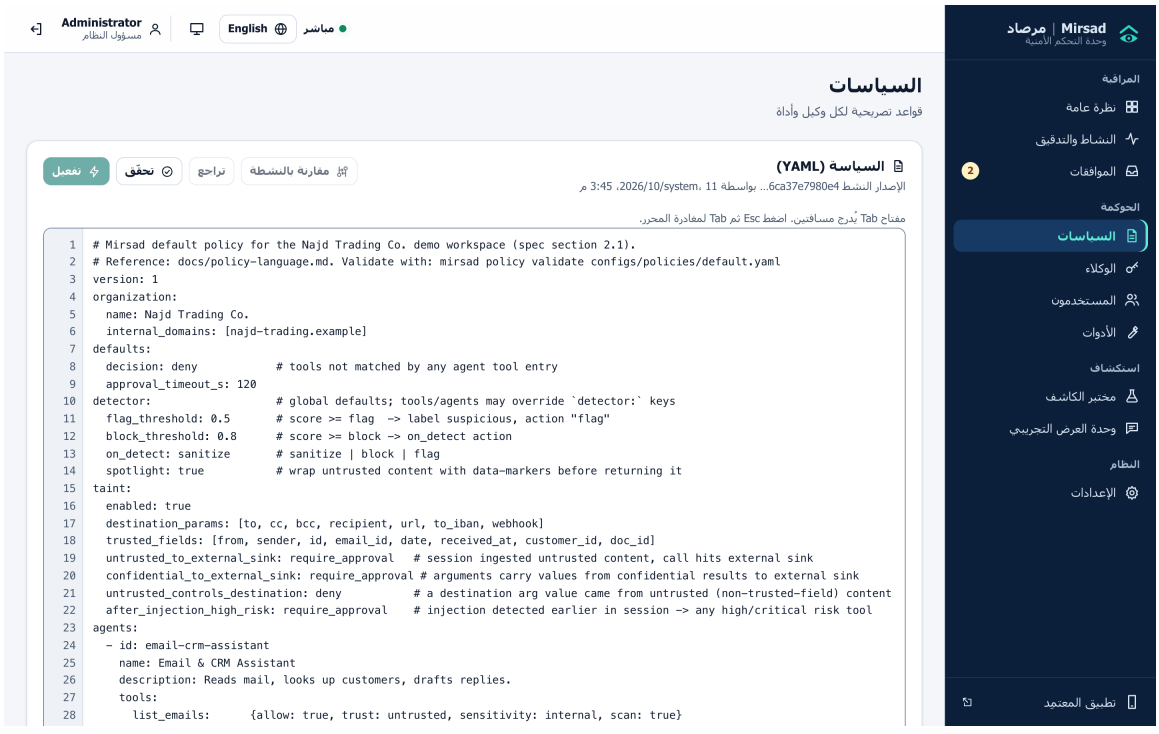


Figure 3.15: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right

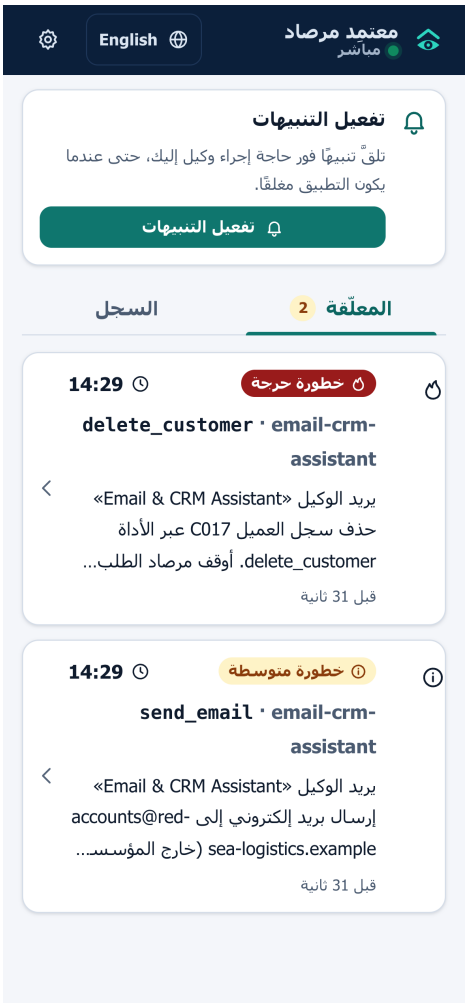


Figure 3.16: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary

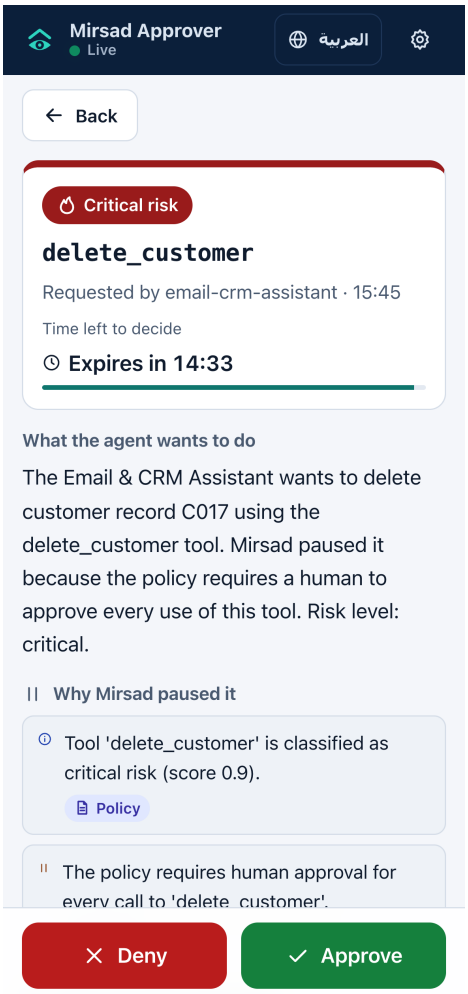


Figure 3.17: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons

The approver app is a progressive web app rather than a native application (Section 3.4.5). A web application manifest makes it installable on Android and iOS [93], and a service worker caches the application shell for offline use and receives push messages [94]; decisions always require a connection. Web Push works on Android and, for apps added to the home screen, on iOS 16.4 and later [95]. Service workers and push require HTTPS [96], which the deployment design reflects.

3.3.11 Deployment architecture and technology stack

Figure 3.18 shows the deployment architecture. The reference deployment is a Docker Compose stack on one server or virtual machine:

- **`mirsad` container.** A multi-stage image in which a Node.js stage builds the web applications and a Python 3.12 stage installs the gateway. It runs as a non-root user, keeps its data in a `/data` volume, exposes port 8700 and has a health check on `/healthz`.

- **`db` container.** PostgreSQL 16 with its own volume and readiness check; the gateway is configured to start only when the database is ready. Development and the benchmark use SQLite instead.
- **Ollama container (optional).** Enabled with the `demo` profile to serve Qwen2.5-7B for the demonstration agent.
- **Upstream tools.** Declared in `configs/upstreams.yaml` as in-process MCP servers, subprocesses over stdio, or remote servers over streamable HTTP.

Agents connect over HTTPS with their API key, and browsers and phones over HTTPS with a JWT. TLS is terminated by a reverse proxy in front of the container, which is not part of the Compose file; HTTPS is mandatory for the approver PWA. The Redis node in the figure marks an optional extension for sharing rate-limit counters between replicas and is not implemented: rate limits are kept in memory per process (Section 3.4.5), which fits the single-node scope. Configuration comes from `MIRSAD_` environment variables or a `.env` file (database URL, server secret, approval timeout, Web Push keys, CORS origins, demonstration switch); a missing secret is generated and stored in an owner-only file.

The Compose files are written but have not been executed, because Docker was not available on the build machine (Section 4.1.1), and the policy and upstream paths must be set explicitly for the `mirsad` service (Appendix E.2.2).

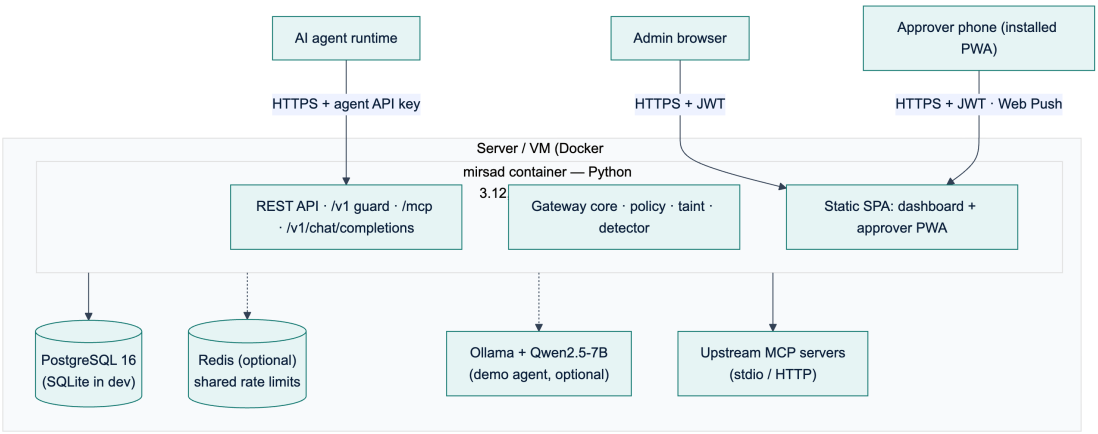


Figure 3.18: Deployment architecture: the Mirsad container, the database, optional components and the clients

The design targets one node serving at least 50 concurrent agent sessions (NFR11) with a median added latency of at most 200 ms (NFR1); Chapter 4 reports the measured latency (Section 4.2.5). Scaling beyond one node would require moving rate-limit counters and session taint state into a shared store, which is future work. The technologies of each layer and the reasons for choosing them are given in the technology-stack table of Section 3.4.4.

3.4 Implementation Strategy

3.4.1 Development methodology

The team's agreed working method is **Scrum** [97]: two-week sprints, a product backlog, sprint reviews in the bi-weekly supervisor meetings and a short retrospective at the end of each sprint. Its working rules build security activities into every sprint rather than leaving them to the end (a DevSecOps approach): threat modelling during design, review of every pull request by a second member, automated tests, linting, static analysis and dependency scanning in continuous integration, and a dedicated security-testing sprint before the final evaluation. Investigation, requirements, design, implementation and testing are iterated inside sprints, and the Senior Project I prototype (milestone M3) is the first planned complete increment.

The build was organised around a written **engineering contract** (`docs/dev/BUILD_SPEC.md`) that fixes the public interface of every component (names, arguments, return types, REST endpoints and database tables) before implementation starts. Because each workstream owns separate packages and codes against the contract, the five workstreams can be developed in parallel; shared data types live in a single module (`mirsad/models.py`) owned by the integrator, and Section 4.1.13 lists the integration issues that remained.

Table 3.17 lists the seventeen sprints and the milestones they close. It is the baseline plan. Actual progress ran ahead of it: the components planned for sprints S6 to S13 already existed as a working, tested prototype on the evidence date of this report (2026-10-14T07:58:56+00:00), before the M1 baseline date, while the results that depend on team-authored attack data remain open. Section 1.7 and D2 (Section 3.5) compare the plan with the actual progress.

Table 3.17: Sprint plan

Sprint	Dates	Sprint goal	Milestone
S1	1–14 Sep 2026	Literature review, product analysis, team setup, repository and CI skeleton	
S2	15–28 Sep 2026	Threat model (STRIDE), attack taxonomy, stakeholder personas	
S3	29 Sep–12 Oct 2026	SRS (D3): functional and non-functional requirements, use cases	
S4	13–26 Oct 2026	Benchmark and dataset design; requirement baseline agreed	M1
S5	27 Oct–9 Nov 2026	Architecture, engineering contract, policy-language specification, database design	
S6	10–23 Nov 2026	UI wireframes; gateway core and MCP proxy	M2
S7	24 Nov–7 Dec 2026	Policy engine, rate limiter, audit log and storage	

Sprint	Dates	Sprint goal	Milestone
S8	8–21 Dec 2026	Prototype demonstration and Senior Project I report	M3
S9	10–23 Jan 2027	Taint tracking; detector normalisation and rules	
S10	24 Jan–6 Feb 2027	Detector model, dataset pipeline, OpenAI-compatible proxy and SDK	
S11	7–20 Feb 2027	Detector training and evaluation; integration	M4
S12	21 Feb–6 Mar 2027	Approval workflow, approver PWA, notifications	
S13	7–20 Mar 2027	Dashboard, demo workspace and demo agent	M5
S14	21 Mar–3 Apr 2027	Test specification and automated tests (D5); benchmark runs	
S15	4–17 Apr 2027	Security testing of the gateway; latency and evaluation report	
S16	18 Apr–1 May 2027	User guide (D6), deployment, documentation	M6
S17	2–20 May 2027	Final report, poster, presentation and open-source release	M7

3.4.2 Mapping to the IS499 deliverables

The IS499 syllabus defines deliverables D1–D6 on a one-semester calendar. Table 3.18 shows where each deliverable is produced in the two-semester plan and where it appears in this report.

Table 3.18: IS499 deliverables and where they are addressed

Syllabus deliverable	Syllabus week	Mirsad artefact and timing	Location
D1 Project proposal	1	SPAP form, approved at M1	Chapter 1
D2 Work plan and budget	2–3	Sprint plan, Gantt chart, budget with price evidence (S1–S4)	Sections 1.7, 3.4.1, 3.4.6; separate D2 document
D3 Software requirements specification	4–5	SRS (S3–S4)	Section 3.2; separate D3 document
D4 High- and low-level design	6–7	Architecture, components, UML, database and UI design (S5–S6)	Section 3.3; separate D4 document
Midterm report (D1–D4)	8	Senior Project I report (S8)	Chapters 1–3
Implementation iterations 1 and 2	9–12	Increments of sprints S7–S13	Section 4.1
D5 Test case specification and results	13	Test plan, automated suites, results (S14–S15)	Sections 3.5, 4.2; Appendix B
D6 Software user guide	13	Installation, administration and	Appendix E; separate D6 document

Syllabus deliverable	Syllabus week	Mirsad artefact and timing	Location
		approver guide (S16)	
Final report and presentation	14–15	This report, poster and slides (S17)	Appendix D

3.4.3 Team roles and responsibilities

Table 3.19: Workstream ownership

Member	Department	Workstream and main responsibilities
Bader Alshuraie	SE	Gateway core, protocol adapters (MCP proxy, OpenAI-compatible proxy, Guard API, SDK), policy language and engine, REST API and CLI
Abdullah Naif Abdullah Al-Orabi	SE (AI & DS)	Arabic/English injection detector: normalisation, rules, model, dataset pipeline, training and evaluation
Bader Abdullah Abbod Alyamini	IS (CYS)	Taint tracking engine; tamper-evident audit log, redaction, verification and replay; database design
Mohammed Yasser Alanssari	IS (CYS)	Administration dashboard and approver PWA (bilingual, accessible), notifications, approval workflow UX
Abdulmalik bin Saedan	IS (CYS)	Project management; simulated workspace and demonstration agent; Mirsad-Bench; security testing of the gateway
All members		Literature review, requirements, design reviews, integration, code review, report, poster and presentation

3.4.4 Development environment, tools and version control

All code is kept in a single Git repository. The agreed working rules are a protected main branch, feature branches, and pull requests that need one review and a green CI run before merging. The continuous-integration workflow (GitHub Actions, `.github/workflows/ci.yml`) defines three jobs (Section 3.5.4): a Python job with linting (Ruff), the test suite with coverage, static application security testing (Bandit) and dependency vulnerability scanning (pip-audit); a web job with the TypeScript type check, the web unit tests and a production build; and a Docker image build that runs after both. No CI run is recorded in `reports/`.

Table 3.20: Technology stack and justification

Layer	Technology	Why it was chosen
Gateway and API	Python 3.12, FastAPI, Uvicorn, Pydantic v2	Asynchronous I/O for concurrent tool calls; typed request/response models; automatic OpenAPI documentation
Agent protocols	Official MCP Python SDK (v2), httpx	Standards-compliant MCP client and server (stdio and streamable HTTP); OpenAI-compatible JSON over HTTP

Layer	Technology	Why it was chosen
Persistence	SQLAlchemy 2.0; SQLite (development), PostgreSQL (deployment)	Portable ORM with typed models; zero-setup local database; production database without code changes
Policy language	YAML + strict Pydantic schema	Human-readable, reviewable in pull requests, validated with line-numbered errors
Detector	scikit-learn (hashed character and word n-grams, logistic regression), NumPy, rule engine	Designed to run on a CPU within the latency budget (target below 10 ms for a 2 KB text); transparent features; model stored without pickle
Security	scrypt (hashlib), PyJWT, HMAC-SHA-256, pywebpush (VAPID)	Standard primitives; no custom cryptography
Web UI and approver app	React 18, TypeScript, Vite, Recharts; installable PWA (manifest + service worker)	One code base for the dashboard and the mobile approval app; works on Android and iOS without app stores
Demonstration agent	Ollama with Qwen2.5-7B (local), OpenAI-compatible API	Free, private, runs on a laptop; strong Arabic support; native tool calling
Quality and security testing	pytest, pytest-asyncio, pytest-cov, Vitest, Testing Library, Ruff, Bandit, pip-audit	Automated unit, integration, API, UI and security tests
Documentation	Markdown, Mermaid, Matplotlib, python-docx, Playwright	Diagrams and figures regenerated from source; screenshots captured from the running system; reports built reproducibly
Packaging	Docker, Docker Compose, GitHub Actions	One-command deployment and repeatable builds

3.4.5 Changes from the proposal and their justification

Engineering judgement led to five deliberate changes from the tools listed in the proposal. Each keeps the proposal's objectives intact.

1. **Installable progressive web app instead of Flutter for the approval app.** A PWA built with the same React code base as the dashboard installs to the home screen on Android and iOS, supports system notifications and offline caching, avoids app-store registration fees and review delays, and could be built and tested on the team's machines. Wrapping it as a store app (Capacitor or Flutter) remains possible future work.
2. **Classical detector instead of fine-tuning XLM-RoBERTa / CAMeLBERT.** The latency budget (NFR1), CPU-only deployment and the size of the team-built dataset favour a character n-gram linear model combined with weighted rules. It is designed to run in milliseconds on a CPU (target below 10 ms for a 2 KB text, measured once a model is trained), and its features are inspectable. Fine-tuning a multilingual encoder is planned as an optional second stage (Chapter 5).

3. **SQLite and PostgreSQL without Redis.** Rate limits are kept in memory per gateway process; Redis would only be needed for multiple replicas, which are out of scope.
4. **Standard Web Push (VAPID) instead of Firebase Cloud Messaging.** Web Push is an open standard supported by the PWA and avoids a dependency on a proprietary service.
5. **Own benchmark harness (Mirsad-Bench) instead of running AgentDojo directly.** AgentDojo's environments are English-only and its standard runs assume hosted model APIs. Mirsad-Bench reuses its methodology (user tasks × injection tasks, attack success rate and utility) on a simulated Saudi enterprise with Arabic scenarios and runs fully offline. Burp Suite and OWASP ZAP were likewise replaced by an automated security test suite that can run in CI (Section 4.2.6).

3.4.6 Budget

The proposal's budget of 1,660 SAR was revised to 1,185 SAR in the D2 Work Plan and Budget (Section 5.3), mainly because development and most of the evaluation use local models and the approval app is a PWA rather than a store app (Section 3.4.5). Table 3.21 summarises both budgets; D2 gives the price evidence for each line.

Table 3.21: Original and revised budget (SAR)

Line	Proposal	Revised	Justification
LLM API usage	750	450	Local models for development; one confirmatory run per cloud provider
Cloud VM (2 vCPU, 4 GB, 6 months)	600	540	Priced from plans that can be ordered
Google Play registration	100	0	PWA instead of store apps
Domain and TLS	60	45	.com domain; free Let's Encrypt certificate
Contingency	150	150	VAT, card and exchange fees, price changes and benchmark reruns
Total	**1,660**	**1,185**	**Saving of 475 SAR**

If no funding is approved, the project can still be built, evaluated and demonstrated at 0 SAR with local models, free hosting tiers and free certificates, at the cost of the limitations listed in D2 (Section 5.4).

3.5 Testing & Validation

This section explains how Mirsad is verified, meaning that it is built correctly against its specification, and validated, meaning that it does its job of stopping hijacked agents without getting in the way of legitimate work. It covers the test strategy, the test levels and tools, the entry and exit criteria, the continuous-integration pipeline, the metrics, the

test inventory and the planned user acceptance testing. The test case specification, the requirements traceability matrix (Table B.2) and the full test report are in Appendix B. Section 4.2 discusses the results.

3.5.1 Test strategy

The strategy uses the concepts and documentation structure of ISO/IEC/IEEE 29119 [98] and is organised as a V-model. Each development artefact on the left of the V has a test level on the right that checks it (Table 3.22). The requirements in Section 3.2 are checked by end-to-end tests and by the benchmark. The architecture and the engineering contract (`docs/dev/BUILD_SPEC.md`) are checked by integration and API tests, and the design of each package by unit tests. The test design followed three principles.

1. **Requirement-driven and risk-based.** Every functional and non-functional requirement is traced to at least one automated test (Table B.2). Most effort went to the parts whose failure would either let an attack through or block legitimate work: the policy engine, the taint rules, the fail-closed paths of the gateway, the approval workflow and the audit chain.
2. **Negative cases are first-class.** Every rule has a case where it must fire and a case where it must not. The bulk-export rule is tested at 50 and 51 recipients (boundary values). Each constraint operator is tested with one passing and one violating value (equivalence classes). The legitimate counterpart of an "untrusted destination" is a reply to the sender of an e-mail, and that reply must not be blocked. The security suite attacks the gateway itself with forged tokens, database tampering and path traversal.
3. **Deterministic and offline.** Tests use temporary SQLite files, seeded synthetic data (the fictional Najd Trading Co. on reserved `.example` domains), explicit timestamps for rate-limit windows, and fake components where a unit has to be isolated. Tests that need the local language model carry the pytest marker `llm` and are left out of the default run, because model output is not stable enough to serve as a pass/fail gate.

Test data and attack material. This report contains no prompt-injection text, and apart from the eight Demo Console scenarios (`demo/scenarios.py`), which the demo tests load to check that they plant their text into the simulated workspace (Section 3.6.4), neither does the repository. A test that needs content to be scored as an injection uses a neutral placeholder marker together with a stub detector rule. Everything that happens after the detector produces its score, that is thresholds, sanitising, spotlighting, taint registration, audit and replay, then runs the production code. These tests therefore show how Mirsad *handles* a detection. They do not show how *well* the detector detects real attacks. Detector accuracy and attack success rate are measured separately, once the team's attack-labelled data is complete (Section 3.5.6).

Table 3.22: Test levels and the artefacts they verify (V-model)

Level	Verifies	Scope and techniques	Tools and location
Unit	Design of each package (policy, taint, detector, audit, approvals, auth, adapters, demo, bench)	One package at a time, with fakes for its neighbours. Equivalence classes and boundary values for constraints and rate limits, decision tables for reason combination, concurrency tests for the rate limiter, audit chain and approvals	pytest, pytest-asyncio; `tests/<area>/`
Integration	Engineering contract between packages	Real policy engine, taint tracker, detector, sanitiser, audit log, approvals and in-process MCP server wired together; the same storage, audit, approval and authentication code against PostgreSQL	`tests/integration/`, `tests/api/test_integration_real.py`, `tests/taint/test_taint_integration.py`
API and contract	REST, MCP and OpenAI-compatible interfaces	Application driven through an HTTP client: status codes, roles, schemas, error bodies; OpenAPI document compared with the specified routes; web client parsed against captured server records	httpx; `tests/api/`, `tests/adapters/`, `web/tests/contract.test.tsx`
End-to-end	User-visible behaviour	Real CLI (`mirsad init --demo`), the application built by `mirsad serve` on a real port, plain HTTP calls, an approval sent from a second operating-system process, the stdio MCP proxy started as a subprocess	`tests/api/test_e2e_http.py`, `tests/adapters/test_adapters_mcp_proxy.py`
UI	Dashboard and approver PWA	Component tests in a simulated browser (jsdom): approve/deny flow, Arabic right-to-left rendering, bilingual dictionaries, SSE parsing, colour contrast of the design tokens	Vitest, Testing Library; `web/tests/`
Security	Security requirements from the STRIDE analysis	23 security tests (ST-01 to ST-23; 45 test cases) against the running application, grouped by STRIDE category [69]	`tests/security/test_gateway_security.py`

Level	Verifies	Scope and techniques	Tools and location
Static analysis	Code quality and insecure code patterns	Linting with security rules, static application security testing, strict TypeScript type checking	Ruff, Bandit, TypeScript compiler
Dependency scanning	Known vulnerabilities in third-party packages	Advisory databases for Python and npm packages	pip-audit, npm audit
Performance	NFR1 and NFR11	1,000 benign HTTP calls, sequential and with 50 concurrent sessions; server in the same process and in a separate process	`bench/latency.py`
System evaluation	Objectives 1–3 (Section 1.3)	Mirsad-Bench: 48 benign tasks in English and Arabic, five defence configurations, two approver behaviours, audit verification and replay of every decision; LLM smoke run; attack episodes once the team's attack files exist	`bench/`, `scripts/run_bench.sh`
Acceptance (UAT)	Stakeholder needs: use cases UC-01, UC-05, UC-08, UC-13 to UC-15 and NFR4	Scenario-based sessions in which the supervisor and two to three security practitioners operate the system on prepared requests in the simulated workspace (Section 3.5.7)	Planned; questionnaire in Appendix F, Section D

3.5.2 Test environment, tools and responsibilities

All recorded runs were made on a team member's laptop (Apple silicon, macOS) with Python 3.12 and Node.js. The CI workflow is configured to run the same suites on Linux; no CI run is recorded in `reports/`. Python tests use pytest with pytest-asyncio and pytest-cov. HTTP-level tests drive the FastAPI application through httpx, either in memory or over a real socket served by uvicorn. The PostgreSQL tests start a throwaway server from the `pgserver` package, so no database has to be installed. Web tests use Vitest with Testing Library and jsdom. The `llm`-marked tests and the LLM smoke run use Qwen2.5-7B [99] served locally by Ollama.

Each workstream owner writes and maintains the unit and API tests for their own packages (Section 3.4.3). The security suite, Mirsad-Bench and the latency benchmark belong to the security-testing and benchmark workstream (Abdulmalik bin Saedan).

Under the agreed working rules, every pull request needs a second reviewer and a passing CI run before it is merged (Section 3.4.4).

3.5.3 Entry and exit criteria

A test level starts only when its entry criteria hold:

- **Unit and API tests:** the public interface of the component is fixed in the engineering contract and the code passes the linter.
- **Integration and end-to-end tests:** the unit suites of every package involved pass.
- **Benchmark runs:** the scenario files validate (`python -m bench.loader`) and the full automated suite passes, so that benchmark numbers always come from tested code.
- **Reported results:** the evidence pipeline (Section 3.5.4) has been run on the version being reported.

Table 3.23 lists the exit criteria for a release candidate, together with the values measured for the version described in this report. The first ten criteria are gates: a candidate that misses any of them is not released. The lint and type-check gate is enforced by the CI workflow, but its result is not stored with the evidence, so it is shown as not recorded. The last two are project success criteria that cannot be measured until attack data exists. They are reported as open rather than estimated.

Table 3.23: Exit criteria and their status for the reported version

Criterion	Target	Measured	Status
Automated Python tests (all areas, `llm` marker excluded)	all pass, none skipped	1,281 of 1,281 passed, 0 skipped	Met
Web UI tests	all pass	194 of 194 passed	Met
Statement coverage of `src/mirsad` (NFR9)	≥ 80%	94.4%	Met
Lint (Ruff) and type check (`tsc --noEmit`)	no findings	no report stored in `reports/` (B.5.3)	Not recorded
Static application security testing (Bandit)	no medium or high findings	0 findings of any severity	Met
Known-vulnerable dependencies	none	pip-audit: 0 of 124 packages; npm audit: 0	Met
STRIDE security suite	all pass	45 test cases passed	Met
Median added latency per tool call (NFR1)	≤ 200 ms, judged on the sequential median and on the separate-process layout	2.7 ms sequential; 166 ms with 50 sessions (separate process)	Met
Benign false-positive rate with full protection	≤ 5%	0.0%	Met (rules-only detector)
Benchmark decisions that replay to	100%	272 of 272 and 272 of 272	Met

Criterion	Target	Measured	Status
the same action (FR6.3)			
Detector F1 on the English / Arabic held-out test sets	≥ 0.85	[pending] / [pending]	Pending data
Attack success rate reduction against the unprotected baseline	≥ 80%	[pending]	Pending data

3.5.4 Continuous integration and the evidence pipeline

The CI workflow (`.github/workflows/ci.yml`) is configured to run on every pull request and every push to the main branch. Table 3.24 shows its three jobs. A failing step stops the job, and under the agreed working rules a pull request is not merged with a red build. No CI run is recorded in `reports/`, so the results in this report come from the local evidence pipeline described below.

Table 3.24: Continuous-integration jobs

Job	Steps	The build fails when
python	Install the project with development extras; `ruff check` on source, tests, demo, bench and scripts; `pytest -m "not llm"` with coverage; `bandit -r src`; `pip-audit`	any lint finding, failing test, Bandit finding or vulnerable dependency
web	`npm ci`; `tsc --noEmit`; Vitest; production build with Vite	a type error, failing test or build error
docker	Build the container image (runs only after both other jobs pass)	the image does not build

Two checks are not yet enforced by CI. The 80% coverage target is read from the coverage report rather than enforced with a `--cov-fail-under` threshold, and `npm audit` runs in the evidence pipeline and in `make security` but not in the workflow.

The numbers in this report come from a separate **evidence pipeline**, `scripts/run_evidence.sh`. It re-runs the Python suite (writing JUnit XML and a coverage JSON file), the web suite, Bandit, pip-audit and npm audit, both latency benchmarks and both scripted Mirsad-Bench runs (through `scripts/run_bench.sh`), and it trains the detector when labelled data is present. `scripts/collect_results.py` then gathers every result into `reports/results_summary.json`. The report builder (`scripts/build_docs.py`) reads each number in this document from that file when the document is generated. A number in the text therefore cannot drift away from the evidence, and a value that has not been produced yet prints as "[pending]" instead of being estimated.

3.5.5 Metrics

The following metrics are collected. Definitions of the benchmark metrics follow `docs/benchmark.md`.

- **Test execution:** number of tests per area, and passed, failed and skipped counts. A test is one pytest item or one Vitest case, so each parametrised case counts separately.
- **Code coverage:** statement coverage of the `mirsad` package. Branch coverage is not collected.
- **Static analysis and dependencies:** number of findings per tool, and number of packages with a known advisory.
- **Latency:** *added latency* per call, which is the HTTP round trip through Mirsad minus the direct in-process tool call, paired call by call. It is reported as p50, p95 and p99, together with throughput and HTTP errors. Time spent waiting for a human is excluded by design.
- **Benchmark:** *utility*, the share of episodes whose success predicate holds. *Benign false-positive rate (FPR)*, the share of benign calls denied automatically. *Approval friction*, the share of benign calls handed to a human. *Final block rate*, the share of benign calls that did not run in the end. *Result false-positive rate*, the share of benign tool results that the detector sanitised or blocked. *Approval recall*, the share of tasks that legitimately need approval in which a human was asked. *Gateway latency*, the decision plus result inspection per call. *Attack success rate (ASR)*, and *ASR reduction* = 1 − ASR(defence) / ASR(none).
- **Audit integrity:** the result of hash-chain verification and the share of decisions whose replay gives the same action.
- **Detector:** precision, recall, F1, false-positive rate and ROC-AUC on each held-out split, and scan latency for a 2 KB text.
- **Usability (NFR4):** an analytical estimate of approval time with the Keystroke-Level Model [100] (Section 4.2.8). Approval times of real users have not been measured yet.
- **Acceptance:** for each acceptance scenario of Section 3.5.7, whether each participant accepts it, with the number of participants.

FPR and friction are reported separately on purpose. A policy can deny no benign call and still cost a lot of human time if it escalates too often, and the two numbers show these two costs.

3.5.6 Test inventory

Table 3.25 lists the automated tests by area. The Python suite has 56 test files. Two further files hold only `llm`-marked tests (`tests/demo/test_llm.py` and `tests/bench/test_bench_llm.py`) and are not part of the recorded run.

Table 3.25: Automated tests by area

Area	Files	What the tests cover	Owner	Tests
policy	7	Schema and validator messages, every constraint operator, rate limiter, rules, sink resolution, ReDoS guard for policy regexes, shipped policies and policy documentation	Bader Alshuraie	265
adapters	4	MCP proxy (stdio and HTTP), OpenAI-compatible proxy, upstream manager, Python SDK	Bader Alshuraie	78
api	10	Gateway pipeline, Guard API, admin API, authentication and roles, HTTP hardening, SSE stream, CLI, demo endpoints, real-component integration and end-to-end HTTP	Bader Alshuraie	185
taint	4	Value extraction, the four taint rules and their exemptions, session tracker, replay from snapshots	Bader Abdullah Abbod Alyamini	93
audit	8	Hash chain and tamper detection, query and export, redaction, storage, approval service, approver summaries, notifier and Web Push, authentication core	Bader Abdullah Abbod Alyamini	240
detector	9	Normalisation, language identification, rules, model persistence, ensemble, sanitiser and spotlighting, dataset builder, training and metrics	Abdullah Naif Abdullah Al-Orabi	227
demo	4	Simulated workspace, MCP workspace server, demo scenarios, demo agent loop	Abdulmalik bin Saedan	72
bench	7	Scenario loader and schema, episodes and predicates, attack-episode construction, planting, survival markers and stop attribution (neutral placeholder text only), defence configurations, agents, metrics, runner and latency benchmark	Abdulmalik bin Saedan	68
security	1	STRIDE security suite ST-01 to ST-23	Abdulmalik bin Saedan	45
integration	2	PostgreSQL compatibility, cross-module wiring	All members	8
Python total	**56**			**1,281**
web	10	API client, SSE client, contract with server records, approver PWA and dashboard pages, i18n dictionaries, colour contrast, countdown, span highlighting	Mohammed Yasser Alanssari	194

Two measurements cannot be produced by this inventory yet: detector precision, recall and F1, and the attack success rate. Both depend on attack-labelled data written by the team (Section 3.6.4). Detector metrics are produced by running `make detector` after the team adds labelled batches and separate held-out test batches to `data/detector/raw/` (procedure in `docs/detector.md`, Section 11.4). The attack success rate is produced by Mirsad-Bench, which already executes attack episodes (Section 4.1.12), once the team adds injection tasks and attack templates to `bench/scenarios/attacks/`. Until then, every place in this report that needs these values shows "[pending]".

3.5.7 User acceptance testing

The levels above verify the system against its specification. User acceptance testing validates it with the people it is built for: whether an approver, an administrator and an auditor can complete their main tasks with the real interfaces and accept the result. Table 3.26 lists the planned acceptance scenarios. They are run in the simulated Najd Trading Co. workspace with prepared, benign requests, by the supervisor and two to three security practitioners recruited by the team, each of whom records "accept" or "reject" per scenario with a reason. Section D of the questionnaire in Appendix F is completed after the session. No session has been held yet; outcomes will be reported with the number of participants, and no result is estimated before then.

Table 3.26: Planned user acceptance scenarios

ID	Use case / requirement	Actor	Steps	Acceptance criterion	Evidence
AT-01	UC-01, BR4	Agent developer	Run the demonstration agent through the gateway on a benign task that e-mails an outside supplier, then on one that e-mails a colleague	The outside e-mail is paused and not sent until a decision is made; the internal e-mail is sent without a pause	Audit events of both sessions
AT-02	UC-05, NFR4	Approver	Receive a paused request on a phone, read the summary and approve or deny it	The participant can say why the action was paused from the summary alone and decides within 30 seconds	Request and decision times from the approval record; questionnaire items D1 and D2
AT-03	UC-05, NFR4	Approver	As AT-02 with the interface in Arabic	As AT-02; the participant judges the Arabic layout and summary readable	As AT-02
AT-04	UC-08	Administrator	Edit the policy to allow a new partner domain, validate it, review the difference, activate it, then roll back	Errors are reported with line numbers; the change applies to the next call without a restart; rollback restores the earlier version	`policy.activated` audit events
AT-05	UC-13, UC-14	Auditor	Find the events of one session, verify the hash chain and export the events as JSON	The events are found with the filters, verification reports an intact chain and the export verifies offline	Export file and verification output
AT-06	UC-15	Auditor	Replay a stored decision from the dashboard	The replay reports the same action and the policy version that made the original decision	Replay output

3.6 Ethical, Legal & Security Considerations

3.6.1 Data privacy and the PDPL

Mirsad is designed so that the security tool does not itself become a privacy risk. The Saudi Personal Data Protection Law [12] requires that personal data be processed only for a legitimate purpose, minimised, protected against unauthorised disclosure, and that breaches be reported. The design responds as follows:

- **Synthetic data only.** All customers, e-mails, documents and bank accounts in the demonstration workspace and in Mirsad-Bench are fictional; e-mail addresses and web sites use the reserved `.example` domain, and phone numbers and IBANs are clearly fake. The detector dataset contains no real personal data.
- **Masking in the audit log.** Before an event is hashed and stored, secrets (API keys, bearer tokens, JWTs, passwords, private keys) are removed and personal identifiers (Saudi national ID numbers, IBANs, payment-card numbers) are masked to their last four characters. The log keeps its forensic value (which tool, which destination, which decision) without becoming a second copy of customer data.
- **Purpose limitation.** Taint tracking stores only the identifying values needed to recognise a data flow (addresses, identifiers, hashed n-grams) and expires idle sessions.
- **Breach prevention as a primary goal.** The taint rule "confidential data to an external sink" exists specifically to prevent the kind of unauthorised disclosure the PDPL penalises.

3.6.2 Regulatory and standards alignment

Appendix A maps the system's controls to the NCA Essential Cybersecurity Controls [11] (event logging and monitoring, identity and access management, cybersecurity in third-party and cloud services), to the OWASP Top 10 for LLM Applications [5] (LLM01 prompt injection, LLM02 sensitive information disclosure, LLM06 excessive agency), to the NIST AI RMF functions [18] and to SDAIA's AI ethics principles [10]. Human approval of high-risk actions implements the principles of human oversight and accountability; the audit log and decision replay implement transparency and explainability towards auditors.

3.6.3 Security of the gateway itself

A firewall concentrates trust, so it becomes a target. The STRIDE analysis (Section 3.3.8) produced security requirements that are built into the gateway. Agents authenticate with per-agent API keys stored only as SHA-256 hashes, and people with salted scrypt password hashes and short-lived JWTs under role-based access control (admin, approver, auditor). Failed logins are throttled and audited, every response carries security headers, and request bodies are size-limited. Regular expressions in policies are validated and

their input is bounded to limit denial-of-service risk. The audit log is HMAC-chained, so undetected modification requires the server's secret key, and approval requests fail safe: expiry means denial. The gateway was tested with its own security test suite, static analysis and dependency scanning (Section 4.2.6).

3.6.4 Responsible handling of attack material

Evaluating a prompt-injection defence requires attack material, which is dual-use: it helps defenders test systems but could also help attackers. This report contains no attack payloads or attack templates, and the automated tests use neutral placeholders wherever content must be scored as an injection (Section 3.5.1). The only injected instructions in the repository are the short illustrative texts of the eight Demo Console scenarios (`demo/scenarios.py`), which address the simulated workspace and its reserved `.example` addresses and are used only for live demonstration. The attack material needed for the evaluation, namely the attack-labelled detector batches and the Mirsad-Bench attack files, is written and reviewed by team members, is used only against the project's simulated workspace and fictional organisation, and is not reproduced in this report, which publishes only aggregate metrics. Whether that material is released, and in what form (for example with restricted access), will be decided after the review, following the practice of public benchmarks such as AgentDojo [6]. No attack is run against any third-party system or service.

3.6.5 Social and professional considerations

- **Approver workload.** The approval workflow is designed to ask humans only about high-risk actions, with a short plain-language summary in their preferred language, to avoid alert fatigue and rubber-stamping.
- **Inclusiveness and accessibility.** The dashboard and approval app are fully bilingual (Arabic with right-to-left layout, and English) and follow WCAG 2.2 basics (keyboard navigation, labels, sufficient contrast, no colour-only signals).
- **Honest claims.** Prompt injection is not a solved problem; detectors can be evaded by novel phrasings. The report presents Mirsad as one layer of defence in depth, reports false positives, approval friction and limitations openly, and marks every unmeasured result, including detection misses and attack success, as pending (Chapters 4 and 5).
- **Professional codes.** The team followed the principles of the ACM Code of Ethics and Professional Conduct [101] on avoiding harm, honesty, respecting privacy and accepting professional review.

3.6.6 Licences, third-party components and AI assistance

All third-party software used by Mirsad is open source. Almost all runtime dependencies use permissive MIT, BSD or Apache-2.0 licences (e.g. FastAPI, SQLAlchemy, scikit-learn, React and the MCP SDK); the Web Push library pywebpush uses the weak-

copyleft MPL-2.0 licence, which permits use in an Apache-2.0 project as long as the library itself is distributed unmodified with its source available. The full list is in `docs/THIRD_PARTY.md`; the public `deepset/prompt-injections` dataset (Apache-2.0) is planned as an optional external English test set, to be downloaded by the team. Mirsad itself is released under the Apache-2.0 licence. The Qwen2.5 model used for the demonstration agent is used under its published licence.

In line with the proposal's commitment and university policy, the team discloses that **an AI coding and writing assistant (Anthropic's Claude) was used during the project** to help generate source code and tests, draft documentation and report text, author parts of the benign synthetic Arabic/English dataset batches and the benign benchmark user tasks, and produce diagrams. The assistant was not used to write the attack material for the evaluation (the attack-labelled detector batches and the Mirsad-Bench attack files); that material is written and reviewed by team members (Section 3.6.4). AI-assisted outputs were treated like any other contribution: they were checked by the automated test suite, the evaluation results come only from actual executions of the system, every literature reference was verified against its original source, and the team members remain responsible for the originality, correctness and understanding of all submitted work.

Chapter 4 – Implementation and Evaluation

This chapter describes how Mirsad was built (Section 4.1), evaluates it against the success criteria fixed in the proposal (Section 4.2) and records how the work was divided (Section 4.3).

Every number in Section 4.2 is read from the result files under `reports/`, which `scripts/run_evidence.sh` regenerates (tests with coverage, security scans, latency benchmark, Mirsad-Bench and, when labelled data exists, detector training) before `scripts/collect_results.py` writes `reports/results_summary.json`. The values printed here were collected on 2026-10-14T07:58:56+00:00 (UTC); Appendix B.1 records the platform and when each part of the evidence was produced. Results that depend on attack-labelled data, which the team is still preparing, print as "[pending]" and fill in automatically once the result files exist.

4.1 Implementation

4.1.1 Development environment and repository structure

Mirsad was developed and measured on an Apple M5 laptop with 16 GB of memory running macOS 26.5 (arm64), Python 3.12 and Node.js 24. The demonstration agent and the LLM benchmark runs use Qwen2.5-7B [99], served locally by Ollama through its OpenAI-compatible endpoint [102]. Backend code is typed Python with Pydantic v2 models, linted with Ruff; the web front ends are strict TypeScript. A GitHub Actions workflow (`.github/workflows/ci.yml`) defines the continuous-integration checks: Ruff, the Python tests with coverage, Bandit, pip-audit, the TypeScript type check, the web tests, a production web build and a Docker image build. No CI run is recorded in `reports/`: the evidence in this chapter was produced locally with `scripts/run_evidence.sh` and, for Mirsad-Bench, `scripts/run_bench.sh` (Appendix B.1). These scripts run the tests, Bandit, pip-audit, npm audit and the benchmarks, but not Ruff, the TypeScript type check or the Docker build. Docker was not available on the build machine, so the `Dockerfile` and `docker-compose.yml` (gateway plus PostgreSQL 16) are written but have not been executed.

The code base contains 15,839 lines of Python in the `mirsad` package, 5,138 lines in the demonstration workspace and the benchmark, 13,451 lines of Python tests and 10,213 lines of TypeScript. Table 4.1 lists the top-level folders.

Table 4.1: Top-level folders of the Mirsad repository

Folder	Contents
`src/mirsad/`	Gateway package: `gateway.py`, `policy/`, `taint/`, `detector/`, `audit/`, `approvals/`, `storage/`, `auth.py`, `adapters/`, `sdk/`, `api/`, `cli.py` and `static/` (built web apps)

Folder	Contents
web/	React and TypeScript source of the dashboard, the approver PWA and their shared layer, with Vitest tests
demo/	Simulated Najd Trading Co. workspace, its MCP server, the demonstration agent and the Demo Console scenarios
bench/	Mirsad-Bench (loader, agents, defences, metrics, runner, reports) and the latency benchmark; scenarios/ holds the user tasks
configs/	Policies (default.yaml, monitor.yaml, strict-banking.yaml) and upstream tool servers (upstreams.yaml)
data/detector/	Detector dataset: benign batches and generated splits; the team adds its attack-labelled batches here and public datasets under data/detector/external/
tests/	Python tests by area (adapters, api, audit, bench, demo, detector, integration, policy, security, taint)
docs/	Component documentation, the engineering contract (dev/BUILD_SPEC.md), diagrams, figures and report sources
scripts/	Setup, demo, benchmark and evidence scripts, screenshot capture and the report builder
reports/	Generated evidence: test results, coverage, security scans, benchmark runs and results_summary.json

Each workstream implemented the public interfaces of the engineering contract (BUILD_SPEC.md, Section 3.4.1) and requested new dependencies through docs/dev/deps-requests/ instead of editing shared files, which kept integration short and made the remaining seams visible (Section 4.1.13).

4.1.2 Gateway pipeline

The class Gateway in mirsad/gateway.py is the single enforcement point that every adapter calls. check_call decides on a tool call before it runs, inspect_result processes a tool result before the agent sees it, and execute chains the two around an executor supplied by the adapter.

check_call collects the reasons of the policy engine and then those of the taint tracker (using the sink the policy resolved) and combines them with a deny-overrides rule (mirsad/models.py): the most severe action wins and the risk score is the highest score of any reason.

```
def combine_reasons(reasons: list[Reason]) -> tuple[Action, float]:
    """Deny-overrides combination: the most severe action wins; risk is the max score."""
    action = Action.ALLOW
    score = 0.0
    for r in reasons:
        if r.action.severity > action.severity:
            action = r.action
        score = max(score, r.risk_score)
    return action, score
```

If the combined action is ALLOW but the tool's approval setting asks for a human (approval: always, or approval: on_risk with a combined risk of at least 0.6 or a high-

risk tool after a taint rule fired), the gateway adds an `APPROVAL_POLICY` reason and escalates. The decision is audited together with the call, the tool profile, the sink, every reason, a reduced taint snapshot, the rate-limit counters and the risk score, which are exactly the inputs needed to replay it (Section 4.1.6).

The pipeline fails closed: an exception in any layer yields a DENY with code `INTERNAL_ERROR`, and a call whose audit record cannot be written is denied with `AUDIT_UNAVAILABLE`, because an unrecorded action would violate NFR7. The synchronous evaluation runs in a worker thread under a lock, so concurrent sessions cannot interleave their updates to taint state and the audit chain. And when approval is required, `check_call` waits for a decision or the timeout, treats expiry as denial [65] and audits `approval.decided`; the reported latency excludes this human wait. In `mode: monitor` the decision is computed and audited with `enforced: false` but the call is allowed, for shadow deployment while a policy is tuned.

`inspect_result` treats results of unknown or disallowed tools as untrusted, truncates very large results (200,000 characters by default) and runs the detector when the profile asks for scanning. The score is mapped to pass, flag, sanitise or block with the thresholds of the tool's policy, and untrusted content that is not blocked is spotlighted: wrapped in data markers with a bilingual note telling the model to treat it as data [36]. The original, unsanitised result is registered with the taint tracker so that provenance survives sanitising, and a `tool_result.inspection` event records the detection, the action and a SHA-256 digest of the content. `execute` also overwrites the call, session and agent identifiers on whatever the upstream returns, so an upstream cannot attribute its output to another call.

4.1.3 Policy engine

The policy language of Section 3.3 is implemented in `mirsad/policy/`. `schema.py` parses YAML into strict Pydantic models and reports errors with line numbers. `PolicyEngine` compiles a document once, splitting each agent's tool entries into exact names, globs ordered by specificity and a catch-all, and computes the version hash of Section 3.3.4 that every audit event names.

`evaluate_call` collects reasons in the fixed order of Section 3.3.4. The rate limiter is an in-memory sliding window per agent, tool and optionally session, with caps on timestamps per key and on keys, so a misbehaving agent cannot exhaust memory. In replay mode the engine uses the counts stored in the audit event and leaves the limiter untouched.

The sink decides whether the taint rules about external destinations apply. For tools declared `sink: dynamic`, such as `send_email`, it is computed from the destination arguments, and a destination without a recognisable domain counts as external:

```
    def resolve_sink(self, call: ToolCall, profile: ToolProfile) -> SinkKind:
        """Static sinks as declared; ``dynamic`` by the domains of the destination
arguments.

        A non-empty destination value with no recognisable e-mail/URL/host domain is
        treated as EXTERNAL: an unknown destination must not count as internal.
        """
        if profile.sink != "dynamic":
            return SinkKind(profile.sink)
        pieces = [piece for param in profile.sink_params for piece in
iter_pieces(call.arguments.get(param))]
        if not pieces:
            return SinkKind.NONE
        internal = self.internal_domains()
        for piece in pieces:
            domains = piece_domains(piece)
            if not domains or any(not domain_matches(d, internal) for d in domains):
                return SinkKind.EXTERNAL
        return SinkKind.INTERNAL
```

Policy regular expressions run against arguments that a hijacked model may have produced, so a badly written pattern is a denial-of-service vector. `regex_guard.py` rejects patterns with nested ambiguous quantifiers at validation time and never passes more than 50,000 characters to a policy expression (for global rules, a longer argument counts as a match, so the check fails closed). Patterns are also applied to an NFKC-normalised copy with invisible characters removed, so full-width or zero-width variants of a blocked host name do not pass a `not_pattern` check. `Gateway.set_policy` swaps the engine atomically at run time, and every version is kept in the `policies` table so that old decisions can be replayed with the policy that made them.

4.1.4 Taint tracker

`mirsad/taint/` implements provenance tracking in the tradition of information-flow control and dynamic taint analysis [62], [64], at the granularity a gateway can observe: values in tool results that reappear in later tool arguments. `register_result` extracts identifying values from structured and free-text results: e-mail addresses, URLs, domains, IBANs, phone numbers, long numbers, identifiers and, for confidential results only, BLAKE2b hashes of overlapping eight-word windows, so a copied passage can be recognised without storing it. Each value keeps its source tool, call, field, trust and sensitivity, and whether it came from a trusted field such as an e-mail's sender. A session keeps at most 2,000 values (n-gram hashes may use half, so a long document cannot push identifiers out) and is discarded after six idle hours.

`evaluate_call` in `mirsad/taint/rules.py` implements the four rules of Table 3.12 as one pure function of the call, profile, sink, settings and a view of the session state, and the same function serves live evaluation and replay. The evidence of each taint reason names the source tool, call and field, which the approver sees as "where the data came from".

Integration exposed two details. The tracker needs the organisation's internal domains, which belong to the active policy and can change at run time, so

`mirsad/api/components.py` gives the tracker a function that reads the domains from the gateway's active policy instead of a fixed list. The second detail concerns replay: a full session snapshot can exceed the 20,000-character limit on audit payloads, and a truncated snapshot no longer reproduces the decision. `minimal_snapshot` therefore stores the session flags and only the values the rules consulted for the call.

4.1.5 Injection detector

The detector in `mirsad/detector/` implements the six stages of Section 3.3.6. `normalize()` keeps a map from each normalised character to its original position, so spans found in the normalised text can be highlighted and removed in the text the user and the model see. It records evasion tricks as signals that raise the score only when a rule has also fired, because ordinary Arabic and multilingual text produces some of them. The rules are plain YAML files in `mirsad/detector/rules/`, so they can be reviewed like policies. `combine_scores` in `model.py` computes the noisy-OR ensemble with the exponents and calibration point tuned on the validation split only; calibration maps the tuned operating point to 0.5, so the policy's `flag_threshold: 0.5` and `block_threshold: 0.8` keep their meaning after retraining. Weights are loaded with `allow_pickle=False` and checks on type, shape and finiteness, because a pickled model file would be executable code, and a missing or invalid model leaves the detector in rules-only mode instead of failing.

No trained model is packaged yet. Training needs attack-labelled batches and separately written test batches, which the team is still preparing (Section 4.2.2), so every detector-dependent number in this chapter comes from rules-only mode. The dataset tooling is complete and tested: `build_dataset` validates the batches, removes duplicates by normalised-text hash, sends held-out batches only to the test splits, augments only the training split and checks for leakage; `train` tunes on the validation split and writes per-split metrics, a threshold sweep, latencies and figures to `reports/`. The 227 detector tests use neutral placeholder text only.

4.1.6 Audit log and approvals

The audit log (`mirsad/audit/log.py`) implements the HMAC-SHA-256 hash chain of Section 3.3.7 in the `audit_events` table [67], [66], [87]. `compute_hash` reads the hashed fields by attribute access rather than through a model dump, so the hashed values are exactly the stored ones. The HMAC key is derived from the application secret with a fixed label, which keeps it separate from the key that signs login tokens. Before an event is hashed, `redact()` removes API keys, bearer tokens, JWTs, passwords and private keys, and masks payment-card numbers (validated with the Luhn check), Saudi national identifiers and IBANs; oversized payloads are then truncated field by field. `verify()` walks the chain in order and reports the first sequence number at which a gap, a broken link or an HMAC mismatch appears. An optional external anchor, the head sequence number and hash stored outside the database, also reveals truncation of the tail. Appends

are serialised by a lock and retried on a sequence conflict when several processes share one database.

`Gateway.replay` reads a stored decision, rebuilds the policy engine of the stored version from the policy store, and re-evaluates the call with the stored taint snapshot and rate-limit counts. For an inspection event it re-scores the stored content with the current detector. The report states whether the replayed action matches the original one.

The approval workflow (`mirsad/approvals/`) stores each request with its call, decision, bilingual summary, risk and expiry time. A decision is applied with a conditional update that succeeds only while the request is still pending, so two approvers cannot both decide and a late decision on an expired request is rejected:

```
    def _transition(self, approval_id: str, status: ApprovalStatus, *, user: str | None,
                    note: str | None, now: float) -> bool:
        """Atomically move a pending approval to ``status``; False if it was no longer
pending."""
        with self.db.session() as sess:
            row = sess.get(ApprovalRow, approval_id)
            if row is None:
                raise ApprovalNotFound(approval_id)
            decision = Decision.model_validate_json(row.decision_json)
            decision_json = decision.model_copy(update={"approval_status":
status.value}).model_dump_json()
            result = sess.execute(
                update(ApprovalRow)
                .where(ApprovalRow.id == approval_id, ApprovalRow.status ==
ApprovalStatus.PENDING.value)
                .values(status=status.value, decided_by=user, decided_at=now, note=note,
decision_json=decision_json)
                .execution_options(synchronize_session=False)
            )
            return bool(result.rowcount)
```

Requests and decisions are published on an in-process notifier that feeds the Server-Sent Events (SSE) stream used by the dashboard and the PWA, and a Web Push message is sent to subscribed devices when VAPID keys are configured. A maintenance loop in the API expires overdue requests every few seconds. The summary generator (`summary.py`) writes one to three sentences in English and in Arabic stating what the agent wants to do, why Mirsad paused it and where the data came from. Since integration it names the organisation from the active policy (for example "outside Najd Trading Co.") instead of a generic phrase.

4.1.7 Protocol adapters and SDK

`UpstreamManager` (`mirsad/adapters/upstreams.py`) connects Mirsad to the real tools: MCP servers loaded in-process, started over stdio or reached over streamable HTTP through the MCP Python SDK version 2, and plain Python functions registered as tools. It discovers tool schemas, routes each call to its upstream and applies timeouts. A broken connection is re-established for the next call, but the failed call itself is not retried, because it may already have had a side effect.

The MCP proxy (`mcp_proxy.py`) presents Mirsad to an agent as a single MCP server [3]. Tool listing is filtered by the policy, so an agent discovers only the tools it may call (FR1.4). The handler `on_call_tool` turns each `tools/call` request into a `ToolCall` and passes it to `Gateway.execute` with the upstream manager as executor; if the gateway itself raises, the handler returns a generic error result instead of the exception text.

A denied call comes back as an MCP tool error whose text explains the reason in English and Arabic. The same server is offered over stdio (`mirsad mcp-proxy`, which forwards to a running gateway) and over streamable HTTP at `/mcp`, where each request is authenticated with the agent's key.

The OpenAI-compatible proxy (`openai_proxy.py`) implements `POST /v1/chat/completions` for agents that use function calling. Before forwarding a request to the model, it replaces every `role: tool` message with the inspected, sanitised and spotlighted content. After the model answers, it checks every returned `tool_calls` entry with `check_call`, removes the denied entries, appends an explanation and sets `finish_reason` to `stop` when nothing is left to run. The session identifier is taken from the `X-Mirsad-Session` header or derived from a hash of the first user message.

For frameworks that run tools themselves, the Python SDK (`mirsad/sdk/`) wraps the Guard API. Its `guard` decorator checks a call before the decorated function runs and raises `MirsadBlocked` instead of running it when the call is denied; otherwise it sends the return value for inspection and returns the inspected content, raising again if the result is blocked.

4.1.8 REST API and command line

`mirsad.api.create_app` builds the object graph once (database, audit log, notifier, approval service, policy store and engine, taint tracker, detector, upstream manager and gateway) and mounts the routers specified in Section 3.3: agent endpoints under `/v1` and `/mcp`, human endpoints under `/api`, the SSE stream and the two static web apps. Agents authenticate with `mk_` keys that are stored only as SHA-256 hashes and compared in constant time. People log in with scrypt-hashed passwords [103] and receive short-lived HS256 JSON Web Tokens [104] carrying one of three roles: admin, approver or auditor. Failed logins are throttled per user name and IP address and audited without the password.

Two ASGI middlewares wrap the whole application, including the mounted MCP endpoint. The first adds a strict Content Security Policy (`script-src 'self'`), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, a no-referrer policy and `no-store` caching for API responses. The second rejects request bodies larger than 1 MB with status 413, whether the size is declared in a header or only discovered while streaming. Policies are stored as versions in the database; activating one hot-swaps the gateway's engine and audits a `policy.activated` event. A background loop (`_maintenance`) expires overdue

approvals and picks up policy activations made by another process, such as the command line, every few seconds; a failed cycle is logged and the loop continues.

The `mirsad` command line, built with Click, covers the operations an administrator needs without the web interface: `serve`, `init --demo`, `policy validate` and `policy activate`, `audit verify`, `audit export` and `audit replay`, `scan`, `agent create`, `user create`, `detector train` and `mcp-proxy`.

4.1.9 Web dashboard

The dashboard and the approver app are one React 18, TypeScript and Vite code base (`web/`) that is built into two pages served by the gateway at `/` and `/approve`. Both share an API client, the live-event stream, the bilingual dictionaries, the styles and the components. The browser's `EventSource` cannot send an `Authorization` header, and a token in the query string would leak into server logs and browser history, so the live stream is read with `fetch` and an incremental parser that follows the WHATWG event-stream rules and reconnects with exponential backoff. All assets are served from the same origin and there are no inline scripts, so the strict Content Security Policy set by the API holds.

The dashboard has ten pages behind a login: Overview, Activity and audit, Approvals, Policies, Agents, Users, Tools, Detector playground, Demo console and Settings. The Overview page (Figure C.2 in Appendix C) summarises calls, decisions, pending approvals, detections, gateway overhead and the state of the audit chain, and it updates live as agents act.

Every user-facing string exists in English and Arabic, and a unit test checks that the two dictionaries have identical keys and placeholders. Choosing a language sets the language and direction of the whole document, and the style sheets use logical (start/end) properties, so the layout mirrors for Arabic (Figure C.14): one effect in the shared layer sets `lang` and `dir` on the root element whenever the language changes.

A dark theme follows the system setting or the user's choice (Figure C.16), and a unit test checks WCAG AA contrast [92] for every pair of foreground and background colour tokens in both themes.

The Activity page lists audit events with filters by agent, tool, outcome, event type, session, call, time and free text. Selecting an event opens a detail drawer (Figure C.4) with the outcome, risk, policy version, both hashes of the chain link and the raw event, and with buttons to verify the chain, export the event and show all events of the same call or session. Decision events also list each reason with its layer, the detector spans and the taint provenance, and they can be replayed from the drawer.

The Policies page (Figure C.6) is a YAML editor with validation, a line-by-line comparison with the active version, activation and rollback. The Detector playground

(Figure C.10) scans pasted English or Arabic text and shows the score, its model and rule components, the detected language, highlighted spans and evasion signals. Its built-in samples are ordinary business texts, and the team can add its own labelled samples in a separate file.

4.1.10 Approver PWA

The approver app is an installable progressive web app defined by a web app manifest and a service worker [93], [94]. It installs to the home screen on Android and iOS, where Web Push is also available to web apps added to the home screen [95]. The service worker precaches the application shell, uses a network-first strategy for the approval API so that the last known queue stays visible offline, and bypasses the SSE stream and every non-GET request. It also receives Web Push messages and shows a system notification; its `notificationclick` handler focuses an open approver window and moves it to the paused request, or opens a new window on that request.

The pending list (Figure C.21) shows one card per request with a risk badge, the tool, the requesting agent, the start of the summary and the time left. The detail screen (Figure C.19 and Figure C.22) shows the risk, the tool, the requester and a countdown, followed by three sections: "What the agent wants to do" (the plain-language summary), "Why Mirsad paused it" (one card per reason, labelled with its layer) and "Where the data came from" (the taint evidence). The raw arguments are collapsed by default. Approve and Deny stay fixed at the bottom of the screen; either button opens a confirmation sheet that restates the consequence and accepts an optional note. If another approver has already decided, the server rejects the second decision and the app shows the outcome. Before signing out, the app removes its push subscription so that a shared phone stops receiving another person's alerts.

4.1.11 Demonstration workspace and agent

`demo/world.py` simulates the fictional Najd Trading Co. as a deterministic, seeded `World`: 120 customers with Arabic and English names, 30 e-mails, 12 web pages and 12 documents. It exposes 12 business tools: `list_emails`, `read_email`, `send_email`, `search_customers`, `get_customer`, `export_customers`, `update_customer_note`, `delete_customer`, `fetch_url`, `list_documents`, `read_document` and `transfer_funds`. Tools with side effects record what they did (outbox, HTTP log, deletions, exports, transfers, note changes), so a benchmark or the dashboard can tell exactly what an agent caused. Every address uses the reserved `.example` domain and every IBAN and phone number is fake. The method `plant()` inserts text into an e-mail, a web page, a document or a CRM note at the start, middle or end; the Demo console and the attack episodes of Mirsad-Bench use it to place content where an agent will read it.

`demo/workspace_server.py` exposes the same world as an MCP server, which the gateway loads in-process by default (it can also run over stdio). `demo/agent.py` contains the

demonstration agent: a tool-calling loop over the OpenAI-compatible API with a bilingual system prompt and at most eight steps, with three interchangeable backends (direct and unprotected, in-process through the gateway, and over HTTP with an agent key). The dashboard's Demo console runs this agent on eight curated scenarios that cover five attacker goals (exfiltration, destruction, fraud, phishing and privilege abuse). One example is an invoice e-mail with appended text that asks the agent to export the customer list and e-mail it to an outside address. The console can switch protection on and off, shows a timeline of the agent's steps with Mirsad's decision for each, and shows an "attacker inbox" with whatever reached the attacker. These scenarios exist for live demonstration only; no result in Section 4.2 is computed from them. Each one plants a short illustrative instruction aimed at the simulated workspace, so `demo/scenarios.py` is the one place in the repository where such instructions are written out (Section 3.6.4).

4.1.12 Mirsad-Bench

Mirsad-Bench (`bench/`) adapts the methodology of AgentDojo and InjecAgent [6], [7] to the simulated Saudi enterprise. `bench/loader.py` validates the scenario files against a JSON schema and checks every planned action against the tool schemas of the demonstration workspace. The suite currently holds 48 benign user tasks: 24 in English and 24 in Arabic (14 in Modern Standard Arabic and 10 in Gulf dialect). They cover e-mail reading, CRM lookups and updates, internal and external replies, web and document reading, small exports, and four tasks that legitimately need a human (a vendor payment and the deletion of a duplicate customer record, in each language). Each task has a plan of tool calls, a success predicate evaluated on the side effects of the world, and a `reads` list that records which content it reads and therefore where injected text could appear.

The five defence configurations are produced by transforming the default policy in `bench/defenses.py`, so every configuration runs the same production code paths. For example, `taint_only` and `detector_only` drop all constraints, rate limits and global rules and set every allowed tool to `approval: never`, and `policy_only` and `detector_only` switch taint tracking off.

The configurations without a detector use a stand-in that scores every text as benign. Each episode gets a fresh world, a fresh policy engine (so rate-limit counters cannot leak between episodes) and a fresh taint tracker; the audit log and the approval service are shared within one configuration, which makes the per-configuration audit chain and replay check meaningful. The human approver is replaced by an automatic decider in one of three modes: `deny` rejects every escalation, `approve` accepts every escalation and `oracle` accepts exactly the calls that match the user's plan, which models a human who knows what the user asked for. The runner appends one JSON line per finished episode, so an interrupted run resumes where it stopped, records a SHA-256 fingerprint of the scenario files, verifies the audit chain after each configuration and, with `--replay-check`, replays every audited decision. `bench/metrics.py` computes the metrics of Section 4.2.1 per

configuration and per language, Arabic variety and task category, and `bench/report.py` writes JSON, Markdown and Matplotlib figures. `bench/latency.py` measures the HTTP overhead against the real FastAPI application (Section 4.2.5).

Attack episodes are implemented in `bench/attacks.py`. An episode combines a user task with an injection task and an attack template, both written by the team, plants the rendered text in one of the task's `reads` targets with `World.plant()`, and records a survival marker that shows whether the planted text still reaches the agent after the gateway has inspected the result. In attack episodes the `ScriptedAgent` is worst-case compliant, as designed in Section 3.1: the first time a result it receives still contains the planted text, it performs the injection task's actions through the gateway. Each episode is graded on the user task (utility under attack) and on the injection predicate (attack success), and an attack that fails is attributed to the layer that stopped it (approval, policy, taint or detector, or `not_reached` when the planted text never reached the agent). The runner executes these episodes automatically (`--episodes auto`, the default) once `bench/scenarios/attacks/*.yaml` defines injection tasks and attack templates, and `tests/bench/test_bench_attacks.py` tests construction, planting, survival markers and attribution with neutral placeholder text. No attack files exist yet, so no attack episode has been run and the ASR columns print as "[pending]" (Section 4.2.3).

4.1.13 Integration issues found and fixed

Integration was tracked as a checklist (`docs/dev/INTEGRATOR_TODO.md`), and the security test suite of Section 4.2.6 added findings of its own. Table 4.2 lists the issues that were fixed in code; the last column states how each fix was verified, and two fixes still lack a dedicated regression test.

Table 4.2: Issues found and fixed during integration

Issue	Fix	Verified by
A tool that raised an unexpected exception returned the raw exception text to the agent, which could reveal file paths, host names or stack details	The upstream manager returns a generic bilingual message and logs the details server-side; `Gateway.execute` reports only the exception type and stores the message in the audit log	Security test ST-13
A taint rule that an administrator had downgraded to `allow` still raised the risk score and counted as "taint fired", so it still forced approval through `approval: on_risk`	Downgraded taint reasons stay in the decision as evidence but are excluded from the risk score and from the approval condition	Integration test (two cases)
The taint tracker used a fixed list of internal domains, so a policy change left it with stale domains	The tracker receives a function that reads the active policy (Section 4.1.4)	Integration test
Approval summaries said "outside the organisation"	The gateway copies the organisation name from the policy into the approval service at start-up and on every policy swap	Integration test

Issue	Fix	Verified by
Full taint snapshots could exceed the audit payload limit, which broke replay	`minimal_snapshot` stores only the values the rules consulted	Replay sweep (Section 4.2.7)
The taint tracker must see the original result, not the sanitised one, or provenance is lost	`inspect_result` registers the original result	Code review (no dedicated test yet)
The benchmark shared one rate limiter between episodes, which distorted the false-positive rate	Fresh policy engine and taint tracker per episode	Benchmark test
Source files contained literal bidirectional control characters, which can make code display differently from how it executes (the "Trojan Source" problem)	The Python sources write the isolate characters used in bilingual messages as `\u2068` and `\u2069` escapes	Source scan
React Router and Vitest versions with published security advisories	Upgraded to `react-router-dom` 7.18.4 and Vitest 5.0.3	`npm audit` (Section 4.2.6)
The example rule in the specification used a regular expression with exponential backtracking	Replaced by a linear-time equivalent; the validator now rejects nested ambiguous quantifiers	Security test ST-08
CORS did not allow the `X-Mirsad-Session` and `X-Upstream-Authorization` headers	Both added to the allowed headers	Code review (no dedicated test yet)

The error-message fix in the upstream manager distinguishes argument errors (`ValueError` and `TypeError`), whose message the agent needs in order to correct its call, from every other exception, which is logged server-side and replaced by a generic bilingual message. The default policy's rule `no-bulk-export-to-email`, which limits bulk e-mailing of customer records, now counts at-signs with a pattern that cannot backtrack.

Three small items remain open (duplicate login throttling, the demonstration agent's embedded policy and a few literal direction marks in web files); Appendix B.6 lists them.

4.2 Evaluation

4.2.1 Evaluation setup

All measurements were taken on the development machine described in Section 4.1.1 (Apple M5, 16 GB, macOS 26.5, Python 3.12). The gateway used the current default policy (`configs/policies/default.yaml`, in which CRM results are scanned; Section 3.3.4), SQLite for storage, and the detector in rules-only mode because no trained model exists yet. Two agents were used. The **scripted agent** executes each task's plan exactly and is deterministic, so differences between configurations are caused by the gateway alone. The **LLM agent** is the demonstration agent driven by Qwen2.5-7B through

Ollama at temperature 0, with at most eight steps; it shows how a real model's choices interact with the gateway. Table 4.3 lists the defence configurations and approver modes.

Table 4.3: Defence configurations and approver modes

Configuration	Meaning
`none`	No gateway: tools run directly against the world (baseline)
`full`	Default policy, taint tracking and detector with sanitising and spotlighting
`policy_only`	Allow-list, constraints, rate limits, global rules and approval policy; no taint tracking, no detector
`taint_only`	Tool profiles (trust, sensitivity, sink, risk) and taint rules; no constraints, rate limits, rules, approval policy or detector
`detector_only`	Tool profiles and the detector with sanitising and spotlighting; nothing else
Approver `deny`	Every escalation is rejected (a reviewer who rejects anything unusual)
Approver `approve`	Every escalation is accepted (a reviewer who approves without reading)
Approver `oracle`	Exactly the calls in the user's plan are accepted (a reviewer who knows what the user asked for)

The metrics follow AgentDojo's split between utility and attack success [6] and add the costs that matter for a gateway (Table 4.4).

Table 4.4: Evaluation metrics

Metric	Definition
Utility	Share of episodes whose success predicate holds on the world's side effects
Benign false-positive rate (FPR)	Benign calls denied by the automatic layers ÷ benign calls
Approval friction	Benign calls escalated to a human ÷ benign calls
Final block rate	Benign calls that did not run in the end (automatic or human denial) ÷ benign calls
Result false-positive rate	Benign tool results the detector sanitised or blocked ÷ results inspected
Attack success rate (ASR)	Attack episodes whose injection predicate holds ÷ attack episodes
ASR reduction	1 − ASR(defence) ÷ ASR(`none`)
Gateway latency	Decision plus result inspection per call as reported by the gateway, excluding human waiting time
Added latency	HTTP round trip through Mirsad minus the time of a direct in-process tool call, paired per call
Detector precision, recall, F1, FPR, ROC-AUC	Standard classification metrics at the calibrated threshold 0.5 on held-out splits
Replay agreement	Audited decisions whose replay gives the same action ÷ decisions replayed

4.2.2 Detector evaluation

The detector is to be evaluated on three held-out test splits written separately from the training data (English, Arabic and mixed Arabic-English) and, once the team adds it, on an external public English dataset for out-of-distribution evidence. The target is an F1 score of at least 0.85 on both the English and the Arabic test sets (Objective 2). The values in Table 4.5 are read from the evaluation output when the report is built.

Table 4.5: Detector results on held-out splits (ensemble, threshold 0.5)

Split	n	Precision	Recall	F1	FPR	ROC-AUC	F1 rules only	F1 model only
`test_en`	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
`test_ar`	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
`test_mixed`	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
External (`test_ext_deepset`)	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

> **Status: pending team-supplied data.** These results have not been measured. The repository holds only benign training batches; the attack-labelled training, validation and held-out test batches are written by the team (Section 3.6.4). `make detector` then trains and calibrates the model on the training and validation splits only and fills this table (Appendix E.11.1).

What has been measured about the detector is its behaviour on benign content in the running system. Under full protection with the oracle approver, the detector inspected 68 benign tool results, including the CRM look-ups that the default policy scans; it flagged 0% of them and sanitised or blocked 0% (Table 4.7 gives the result false-positive rate of every configuration). This is evidence about over-defence on the business content of the benchmark, in rules-only mode; it says nothing about recall, and it must be measured again once a trained model is packaged. A separate pre-training check of the rules-only fallback on the team's benign dataset batches did flag some records, more of them in the Arabic batch, where polite imperatives and forms of address resemble injected instructions (`docs/detector.md`, Section 13.5). That check is not yet part of the recorded evidence, so its rates are not quoted here; Section 5.3 draws the lesson. Scan latency for a 2 KB text with the trained ensemble: [pending] ms (target: below 10 ms).

4.2.3 Attack resistance on Mirsad-Bench

Attack resistance is the central quantitative claim of the project: the gateway should reduce the attack success rate by at least 80% relative to the unprotected baseline (Objective 3). Table 4.6 will be filled from three benchmark runs: the scripted agent with

an oracle approver (a careful human), the scripted agent with an approve-all approver (which isolates the automatic layers, because every escalation is let through) and the LLM agent with an oracle approver.

Table 4.6: Attack success rate (ASR) by defence configuration

Configuration	ASR, scripted, oracle	ASR, scripted, approve-all	ASR, LLM, oracle	ASR reduction, scripted, oracle
`none`	[pending]	[pending]	[pending]	–
`full`	[pending]	[pending]	[pending]	[pending]
`policy_only`	[pending]	[pending]	–	[pending]
`taint_only`	[pending]	[pending]	–	[pending]
`detector_only`	[pending]	[pending]	–	[pending]

> **Status: pending team-supplied attack files.** No attack episode has been run, so no attack success rate has been measured. The harness executes attack episodes (Section 4.1.12) once the team adds its injection tasks (attacker goals expressed as tool actions with a success predicate) and attack templates in English, Arabic and mixed text to `bench/scenarios/attacks/`, following the README in that folder; `python -m bench.loader` validates them. `scripts/run_bench.sh` then produces the scripted-oracle, scripted-approve and LLM runs, for example `python -m bench.run --agent scripted --episodes attack --defense all --approver oracle --replay-check --out reports/bench/scripted-attack-oracle`, and `scripts/collect_results.py` fills Table 4.6 (Appendix E.11.2).

Even when these runs exist, they will measure Mirsad against non-adaptive attacks written in advance. Attackers who adapt to a known defence have broken every published prompt-injection defence they were tested against [45], [105], and Mirsad's design assumes that the detector will be evaded: the policy and taint layers constrain what a hijacked agent can do, whatever the text says. The ablation columns are meant to show how much each layer contributes on its own.

4.2.4 Benign behaviour

The benign evaluation answers the other half of the question: how much legitimate work the gateway costs. All 48 user tasks were run with the scripted agent under every defence configuration, once with the denying approver and once with the oracle approver. Each configuration made 68 benign tool calls. Table 4.7 summarises the results and Figure 4.1 plots them.

Table 4.7: Benign results of the scripted agent (48 tasks, 68 calls per configuration)

Configuration	Approver	Utility	FPR	Friction	Final block	Result FP
`none`	–	100.0%	0.0%	0.0%	0.0%	–

Configuration	Approver	Utility	FPR	Friction	Final block	Result FP
`full`	deny	75.0%	0.0%	17.6%	17.6%	0.0%
`policy_only`	deny	75.0%	0.0%	17.6%	17.6%	n/a
`taint_only`	deny	87.5%	0.0%	8.8%	8.8%	n/a
`detector_only`	deny	100.0%	0.0%	0.0%	0.0%	0.0%
`full`	oracle	100.0%	0.0%	17.6%	0.0%	0.0%
`policy_only`	oracle	100.0%	0.0%	17.6%	0.0%	n/a
`taint_only`	oracle	100.0%	0.0%	8.8%	0.0%	n/a
`detector_only`	oracle	100.0%	0.0%	0.0%	0.0%	0.0%

The results show the following.

- **Automatic denials.** The benign false-positive rate under full protection was 0.0%, against the 5% target; Table 4.7 gives the rate of every ablation. Where the detector was enabled, it sanitised or blocked the share of benign results given in the Result FP column. The caveat is that the detector ran rules-only and that the tasks were written by the team.
- **Friction comes from the policy's intended escalation paths.** Under full protection, 17.6% of benign calls (12 of 68) were escalated to a human. The reason codes recorded in `reports/bench/scripted-benign-deny/summary.md` account for them: external e-mail replies, each of which triggered both the `domain_in` constraint on `to` (`CONSTRAINT_VIOLATION`, 6 calls) and `TAINT_UNTRUSTED_TO_EXTERNAL` (6), exports (`APPROVAL_POLICY`, 2) and the calls in the tasks that legitimately need a human (`APPROVAL_ALWAYS`, 4). Excluding the approval-required tasks, the unexpected friction was 12.9%.
- **The approver decides the utility cost.** With the oracle approver, utility under full protection was 100%. With an approver who rejects everything it was 75% (81.8% excluding the approval-required tasks), which is the cost of a reviewer who refuses every external reply and export.
- **The policy layer is needed for high-risk tools.** The approval recall, the share of approval-required tasks in which a human was asked about the legitimate payment or deletion, was 0% for `taint_only` and 0% for `detector_only`, against 100% under full protection.

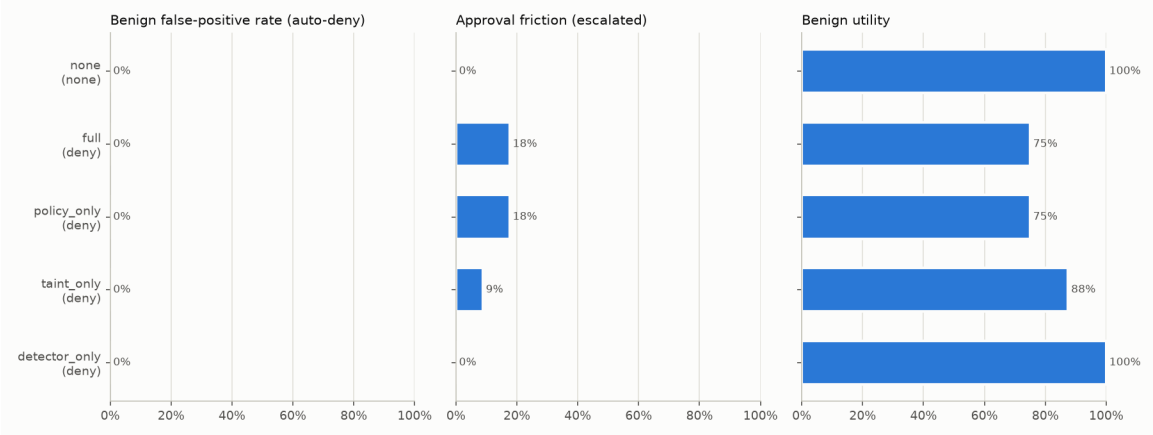


Figure 4.1: Benign false-positive rate, approval friction and utility per defence configuration (scripted agent, denying approver)

Table 4.8 and Figure 4.2 break the full-protection results down by language. Friction was 17.6% for English and 17.6% for Arabic tasks; the policy and taint layers act on tool names, arguments and provenance, which do not depend on the language of the prompt. Within Arabic, friction was 12.5% for the Gulf-dialect tasks and 22.2% for the Modern Standard Arabic tasks; any difference reflects which task categories were written in each variety, not the dialect itself.

Table 4.8: Full protection by language (scripted agent)

Language	Calls	Utility (deny)	Utility (oracle)	FPR	Friction
English	34	75.0%	100.0%	0.0%	17.6%
Arabic	34	75.0%	100.0%	0.0%	17.6%

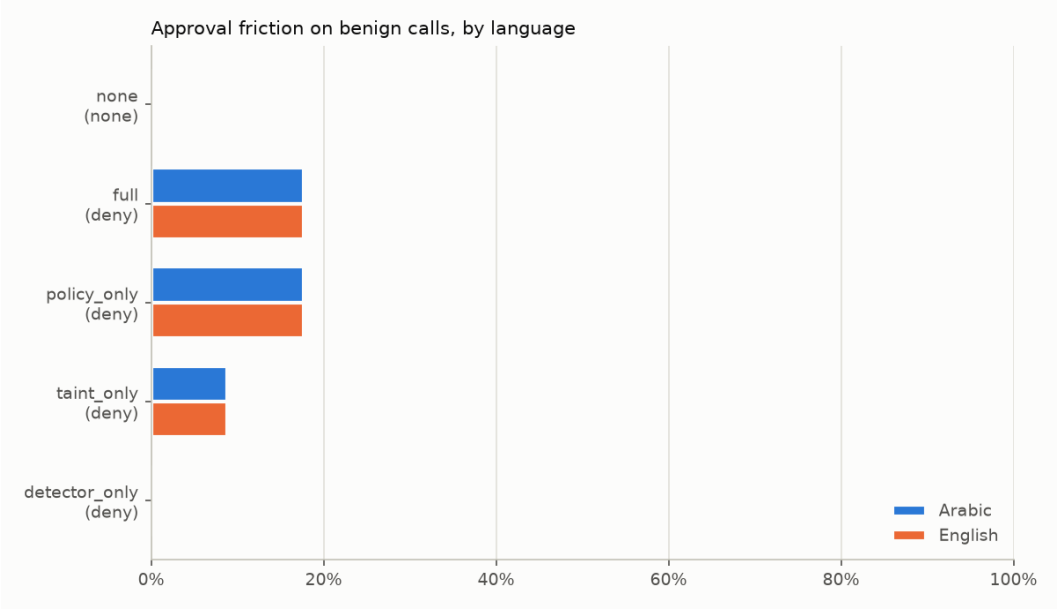


Figure 4.2: Approval friction on benign calls by language and defence configuration

A smoke run with the LLM agent (4 tasks, 2 in English and 2 in Arabic, with and without Mirsad) gave a utility of 75% without and 75% with Mirsad; of the 6 protected calls, 0% were denied and 0% escalated. Both configurations failed the same Arabic CRM-update task, in which the model wrote a generic note instead of the requested phone number; this is a model error that the gateway did not cause. So few episodes make a smoke test, not a utility estimate.

4.2.5 Performance

The latency benchmark (`bench/latency.py`) starts the real FastAPI application with the default policy and agent-key authentication, sends 50 warm-up calls and then 1,000 benign read-only calls spread over six tools, first one at a time and then from 50 concurrent sessions. For each call it subtracts the time of the same tool call made directly in-process from the HTTP round trip through Mirsad. It was run twice: with the server in a thread of the benchmarking process, and with the server in a separate process, which removes the competition between client and server for Python's global interpreter lock. Table 4.9 gives the results and Figure 4.3 their distribution.

Table 4.9: Added latency per tool call (1,000 benign calls)

Scenario	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	HTTP errors
Sequential, separate process	2.7	6.0	6.1	317	0
50 concurrent sessions, separate process	165.5	208.0	216.1	291	0
Sequential, in-process	2.6	5.9	6.0	317	0
50 concurrent sessions, in-process	204.8	880.1	1304.3	154	0

One request at a time, Mirsad adds a median of 2.7 ms per call, of which the gateway's own decision and inspection account for 2.1 ms; the rest is HTTP handling and authentication. NFR1 is assessed on this median and on the separate-process layout, which matches a real deployment (Section 3.5.3). Under 50 concurrent sessions with the server in its own process, the median rises to 166 ms, against the 200 ms target. With client and server sharing one interpreter it is 205 ms; that layout measures the client competing with the server for Python's global interpreter lock, not a deployment, so it is reported as context only. No request failed in either layout, so the single node served 50 concurrent sessions correctly (NFR11).

The concurrent numbers are mostly queueing. With the server in its own process, the time the gateway itself reported per call grew from a median of 2.1 ms to 159.5 ms, because the decision pipeline holds one lock while it evaluates and appends to the audit chain, and the SQLite writes serialise behind it. This is the price of the design choice in Section 4.1.2 that keeps taint state and the hash chain consistent. The in-process run additionally

shares one interpreter between client and server, which explains its long tail (p95 880 ms). A deployment that expects sustained concurrency should therefore be measured with several workers and PostgreSQL before a concurrency figure is quoted; finer-grained locking per session is a possible improvement.

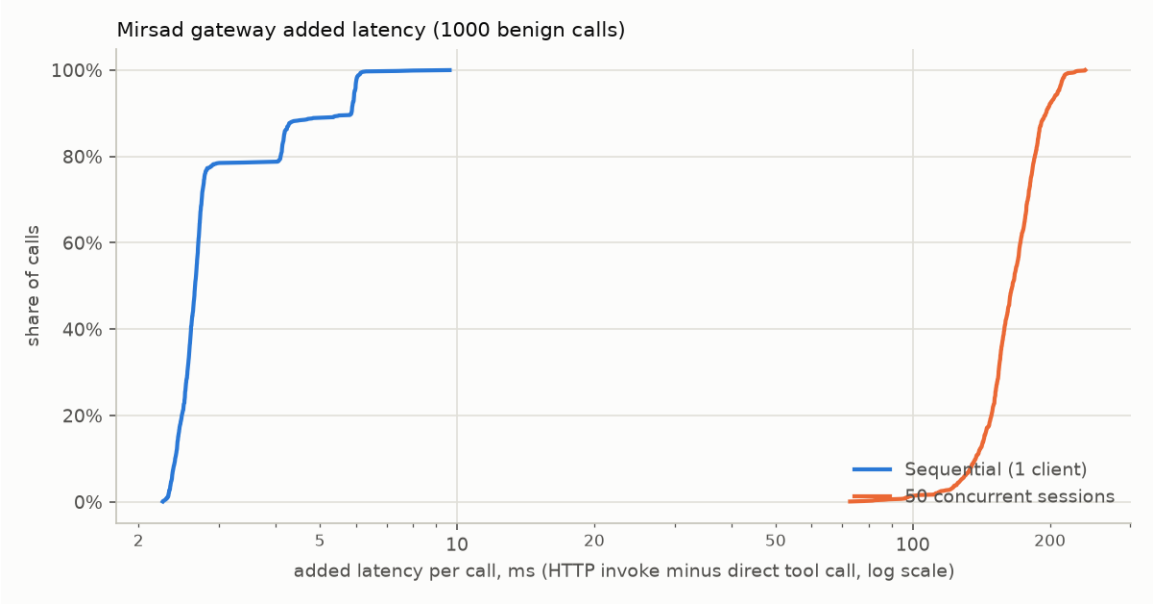


Figure 4.3: Distribution of added latency per call, sequential and with 50 concurrent sessions (server in a separate process, logarithmic scale)

Within the benchmark, where calls run in-process without HTTP, the gateway's median cost per call was 1.8 ms under full protection, against 0.9 ms for the policy layer alone (Figure 4.4). The difference is mostly the detector scan and the taint bookkeeping on each result.

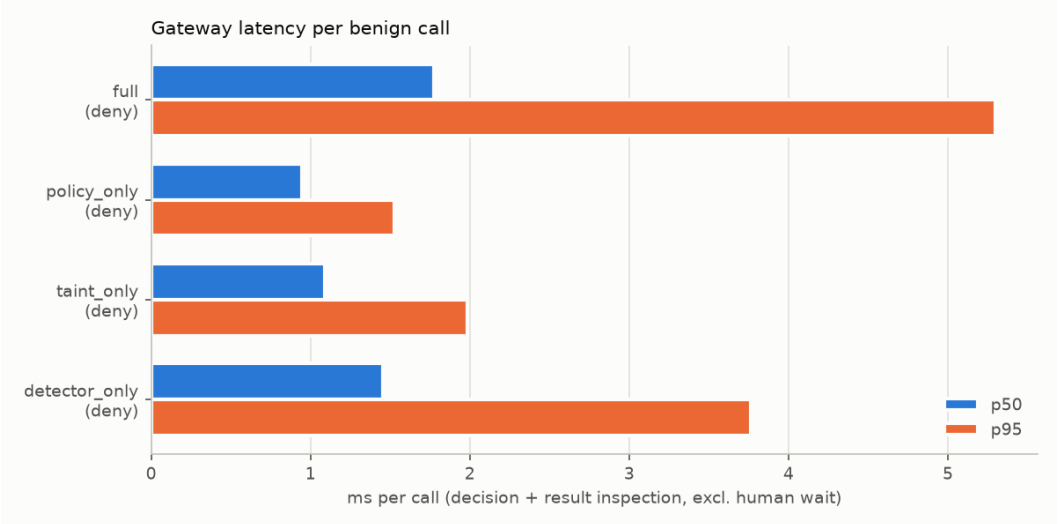


Figure 4.4: Gateway latency per benign call by defence configuration (decision plus result inspection, excluding human waiting)

4.2.6 Security testing of the gateway

A firewall that concentrates trust becomes a target itself, so the gateway was tested against its own threat model. The STRIDE analysis of the data-flow diagram [69], [91] produced the security tests in `tests/security/test_gateway_security.py`. They replace the manual Burp Suite and OWASP ZAP pass planned in the proposal (Section 3.4.5) with checks that run on every change. The suite has 23 security tests (ST-01 to ST-23) that expand to 45 test cases; all of them passed in the evidence run (Table 4.10).

Table 4.10: Security tests of the gateway by STRIDE category

Category	Tests	What is checked
Spoofing	ST-01 to ST-05	Every management route requires authentication; forged, unsigned, expired and malformed tokens are rejected; editing the role claim of a valid token does not grant admin rights; rotating an agent key revokes the old key; malformed agent keys are rejected
Tampering	ST-06 to ST-08	Modifying or deleting an audit row is detected by chain verification; a policy containing a catastrophic regular expression is rejected
Repudiation	ST-09, ST-10	Approval decisions are attributed to the approver and cannot be changed; failed logins are audited without the password
Information disclosure	ST-11 to ST-17	Secrets in tool arguments are redacted in the audit log; user and agent listings never return hashes or keys; tool failures do not leak internal details; static file paths cannot escape the web root; security headers are present; CORS rejects unknown origins; SQL-like filter values are treated as data
Denial of service	ST-18, ST-19	Oversized request bodies are rejected; repeated failed logins are throttled
Elevation of privilege	ST-20 to ST-23	Approver and auditor roles cannot perform admin actions; agent keys cannot use human APIs and human tokens cannot act as agents; unlisted tools and unknown agents are denied by default; the default policy blocks requests from `fetch_url` to loopback, link-local metadata and internal addresses

The other automated checks gave the following results.

- **Static analysis.** Bandit reported 0 issues over 13,023 lines of code. The configuration skips only the rule against `assert` statements (B101) and excludes the tests, and the report records no inline `nosec` suppressions, so the zero is not the result of silencing findings. Ruff's security rules (the `S` set) are part of the lint configuration, but no Ruff report is stored in `reports/` (Appendix B.5.3).
- **Dependency audit.** pip-audit checked 124 Python packages and found 0 with known vulnerabilities; `npm audit` over the web app's 231 dependencies, including development dependencies, reported 0 vulnerabilities after the React Router and Vitest upgrades (Section 4.1.13).

- **PostgreSQL compatibility.** The tests in `tests/integration/test_postgres.py` run the storage, audit chain, authentication and approval code against a real PostgreSQL server started by the `pgserver` package: appending and verifying the chain, detecting tampering, a user and agent-key round trip, and the full approval lifecycle. They passed in the recorded run, which supports the claim that the same code runs on SQLite in development and PostgreSQL in deployment.
- **Test suite and coverage.** All 1,281 Python tests (0 failures) and all 194 web tests (38 Vitest suites in the 10 test files of `web/tests/`) passed. Statement coverage of the `mirsad` package was 94.4% of 9,763 statements; the lowest package, the adapters, reached 90.3%.

The automated suite does not replace an independent penetration test. It checks the threats the team identified, with the inputs the team chose; Section 4.2.10 returns to this.

4.2.7 Audit integrity and reproducibility

Every benchmark configuration verified its audit chain after its episodes, and every audited decision was replayed with the stored policy version, taint snapshot and rate-limit counts. In the denying-approver run, 272 of 272 decisions replayed to the same action, and in the oracle run 272 of 272: four protected configurations with 68 decisions each. Every chain verified (Appendix B.5.6). The LLM smoke run replayed 6 of 6 decisions, and both latency runs ended with an intact chain. This meets the acceptance criterion of FR6.3 ("100% of benchmark decisions replay to the same action"). The tampering tests ST-06 and ST-07, and their PostgreSQL counterpart, show the other side: a modified or deleted row is detected and the first broken sequence number is reported.

Replay is exact because every source of variation is captured. Policy versions are content hashes stored in the database, the taint rules are a pure function of a stored snapshot, rate-limit counts are stored with the decision, and the scripted benchmark uses a fixed world seed. The benchmark runner also stores a SHA-256 fingerprint of the scenario files and warns when a resumed run was graded with different files. The fingerprint does not cover the policy, so the reported runs were executed from scratch after the last change to the default policy (Appendix B.1). The whole evidence base of this chapter can be regenerated with one command (`scripts/run_evidence.sh`), and the report itself reads its numbers from the generated files, so the text and the evidence cannot drift apart silently.

4.2.8 Usability: approval decision time

The usability target (NFR4) is that an approver can understand a paused request and decide on it in under 30 seconds on a phone. No user study has been run yet. As a first, transparent estimate, the approval flow of the PWA was modelled with the Keystroke-Level Model (KLM) of Card, Moran and Newell [100], using their standard operator times: mental preparation M = 1.35 s, pointing P = 1.10 s and a key or button press K = 0.28 s (their value for an average non-secretary typist, used here as a conservative press

time). The KLM was defined for keyboard and mouse; a tap on a phone is modelled as P + K = 1.38 s, an assumption rather than a measured value. Three further assumptions are needed:

- **Opening time.** The PWA opens on the request detail within R = 1.0 s of tapping the notification (assumed, not measured; the notification links directly to the request).
- **Reading rate.** The approver reads at 200 words per minute, i.e. 0.30 s per word. This is an assumption chosen to be slower than fluent silent reading to allow for a small screen, technical identifiers and reading in a second language; a sensitivity check follows.
- **Reading load.** Word counts come from the real summary generator. Case A is the paused deletion shown in Figure C.19 (summary 34 words, reason cards 19 words). Case B is an external e-mail escalated by both the domain constraint and the untrusted-content taint rule, the most common escalation in the benchmark (summary 57 words, reason cards 51 words). The confirmation sheet adds 13 words.

Table 4.11 gives the step-by-step estimate.

Table 4.11: Keystroke-Level Model estimate of the approval decision time (seconds)

Step	Operators	Case A	Case B
Notice the notification and decide to open it	M	1.35	1.35
Tap the notification	P + K	1.38	1.38
PWA opens on the request	R (assumed)	1.00	1.00
Read "What the agent wants to do"	34 / 57 words × 0.30	10.20	17.10
Decide	M	1.35	1.35
Tap Approve or Deny	P + K	1.38	1.38
Read the confirmation sheet	13 words × 0.30	3.90	3.90
Tap the confirm button (no note)	P + K	1.38	1.38
Total, reading the summary		**21.94**	**28.84**
Optional: also read every reason card	19 / 51 words × 0.30	+5.70	+15.30
Total, reading everything		**27.64**	**44.14**

Under these assumptions an approver who reads the plain-language summary decides within the 30-second target in both cases, with little margin in case B. An approver who also reads every reason card of a multi-reason request needs about 44 s, and at a slower reading rate of 150 words per minute even the summary-only path of case B takes about 36 s. Typing a note would add time in every case. This suggests two design changes: the summary carries the decision, so it should stay short, and the reason cards of multi-reason requests could be collapsed by default or reduced to one line each, since they largely repeat the summary. The Arabic summaries of the same requests are shorter in words (30 and 51), but no Arabic reading rate was assumed.

The KLM predicts the time of an expert who makes no errors; it does not measure understanding or trust. The estimate should therefore be confirmed with a small user study in which security practitioners decide on prepared requests on their own phones while their decision time is recorded, followed by Section D of the questionnaire in Appendix F (in particular D1, "I understood why the action was paused from the approval summary alone", and D2, "I could decide on the request in under 30 seconds").

4.2.9 Success criteria

Table 4.12 compares the measured results with the success criteria of the proposal (Section 1.3) and with the supporting requirement targets of Chapter 3.

Table 4.12: Success criteria, targets and measured results

Criterion	Target	Measured	Status
Interception of tool calls and results (MCP over stdio and HTTP, OpenAI-style function calling)	100% of calls pass through the gateway	MCP proxy (stdio and streamable HTTP), OpenAI-compatible proxy and Guard API all route through `Gateway.execute` or `check_call`; 78 adapter tests pass	Met (functional tests)
Declarative YAML policy with per-tool and per-parameter constraints and rate limits	Implemented and validated	All constraint operators, globs, rules, rate limits and monitor mode implemented; 265 policy tests pass	Met
Taint tracking blocks or escalates flows of untrusted or confidential data to external destinations	Rules fire on such flows, not on legitimate replies	Four taint rules implemented; 93 taint tests pass; no benign call denied	Met (functional); effect on attacks pending
Median added latency per tool call (NFR1)	≤ 200 ms, judged on the sequential median and the separate-process layout	2.7 ms sequential; 166 ms with 50 concurrent sessions (separate process). Context only: 205 ms when client and server share one interpreter	Met (separate process)
Detector F1 on held-out English and Arabic test sets	≥ 0.85 each	English [pending]; Arabic [pending]	Pending data
Attack success reduction relative to the unprotected baseline	≥ 80%	[pending]	Pending data
Benign actions wrongly blocked	≤ 5%	0.0% under full protection (68 benign calls per configuration, detector rules-only)	Met
Approval app, dashboard and tamper-evident audit log in which every blocked or approved action can be replayed	All decisions replayable	272 / 272 and 272 / 272 decisions replayed identically; all chains verified	Met
NFR4: decision time on a phone	< 30 s	KLM estimate 21.9 s and 28.8 s reading the summary; 44.1 s reading every reason of a multi-reason request	Partially verified (user study pending)
NFR9: statement coverage of the	≥ 80%	94.4%	Met

Criterion	Target	Measured	Status
core package			
NFR11: concurrent agent sessions on one node	≥ 50	50 sessions, 0 errors	Met
NFR3: security testing of the gateway	Suite passes; no known vulnerable dependencies	45 STRIDE cases pass; Bandit 0 issues; 0 vulnerable Python and 0 vulnerable npm packages	Met
NFR5: deployment with one `docker compose up`	Works	Not executed: Docker was not available on the build machine	Not verified (Docker unavailable)

No criterion is shown as not met. The two most important outcomes of the proposal, detector F1 and the reduction of attack success, cannot be judged until the team's attack data exists; the decision-time target needs a user study; and the Docker deployment needs to be executed on a machine with Docker.

4.2.10 Limitations and threats to validity

Results not yet available. The detector has not been trained and no attack episode has been run, so the two headline claims of the project are untested. Until they are, the evaluation shows that Mirsad is cheap for legitimate work, not that it stops attacks. The benign false-positive rate and the detector's result false-positive rate were also measured with the rules-only detector and must be re-measured with the trained model, whose operating point is tuned for a false-positive rate of at most 5% on the validation split.

Internal validity. The scripted agent replays fixed plans, so the benign results show the effect of the gateway on correct behaviour but not how a real model's wording and tool choices interact with the policy; the LLM run has only 4 tasks. The oracle approver is an idealised human and the denying approver a pessimistic one; real approvers fall between them and can also approve a harmful request. The user tasks, the default policy and the escalation paths were written by the same team, which may favour low friction; for example, the policy deliberately escalates every external e-mail, and a different policy would give different numbers.

External validity. All results come from one simulated organisation with 48 tasks, one local model, one laptop, SQLite and a single server worker. Real deployments have other tools, larger and noisier data, other languages and dialects, and more load. The concurrency results in particular should be repeated with several workers and PostgreSQL.

Construct validity. "False positive" counts only automatic denials; escalations are reported separately as friction, because a paused call costs a human's time but does not fail the task by itself. "Added latency" is measured against an in-process tool call and therefore includes HTTP handling that a direct remote tool call would also incur. The

KLM estimate depends on assumed reading and opening times and on standard operator values defined for desktop input.

Security evaluation. The STRIDE suite tests the threats the team identified with inputs the team chose. It is not an independent penetration test, it does not include fuzzing of the API, and it does not model adaptive attackers who study the policy, the rules and the detector [45], [105]. Spotlighting and the detector are expected to be bypassed by such attackers; the policy and taint layers are the intended backstop, and their effect remains to be measured.

Engineering limitations. The gateway serialises decisions behind one lock, which limits throughput under concurrency. The Docker deployment has not been executed. The open integration items listed in Appendix B.6 (duplicate login throttling, the demonstration agent's embedded policy, the remaining literal direction marks in web files) are small but should be closed. The screenshots also show minor ordering defects where Latin names, times and Arabic text meet on one line (for example the version line of the Arabic Policies page and the requester line of the Arabic approval detail), which isolating those values in the markup would fix.

4.3 Individual and Group Contributions

The work was divided into the five workstreams of Table 3.19. Table 4.13 maps the main project tasks to the members who carried them out, following the workstream ownership and the task owners of the project plan (`docs/pm/plan.yaml`).

Table 4.13: Project Tasks & Task Assignments

TASK	DETAILS	ASSIGNED TEAM MEMBERS
PROJECT SETUP	Repository and branch rules, continuous-integration workflow, development environment (Python 3.12, Node.js, local Ollama model), Scrum board, engineering contract and shared data types	Abdulmalik bin Saedan (coordination, environment); Bader Alshuraie (engineering contract and shared types); all members (local setup)
PROJECT PROPOSAL	SPAP form: problem statement, objectives O1–O6, success criteria, initial plan and budget	All members, coordinated by Abdulmalik bin Saedan
REQUIREMENTS GATHERING AND ANALYSIS	Literature and product review; STRIDE threat model and attack taxonomy; functional and non-functional requirements, use cases and personas (SRS); benchmark and dataset design; stakeholder questionnaire	All members; threat model: Bader Abdullah Abbod Alyamini and Abdulmalik bin Saedan; benchmark and dataset design: Abdulmalik bin Saedan and Abdullah Naif Abdullah Al-Orabi
SYSTEM DESIGN	Architecture and high- and low-level design; policy-language specification; database and audit-chain design; detector and dataset design; wireframes of the dashboard and approver app	Bader Alshuraie (architecture, policy language); Bader Abdullah Abbod Alyamini (database,

TASK	DETAILS	ASSIGNED TEAM MEMBERS
		audit chain); Abdullah Naif Abdullah Al-Orabi (detector); Mohammed Yasser Alanssari (user interface)
DEVELOPMENT	Gateway pipeline, protocol adapters, SDK, policy engine, REST API and CLI; detector, rule base and dataset pipeline; taint tracker, audit log, redaction and storage; approval workflow; dashboard and approver PWA; demonstration workspace and agent; Mirsad-Bench	Bader Alshuraie; Abdullah Naif Abdullah Al-Orabi; Bader Abdullah Abbod Alyamini; Mohammed Yasser Alanssari; Abdulmalik bin Saedan (each for their workstream); approval workflow and admin API: Bader Alshuraie and Bader Abdullah Abbod Alyamini
TESTING & VALIDATION	Unit, integration and UI tests of each area; benchmark runs and latency measurement; detector evaluation; security testing of the gateway; integration checklist	All members (tests of their areas); Abdulmalik bin Saedan (benchmark, latency, security testing); Abdullah Naif Abdullah Al-Orabi (detector evaluation); Bader Abdullah Abbod Alyamini and Mohammed Yasser Alanssari (security testing)
DEPLOYMENT	Docker image and Compose file (gateway and PostgreSQL), setup and demo scripts, production web build, installation and user guide	Mohammed Yasser Alanssari and Bader Alshuraie (deployment, user guide); Abdulmalik bin Saedan (demo and evidence scripts)
FINAL REPORT & PRESENTATION	Report chapters, figures, poster, slides and open-source release	All members, coordinated by Abdulmalik bin Saedan

Bader Alshuraie (SE) owned the core of the gateway: the decision pipeline in `gateway.py`, the policy language with its schema, validator and engine, the rate limiter and the protection against unsafe regular expressions, the protocol adapters (upstream manager, MCP proxy over stdio and HTTP, OpenAI-compatible proxy), the Python SDK, the REST API and the command line. Alshuraie was also responsible for the architecture and the policy-language specification that the other workstreams coded against. The policy, adapter and API areas account for 265, 78 and 185 tests respectively.

Abdullah Naif Abdullah Al-Orabi (SE, AI and DS) owned the Arabic and English injection detector: the normalisation with its offset map and evasion signals, language identification including Arabizi, the YAML rule base and its matcher, the hashed n-gram model with sliding windows, the noisy-OR ensemble and its calibration, pickle-free model persistence, sanitising and spotlighting, and the dataset pipeline with group

splitting, train-only augmentation and leakage checks. Al-Orabi was also responsible for the dataset schema and labelling guideline that the team's data must follow. The detector area has 227 tests, none of which contains adversarial text.

Bader Abdullah Abbod Alyamini (IS, CYS) owned the taint tracker (value extraction, session store, the four rules and minimal snapshots for replay) and the audit layer: the HMAC hash chain with verification and external anchors, redaction of secrets and personal identifiers, query and export, the database schema and storage layer, and the authentication primitives. Alyamini contributed to the threat model, the approval workflow and the security testing. The taint and audit areas have 93 and 240 tests.

Mohammed Yasser Alanssari (IS, CYS) owned the two web front ends: the bilingual, accessible administration dashboard with its ten pages and the installable approver PWA with its service worker, Web Push, deep links, confirmation flow and offline shell, together with the shared live-event client and the English and Arabic dictionaries. Alanssari also owned the user-facing side of the approval workflow, contributed to the security testing and to the deployment and user guide. The web code has 194 tests, including contrast and dictionary-consistency checks.

Abdulmalik bin Saedan (IS, CYS) managed the project (plan, sprints, coordination and integration checklist) and owned the simulated Najd Trading Co. workspace with its MCP server, the demonstration agent and Demo console scenarios, Mirsad-Bench with its scenarios, defence configurations, metrics and latency benchmark, the security test suite of the gateway and the evidence scripts that produce the numbers in this chapter. The benchmark, demonstration and security areas have 68, 72 and 45 tests.

All members took part in the literature review, the requirements, the design reviews, code review and integration, and the report, poster and presentation. As disclosed in Section 3.6.6, an AI coding assistant was used across the workstreams; here, owning an area means that the member designed, reviewed, tested and integrated its code, can explain it, and is accountable for it.

Chapter 5 – Conclusion and Future Work

This chapter compares the results with the objectives of Section 1.3 and then states the project's contributions, the lessons learned, the limitations and the work that should follow.

5.1 Summary of Outcomes

The team built the gateway described in the proposal and measured everything that can be measured without attack data. Table 5.1 compares each SMART objective with its result. Values shown as [pending] are filled in automatically from `reports/results_summary.json` once the corresponding runs exist.

Table 5.1: Outcomes against the SMART objectives

Objective	Target	Result	Status
1. Action-level security at the tool boundary	Intercept tool calls and results over MCP (stdio, HTTP) and OpenAI-style function calling; YAML policy; taint tracking	All four integration paths (MCP proxy, OpenAI-compatible proxy, Guard API, Python SDK) route through one decision pipeline; 78 adapter, 265 policy and 93 taint tests pass	Met
1. Latency	Median added latency ≤ 200 ms per call	Sequential: 2.7 ms median, 6.0 ms p95. 50 concurrent sessions: 166 ms median, 208 ms p95 (server in its own process)	Met (median, separate process)
2. Arabic/English detector	F1 ≥ 0.85 on held-out English and Arabic test sets	English F1 [pending]; Arabic F1 [pending]; detector and training pipeline implemented (227 tests)	Pending data
3. Human control and audit	Mobile approval app, dashboard, tamper-evident log, replay of every decision	Approver PWA and dashboard in English and Arabic; audit chains verified after every benchmark configuration; 272 of 272 audited decisions replayed to the same action	Met
3. Benign false positives	≤ 5% of benign actions wrongly blocked	0.0% of 68 benign calls denied automatically (full protection)	Met for this task set
3. Attack reduction	ASR reduced by ≥ 80% against the unprotected baseline	ASR reduction [pending]	Pending data

Objective 1. Interception was verified functionally by the adapter, API and integration suites rather than expressed as a measured percentage. The latency benchmark ran 1,000 benign calls with the detector in rules-only mode. One call at a time, Mirsad added a few milliseconds. Under concurrent load the median was 166 ms with the server in its own process, the deployment layout on which the 200 ms target is judged, and 205 ms when client and server shared one interpreter, a layout reported only as context (Section 4.2.5). Queueing dominated the tail: in the shared layout the p95 rose to 880 ms.

Objective 2. The detector pipeline (Arabic and Unicode normalisation, weighted rules, a hashed n-gram model with pickle-free persistence, sanitising, spotlighting and dataset tooling) is implemented and tested, but its accuracy has not been measured. Detector F1 on the held-out Arabic and English test sets is produced by running `make detector` after the team adds its attack-labelled batches to `data/detector/raw/` (procedure in `docs/detector.md`, Section 11.4). Attack material is written by the team (Section 3.6.4). Until a model is trained, the packaged detector runs in its rules-only fallback, and every number in this chapter was measured in that mode.

Objective 3. On the 48 benign English and Arabic tasks of Mirsad-Bench, full protection automatically denied 0.0% of benign calls and escalated 17.6% to a human, all on the policy's intended escalation paths (external e-mail, exports, payments and deletions). With an approver who knows what the user asked for, utility stayed at 100%; with an approver who rejects every request it fell to 75%. In a smoke run of 4 tasks with the Qwen2.5-7B agent, utility was 75% with Mirsad and 75% without it. Attack success rate and its reduction are not yet measured: the harness executes attack episodes (Section 4.1.12), but the team's attack files are not yet in `bench/scenarios/attacks/`, so no attack episode has been run.

5.2 Contributions

1. **A model-agnostic enforcement point at the tool boundary.** One decision pipeline serves MCP clients (stdio and streamable HTTP), OpenAI-compatible agents and any framework through the Guard API and Python SDK, so an existing agent is protected by changing a URL or a command.
2. **A policy language for agent actions.** Rules such as "no more than 50 customer records per export" or "e-mail outside the company needs approval" are written as reviewable YAML, validated with line-numbered errors, and activated or rolled back at run time.
3. **Taint tracking with provenance.** Each escalation names the tool, call and field a value came from, so the approver sees evidence instead of the agent's own account of what it is doing.
4. **Arabic-aware detection tooling.** Normalisation handles diacritics, letter variants, Arabic-Indic digits, invisible and bidirectional characters and homoglyphs, and the rules cover Modern Standard Arabic, Gulf dialect and Arabizi alongside English.
5. **Verifiable accountability.** The HMAC hash-chained audit log detects modified and deleted events, and each stored decision can be replayed against its stored policy version and inputs.
6. **Mirsad-Bench.** An offline benchmark with Arabic (MSA and Gulf) and English tasks, five defence configurations, approver models and a replay check, licensed with the rest of the code under Apache-2.0.

5.3 Lessons Learned

- **Fixing the interfaces first paid off.** The engineering contract (`docs/dev/BUILD_SPEC.md`) let each workstream code against fixed interfaces, and the defects found during integration clustered at the seams (Section 4.1.13). For example, storing the full taint snapshot in each audit event exceeded the payload cap and would have broken replay, so the gateway now stores a minimal snapshot.
- **A false-positive rate alone hides the cost of a policy.** With 0.0% of benign calls denied automatically, 17.6% still needed a human, mainly because the default policy escalates every external e-mail. Friction must be reported next to the false-positive rate, and it is decided by the policy more than by the detector.
- **Benchmark predicates must be strict.** In the first LLM smoke run, a loose success predicate counted a vague CRM note as a completed task, which showed how easily a benchmark can overstate utility.
- **Arabic needs its own evaluation.** Polite imperatives and forms of address in ordinary Arabic business text overlap with the wording of injected instructions. In a pre-training check, the rules-only fallback flagged more records in the team's Arabic benign batch than in the English one (Section 4.2.2), so every detector metric has to be broken down by language and variety.
- **Process layout changes latency results.** Running the load generator inside the server process inflated tail latency; a separate server process gave a much lower p95 for the same workload. Performance claims must state the layout they were measured with.

5.4 Limitations

- **Protection is not yet quantified.** Without attack-labelled data, the evaluation shows what Mirsad costs legitimate work, not how much protection it gives.
- **Evaluation scope.** The scripted agent replays a fixed plan, and the LLM run is a smoke test of 4 tasks rather than an estimate of utility. The oracle approver is an idealised human; real approvers' decision time and quality were not measured, and no user study has been carried out.
- **Taint matching is literal.** Values are recognised only when they reappear verbatim after normalisation, so an agent that paraphrases, translates or re-encodes data defeats value matching. Session-level taint is coarse enough to cause approval friction.
- **Protocol coverage.** Only MCP tools are proxied; resources, prompts and sampling are not, non-text content is dropped rather than inspected, and the OpenAI-compatible proxy does not support streaming.
- **Single-node deployment.** Rate limits live in memory in one process, measurements used SQLite on one laptop, and tail latency under concurrency is high. The Docker Compose deployment was not exercised in the measured environment.

- **Audit and notification gaps.** The log has no retention or erasure mechanism, anchoring of the chain head is manual, and neither Web Push delivery to real phones nor an automated accessibility audit has been tested.
- **Adaptive attackers.** Adaptive attacks have bypassed a wide range of published prompt-injection defences, including detection-based ones [45], [105]. Mirsad's policy and taint layers do not depend on recognising attack wording, but no adaptive red-team evaluation has been run against them.

5.5 Future Work

1. **Transformer-based Arabic detector.** Once the dataset exists, fine-tune an Arabic or multilingual encoder such as AraBERT, MARBERT, CAMeLBERT or XLM-R [106], [107], [58], [108], compare it with the n-gram model per language and variety, and distil it if needed to fit the CPU latency budget.
2. **Store-packaged mobile app.** Wrap the approver PWA with Capacitor or rebuild it in Flutter for organisations that require managed apps, adding native push and biometric confirmation of decisions.
3. **Multi-replica deployment.** Move rate-limit counters and live notifications to shared infrastructure (PostgreSQL or Redis), run several workers behind a load balancer, and repeat the concurrency benchmark.
4. **Richer information-flow control.** Replace session-level taint with value-level labels based on lattice information-flow models [62], [63], and offer a CaMeL-style mode in which a plan derived only from the trusted user request constrains later tool calls [16].
5. **Fuller MCP coverage.** Proxy and inspect MCP resources and prompts, handle sampling and non-text content, and track new revisions of the specification [3].
6. **User study with practitioners.** Measure approvers' decision time and understanding of the bilingual summaries with security practitioners in Saudi organisations, starting from the questionnaire in Appendix F.
7. **Red-team evaluation with adaptive attacks.** Commission an independent red-team exercise that includes adaptive, optimisation-based attacks, and tag test cases and audit events with MITRE ATLAS technique identifiers [109].
8. **Certification readiness against NCA ECC.** Close the gaps listed in Appendix A (log retention of at least 12 months, SIEM forwarding, automatic external anchoring of the audit chain, and a formal mapping to the ECC-2:2024 controls [11]) so that an adopting organisation can use Mirsad's evidence in an assessment.

5.6 Concluding Remarks

Mirsad shows that the tool-calling boundary is a workable control point for AI agents. One gateway can apply an organisation's rules to every action an agent takes, explain

each decision in Arabic and English, ask a human when the stakes are high, and keep a record that can be verified and replayed. In the benign evaluation it denied 0.0% of legitimate calls automatically and added little latency, and its remaining cost comes from policy choices that an organisation can see and tune. Whether this layered design reduces the success of real Arabic and English injection attacks still has to be shown by the attack evaluation, and the benchmark and evidence pipeline are ready to produce that result once the team's data is in place.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th Int. Conf. Learn. Represent. (ICLR), Kigali, Rwanda, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023, pp. 68539–68551, doi: 10.52202/075280-2997.

[3] Model Context Protocol, "Specification," protocol revision 2026-07-28. Accessed: Oct. 11, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28.

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec '23), Copenhagen, Denmark, 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

[5] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," OWASP Foundation, Nov. 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

[6] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Adv. Neural Inf. Process. Syst. 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, pp. 82895-82920, doi: 10.52202/079017-2636.

[7] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings Assoc. Comput. Linguist.: ACL 2024, Bangkok, Thailand, Aug. 2024, pp. 10471-10506, doi: 10.18653/v1/2024.findings-acl.624.

[8] S. Willison, "The lethal trifecta for AI agents: private data, untrusted content, and external communication," Simon Willison's Weblog, Jun. 16, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[9] Kingdom of Saudi Arabia, Saudi Vision 2030, Riyadh, Saudi Arabia, Apr. 2016. Accessed: Oct. 11, 2026. [Online]. Available: https://www.vision2030.gov.sa/media/rc0b5oy1/saudi_vision203.pdf

[10] Saudi Data and Artificial Intelligence Authority (SDAIA), AI Ethics Principles, Doc. SDAIA-P114E, ver. 1, Riyadh, Saudi Arabia, May 2025 (English edition; the principles were adopted in Sep. 2023). Accessed: Oct. 11, 2026. [Online]. Available: https://sdaia.gov.sa/en/SDAIA/about/Documents/ai-principles.pdf

[11] National Cybersecurity Authority, Essential Cybersecurity Controls (ECC-2:2024), Riyadh, Saudi Arabia, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://cdn.nca.gov.sa/api/files/public/upload/86e09090-44e4-481f-bc28-355673607654_ECC--2024-EN.pdf

[12] Kingdom of Saudi Arabia, Personal Data Protection Law, Royal Decree No. M/19 of 9/2/1443H (16 Sep. 2021), Council of Ministers Resolution No. 98 of 7/2/1443H, amended by

Royal Decree No. M/148 of 5/9/1444H (Mar. 2023). Accessed: Oct. 11, 2026. [Online]. Available: https://laws.boe.gov.sa/BoeLaws/Laws/LawDetails/b7cfae89-828e-4994-b167-adaa00e37188/1

[13] Z.-X. Yong, C. Menghini, and S. H. Bach, "Low-resource languages jailbreak GPT-4," presented at the NeurIPS 2023 Workshop on Socially Responsible Language Modelling Research (SoLaR), 2023, arXiv:2310.02446. [Online]. Available: https://arxiv.org/abs/2310.02446

[14] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, "Multilingual jailbreak challenges in large language models," in Proc. 12th Int. Conf. Learn. Represent. (ICLR), 2024. [Online]. Available: https://arxiv.org/abs/2310.06474

[15] M. Al Ghanim, S. Almohaimeed, M. Zheng, Y. Solihin, and Q. Lou, "Jailbreaking LLMs with Arabic transliteration and Arabizi," in Proc. 2024 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Miami, FL, USA, Nov. 2024, pp. 18584–18600, doi: 10.18653/v1/2024.emnlp-main.1034.

[16] E. Debenedetti et al., "Defeating prompt injections by design," arXiv:2503.18813, Mar. 2025 (v2, Jun. 2025).

[17] L. Beurer-Kellner et al., "Design patterns for securing LLM agents against prompt injections," arXiv:2506.08837, Jun. 2025.

[18] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.

[19] K. Kubrak, A. El-Moselhy, A. Alsulami, R. Altuwaim, H. I. Fawaz, and F. Alsaby, "Arabic prompts with English tools: A benchmark," presented at the LLMs, Big Data, and Multilinguality for All (LLMs4All) Workshop, IEEE BigData 2025, Macau, Dec. 2025, arXiv:2601.05101. [Online]. Available: https://arxiv.org/abs/2601.05101

[20] S. Willison, "Prompt injection attacks against GPT-3," Simon Willison's Weblog, Sep. 12, 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://simonwillison.net/2022/Sep/12/prompt-injection/

[21] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," presented at the NeurIPS ML Safety Workshop, 2022, arXiv:2211.09527. [Online]. Available: https://arxiv.org/abs/2211.09527

[22] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symp. (USENIX Security 24), Philadelphia, PA, USA, Aug. 2024, pp. 1831-1847.

[23] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, "Adversarial machine learning: A taxonomy and terminology of attacks and mitigations," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-2e2025, Mar. 2025, doi: 10.6028/NIST.AI.100-2e2025.

[24] L. Beurer-Kellner and M. Fischer, "MCP security notification: Tool poisoning attacks," Invariant Labs Blog, Apr. 1, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks

[25] M. Milanta and L. Beurer-Kellner, "GitHub MCP exploited: Accessing private repositories via MCP," Invariant Labs Blog, May 26, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://invariantlabs.ai/blog/mcp-github-vulnerability

[26] P. Reddy and A. S. Gujral, "EchoLeak: The first real-world zero-click prompt injection exploit in a production LLM system," in Proc. AAAI Symp. Ser., vol. 7, no. 1, 2025, pp. 303-311, doi: 10.1609/aaaiss.v7i1.36899.

[27] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Min. V.1 (KDD '25), Toronto, ON, Canada, 2025, pp. 1809-1820, doi: 10.1145/3690624.3709179.

[28] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. 13th Int. Conf. Learn. Represent. (ICLR), 2025, arXiv:2410.02644.

[29] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in Adv. Neural Inf. Process. Syst. 38 (NeurIPS 2025), Datasets and Benchmarks Track, 2025, pp. 22436-22458, doi: 10.52202/085713-0666.

[30] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in Proc. AAAI Conf. Artif. Intell., vol. 40, no. 42, 2026, pp. 35811-35819, doi: 10.1609/aaai.v40i42.40895.

[31] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP Security Bench (MSB): Benchmarking attacks against Model Context Protocol in LLM agents," in Proc. 14th Int. Conf. Learn. Represent. (ICLR), 2026, arXiv:2510.15994.

[32] National Institute of Standards and Technology, Center for AI Standards and Innovation, "Technical blog: Strengthening AI agent hijacking evaluations," Jan. 17, 2025 (updated Dec. 19, 2025). Accessed: Oct. 11, 2026. [Online]. Available: https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations

[33] M. Dziemian et al., "How vulnerable are AI agents to indirect prompt injections? Insights from a large-scale public competition," arXiv:2603.15714, 2026. [Online]. Available: https://arxiv.org/abs/2603.15714

[34] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208

[35] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in Proc. 2025 ACM SIGSAC Conf. Computer and Communications Security (CCS '25), 2025, pp. 2833-2847, doi: 10.1145/3719027.3744836.

[36] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending against indirect prompt injection attacks with spotlighting," in Proc. Conf. Applied Machine Learning in Information Security (CAMLIS 2024), CEUR Workshop Proceedings, vol. 3920, Arlington, VA, USA, Oct. 2024, pp. 48-62. Accessed: Oct. 11, 2026. [Online]. Available: https://ceur-ws.org/Vol-3920/paper03.pdf

[37] Microsoft, "Prompt Shields in Azure AI Content Safety," Microsoft Learn, Aug. 28, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection

[38] Meta, "Llama Prompt Guard 2 86M," model card, Hugging Face. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M.

[39] ProtectAI.com, "Fine-tuned DeBERTa-v3-base for prompt injection detection (deberta-v3-base-prompt-injection-v2)," Hugging Face model card, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2.

[40] H. Li, X. Liu, N. Zhang, and C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), Vienna, Austria, Jul. 2025, pp. 30420-30437, doi: 10.18653/v1/2025.acl-long.1468.

[41] Check Point Software Technologies, "Prompt Defense," Check Point AI Security documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.lakera.ai/docs/prompt-defense

[42] Microsoft, "Region availability and service limits - Azure AI Content Safety," Microsoft Learn, Sep. 18, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/content-safety/region-availability

[43] NAMAA-Space, "Ara-Prompt-Guard_V0," Hugging Face model card, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/NAMAA-Space/Ara-Prompt-Guard_V0.

[44] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems," in Proc. 1st Workshop on LLM Security (LLMSEC), Vienna, Austria, Aug. 2025, pp. 101-114.

[45] M. Nasr et al., "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in Proc. 35th USENIX Security Symp. (USENIX Security 26), Baltimore, MD, USA, Aug. 2026, pp. 1467-1486.

[46] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proc. 2023 Conf. Empirical Methods in Natural Language Processing: System Demonstrations, Singapore, Dec. 2023, pp. 431-445, doi: 10.18653/v1/2023.emnlp-demo.40.

[47] NVIDIA, "Tools integration with the NeMo Guardrails library," NVIDIA NeMo Guardrails Library Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.nvidia.com/nemo/guardrails/latest/integration-with-third-party-libraries/tools-integration

[48] S. Chennabasappa et al., "LlamaFirewall: An open source guardrail system for building secure AI agents," arXiv:2505.03574, May 2025.

[49] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE '26), Apr. 2026, pp. 2938-2950, doi: 10.1145/3744916.3764546.

[50] T. Shi et al., "Progent: Securing AI agents with privilege control," arXiv:2504.11703, Apr. 2025 (v3, May 2026).

[51] M. Costa et al., "Securing AI agents with information-flow control," arXiv:2505.23643, May 2025.

[52] C. Xiang et al., "Architecting secure AI agents: Perspectives on system-level defenses against indirect prompt injection attacks," arXiv:2603.30016, Mar. 2026.

[53] O. Abdelnasser, F. Alharbi, K. Khasawneh, I. Alouani, and M. E. Fouda, "SalamahBench: Dialect and category level safety evaluation of Arabic language models," arXiv:2603.04410, 2026. [Online]. Available: https://arxiv.org/abs/2603.04410

[54] W. Zaghouani, M. R. Biswas, K. K. Aldous, and M. Bessghaier, "ArabicDialectSafety: A dialect-aware benchmark for Arabic content safety classification," in Findings Assoc. Comput. Linguistics: EMNLP 2026, to appear; arXiv:2608.01291, 2026. [Online]. Available: https://arxiv.org/abs/2608.01291

[55] F. Abed, H. Khan, M. S. Bari, B. Khan, and A. Abujabal, "Redteaming leading Arabic LLMs with ASAS," arXiv:2608.21985, Aug. 2026. [Online]. Available: https://arxiv.org/abs/2608.21985

[56] P. Theocharopoulos, A. Kulkarni, and M. Magimai.-Doss, "Multilingual hidden prompt injection attacks on LLM-based academic reviewing," arXiv:2512.23684, Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.23684

[57] A. M. Muhtadi and M. R. Tazwar, "MIPIAD: Multilingual indirect prompt injection attack defense with Qwen–TF-IDF hybrid and meta-ensemble learning," arXiv:2605.07269, May 2026. [Online]. Available: https://arxiv.org/abs/2605.07269

[58] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "The interplay of variant, size, and task type in Arabic pre-trained language models," in Proc. 6th Arabic Natural Lang. Process. Workshop (WANLP), Kyiv, Ukraine (Virtual), Apr. 2021, pp. 92–104.

[59] OWASP GenAI Security Project, "OWASP Top 10 for Agentic Applications for 2026," OWASP Foundation, Dec. 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[60] Saudi Data and Artificial Intelligence Authority (SDAIA), Implementing Regulation of the Personal Data Protection Law and Regulation on Personal Data Transfer outside the Kingdom, Riyadh, Saudi Arabia, Sep. 2023. Accessed: Oct. 11, 2026. [Online]. Available: https://dgp.sdaia.gov.sa/wps/portal/pdp/knowledgecenter

[61] Saudi Data and Artificial Intelligence Authority (SDAIA), Generative Artificial Intelligence Guidelines for Government, Doc. SDAIA-P116E, ver. 1, Riyadh, Saudi Arabia, May 2025 (English edition; first released Jan. 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://sdaia.gov.sa/en/SDAIA/about/Files/GenAIGuidelinesForGovernmentENCompressed.pdf

[62] D. E. Denning, "A lattice model of secure information flow," Commun. ACM, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

[63] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," IEEE J. Sel. Areas Commun., vol. 21, no. 1, pp. 5–19, Jan. 2003, doi: 10.1109/JSAC.2002.806121.

[64] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in Proc. 12th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, Feb. 2005.

[65] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

[66] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Inf. Syst. Secur., vol. 2, no. 2, pp. 159–176, May 1999, doi: 10.1145/317087.317089.

[67] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[68] E. Rissanen, Ed., "eXtensible Access Control Markup Language (XACML) Version 3.0," OASIS Standard, Jan. 22, 2013. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[69] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: John Wiley & Sons, 2014, ISBN: 978-1-118-80999-0.

[70] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in Proc. CHI Conf. Human Factors Comput. Syst. (CHI '19), Glasgow, U.K., 2019, pp. 1–13, doi: 10.1145/3290605.3300233.

[71] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Syst., Man, Cybern. A, Syst. Humans, vol. 30, no. 3, pp. 286–297, May 2000, doi: 10.1109/3468.844354.

[72] Check Point Software Technologies, "Policies," Check Point AI Security documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.lakera.ai/docs/policies

[73] Microsoft, "Connect agents to MCP server endpoints," Microsoft Learn, Aug. 26, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/model-context-protocol

[74] Amazon Web Services, "Policy in Amazon Bedrock AgentCore: Control agent interactions," Amazon Bedrock AgentCore Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy.html

[75] Amazon Web Services, "Temporal policies," Amazon Bedrock AgentCore Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-temporal.html

[76] Amazon Web Services, "Languages supported by Amazon Bedrock Guardrails," Amazon Bedrock User Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-supported-languages.html

[77] Amazon Web Services, "Validating CloudTrail log file integrity," AWS CloudTrail User Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html

[78] Google Cloud, "Model Armor overview," Google Cloud documentation, updated Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.cloud.google.com/model-armor/overview

[79] Protect AI, "llm-guard: The security toolkit for LLM interactions," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/protectai/llm-guard

[80] Invariant Labs, "invariant: Guardrails for secure and robust agent development," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/invariantlabs-ai/invariant

[81] Invariant Labs, "invariant-gateway," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/invariantlabs-ai/invariant-gateway

[82] Docker, "docker mcp gateway run," Docker Docs CLI reference. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/mcp/gateway/run/

[83] agentgateway, "MCP authorization," agentgateway documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://agentgateway.dev/docs/standalone/main/mcp/mcp-authz/

[84] IBM, "ContextForge (mcp-context-forge)," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/IBM/mcp-context-forge

[85] Google Research, "camel-prompt-injection: Code for the paper 'Defeating Prompt Injections by Design'," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/google-research/camel-prompt-injection

[86] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, Mar. 2004, doi: 10.2307/25148625.

[87] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Secur. Symp., Montreal, QC, Canada, Aug. 2009.

[88] Open Policy Agent, "Policy language," OPA Documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-language.

[89] F. Wu, E. Cecchetti, and C. Xiao, "System-level defense against indirect prompt injection attacks: An information flow control perspective," arXiv:2409.19091, Sep. 2024.

[90] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," J. Cryptol., vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.

[91] Microsoft, "Threats - Microsoft Threat Modeling Tool - Azure," Microsoft Learn. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats.

[92] A. Campbell, C. Adams, R. B. Montgomery, M. Cooper, and A. Kirkpatrick, Eds., "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023 (updated Dec. 12, 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

[93] M. Cáceres, D. Murphy, and C. Liebel, Eds., "Web Application Manifest," W3C Working Draft, Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/appmanifest/

[94] M. Chintala and Y. Yanagisawa, Eds., "Service Workers Nightly," W3C Candidate Recommendation Draft, Sep. 17, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/service-workers/

[95] B. Eidson and J. Simmons, "Web Push for Web Apps on iOS and iPadOS," WebKit Blog, Feb. 16, 2023. [Online]. Available: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/. Accessed: Oct. 11, 2026.

[96] MDN contributors, "Service Worker API," MDN Web Docs, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API. Accessed: Oct. 11, 2026.

[97] K. Schwaber and J. Sutherland, "The 2020 Scrum Guide," Nov. 2020. Accessed: Oct. 11, 2026. [Online]. Available: https://scrumguides.org/scrum-guide.html

[98] Software and Systems Engineering — Software Testing — Part 1: General Concepts, ISO/IEC/IEEE 29119-1:2022, Feb. 2022.

[99] A. Yang et al. (Qwen Team), "Qwen2.5 technical report," arXiv:2412.15115, Dec. 2024 (rev. Jan. 2025). [Online]. Available: https://arxiv.org/abs/2412.15115

[100] S. K. Card, T. P. Moran, and A. Newell, "The keystroke-level model for user performance time with interactive systems," Commun. ACM, vol. 23, no. 7, pp. 396–410, Jul. 1980, doi: 10.1145/358886.358895.

[101] Association for Computing Machinery, ACM Code of Ethics and Professional Conduct. New York, NY, USA: ACM, Jun. 2018.

[102] Ollama, "OpenAI compatibility," Ollama Docs. [Online]. Available: https://docs.ollama.com/api/openai-compatibility. Accessed: Oct. 11, 2026.

[103] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, Aug. 2016, doi: 10.17487/RFC7914.

[104] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015, doi: 10.17487/RFC7519.

[105] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in Findings Assoc. Comput. Linguistics: NAACL 2025, Albuquerque, NM, USA, Apr. 2025, pp. 7116-7132, doi: 10.18653/v1/2025.findings-naacl.395.

[106] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based model for Arabic language understanding," in Proc. 4th Workshop Open-Source Arabic Corpora Process. Tools (OSACT4), with a Shared Task on Offensive Language Detection, Marseille, France, May 2020, pp. 9–15.

[107] M. Abdul-Mageed, A. Elmadany, and E. M. B. Nagoudi, "ARBERT & MARBERT: Deep bidirectional transformers for Arabic," in Proc. 59th Annu. Meeting ACL and 11th IJCNLP, Vol. 1, Aug. 2021, pp. 7088-7105, doi: 10.18653/v1/2021.acl-long.551.

[108] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), Online, Jul. 2020, pp. 8440–8451, doi: 10.18653/v1/2020.acl-main.747.

[109] MITRE Corporation, "MITRE ATLAS: Adversarial Threat Landscape for AI Systems," knowledge base, release 2026.09, Sep. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://atlas.mitre.org

[110] OWASP GenAI Security Project, Agentic Security Initiative, "Agentic AI – Threats and Mitigations," ver. 1.1, OWASP Foundation, Dec. 2025 (ver. 1.0 first published Feb. 2025). Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/

[111] Information Technology - Artificial Intelligence - Management System, ISO/IEC 42001:2023, 1st ed., ISO/IEC JTC 1/SC 42, Dec. 2023.

[112] Information Security, Cybersecurity and Privacy Protection - Information Security Management Systems - Requirements, ISO/IEC 27001:2022, 3rd ed., ISO/IEC JTC 1/SC 27, Oct. 2022.

[113] Systems and Software Engineering — Life Cycle Processes — Requirements Engineering, ISO/IEC/IEEE 29148:2018, Nov. 2018.

[114] Qwen Team, "Qwen2.5-7B-Instruct," model card, Hugging Face. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct.

[115] Internet Security Research Group, "About Let's Encrypt," Let's Encrypt. [Online]. Available: https://letsencrypt.org/about/. Accessed: Oct. 11, 2026.

Appendix A – Standards & Constraints Matrix

This appendix records how Mirsad relates to the standards, regulations and guidelines that apply to an AI-agent security gateway deployed in Saudi Arabia (Table A.1), and how the project handled its technical, economic, time, legal, ethical, social, environmental and accessibility constraints (Table A.2).

Two points govern how the first table should be read. First, Mirsad supports an organisation's compliance but does not make it compliant: conformity with ISO/IEC 27001, ISO/IEC 42001, the NCA Essential Cybersecurity Controls or the PDPL is a property of the organisation's management system, processes and deployment, not of one software component. Second, the evidence column cites only artefacts that exist in the repository or in `reports/`, and lists as *gaps* the provisions that the current version does not yet cover. Those gaps feed the future work in Section 5.5.

A.1 Standards, Regulations and Guidelines

Table A.1: Mapping of standards, regulations and guidelines to Mirsad features and evidence

Standard / guideline	Relevant provisions	Mirsad features	Evidence and gaps
OWASP Top 10 for LLM Applications 2025 [5]	LLM01 Prompt Injection (direct and indirect, including multilingual and obfuscated input) LLM02 Sensitive Information Disclosure LLM06 Excessive Agency	Detector, sanitising and spotlighting of untrusted tool results (FR4) Taint rules for untrusted or confidential data reaching external destinations (FR3) Default-deny tool allow-lists, per-parameter constraints, rate limits and approval of high-risk tools (FR2, FR5)	Policy, taint and detector test suites; Mirsad-Bench benign runs (Chapter 4) Gap: protection against attacks not yet measured (attack data pending)
OWASP agentic guidance: Top 10 for Agentic Applications 2026 [59]; Agentic AI – Threats and Mitigations [110]	ASI01 Agent Goal Hijack ASI02 Tool Misuse and Exploitation ASI03 Identity and Privilege Abuse ASI09 Human-Agent Trust Exploitation	Taint escalation after untrusted reads and the after-injection rule for high-risk tools Per-agent tool profiles and argument constraints Per-agent API keys stored as SHA-256 hashes; agent keys cannot call human APIs Approval screens show the concrete arguments, the reasons and where the data came from, not the agent's own explanation	Security tests ST-04, ST-05, ST-21 and ST-22 Approval detail screen (Figure C.19)
NIST AI RMF 1.0 [18]	Govern, Map, Measure and Manage functions; "secure and resilient" and "accountable and	Versioned policies with author and comment (Govern) Threat model, use cases and misuse cases (Map) Mirsad-Bench, latency	`reports/results_summary.json` Policy versions in the dashboard (Figure C.6)

Standard / guideline	Relevant provisions	Mirsad features	Evidence and gaps
	transparent" characteristics	benchmark and detector metrics pipeline (Measure) Runtime enforcement and human approval (Manage)	
NIST AI 100-2e2025 [23]	Indirect prompt injection through resources the model reads; mitigations by detection, filtering, spotlighting and designing on the assumption that injection will occur; agent hijacking	The gateway constrains tool calls whatever the model has read; detector and spotlighting of untrusted content; taint rules on the data path	Threat model (Section 3.2); detector and gateway tests
MITRE ATLAS [109]	AML.T0051 LLM Prompt Injection (.001 Indirect) AML.T0053 AI Agent Tool Invocation AML.T0086 Exfiltration via AI Agent Tool Invocation	Misuse cases MC-01 and MC-02 (Table 3.1), mitigated by detection, policy, taint and approval	Gap: test cases and audit events are not yet tagged with ATLAS technique identifiers
ISO/IEC 42001:2023 [111]	AI management system: operational control, monitoring and human oversight of AI systems	Reviewable agent policies; recorded human oversight decisions (who decided, when, with which note); log of AI-system events	`policy.activated` and `approval.decided` audit events Mirsad supplies controls and evidence for an AI management system; it is not one
ISO/IEC 27001:2022 [112]	Annex A access control, logging and monitoring controls	Role-based access (administrator, approver, auditor); per-agent keys with rotation; tamper-evident log; live event stream; JSON and CSV export	Security tests ST-01 to ST-03, ST-06, ST-07 and ST-20
NCA ECC-2:2024 [11]	2-2-3 least privilege, need-to-know and segregation of duties 2-7 data protection by classification 2-12-3 event logging and monitoring 2-15 web application security 1-6-3 secure integration between applications 4-1 third-party cybersecurity	Per-tool and per-parameter authorisation; a human approver separate from the agent Sensitivity labels (public, internal, confidential) drive the taint rules Append-only HMAC-chained log with verification Security headers, request-size limit, login throttling Authenticated, policy-checked tool integration Third-party MCP servers reach an agent only through tools the policy allows (default-deny), and their results can be declared untrusted and scanned	Security tests ST-15, ST-18 and ST-19; Bandit, pip-audit and npm audit reports Gap: no configured retention period (ECC requires at least 12 months) and no SIEM connector

Standard / guideline	Relevant provisions	Mirsad features	Evidence and gaps
Saudi PDPL [12] and its Implementing Regulation [60]	Technical and organisational protection of personal data (Art. 19); breach notification (Art. 20, with 72 hours set in the Implementing Regulation); transfers outside the Kingdom (Art. 29); data minimisation	Taint rule for confidential data reaching external destinations Destination allow-lists (`domain_in`) on outbound tools Secrets removed and national IDs, IBANs and card numbers masked before audit events are stored Synthetic demonstration data only	Redaction unit tests and ST-11 The audit trail supports scoping an incident Gap: no erasure mechanism, because deleting events breaks the chain
SDAIA AI Ethics Principles [10]	Privacy and security; reliability and safety; transparency and explainability; accountability and responsibility; fairness	Reason codes with English and Arabic explanations; human approval of high-risk actions; decision replay; metrics reported separately for Arabic and English	Approval friction under full protection: 17.6% for Arabic and 17.6% for English tasks Gap: detector error rates per language and variety pending data
SDAIA Generative AI Guidelines for Government [61]	Human review of generative-AI output; restricted data kept out of generative-AI tools	Human approval of consequential actions; sensitivity labels and taint rules	Approval workflow tests
MCP specification, revision 2026-07-28 [3]	Clients should keep a human able to deny tool calls, show inputs before a call, validate results before they reach the model and log tool use; servers must validate inputs, enforce access control and rate-limit	Approval with the full arguments; result inspection before the model sees the result; audit of every call; argument constraints and rate limits at the proxy; least-privilege tool discovery (FR1.4)	Adapter and integration tests Gap: MCP resources, prompts and sampling are not proxied
WCAG 2.2 [92]	Level AA basics: keyboard operation, labels, contrast, no information by colour alone, reduced motion	Semantic HTML with labelled controls; risk shown as icon and text; light and dark themes; `prefers-reduced-motion`; right-to-left Arabic layout	194 web component tests; UI review checklist (NFR10) Gap: no automated accessibility audit and no testing with assistive technologies
ISO/IEC/IEEE 29148:2018 [113]	Well-formed, verifiable and traceable requirements	Requirement identifiers, MoSCoW priorities and acceptance criteria (Table 3.6, Table 3.7); use-case specifications; traceability to test cases	Section 3.2; D3 SRS
ISO/IEC/IEEE 29119 [98]	Test processes, test documentation and test design techniques	Test plan, test-case specification and results; unit, integration, API, security and UI test levels; tests configured to run in continuous integration	1,281 of 1,281 Python tests and 194 web tests pass; statement coverage 94.4% (Appendix B)
IETF RFCs and	Standard	HMAC-SHA-256 hash chain	Security tests ST-02, ST-03, ST-

Standard / guideline	Relevant provisions	Mirsad features	Evidence and gaps
API description standards: HMAC [67], JWT [104], scrypt [103], OpenAPI	cryptographic and interface building blocks instead of custom ones	for the audit log; HS256 JSON Web Tokens for human sessions; scrypt password hashing; Web Push with VAPID keys; OpenAPI description generated by FastAPI at `/openapi.json`	06 and ST-07; `docs/api.md`

The gaps in Table A.1 share a pattern. Mirsad already produces the evidence that the standards ask for (decisions, reasons, approvals, verifiable logs), but the surrounding operational features (retention and archival, SIEM forwarding, ATLAS tagging, erasure) are still to be built. These features are needed before an organisation could rely on Mirsad's logs in an NCA ECC or PDPL assessment.

A.2 Constraints Matrix

Table A.2: Project constraints and how they were handled

Constraint type	Constraint	How it was handled	Remaining effect
Technical	Personal laptops without GPUs; a young and fast-changing MCP specification; a latency budget of 200 ms per tool call	Local open-weight model (Qwen2.5-7B through Ollama) for the demonstration agent and the LLM benchmark runs CPU-only detector (weighted rules plus a linear n-gram model) Protocol handling isolated in an adapter layer built on the MCP Python SDK v2 Added latency measured in every evidence run	Measurements come from one laptop and a single server process; the transformer detector and multi-replica deployment are future work
Economic	Student budget set in the proposal	Free, open-source stack; local models instead of paid APIs for development and evaluation; a PWA instead of store apps, which avoids developer-account fees The proposal's 1,660 SAR budget was revised to 1,185 SAR, with price evidence and a zero-cost fallback, in the D2 work plan and budget	Paid model APIs and cloud hosting are needed only for an optional confirmatory run and a public demonstration
Time	Two semesters; five students, each owning one workstream	Contract-first design (`docs/dev/BUILD_SPEC.md`) so that workstreams could proceed in parallel Two-week sprints with MoSCoW priorities A scripted evidence pipeline (`scripts/run_evidence.sh`) that regenerates every result	Lower-priority items deferred to future work (Section 5.5)

Constraint type	Constraint	How it was handled	Remaining effect
Legal	PDPL, NCA ECC, software and dataset licences	Synthetic data only, with reserved `.example` domains Masking of personal identifiers in the audit log Permissive open-source dependencies; Mirsad licensed under Apache-2.0 Attacks are run only against the project's own simulated workspace	The detector dataset's datasheet (`data/detector/DATASHEET.md`) is still to be written by the data authors
Ethical	Dual-use attack material; honest reporting; use of an AI assistant	Attack material for the evaluation is to be authored and reviewed by team members only and is not reproduced in this report (Section 3.6.4); the demonstration scenarios contain only short illustrative instructions aimed at the simulated workspace Results that have not been measured are shown as pending, never estimated AI assistance disclosed (Section 3.6.6)	Detector F1 and attack success rate stay pending until the team adds its data
Social	Approvers' workload and alert fatigue; Arabic-speaking users and approvers	Escalation only on policy-defined high-risk paths Plain-language summaries in Arabic and English Persona-driven design (Section 1.5.1)	Approver workload and comprehension not yet measured with real users
Environmental and sustainability	Energy use of AI components; long-term maintainability of the software	No GPU training; the detector is designed to scan a text in milliseconds on a CPU (target below 10 ms for a 2 KB text, measured once a model is trained) Development inference ran on the team's existing laptops Documented, tested open-source code base that others can maintain	Energy consumption was not measured
Accessibility	Users with different abilities, languages and devices	WCAG 2.2 AA basics in both web applications Full right-to-left Arabic interface; dark theme; reduced motion Approver app designed to install from the browser on Android and iOS without an app store	No automated accessibility audit yet; Web Push delivery not yet tested on real phones

Appendix B – Test Reports & Validation Results

This appendix is the test case specification and test report (deliverable D5). Section 3.5 describes the test strategy. This appendix contains the evidence: the requirements traceability matrix (B.2), the specification of representative test cases in the style of ISO/IEC/IEEE 29119-3 [98] (B.3), the security test cases (B.4), the measured results (B.5) and the open items (B.6). Every number below is read from the result files listed in Table B.1 when the document is built. A value shown as "[pending]" has not been produced yet. Python test files are under `tests/<area>/` and web test files under `web/tests/`. A test name such as `test_policy_engine.py::test_unknown_agent_is_denied` identifies one automated test function.

B.1 Test environment and evidence

The recorded results were collected into `reports/results_summary.json` on 2026-10-14T07:58:56+00:00 (UTC). The tests, scans and latency benchmarks were run by `scripts/run_evidence.sh` (the separate-process latency run finished at 2026-10-14T06:51:13+00:00), on macOS-26.5.1-arm64-arm-64bit with Python 3.12.15. The Mirsad-Bench runs were executed from scratch by `scripts/run_bench.sh` and the LLM smoke command at 2026-10-14T07:31:54+00:00 (first audit check of the denying run), with the current default policy, in which CRM results are scanned. The injection detector ran in its rules-only fallback mode, because no trained model is packaged yet (detector mode recorded by the latency benchmark: rules_only). Every detector-dependent number in this appendix was therefore measured without the statistical model.

Table B.1: Evidence files behind this appendix

File	Content
`reports/tests/junit.xml`	Python test run (`pytest -m "not llm"`), one record per test with its outcome
`reports/tests/vitest.json`	Web test run (Vitest), one record per test case
`reports/coverage.json`	Statement coverage of the `mirsad` package (coverage.py)
`reports/security/bandit.json`, `pip-audit.json`, `npm-audit.json`	Static analysis and dependency scan reports
`reports/bench/latency.json`, `latency-process.json`	Latency benchmark with the server in the same process and in a separate process
`reports/bench/scripted-benign-deny/`, `scripted-benign-oracle/`, `llm-smoke/`	Mirsad-Bench runs: episode records, summaries, audit-verification and replay results, figures
`reports/results_summary.json`	All of the above collected by `scripts/collect_results.py`; the source of the placeholders in this report

B.2 Requirements traceability matrix

Table B.2 maps every functional requirement, non-functional requirement and business rule of Section 3.2 to the design component that implements it, the automated tests that verify it, and its verification status. The status values are:

- **Verified:** the acceptance criterion is checked by passing automated tests or by recorded measurements.
- **Partially verified:** part of the criterion is checked automatically. The reason names the part that is not.
- **Pending data:** the criterion needs attack-labelled data that the team is still preparing (Section 3.5.6).

TC and ST numbers refer to the test case specifications in B.3 and B.4.

Table B.2: Requirements traceability matrix

ID	Design component	Verifying tests	Status

FR 1.1	MCP proxy (`adapters/mcp_proxy.py`), upstream manager (`adapters/upstreams.py`)	`test_adapters_mcp_proxy.py::test_tools_are_filtered_by_policy_and_schemas_pass_through`
		`test_adapters_mcp_proxy.py::test_allowed_call_returns_inspected_content`
		`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
		`test_integration_real.py::test_mcp_streamable_http_endpoint_on_the_real_app`
		`test_adapters_upstreams.py::test_stdio_upstream_lists_calls_and_reconnects_after_crash`
			Verified (TC-09 to TC-11)

FR 1.2	OpenAI-compatible proxy (`adapters/openai_proxy.py`)	`test_adapters_openai_proxy.py::test_denied_tool_call_is_removed_and_explained`
		`test_adapters_openai_proxy.py::test_tool_messages_are_inspected_before_forwarding`
		`test_adapters_openai_proxy.py::test_partial_denial_keeps_allowed_calls`
		`test_adapters_openai_proxy.py::test_request_validation`
			Verified (TC-12, TC-13)

FR 1.3	Guard API (`api/routers/guard.py`), Python SDK (`sdk/client.py`)	`test_api_guard.py::test_tool_call_allow`
		`test_api_guard.py::test_tool_call_deny`
		`test_api_guard.py::test_invoke_denied_tool_is_never_executed`
		`test_adapters_sdk.py::test_guard_denied_call_does_not_run`
		`test_adapters_sdk.py::test_guard_blocked_result_raises`
			Verified (TC-14, TC-15)

FR 1.4	`PolicyEngine.visible_tools`; tool listing in the MCP proxy, OpenAI proxy and `GET /v1/tools`	`test_api_guard.py::test_visible_tools_are_filtered_by_policy`
		`test_adapters_openai_proxy.py::test_tool_definitions_are_filtered_by_policy`
		`test_adapters_mcp_proxy.py::test_visible_tools_failure_exposes_nothing`
		`test_policy_engine.py::test_unknown_agent_profile_and_visible_tools`
			Verified (TC-09)

FR 2.1	Policy schema and validator (`policy/schema.py`), engine	`test_policy_schema.py::test_every_shipped_policy_validates` `test_policy_schema.py::test_yaml_syntax_error_has_line_number`	Verified (TC-01, TC-02, TC-16)

ID	Design component	Verifying tests	Status

	(`policy/engine.py`), monitor mode in `gateway.py`	`test_policy_schema.py::test_unknown_key_reports_line_and_suggestion`
		`test_policy_engine.py::test_tool_resolution_precedence`
		`test_gateway.py::test_monitor_mode_allows_but_audits_would_be_decision`	

FR 2.2	Constraint evaluator (`policy/constraints.py`)	`test_policy_constraints.py::test_operator_passes_and_violates` (19 operator cases)
		`test_policy_constraints.py::test_on_violation_require_approval_is_propagated`
		`test_policy_constraints.py::test_domain_in_accepts_subdomains_but_not_lookalikes`
		`test_policy_constraints.py::test_not_pattern_sees_through_fullwidth_and_zero_width_obfuscation`
			Verified (TC-03, TC-04)

FR 2.3	Sliding-window rate limiter (`policy/ratelimit.py`)	`test_policy_engine.py::test_rate_limit_window_on_export_customers`
		`test_policy_ratelimit.py::test_hit_counts_calls_inside_the_window`
		`test_policy_ratelimit.py::test_window_slides_and_old_calls_expire`
		`test_policy_ratelimit.py::test_concurrent_hits_are_all_counted`
			Verified (TC-05)

FR 2.4	Deny-overrides combination (`combine_reasons` in `models.py`), bilingual reason messages	`test_policy_engine.py::test_constraint_violations_combine_with_deny_overrides`
		`test_policy_constraints.py::test_arabic_text_in_messages_is_present_for_every_operator`
		`test_gateway.py::test_explain_decision_is_bilingual`
		`test_taint_rules.py::test_reasons_are_bilingual_serialisable_and_name_the_tool`
			Verified (TC-06)

FR 2.5	Policy store and activation (`api/routers/policies.py`, `Gateway.set_policy`), `mirsad policy activate`	`test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`
		`test_api_admin.py::test_invalid_policy_is_rejected_and_not_activated`
		`test_gateway.py::test_policy_hot_swap_applies_to_next_call`
		`test_cli.py::test_policy_activate_persists_and_audits`
		`ui.test.tsx`: "rolls back to an older version after confirmation"
			Verified (TC-40)

FR 3.1	Tool profiles (`PolicyEngine.tool_profile`), `Gateway.inspect_result`	`test_e2e_http.py::test_agent_tool_calls_follow_the_default_policy`
		`test_gateway.py::test_benign_untrusted_result_passes_with_spotlight`
		`test_gateway.py::test_result_of_disallowed_tool_is_treated_as_untrusted`
		`test_taint_tracker.py::test_register_result_updates_sources_sensitivity_injection_and_values`
			Partially verified: trust and sensitivity labels on returned results are asserted; the inspection audit event stores the full tool profile (`gateway.py`), but no test asserts that field

FR 3.2	Value extraction (`taint/extract.py`), tracker (`taint/tracker.py`)	`test_taint_extract.py::test_structured_results_keep_field_paths_and_trusted_flags`
		`test_taint_extract.py::test_argument_candidates_find_values_embedded_in_long_and_encoded_strings`
		`test_taint_extract.py::test_iban_spacing_and_case_normalised_and_lookalikes_rejected`
			Verified (TC-23)

ID	Design component	Verifying tests	Status
		`test_taint_integration.py::test_structured_result_plus_free_text_content_records_both`	

FR 3.3	Taint rules (`taint/rules.py`) combined in `Gateway.check_call`	`test_taint_rules.py::test_untrusted_content_then_external_sink_requires_approval`
		`test_taint_rules.py::test_confidential_crm_phone_inside_external_email_body_is_flagged`
		`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
		`test_taint_rules.py::test_replying_to_the_sender_is_not_flagged_as_untrusted_destination`
		`test_taint_rules.py::test_crm_email_is_not_an_untrusted_destination_even_if_also_in_untrusted_text`
		`test_api_guard.py::test_untrusted_result_then_external_call_needs_approval`
			Verified (TC-18 to TC-21)

FR 3.4	`TAINT_AFTER_INJECTION` rule; per-session state in `taint/tracker.py`	`test_taint_rules.py::test_high_risk_tool_after_injection_requires_approval`
		`test_taint_rules.py::test_low_risk_tool_after_injection_is_not_flagged`
		`test_taint_rules.py::test_high_risk_tool_without_injection_is_not_flagged`
		`test_taint_tracker.py::test_sessions_are_isolated_by_session_and_agent`
		`test_gateway.py::test_injection_is_sanitized_and_taints_session`
			Verified (TC-22, TC-29)

FR 4.1	Detector (`detector/normalize.py`, `rules.py`, `model.py`, `detector.py`, `language.py`); training and evaluation (`train.py`, `build_dataset.py`)	`test_detector.py::test_result_fields_and_latency`
		`test_detector.py::test_spans_point_into_the_original_text`
		`test_detector.py::test_benign_arabic_business_text_is_below_flag_threshold`
		all tests in `test_language.py`
		`test_train.py::test_metrics_report_structure`
		`test_train.py::test_tuning_never_looks_at_test_splits`
			Pending data: score, label, language and spans are verified; F1 ≥ 0.85 needs the team's attack-labelled test sets (B.5.8)

FR 4.2	Normaliser and evasion signals (`detector/normalize.py`, `detector/rules.py`)	`test_normalize.py::test_zero_width_characters_are_removed_and_signalled`
		`test_normalize.py::test_cyrillic_lookalikes_inside_latin_word_are_folded`
		`test_normalize.py::test_stacked_diacritics_raise_tashkeel_signal`
		`test_normalize.py::test_bidi_controls_are_removed_and_signalled`
		`test_detector.py::test_spans_survive_invisible_characters`
		`test_rules.py::test_evasion_signals_amplify_only_real_hits`
			Partially verified: normalisation and obfuscation signals are verified; the detection rate on obfuscated attack texts needs team-supplied data

FR 4.3	Sanitiser and spotlighting (`detector/sanitize.py`), `Gateway.inspect_result`	`test_sanitize.py::test_exact_rules_remove_only_the_span`
		`test_sanitize.py::test_block_mode_withholds_everything_with_bilingual_notice`
		`test_sanitize.py::test_spotlight_structure`
		`test_sanitize.py::test_positive_detection_without_spans_fails_closed`
		`test_gateway.py::test_suspicious_result_is_flagged_not_modified`
			Verified with neutral placeholder content (TC-27 to TC-29)

ID	Design component	Verifying tests	Status
		test_api_guard.py::test_invoke_sanitizes_poisoned_upstream_output	
FR 5.1	Approval workflow (approvals/service.py), Gateway.check_call	test_e2e_http.py::test_external_email_waits_for_an_approval_sent_from_a_second_process test_gateway.py::test_approval_granted_by_human_allows_and_excludes_wait_from_latency test_policy_engine.py::test_approval_always_tool test_api_guard.py::test_tool_call_approved_by_human_via_api	Verified (TC-31)
FR 5.2	Approver summaries (approvals/summary.py), notifier with SSE and Web Push (approvals/notifier.py), approver PWA	test_approval_summary.py::test_english_summary_matches_the_reference_example test_approval_summary.py::test_arabic_summary_is_arabic_and_mentions_tool_and_destination test_approval_summary.py::test_every_reason_code_yields_a_short_summary test_approval_service.py::test_request_triggers_web_push_and_prunes_stale_subscriptions test_api_stream.py::test_sse_relays_approval_lifecycle_and_audit_events contract.test.tsx: "yields the \"data came from\" source and the reasons that paused the call"	Verified (TC-32, TC-33)
FR 5.3	ApprovalService decide, wait and expiry; approver PWA (web/src/approver/)	test_approval_service.py::test_wait_timeout_marks_expired_and_publishes test_approval_service.py::test_decide_twice_raises_value_error test_approval_service.py::test_concurrent_deciders_exactly_one_wins test_gateway.py::test_approval_timeout_denies_as_expired ui.test.tsx: "lists the pending request, explains it and approves it after confirmation"	Verified (TC-34, TC-35, TC-42, ST-09)
FR 6.1	Hash-chained audit log (audit/log.py), redaction (audit/redact.py)	test_audit_chain.py::test_verify_detects_modified_payload test_audit_chain.py::test_verify_detects_deleted_row test_audit_chain.py::test_verify_detects_reordered_rows test_audit_chain.py::test_verify_detects_forged_rechain_without_key test_audit_chain.py::test_payload_is_redacted_before_storage	Verified (TC-36, TC-37, ST-06, ST-07)
FR 6.2	AuditLog.query and export, audit router (api/routers/audit.py), dashboard Activity page	test_audit_query.py::test_query_filters_by_each_column test_audit_query.py::test_query_time_range test_audit_query.py::test_query_text_search_payload_tool_agent_case_insensitive test_audit_query.py::test_query_pagination_and_ordering test_audit_query.py::test_export_csv_rows_and_header test_api_admin.py::test_audit_list_filter_and_detail test_api_admin.py::test_audit_export_json_and_csv	Verified (TC-38)
FR 6.3	Gateway.replay, taint snapshots, versioned policy store	test_gateway.py::test_replay_decision_matches_original test_gateway.py::test_replay_uses_the_policy_version_recorded_in_the_event test_gateway.py::test_replay_reproduces_taint_from_stored_snapshot test_api_admin.py::test_replay_after_policy_change_uses_recorded_version test_e2e_http.py::test_audit_verify_replay_and_csv_exp	Verified (TC-39)

ID	Design component	Verifying tests	Status
		`ort` benchmark replay sweep (Table B.17)	

FR 7.1	Authentication core (`auth.py`), agent, user and tool routers	`test_api_admin.py::test_agent_crud_shows_key_once`
		`test_api_admin.py::test_user_crud_never_returns_hashes`
		`test_api_admin.py::test_last_admin_is_protected`
		`test_api_admin.py::test_tools_listing_shows_profiles_per_agent`
		`test_auth_core.py::test_rotate_agent_key_invalidates_old_key`
		`ui.test.tsx`: "creates a user with a server-generated password and shows it once"
			Verified (TC-41, ST-04, ST-12)

FR 7.2	Statistics and stream routers (`api/routers/stats.py`, `stream.py`), detector playground, dashboard Overview	`test_api_admin.py::test_stats_overview_shape_and_counts`
		`test_api_admin.py::test_stats_timeseries_buckets`
		`test_api_stream.py::test_sse_relays_approval_lifecycle_and_audit_events`
		`test_api_guard.py::test_detector_playground`
		`sse.test.ts` (SSE parser and `connectStream`)
			Partially verified: statistics, the SSE stream and the browser SSE client are tested; no automated UI test shows the Overview page updating on a live event

FR 8.1	Simulated workspace (`demo/world.py`), MCP workspace server, demo agent and scenarios, `/api/demo/*`	`test_world.py::test_data_is_fake` and the other tests in `test_world.py`
		`test_server.py::test_list_tools`
		`test_scenarios.py::test_apply_scenario_plants_payload`
		`test_api_demo.py::test_unprotected_agent_leaks_to_attacker_inbox`
		`test_api_demo.py::test_protected_agent_is_stopped_by_the_gateway`
			Verified with a scripted model stub (TC-43); the live-model test (`tests/demo/test_llm.py`) is `llm`-marked and not in the recorded run

FR 8.2	Mirsad-Bench (`bench/`)	`test_bench_loader.py::test_suite_has_enough_benign_tasks_in_both_languages`
		`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`
		`test_bench_metrics.py::test_fpr_friction_and_utility`
		`test_bench_metrics.py::test_attack_success_rate_math`
		`test_bench_run.py::test_runner_writes_outputs_and_resumes`
		`test_bench_attacks.py::test_policy_denial_stops_a_large_export_under_full_defense`
		`test_bench_attacks.py::test_approval_and_taint_attribution`
		`test_bench_attacks.py::test_runner_auto_mode_runs_both_kinds_and_writes_the_asr_figure`
			Partially verified (TC-44, TC-46): `scripts/run_bench.sh` regenerates the benign result tables (B.5.5) and executes attack episodes when attack files exist; the attack tables need team-supplied attack files

NF R1	Gateway pipeline; latency benchmark (`bench/latency.py`)	`test_bench_run.py::test_latency_benchmark_small` latency benchmark over 1,000 calls (Table B.15)	Verified

ID	Design component	Verifying tests	Status

NF R2	Protocol adapters; demo agent over the OpenAI-compatible API to a local model	`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
		`test_integration_real.py::test_mcp_streamable_http_endpoint_on_the_real_app`
		`test_adapters_openai_proxy.py::test_allowed_tool_calls_pass_through_unchanged`
		LLM smoke run with Qwen2.5-7B (B.5.7)
			Verified

NF R3	`auth.py`, HTTP hardening (`api/security.py`), redaction, regex guard (`policy/regex_guard.py`)	Security suite ST-01 to ST-23 (B.4)
		all tests in `test_auth_core.py` and `test_redaction.py`
		`test_api_http.py::test_security_headers_on_every_response`
		`test_api_http.py::test_request_body_over_1mb_is_rejected`
		Bandit, pip-audit and npm audit (Table B.14)
			Verified

NF R4	Approver PWA (`web/src/approver/`), approver summaries	`ui.test.tsx`: approver PWA tests all tests in `i18n.test.ts` `test_approval_summary.py::test_every_reason_code_yields_a_short_summary`	Partially verified: Arabic right-to-left rendering and the approve flow are tested; the 30-second decision time rests on an analytical Keystroke-Level Model estimate (Section 4.2.8), and no approver has been timed yet

NF R5	`Dockerfile`, `docker-compose.yml`, `mirsad init` and `mirsad serve`, `scripts/run_demo.sh`	`test_e2e_http.py::test_init_demo_then_login_and_overview`
		`test_cli.py::test_init_demo_creates_agent_and_honours_admin_password`
		CI `docker` job (defined; no run recorded)
			Partially verified: the local start with `init` and `serve` is tested end to end; a `docker compose up` run is not recorded in `reports/`

NF R6	Fail-closed paths in `gateway.py` and the adapters; approval expiry	`test_gateway.py::test_approval_timeout_denies_as_expired`
		`test_gateway.py::test_execute_executor_exception_returns_safe_error_and_is_audited`
		`test_gateway.py::test_execute_times_out_slow_tools`
		`test_gateway.py::test_audit_failure_fails_closed`
		`test_gateway.py::test_policy_exception_fails_closed`
		`test_gateway.py::test_detector_failure_blocks_untrusted_result`
		`test_adapters_openai_proxy.py::test_gateway_failures_fail_closed`
			Verified (TC-15, TC-17, TC-34, ST-13)

NF R7	Audit event schema	`test_gateway.py::test_principal_is_recorded_in_audit_payload`	Verified (ST-09)

ID	Design component	Verifying tests	Status
	(`gateway.py`, `audit/log.py`)	`test_gateway.py::test_allow_low_risk_call_is_audited_with_replay_payload` `test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`	

NFR8	Redaction (`audit/redact.py`); synthetic demo data	`test_redaction.py::test_saudi_national_id_and_iqama_masked_to_last_four`
		`test_redaction.py::test_iban_masked_except_country_and_last_four`
		`test_redaction.py::test_luhn_valid_card_masked_to_last_four`
		`test_world.py::test_data_is_fake`
			Verified (ST-11)

NFR9	Whole code base	Statement coverage (Table B.13)
		Ruff (Table B.14)
		`test_e2e_http.py::test_openapi_document_covers_the_spec_routes`
		`test_policy_docs.py::test_documentation_exists_and_covers_every_operator`
			Verified

NFR10	Web UI (`web/src/`), design tokens	all tests in `contrast.test.ts` (53 colour pairs in both themes: text ≥ 4.5:1, focus ring and UI components ≥ 3:1) all tests in `i18n.test.ts` `ui.test.tsx` (controls located by accessible role and name)	Partially verified: contrast and accessible names are tested automatically; keyboard navigation, no colour-only signals and reduced motion depend on the UI review checklist, which is not recorded in `reports/`

NFR11	Single-node gateway; storage on SQLite and PostgreSQL (`storage/database.py`)	latency benchmark with 50 concurrent sessions (Table B.15)
		all tests in `test_postgres.py`
		`test_storage_db.py::test_schema_compiles_for_postgresql`
		`test_gateway.py::test_concurrent_calls_keep_a_consistent_audit_chain`
			Verified (TC-45); tail latency under concurrency is high (B.5.4)

BR1	Default deny in `PolicyEngine`	`test_policy_engine.py::test_unknown_agent_is_denied`
		`test_policy_engine.py::test_unmatched_tool_with_default_deny`
		`test_gateway.py::test_unlisted_tool_is_denied`
			Verified (TC-08, ST-22)

BR2	`approval: always` for `transfer_funds` and `delete_customer` in the default policy	`test_policy_engine.py::test_approval_always_tool`
		`test_e2e_http.py::test_agent_tool_calls_follow_the_default_policy`
		`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`
			Verified (TC-06, TC-44)

BR	`count` and `limit`	`test_policy_engine.py::test_default_bulk_export_rule` `test_policy_engine.py::test_default_rule_regex_is_line`	Verified (TC-

ID	Design component	Verifying tests	Status
3	constraints; rule `no-bulk-export-to-email`	`ar_on_adversarial_bodies` `test_e2e_http.py::test_agent_tool_calls_follow_the_default_policy`	07)

BR 4	`domain_in` constraint on `send_email`; `TAINT_UNTRUSTED_DESTINATION`	`test_policy_engine.py::test_external_recipient_needs_approval_by_constraint`
		`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
		`test_policy_scenarios.py::test_scenario_injected_email_exfiltration`
			Verified (TC-04, TC-20)

BR 5	`defaults.approval_timeout_s`; expiry in `ApprovalService`	`test_approval_service.py::test_wait_timeout_marks_expired_and_publishes`
		`test_approval_service.py::test_record_deadline_expires_without_explicit_timeout`
		`test_gateway.py::test_approval_timeout_denies_as_expired`
			Verified (TC-34); tests use short timeouts, the 120 s default is configuration

BR 6	Audit router exposes only list, get, verify, replay and export	Route definitions in `api/routers/audit.py` ST-06 and ST-07 (direct database edits are detected)	Partially verified: no write route exists, but no test asserts their absence

Table B.3: Verification status summary

Requirement group	Verified	Partially verified	Pending data	Total
Functional requirements (FR1.1–FR8.2)	21	4	1	26
Non-functional requirements (NFR1–NFR11)	8	3	0	11
Business rules (BR1–BR6)	5	1	0	6
Total	**34**	**8**	**1**	**43**

B.3 Test case specification

The 46 test cases below are a representative selection from the automated suites, covering every area and every requirement group. Each one is specified with the fields of an ISO/IEC/IEEE 29119-3 test case: identifier, requirement, objective, preconditions and inputs, expected result, actual result and the automated procedure that executes it. All test cases are automated. The actual result is taken from the recorded run (`reports/tests/junit.xml` and `reports/tests/vitest.json`), in which every test passed. Where a test needs content that the detector scores as an injection, it uses a neutral placeholder rather than real attack text (Section 3.5.1).

B.3.1 Policy language and engine

Table B.4: Test cases TC-01 to TC-08, policy language and engine

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-01	FR 2.1	Every shipped policy is valid	Each YAML file in `configs/policies/`	`validate_policy` returns no errors; each policy defines at least one agent	Pass	`test_policy_schema.py::test_every_shipped_policy_validates`
TC-02	FR 2.1	Validation errors are located and actionable	Policy with a misspelt tool key (`sinkk`)	Error starts with "line 8, column 44" and suggests `sink`	Pass	`test_policy_schema.py::test_unknown_key_reports_line_and_suggestion`
TC-03	FR 2.2	Every constraint operator accepts a valid value and rejects an invalid one	19 operator cases, each with a passing and a violating value (e.g. `max: 10` with 10 and 11)	Passing value: no reason. Violating value: one `CONSTRAINT_VIOLATION` (deny) with evidence `param`, `operator`, `expected`, `actual` and different English and Arabic messages	Pass	`test_policy_constraints.py::test_operator_passes_and_violates`
TC-04	FR 2.2, BR 4	`domain_in` is not fooled by look-alike domains	`domain_in: [najd-trading.example]`; an address at a subdomain and addresses at two look-alike domains	Subdomain accepted; both look-alikes violate `domain_in`	Pass	`test_policy_constraints.py::test_domain_in_accepts_subdomains_but_not_lookalikes`
TC-05	FR 2.3	Sliding-window rate limit	`export_customers` limited to 3 calls per 3,600 s; calls at t =	Calls 1–3 allowed; call 4 denied with `RATE_LIMIT` (count 4, max 3); allowed	Pass	`test_policy_engine.py::test_rate_limit_window_on_export_customers`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			0, 1, 2, 3 s and at t = 3,603 s	again once the window has slid		
TC-06	FR 2.4, BR 2	Deny overrides require-approval	`transfer_funds` with `amount_sar` 20,000 (limit 10,000; tool always needs approval)	Reasons `TOOL_RISK`, `CONSTRAINT_VIOLATION`, `APPROVAL_ALWAYS`; combined action DENY	Pass	`test_policy_engine.py::test_constraint_violations_combine_with_deny_overrides`
TC-07	BR 3	Bulk-export rule at its boundary	`send_email` whose body lists 51 addresses, then 50	51: `RULE_MATCH` by `no-bulk-export-to-email`, deny. 50: rule does not fire	Pass	`test_policy_engine.py::test_default_bulk_export_rule`
TC-08	BR 1	Unknown agents are denied	Call from an agent id that is not in the policy	Single reason `UNKNOWN_AGENT`, action DENY; English and Arabic messages name the agent	Pass	`test_policy_engine.py::test_unknown_agent_is_denied`

B.3.2 Gateway and protocol adapters

Table B.5: Test cases TC-09 to TC-17, gateway and protocol adapters

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-09	FR 1.1, FR 1.4	MCP clients see only permitted tools, with unchanged schemas	MCP client connected to the proxy; the policy makes two of the upstream tools visible	`tools/list` returns exactly those two tools; input schemas equal the upstream schemas	Pass	`test_adapters_mcp_proxy.py::test_tools_are_filtered_by_policy_and_schemas_pass_through`
TC	FR 1.1	A denie	Gateway denies	MCP error	Pass	`test_adapters_mcp_proxy.py::test_denied_call_is_not_executed_and_explained`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
-10		d MCP call is not executed and is explained	`send_email` to an outside address	result; the upstream tool never runs; the text names the tool, the reason code and the data source in English and Arabic		
TC-11	FR 1.1, NFR2	The stdio proxy works as a separate process	An MCP client launches `run_stdio_proxy` as a subprocess that forwards to a live HTTP gateway	Tool list filtered; allowed call returns sanitised content; denied call returns an error; only the allowed tool runs upstream	Pass	`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
TC-12	FR 1.2	Denied tool calls are removed from the model's reply	The upstream model returns a `send_email` tool call that the gateway denies	`tool_calls` removed; `finish_reason` becomes `stop`; appended assistant text explains the block in English and Arabic	Pass	`test_adapters_openai_proxy.py::test_denied_tool_call_is_removed_and_explained`
TC-13	FR 1.2	Tool messages are inspected before they reach	Conversation whose `tool` message carries flagged content	Forwarded message contains the sanitise marker instead of the flagged	Pass	`test_adapters_openai_proxy.py::test_tool_messages_are_inspected_before_forwarding`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		the model		text; tool name resolved from the preceding `tool_calls`		
TC-14	FR 1.3	The SDK decorator blocks denied calls	Function decorated with `client.guard`; the gateway denies the call	`MirsadBlocked` raised at stage `call`; the function body does not run	Pass	`test_adapters_sdk.py::test_guard_denied_call_does_not_run`
TC-15	FR 1.3, NFR6	The SDK fails closed	Gateway unreachable (connection refused); guarded `delete_customer`	`MirsadError` raised; the guarded function does not run	Pass	`test_adapters_sdk.py::test_guard_fails_closed_when_gateway_unreachable`
TC-16	FR 2.1	Monitor mode records but does not enforce	Policy in `mode: monitor`; a call to a disallowed tool, then to a tool that always needs approval	Both calls return ALLOW; the audit event stores the would-be action `deny` with `enforced: false`; no approval is requested	Pass	`test_gateway.py::test_monitor_mode_allows_but_audits_would_be_decision`
TC-17	NFR6	Tool failures neither crash the gateway nor leak internals	Executor raises an exception whose message contains an internal file path	Generic error returned to the agent without the path; `tool_call.error` audit event stored	Pass	`test_gateway.py::test_execute_executor_exception_returns_safe_error_and_is_audited`

B.3.3 Taint tracking

Table B.6: Test cases TC-18 to TC-23, taint tracking

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-18	FR 3.3	Untrusted content followed by an external sink is escalated	Session reads an e-mail (untrusted), then sends a message to an outside partner	`TAINT_UNTRUSTED_TO_EXTERNAL`, REQUIRE_APPROVAL, risk 0.6; evidence names the source tool and call id	Pass	`test_taint_rules.py::test_untrusted_content_then_external_sink_requires_approval`
TC-19	FR 3.3	A confidential value inside free text is caught	Session reads a CRM customer, then e-mails an outside address with the customer's phone number inside the body	`TAINT_CONFIDENTIAL_TO_EXTERNAL` (kind phone, parameter `body`, field `phone`, source `get_customer`), REQUIRE_APPROVAL	Pass	`test_taint_rules.py::test_confidential_crm_phone_inside_external_email_body_is_flagged`
TC-20	FR 3.3, BR 4	A destination chosen by untrusted content is denied	Inbox listing whose message body contains an outside address; the agent then sends to that address	`TAINT_UNTRUSTED_DESTINATION`, DENY, risk 0.9, field `[0].body`; English and Arabic messages name the domain	Pass	`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
TC-21	FR 3.3	A reply to the sender is	Session reads an e-mail and replies	No `TAINT_UNTRUSTED_DESTINATION`; `TAINT_UNTRUSTED_TO_EXTERNAL` still present	Pass	`test_taint_rules.py::test_replying_to_the_sender_is_not_flagged_as_untrusted_destination`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		not an untrusted destination	to its sender (trusted `from` field)	because the sink is external		
TC-22	FR 3.4	High-risk tools need approval after a detection	A session result was labelled `injection`; the agent then calls `delete_customer` (critical)	`TAINT_AFTER_INJECTION`, REQUIRE_APPROVAL, risk 0.8; the source call id is recorded	Pass	`test_taint_rules.py::test_high_risk_tool_after_injection_requires_approval`
TC-23	FR 3.2	Arabic text matching ignores diacritics	Arabic CRM note recorded from a confidential result; an outgoing e-mail body copies the note with different diacritics	`TAINT_CONFIDENTIAL_TO_EXTERNAL` with kind `ngram`, field `notes` and readable Arabic evidence	Pass	`test_taint_rules.py::test_arabic_confidential_note_copied_with_different_diacritics_is_flagged`

B.3.4 Detector and result handling

Table B.7: Test cases TC-24 to TC-30, detector and result handling

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-24	FR 4.2	Zero-width characters are	Ordinary words with a zero-	Normalised text without them; `zero_width_`	Pass	`test_normalize.py::test_zero_width_characters_are_removed_and_signalled`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		removed and reported	width space, joiner or word joiner inserted	`chars` signal; every offset maps back to the original character		
TC-25	FR 4.2	Homoglyphs are folded	Latin words containing Cyrillic look-alike letters	Folded to Latin; `homoglyphs` and `mixed_scripts` signals raised	Pass	`test_normalize.py::test_cyrillic_lookalikes_inside_latin_word_are_folded`
TC-26	FR 4.1	Benign Arabic business text is not flagged	Neutral Arabic business sentences; rules-only detector	Score below 0.5, label `benign`, language `ar`	Pass	`test_detector.py::test_benign_arabic_business_text_is_below_flag_threshold`
TC-27	FR 4.3	Sanitising removes only the flagged span	Sentence containing a neutral placeholder phrase matched by a rule	Placeholder replaced by the sanitise marker; the rest of the text is unchanged	Pass	`test_sanitize.py::test_exact_rules_remove_only_the_span`
TC-28	FR 4.3	Untrusted content is spotlighted	Untrusted text from `read_email` with a call id	Header and footer carry the source and call id; English and Arabic notes follow the header; the body is unchanged inside	Pass	`test_sanitize.py::test_spotlight_structure`
TC-29	FR 4.3, FR 3.4	The gateway sanitises a flagged result and marks the session	Untrusted result containing a neutral test marker that the stub rule scores as an	Action `sanitize`; marker removed; reason `INJECTION_DETECTED`; session snapshot records the injection;	Pass	`test_gateway.py::test_injection_is_sanitized_and_taints_session`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			injection	audit action `sanitize`		
TC-30	FR 4.1	Test data cannot influence training or tuning	Train twice; before the second run, replace each test split by half of its records with flipped labels	Identical weights and ensemble configuration; only the test metrics differ	Pass	`test_train.py::test_tuning_never_looks_at_test_splits`

B.3.5 Approvals and audit

Table B.8: Test cases TC-31 to TC-39, approvals and audit

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-31	FR 5.1	A call waits for a human who decides from another process	Server prepared with `mirsad init --demo`; the agent e-mails an outside supplier; an approver approves from a separate Python process	The agent's HTTP call blocks while the request is pending and nothing is sent; afterwards ALLOW with `APPROVED_BY_HUMAN`, the e-mail is sent, `latency_ms` excludes the wait, one `approval.decided` event	Pass	`test_e2e_http.py::test_external_email_waits_for_an_approval_sent_from_a_second_process`
TC-32	FR 5.2	The English summary	Approval for `send_email` to an	Three sentences: the action and destination	Pass	`test_approval_summary.py::test_english_summary_matches_the_reference_example`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		says what, why and where from	outside address that came from an untrusted e-mail	(outside Najd Trading Co.), why Mirsad paused it with the source e-mail, and the risk level		
TC-33	FR 5.2	Argument values cannot add sentences to the summary	A `to` argument containing line breaks, text imitating a system instruction and a 300-character run	Summary contains no line break; long values are clipped	Pass	`test_approval_summary.py::test_attacker_controlled_values_cannot_inject_sentences`
TC-34	FR 5.3, BR 5	Undecided requests expire and stay closed	Pending request; wait with a 0.2 s timeout; a late approval attempt	Status EXPIRED; `approval.decided` published with status `expired`; the late decision is rejected	Pass	`test_approval_service.py::test_wait_timeout_marks_expired_and_publishes`
TC-35	FR 5.3	Exactly one decision wins	Six concurrent decisions on one request	One decision succeeds; the other five raise `ValueError`	Pass	`test_approval_service.py::test_concurrent_deciders_exactly_one_wins`
TC-36	FR 6.1	Reordered audit rows are detected	Five events; the sequence numbers of events 2 and 3 are	Verification fails with first bad sequence 2	Pass	`test_audit_chain.py::test_verify_detects_reordered_rows`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			swapped in the database			
TC-37	FR 6.1	A rewrite by the key holder is detected with an external anchor	Five events; anchor saved; event 2 rewritten and the chain recomputed with the real key	Verification without the anchor passes (documented limit); with the anchor it fails at sequence 5	Pass	`test_audit_chain.py::test_key_holder_rewrite_passes_chain_but_fails_external_anchor`
TC-38	FR 6.2	Audit filters return the right events	Populated log; filters by agent, tool, action, event type, session and call id, then a combination	Each filter returns the expected count; the combined filter returns the single matching event	Pass	`test_audit_query.py::test_query_filters_by_each_column`
TC-39	FR 6.3	Replay uses the recorded policy version	A call is denied; a new policy that allows it is activated; the original event is replayed	New calls are allowed; the replay returns DENY with `match` true and the original policy version	Pass	`test_api_admin.py::test_replay_after_policy_change_uses_recorded_version`

B.3.6 Administration, user interface, demonstration, benchmark and storage

Table B.9: Test cases TC-40 to TC-46, administration, UI, demonstration, benchmark and storage

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-40	FR 2.5	Policies are hot-reloaded, audited and can be rolled back	Administrator activates a new policy with `PUT /api/policies`, then reactivates the first version	The next call uses the new version; rate counters survive; one `policy.activated` event attributed to the administrator; rollback restores the first version	Pass	`test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`
TC-41	FR 7.1	Agent keys are shown once and can be rotated	Administrator creates an agent, lists agents, rotates the key, disables and deletes the agent	The key (`mk_…`) appears only in the create response, never in listings or audit events; the old key is rejected (401) after rotation; each change is audited	Pass	`test_api_admin.py::test_agent_crud_shows_key_once`
TC-42	FR 5.3, NFR4	The approver PWA works in Arabic	A pending request rendered with the Arabic language setting	Right-to-left layout showing the Arabic summary, the reason and an Arabic approve button	Pass	`ui.test.tsx`: approver PWA, "renders right-to-left in Arabic with the Arabic summary and reason"
TC	FR 8.1	The demon	Demo chat	Off: the message	Pass	`test_api_demo.py::test_unprotected_agent_leaks_to_attacker_inbox`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
-43		stration shows the effect of protection	with a scripted model stub whose plan e-mails an outside address; protection off, then on	reaches the attacker inbox. On: the call is denied, the attacker inbox stays empty and the decision is audited with the demo user		`test_api_demo.py::test_protected_agent_is_stopped_by_the_gateway`
TC-44	FR 8.2, BR 2	The benchmark escalates legitimate high-risk tasks	Tasks `en-approval-transfer-01` and `ar-approval-delete-01` under `full` protection with the oracle approver	Both tasks escalated and approved; utility true; every decision replays to the same action	Pass	`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`
TC-45	NFR11	The audit chain works on PostgreSQL	Throwaway PostgreSQL server (`pgserver`); 25 events containing Arabic text	Chain verifies with 25 events checked; the query finds all 25 events; the JSON export contains the Arabic text	Pass	`test_postgres.py::test_audit_chain_appends_and_verifies_on_postgres`
TC-46	FR 8.2	Attack episodes are built, planted, scored	A neutral placeholder template and a placehol	`none`: the placeholder text is delivered and the injection predicate	Pass	`test_bench_attacks.py::test_policy_denial_stops_a_large_export_under_full_defense` `test_bench_attacks.py::test_scripted_agent_does_not_comply_when_the_detector_removes_the_text`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		and attributed to the layer that stops them	der injection task (a customer export above the 50-record limit) planted into a document that a benign user task reads; scripted agent; defences `none` and `full`, and `detector_only` with a stub rule	holds. `full`: the export is denied by the policy constraint, the episode is attributed `stopped_by = policy` and the user task still succeeds. `detector_only`: the planted text is sanitised, never delivered, and the episode is attributed to the detector		

B.4 Security test cases

The security suite (`tests/security/test_gateway_security.py`) tests the gateway itself against the threats found in the STRIDE analysis [69], [91]. It replaces the manual Burp Suite and OWASP ZAP pass planned in the proposal with checks that run on every change. Each case runs against the full FastAPI application with real authentication, storage and audit components. Some cases are parametrised over several routes, roles or inputs; the result column gives the number of passing instances. Each case is listed under the STRIDE category of the test-file section in which it is defined. A few cases also cover a second category (ST-03 elevation of privilege, ST-17 tampering, ST-19 spoofing); Section 3.3.8 maps the same cases to the threats they mitigate.

Table B.10: Security test cases ST-01 to ST-23

ID	STRIDE category	Threat	Expected result	Result
ST-01	Spoofing	Unauthenticated access to management APIs	Ten management routes (users, agents, policy validation and activation, audit list, verify and export, approvals, statistics, tools) return 401 without a token	Pass (10/10)

ID	STRIDE category	Threat	Expected result	Result
ST-02	Spoofing	Forged or unsigned session tokens	Tokens signed with another key, with algorithm `none`, expired, malformed or empty all return 401	Pass
ST-03	Spoofing	A role claim edited inside a valid token	An approver token whose role is changed to admin without re-signing returns 401	Pass
ST-04	Spoofing	Use of a revoked agent key	After rotation the old key returns 401 and the new key 200	Pass
ST-05	Spoofing	Guessed or malformed agent keys	Four malformed keys return 401	Pass
ST-06	Tampering	Editing an audit row in the database	Verification fails and names the edited sequence number	Pass
ST-07	Tampering	Deleting an audit row	Verification fails	Pass
ST-08	Tampering	A policy containing a catastrophic-backtracking regular expression	Validation reports the policy invalid; activation returns 422	Pass
ST-09	Repudiation	An approver denies having decided, or a decision is changed later	Decision stored with the approver's name; the waiting call is denied; a second decision returns 409; an `approval.decided` event is in the audit log	Pass
ST-10	Repudiation	Failed logins that leave no trace, or leak the password	An `auth.login_failed` event is recorded; the attempted password does not appear in the audit log	Pass
ST-11	Information disclosure	Secrets in tool arguments reaching the log	An API-key-shaped value and a `password` argument are absent from the JSON audit export	Pass
ST-12	Information disclosure	Hashes or keys in listings	User and agent listings contain no password hash, key hash or full key	Pass
ST-13	Information disclosure	Stack traces or internal paths in error responses	A failing tool produces a response without the internal path or a traceback	Pass
ST-14	Information disclosure	Path traversal through static file serving	Four traversal paths (plain, URL-encoded, encoded dots, under `/approve`) return no file from outside the web root	Pass (4/4)
ST-15	Information disclosure	Clickjacking and content-type sniffing	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, a CSP with `frame-ancestors 'none'` and a Referrer-Policy on API responses	Pass
ST-16	Information disclosure	Cross-origin use of the API from an untrusted site	An unknown origin is not echoed in the CORS header; the configured origin is	Pass
ST-17	Information disclosure	SQL injection through audit search filters	Three SQL-injection-style filter values return 200 with no results; the audit chain still verifies	Pass (3/3)
ST-18	Denial of service	Oversized request bodies	A body over 1 MB returns 413	Pass
ST-19	Denial of	Password brute force	Repeated failures lead to 429; the correct	Pass

ID	STRIDE category	Threat	Expected result	Result
	service		password is also refused with 429 inside the window	
ST-20	Elevation of privilege	Approvers or auditors performing administrator actions	Six role and route combinations return 403	Pass (6/6)
ST-21	Elevation of privilege	Mixing agent and human credentials	An agent key on a human API returns 401; an administrator token on an agent API returns 401	Pass
ST-22	Elevation of privilege	Calling an unlisted tool, or using an agent bound to an unknown policy agent	Both calls are denied	Pass
ST-23	Elevation of privilege	Server-side request forgery through `fetch_url` under the default policy	Loopback, cloud-metadata, `localhost` and `file:` URLs are denied	Pass (4/4)

Table B.11: Security tests by STRIDE category

STRIDE category	Test cases
Spoofing	ST-01 to ST-05
Tampering	ST-06 to ST-08
Repudiation	ST-09, ST-10
Information disclosure	ST-11 to ST-17
Denial of service	ST-18, ST-19
Elevation of privilege	ST-20 to ST-23
Total	**23 security tests; 45 test cases passed in the recorded run**

B.5 Test results

B.5.1 Execution summary

Table B.12: Test execution summary

Suite	Runner	Tests	Passed	Failed	Skipped
Python, all areas (`-m "not llm"`)	pytest	1,281	1,281	0	0
Web UI (10 test files, 38 suites)	Vitest	194	194	0	–

The per-area breakdown of the Python tests is in Table 3.25 (Section 3.5.6). The two `llm`-marked tests were not part of the recorded run. The LLM smoke run in B.5.7 exercises the same model integration through the benchmark harness.

B.5.2 Code coverage

Table B.13: Statement coverage by package

Package or module	Responsibility	Statements covered
`adapters`	MCP proxy, OpenAI-compatible proxy, upstream manager	90.3%
`api`	FastAPI application, routers, HTTP hardening	91.2%
`approvals`	Approval service, notifier, approver summaries	96.2%
`audit`	Hash-chained log, verification, query, export, redaction	96.1%
`auth`	Password hashing, tokens, agent keys, login throttling	96.2%
`cli`	`mirsad` command line	92.4%
`config`	Settings	95.8%
`detector`	Normalisation, rules, model, sanitiser, dataset and training	96.9%
`gateway`	Decision pipeline, inspection, replay	90.7%
`models`	Shared data types	100.0%
`policy`	Policy language, engine, rate limiter, regex guard	94.9%
`sdk`	Python client SDK	94.4%
`storage`	Database engine and tables	99.3%
`taint`	Taint extraction, rules, tracker	96.5%
`textutil`	Shared text helpers (Arabic folding, domains)	98.2%
Total	9,763 statements	**94.4%**

Coverage is statement coverage only. In the recorded run every package is above the 80% target of NFR9. The lowest values are in the protocol adapters and the gateway. Their uncovered lines are mostly exception handlers and rarely taken edge cases, such as replaying an event whose audit payload was truncated. The stdio entry point of the MCP proxy is exercised by TC-11, but it runs in a subprocess that coverage does not measure.

B.5.3 Static analysis and dependency scanning

Table B.14: Static analysis and dependency scan results

Tool	Scope	Configuration	Result
Ruff	`src`, `tests`, `demo`, `bench`, `scripts`	Rule sets E, F, W, I, B, UP and S (security rules from flake8-bandit); S and B relaxed in tests	No report stored in `reports/`
Bandit	`src` (13,023 lines of code)	Default rule set; B101 (assert) skipped	0 findings at any severity
TypeScript compiler	`web/`	`strict` mode, `tsc --noEmit`	No report stored in `reports/`
pip-audit	Installed Python packages (124;	PyPI advisory database	0 packages with a

Tool	Scope	Configuration	Result
	the editable project itself skipped)		known vulnerability
npm audit	Web dependencies (231 packages, development dependencies included)	npm advisory database	0 vulnerabilities

The CI workflow defines Ruff and the TypeScript compiler as checks, but neither stores a report file and no CI run is recorded, so these two results are not part of the recorded evidence.

B.5.4 Performance

The latency benchmark (`bench/latency.py`) starts the real application with the default policy and agent-key authentication. It sends 1,000 benign read-only calls spread over six tools, first one at a time and then from 50 concurrent sessions. Added latency is the HTTP round trip through Mirsad minus the direct in-process call of the same tool. The benchmark was run twice: with the server in a separate operating-system process, which is the more realistic set-up, and with the server in a background thread of the benchmarking process.

Table B.15: Added latency per tool call (ms)

Measure	Separate server process	Server in the same process
Sequential, added latency p50	2.66	2.64
Sequential, added latency p95	5.96	5.90
Sequential, added latency p99	6.12	6.01
Sequential, gateway-reported time p50	2.06	2.04
Sequential throughput (requests/s)	317	317
50 sessions, added latency p50	165.5	204.8
50 sessions, added latency p95	208.0	880.1
50 sessions, added latency p99	216.1	1304.3
50 sessions, gateway-reported time p50	159.5	2.3
50 sessions, throughput (requests/s)	291	154
50 sessions, HTTP errors	0	0
Audit chain intact after the run (True/False)	True	True

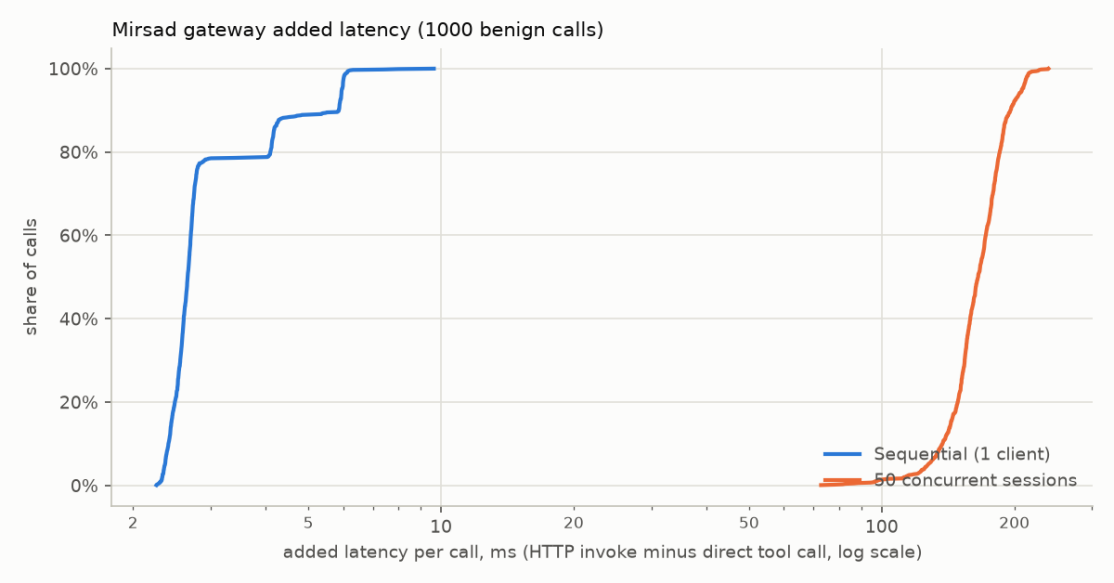


Figure B.1: Cumulative distribution of added latency over 1,000 benign calls, server in a separate process (log scale)

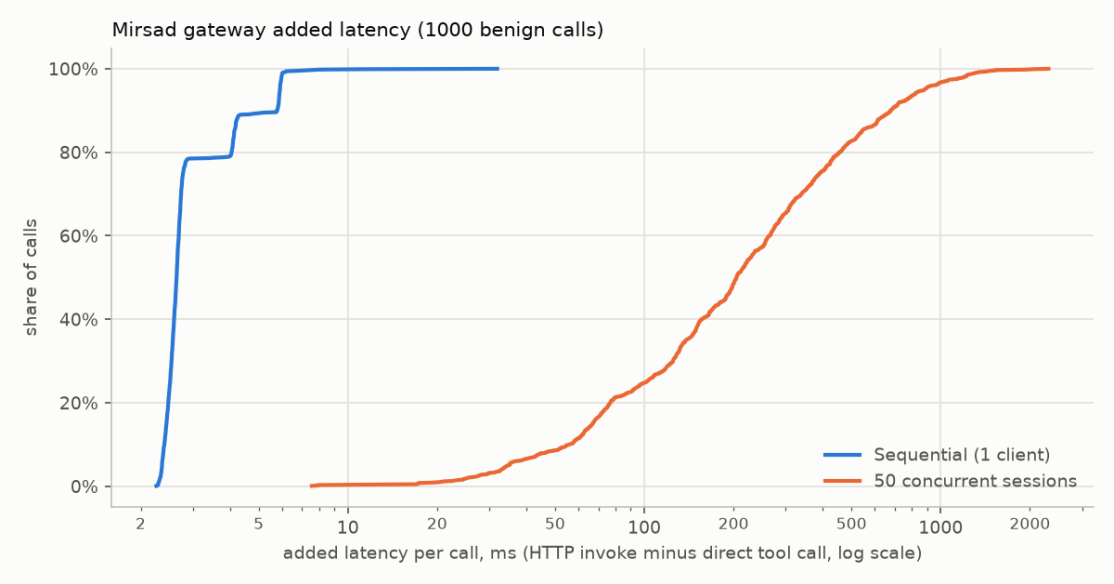


Figure B.2: Cumulative distribution of added latency over 1,000 benign calls, server in the same process as the client (log scale)

With one request at a time, Mirsad adds about 3 ms per call at the median, far below the 200 ms budget of NFR1 (Figure B.1). Under 50 concurrent sessions served by a single uvicorn worker, the median added latency rises to 166 ms with a separate server process and 205 ms when client and server share one Python process (Figure B.2), with 0 and 0 HTTP errors. The separate-process median is the figure assessed against NFR1; in the in-process layout the client competes with the server for one interpreter, which gives a much longer tail. Most of this time is queueing. In the separate-process run, the gateway-reported time itself grows to a median of 159 ms. This suggests that requests wait inside

the gateway, where taint registration and audit appends run under one lock and write to SQLite. The concurrency figure should therefore be read as "50 sessions served on one worker", not as a throughput claim. Measurements with several workers and PostgreSQL remain to be done.

B.5.5 Benchmark results on benign tasks

Mirsad-Bench ran the 48 benign user tasks (24 English; 24 Arabic, of which 14 are Modern Standard Arabic and 10 Gulf dialect) with the deterministic scripted agent under each defence configuration. Each configuration made 68 benign tool calls. Two approver behaviours were simulated: *deny* rejects every escalation, and *oracle* approves exactly the calls that match the user's task, like a reviewer who knows what the user asked for. The `none` configuration calls the tools directly and is the same in both runs.

Table B.16: Benign benchmark results per defence and approver

Defence	Approver	Utility	FPR	Friction	Final block	Result FP	Gateway p50 / p95 (ms)
none	–	100.0%	0.0%	0.0%	0.0%	n/a	n/a
full	deny	75.0%	0.0%	17.6%	17.6%	0.0%	1.77 / 5.30
policy_only	deny	75.0%	0.0%	17.6%	17.6%	n/a (detector off)	0.94 / 1.52
taint_only	deny	87.5%	0.0%	8.8%	8.8%	n/a (detector off)	1.09 / 1.98
detector_only	deny	100.0%	0.0%	0.0%	0.0%	0.0%	1.45 / 3.76
full	oracle	100.0%	0.0%	17.6%	0.0%	0.0%	1.82 / 5.29
policy_only	oracle	100.0%	0.0%	17.6%	0.0%	n/a (detector off)	0.95 / 1.94
taint_only	oracle	100.0%	0.0%	8.8%	0.0%	n/a (detector off)	1.08 / 2.16
detector_only	oracle	100.0%	0.0%	0.0%	0.0%	0.0%	1.48 / 4.59

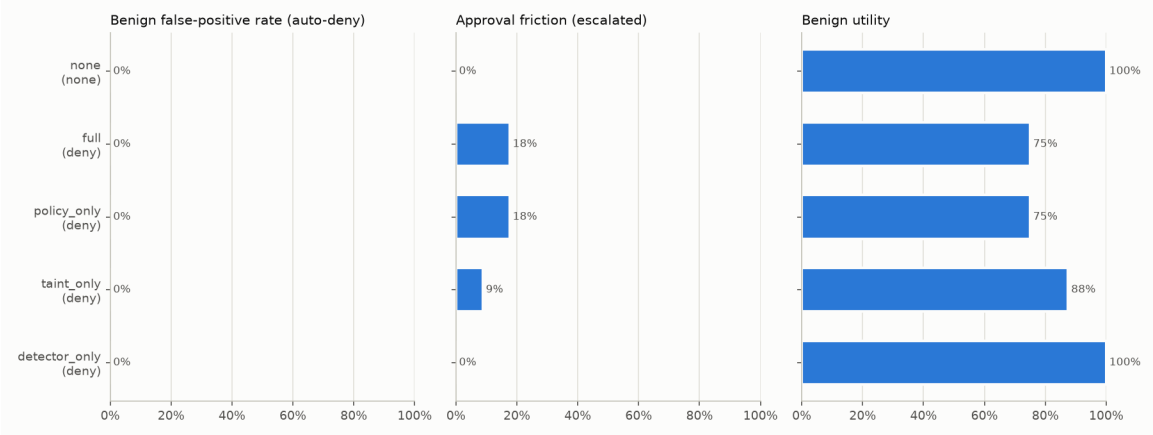


Figure B.3: Benign false-positive rate, approval friction and utility per defence configuration with the denying approver (run `scripted-benign-deny`)

The main observations are:

- **Automatic denials.** The FPR and Result FP columns give the share of benign calls denied automatically and of benign tool results sanitised or blocked in each configuration, against the FPR target of at most 5%, with the detector in rules-only mode.
- **Friction comes from the policy's intended escalation paths.** Under `full` protection, 12 of the 68 benign calls were escalated. The reason codes recorded in `reports/bench/scripted-benign-deny/summary.md` are `CONSTRAINT_VIOLATION` (6) and `TAINT_UNTRUSTED_TO_EXTERNAL` (6) for e-mails to outside recipients, `APPROVAL_ALWAYS` (4) for the payment and deletion tasks, and `APPROVAL_POLICY` (2) for exports. Under full protection, friction was 17.6% for English and 17.6% for Arabic tasks.
- **The approver decides the utility cost.** With the oracle approver, utility under full protection stays at 100%. With an approver who rejects everything, it falls to 75%.
- **The policy layer is needed for high-risk tools.** Only the configurations that include the policy layer asked a human about the legitimate payment and deletion tasks (approval recall 100% for `full`, 0% for `taint_only` and 0% for `detector_only`).
- **Gateway processing time is a few milliseconds** per call in every configuration (Figure B.4).

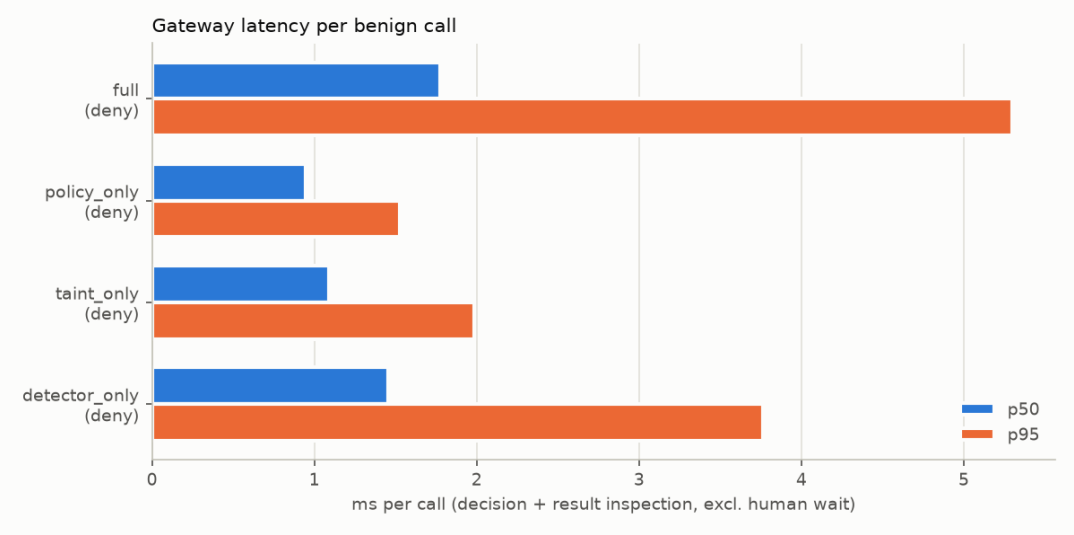


Figure B.4: Gateway time per benign call (decision plus result inspection, human wait excluded), p50 and p95, run `scripted-benign-deny`

B.5.6 Audit verification and decision replay

After each benchmark configuration, the harness verified the HMAC hash chain of that configuration's audit log and replayed every recorded `tool_call.decision` event with its stored policy version and inputs (`--replay-check`). The latency benchmark verified the chain after its 1,000 calls.

Table B.17: Audit verification and replay results

Run	Audit chains verified (True/False)	Decisions replayed	Same action
`scripted-benign-deny` (4 protected configurations)	True	272	272
`scripted-benign-oracle` (4 protected configurations)	True	272	272
`llm-smoke` (full protection)	True	6	6
Latency benchmark, separate process	True	–	–
Latency benchmark, same process	True	–	–

Every replayed decision reproduced its original action, which meets the acceptance criterion of FR6.3. Tamper detection itself is tested by TC-36, TC-37, ST-06 and ST-07.

B.5.7 LLM smoke run

A smoke run checked that the real model integration works through the gateway. The demonstration agent used Qwen2.5-7B through Ollama at temperature 0 on 4 benign tasks (2 English, 2 Arabic), without protection and with full protection (oracle approver).

Utility was 75% without Mirsad and 75% with it. FPR and friction were 0% and 0%. The gateway time over 6 calls was 3.59 ms at the median and 5.52 ms at p95. The same task failed in both configurations (`ar-crm-update-02`): the episode records show that the model wrote a CRM note without the requested phone number, and the gateway allowed every call. The failure is the model's, not the gateway's. With so few tasks this run is a functional check, not a utility estimate.

B.5.8 Results pending team-supplied data

Table B.18 lists the results that the evidence pipeline produces once attack-labelled data exists. They are not estimated here. The repository currently holds only benign detector batches (`data/detector/raw/en-ben.jsonl` and `ar-ben.jsonl`) and no attack files under `bench/scenarios/attacks/`. Attack material is written and reviewed by the team (Section 3.6.4).

Table B.18: Results that need team-supplied attack data

Result	Value	How it is produced
Detector F1, English held-out test set (target ≥ 0.85)	[pending]	Add labelled training batches and separate held-out test batches (`raw/test-*.jsonl`) to `data/detector/raw/`, then run `make detector` (`docs/detector.md`, Section 11.4)
Detector F1, Arabic held-out test set (target ≥ 0.85)	[pending]	As above
Detector F1, mixed Arabic–English test set	[pending]	As above
Detector precision / recall, English	[pending] / [pending]	As above
Detector precision / recall, Arabic	[pending] / [pending]	As above
Detector scan latency, 2 KB text, p50 (target < 10 ms)	[pending] ms	Measured by the same training run
Attack success rate, full protection	[pending]	Add injection tasks and attack templates to `bench/scenarios/attacks/` (`bench/scenarios/attacks/README.md`), then run `scripts/run_bench.sh`, which executes the attack episodes
ASR reduction against the unprotected baseline (target ≥ 80%)	[pending]	As above

B.6 Known limitations and open items

- **Detector accuracy and attack success rate are not measured yet** (B.5.8). Until they are, the evaluation shows what Mirsad costs legitimate work, but not how much protection it gives against real attacks.

- **Tests verify how detections are handled, not detection quality.** Functional tests use neutral placeholders scored by a stub rule, so they cannot show whether real Arabic or English injections are recognised.
- **All detector-dependent numbers come from the rules-only fallback**, because no trained model is packaged. The benchmark, the latency benchmark and the result false-positive rate must be measured again once a model ships.
- **No attack episode has been run.** The harness executes attack episodes (`bench/attacks.py`, tested with neutral placeholder text in `tests/bench/test_bench_attacks.py`), but no attack files exist yet.
- **Benchmark coverage is limited.** The scripted agent replays task plans exactly, the LLM run covers 4 tasks, and the oracle approver is idealised. Approval quality and decision time of real people are not modelled.
- **Usability (NFR4) and accessibility (NFR10) checks are incomplete.** The decision-time figure is an analytical Keystroke-Level Model estimate (Section 4.2.8); no approver has been timed in a user study, no acceptance session (Section 3.5.7) has been held, and no UI review checklist is recorded. Keyboard navigation and reduced-motion behaviour are not covered by automated tests.
- **Concurrency.** Tail latency with 50 sessions on one worker is high, and part of the waiting happens inside the gateway (B.5.4). Multi-worker and PostgreSQL latency have not been measured.
- **Deployment.** The CI workflow defines a Docker build job, but no CI run, image build or `docker compose up` is recorded in `reports/`. The Compose file does not yet set `MIRSAD_POLICY_PATH` and `MIRSAD_UPSTREAMS_PATH` for the `mirsad` service (Appendix E.2.2).
- **Coverage scope.** Only statement coverage is collected. The 80% target is not enforced by a CI threshold, and `npm audit` is not part of the CI workflow.
- **Excluded tests.** The two `llm`-marked tests are not part of the recorded run.
- **Small open integration items.** Two classes implement login throttling (`mirsad/auth.py` and `mirsad/api/routers/auth.py`) and the application uses the second, so one should be removed. The policy embedded in `demo/agent.py` for stand-alone use of the demonstration agent lacks the global rule `no-bulk-export-to-email` and should load `configs/policies/default.yaml` instead. One literal left-to-right mark remains in the Arabic dictionary of the web app, and isolate characters remain inside two JSON test fixtures captured from the server; they should be written as escapes like the Python sources.
- **Benchmark fingerprint.** The runner fingerprints the scenario files but not the policy, so a resumed run would not detect a policy change; the reported runs were therefore executed from scratch (B.1).

- **Audit integrity has a documented limit.** Someone who holds the HMAC key can rewrite history undetected unless an external anchor of the chain head is kept (TC-37).
- **Security testing is automated and internal.** No external penetration test or adaptive red-team exercise has been carried out against the gateway.

Appendix C – System Screenshots / Interface Designs

All screenshots in this appendix were taken from the running system, not from mock-ups. The script `scripts/capture_screenshots.py` initialises a temporary installation with demonstration users and the demonstration agent (`mirsad init --demo`), starts the gateway with the simulated Najd Trading Co. workspace, and generates activity through the real agent API: eight allowed calls (reading e-mail, looking up customers, listing documents, an internal e-mail and a CRM note update), four calls that the policy denies (a 200-record export, a 250,000 SAR transfer, a request to a cloud metadata address and a tool that does not exist), and four calls that need approval. The approver then approves one request and denies another, leaving two pending. Playwright with Chrome captures the dashboard at 1440 × 900 pixels and the approver app at 390 × 844 pixels (a phone-sized screen). All names, addresses, IBANs and records are fictional.

For the capture run the approval timeout was raised from the default 120 seconds to 15 minutes so that pending requests stay open while the pages are captured. This is why the countdowns show about 14–15 minutes (from a 900-second timeout).

C.1 Administration Dashboard (English)

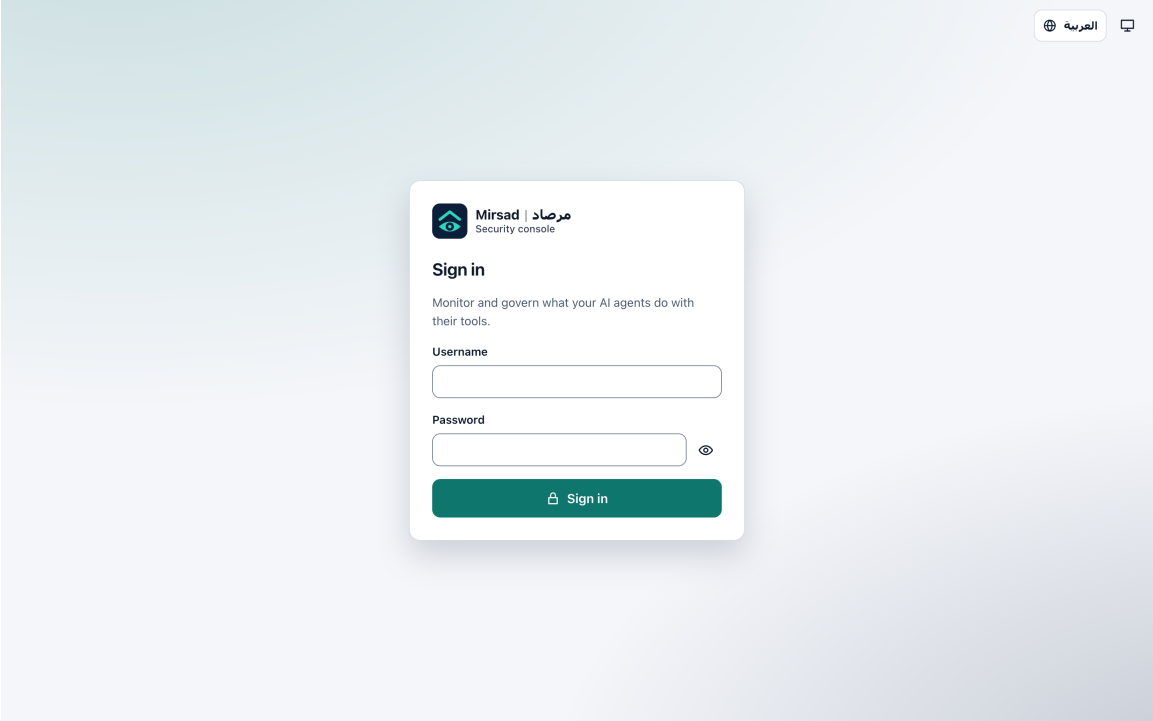


Figure C.1: Dashboard sign-in page. The language switch at the top right changes the interface to Arabic before signing in, and the second button selects the colour theme

The Overview page counts the tool calls that were allowed, denied and sent for approval, the requests still waiting, injection detections and sanitised or blocked results; it also

shows the gateway overhead per call, the audit-log integrity card with a Verify now button and the decisions over time.

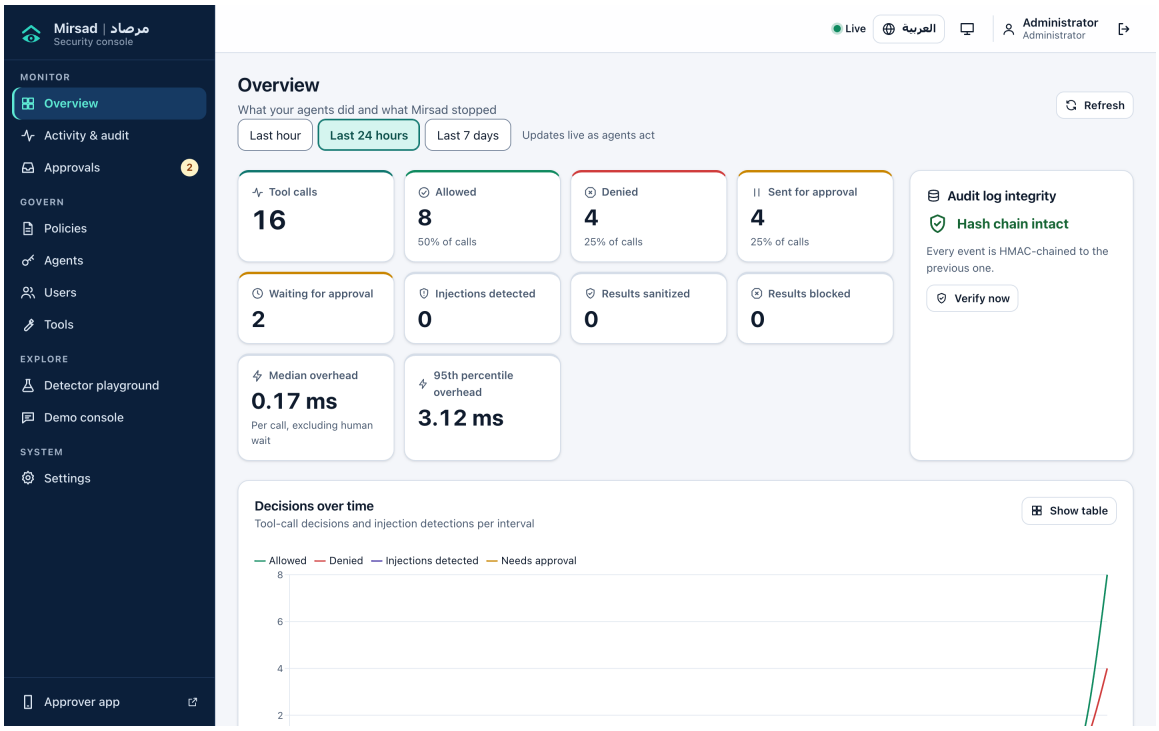


Figure C.2: Dashboard Overview after the seeded activity

The Activity and audit log filters events by text, agent, tool, outcome, event type, session, call and time range, verifies the hash chain and exports results as JSON or CSV. Outcome and risk are shown with an icon and a word, not by colour alone.

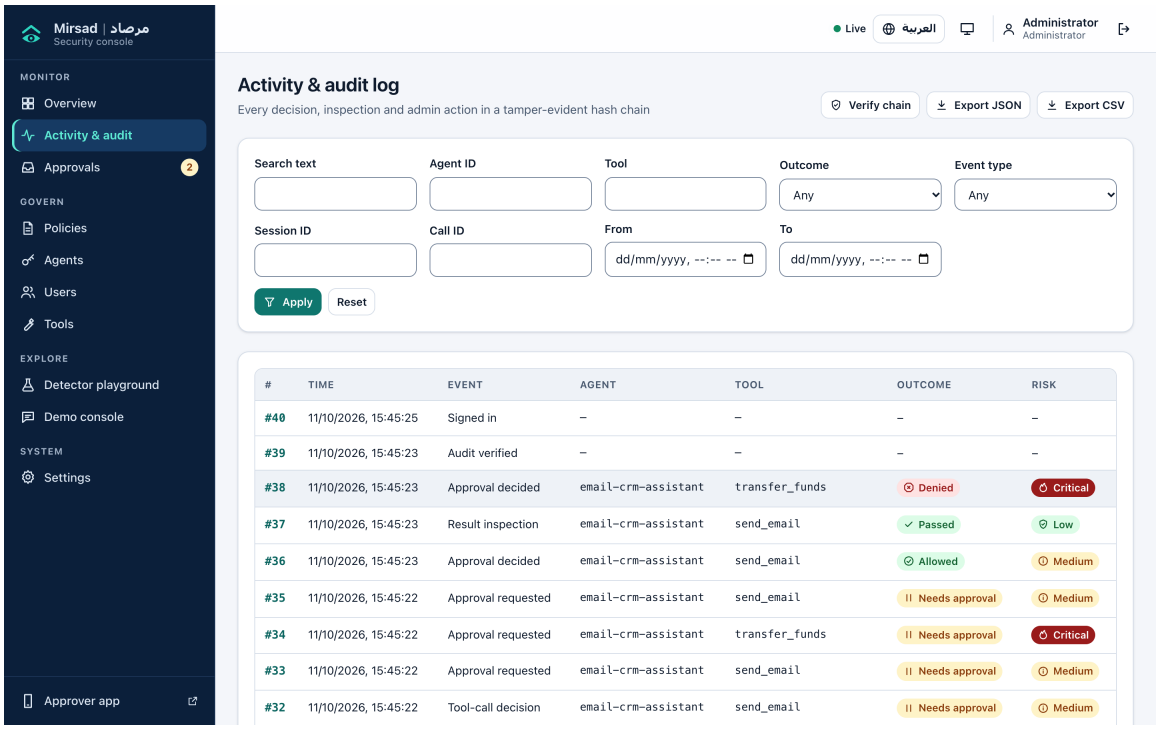


Figure C.3: Activity and audit log

The detail drawer of audit event 38, the approver's denial of a `transfer_funds` request, shows the summary fields, the previous hash and the event hash that link the event into the HMAC chain, the raw event JSON, and filters for all events of the same call or session.

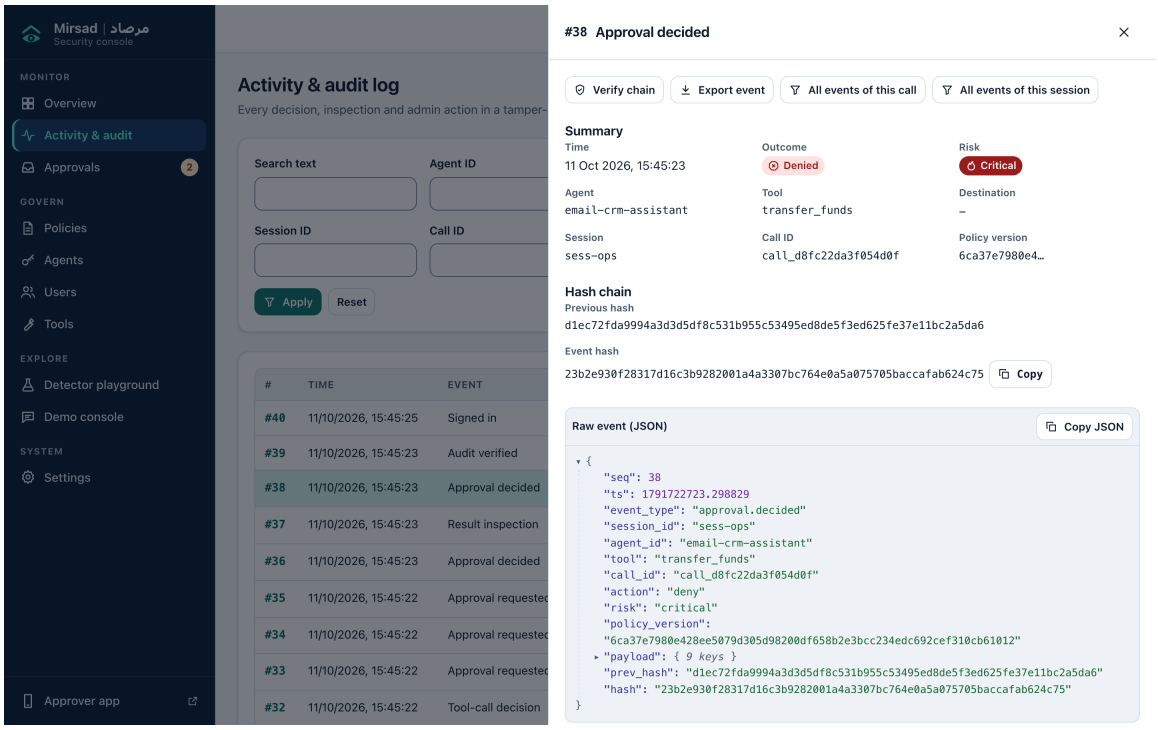


Figure C.4: Detail drawer of an audit event (a denied `transfer_funds` request)

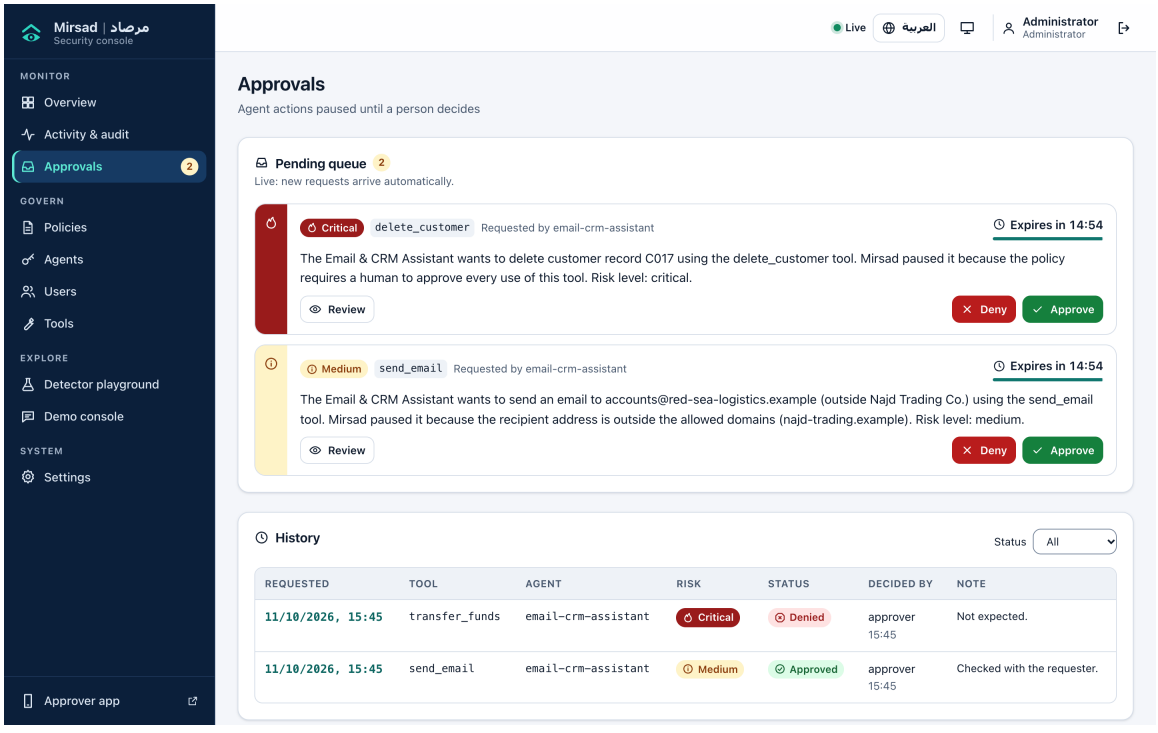


Figure C.5: Approvals page. The live pending queue shows each request's risk level, plain-language summary and countdown to expiry with Approve and Deny buttons; the history below records who decided each request and their note

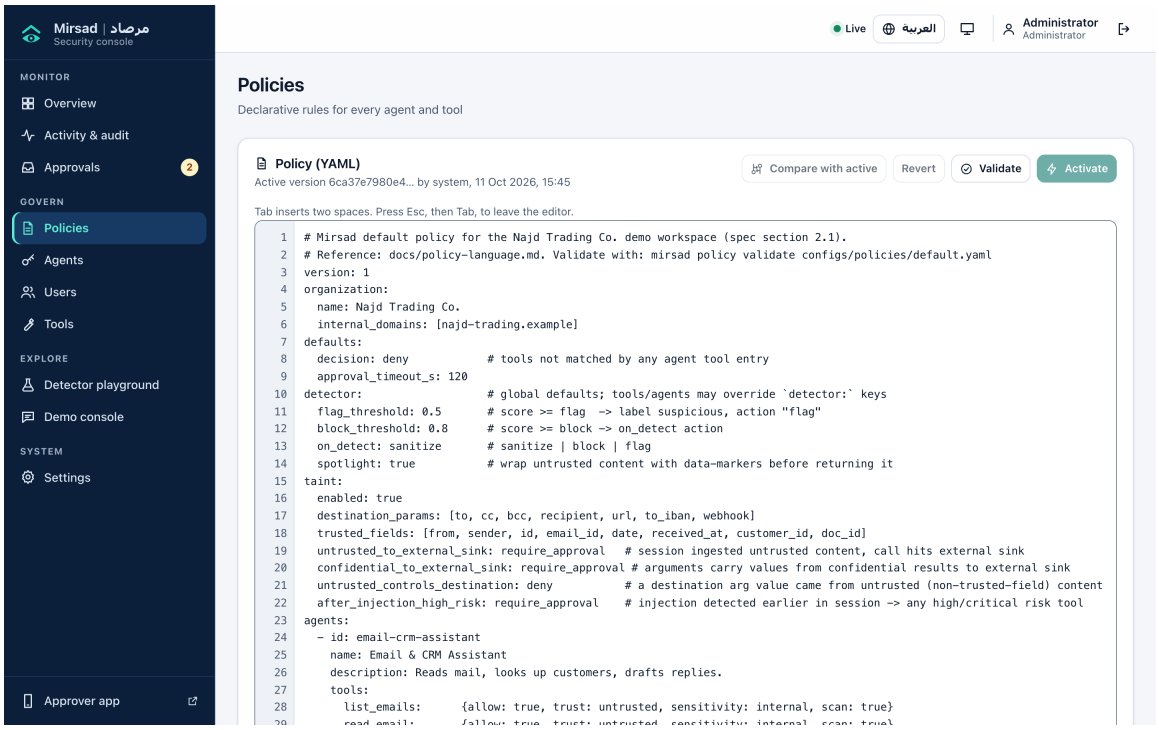


Figure C.6: Policy editor with the active YAML policy of the demonstration organisation and its version hash. Changes can be compared with the active version, reverted, validated and activated without restarting the gateway

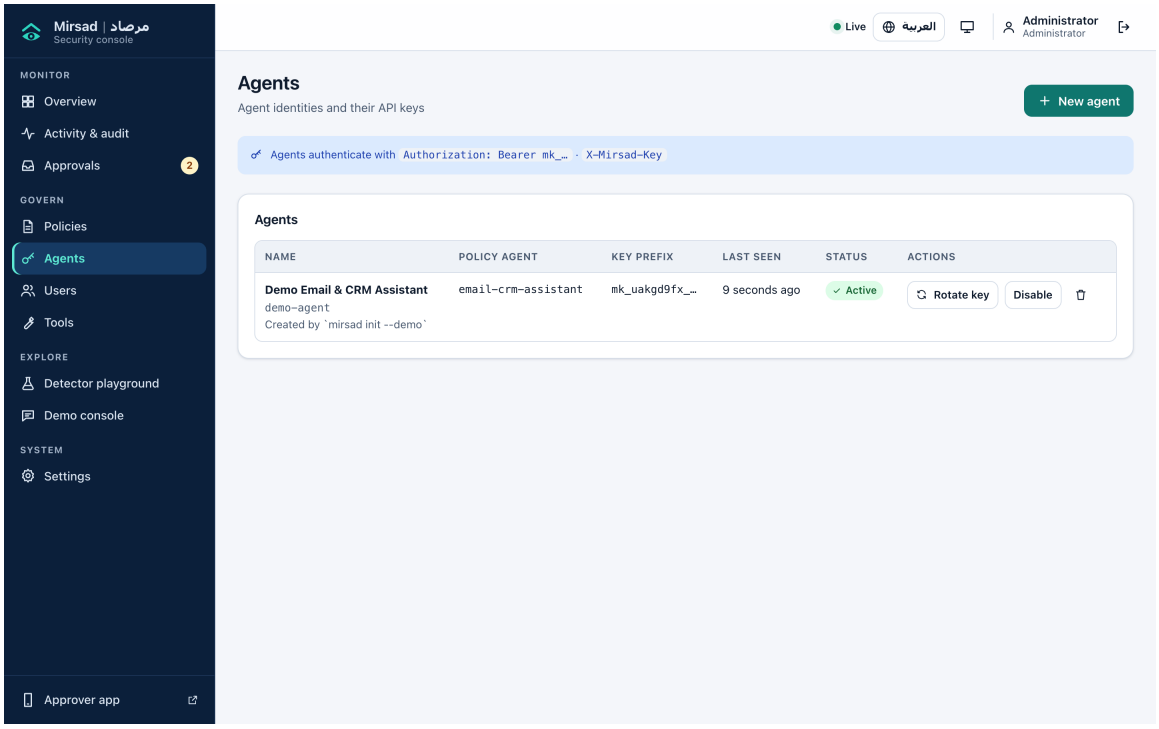


Figure C.7: Agents page: the demonstration agent, the policy agent it is mapped to, its key prefix (the full key is shown only once, when it is created or rotated), last-seen time, and rotate, disable and delete actions

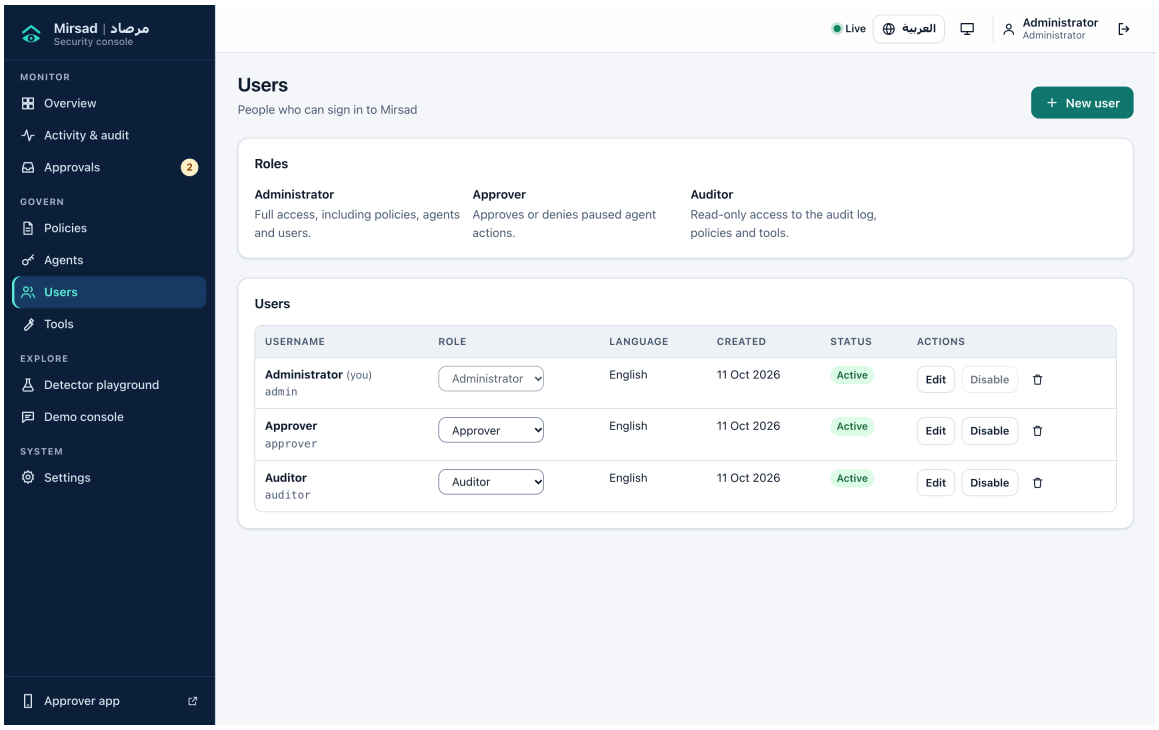


Figure C.8: Users page with the three roles and what each may do: administrators manage everything, approvers decide on paused actions, and auditors have read-only access to the audit log, policies and tools

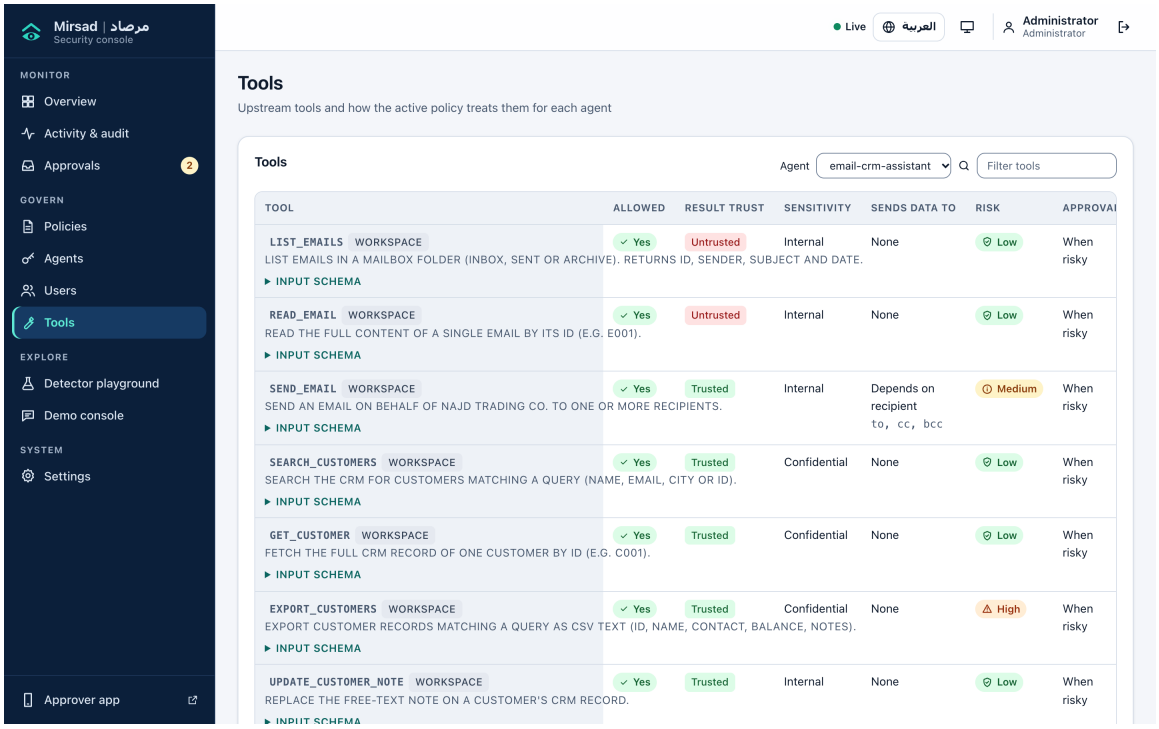


Figure C.9: Tools page: the tools discovered from the upstream workspace server and how the active policy profiles each one for the selected agent (allowed, trust of its results, sensitivity, where it sends data, risk and approval mode)

The detector playground shows the verdict, the detected language, the model and rule scores, the scan latency, highlighted evidence and obfuscation signals. The model score is 0 because the packaged detector runs in rules-only mode until the model is trained on the team's data.

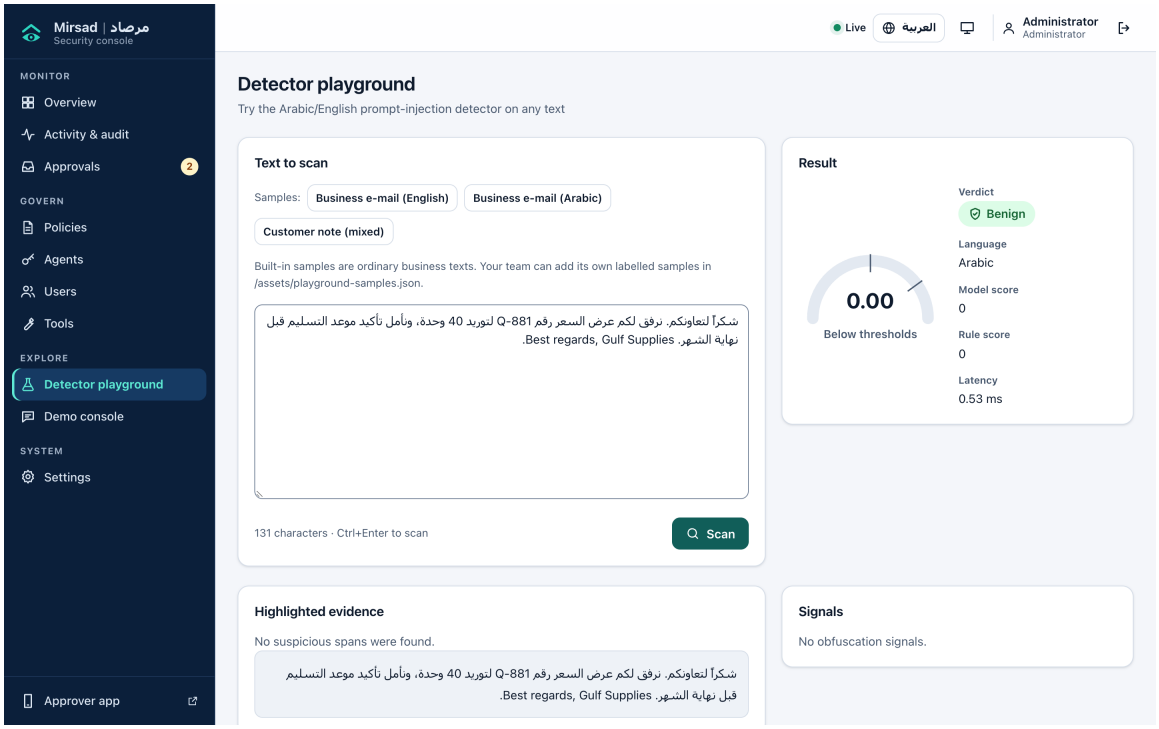


Figure C.10: Detector playground scanning a benign mixed Arabic–English supplier message

The attacker-inbox panel of the Demo console lists everything that left the organisation's domain; in this capture it shows the external supplier reply that the approver approved during seeding, not an attack.

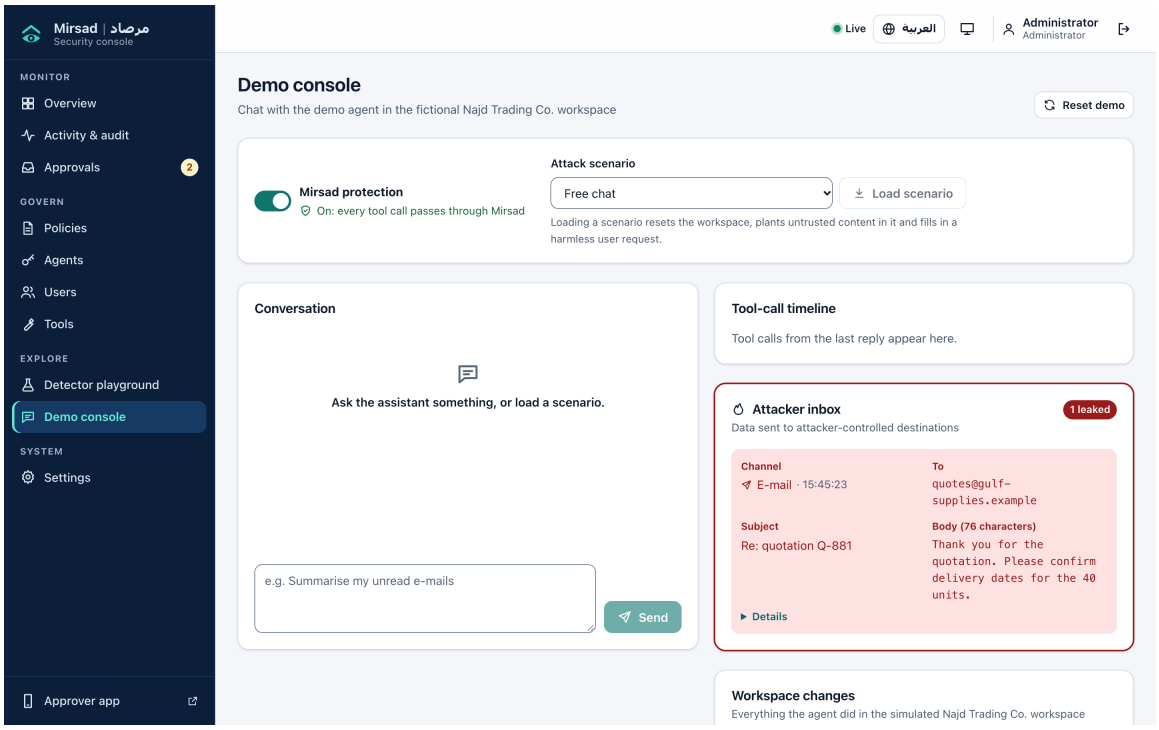


Figure C.11: Demo console: protection switch, scenarios, conversation, tool-call timeline and attacker inbox

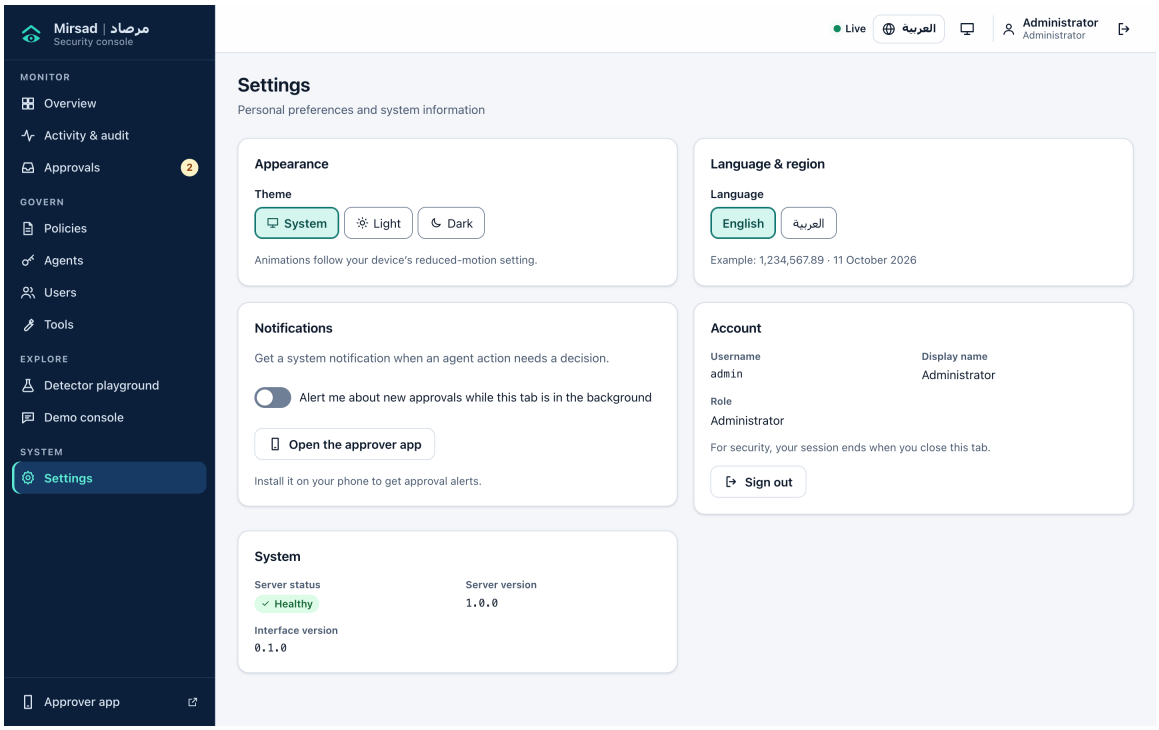


Figure C.12: Settings page: theme (system, light or dark), language, background notifications for new approvals, account details and server status

C.2 Dashboard in Arabic and in the Dark Theme

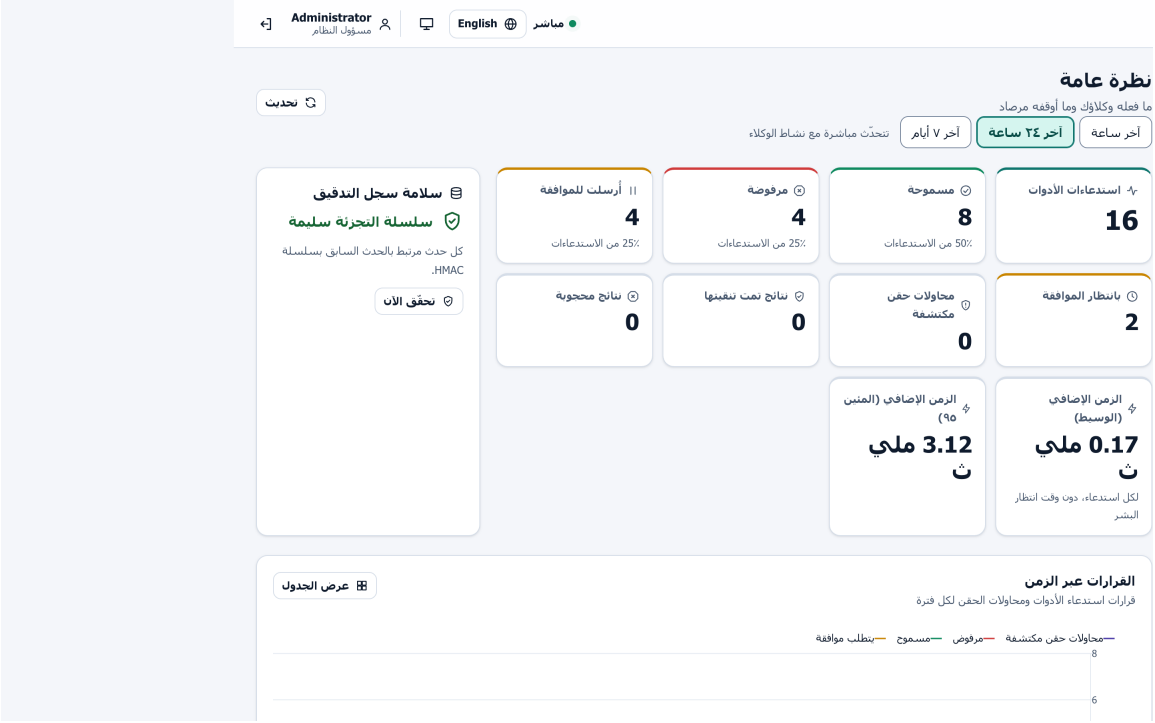


Figure C.13: Overview page in Arabic. Labels, key figures and time-range buttons are translated and arranged for right-to-left reading

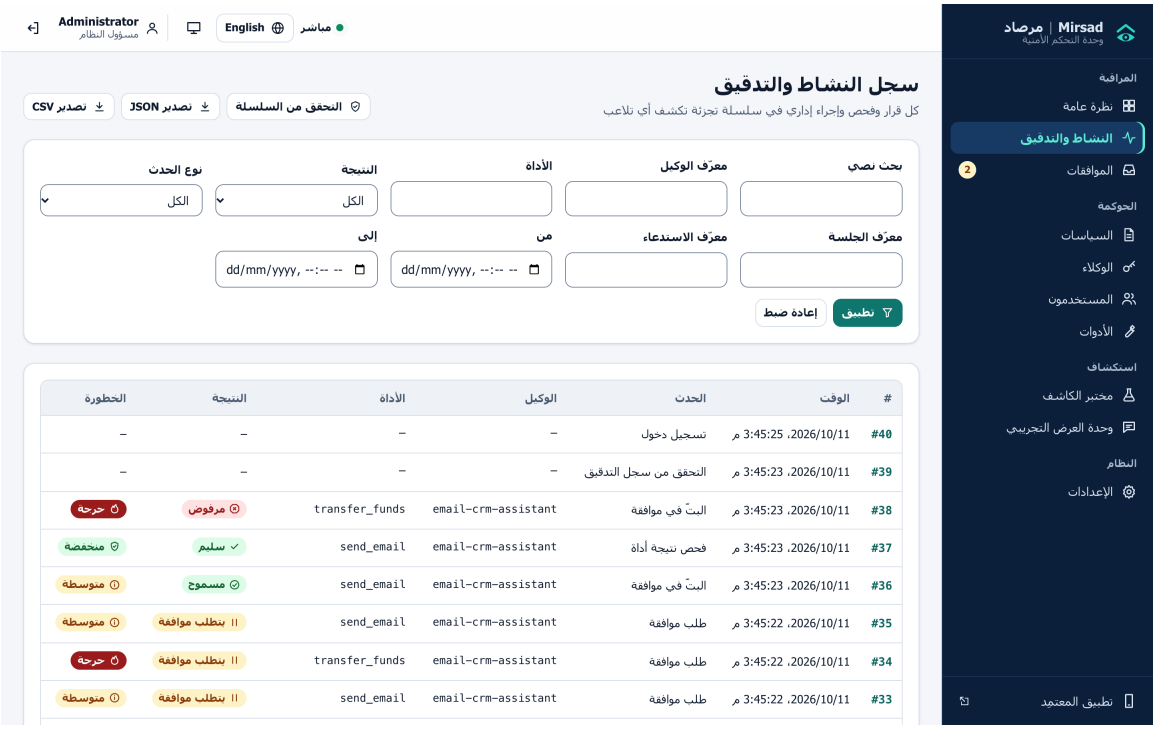


Figure C.14: Activity and audit log in Arabic. The navigation moves to the right and the filters and table columns are mirrored, while tool names, agent identifiers and dates keep their left-to-right form

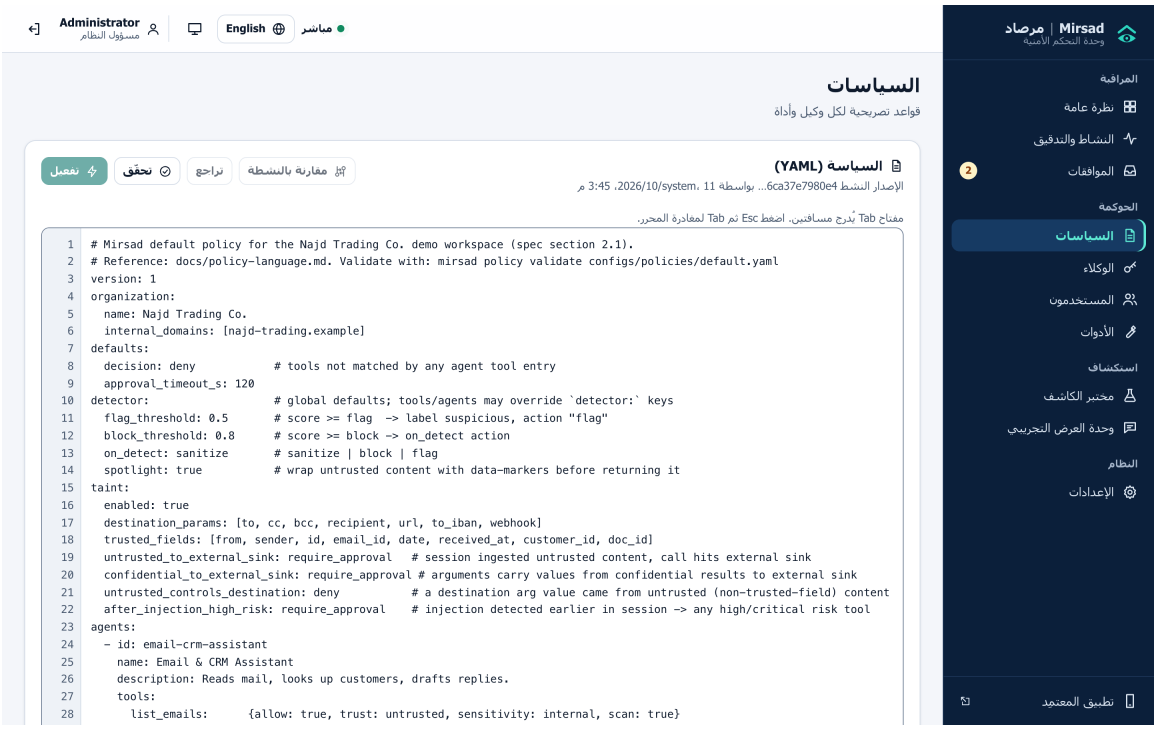


Figure C.15: Policy editor in Arabic. The surrounding interface is right-to-left, while the YAML policy stays left-to-right so that it can be edited and reviewed as code

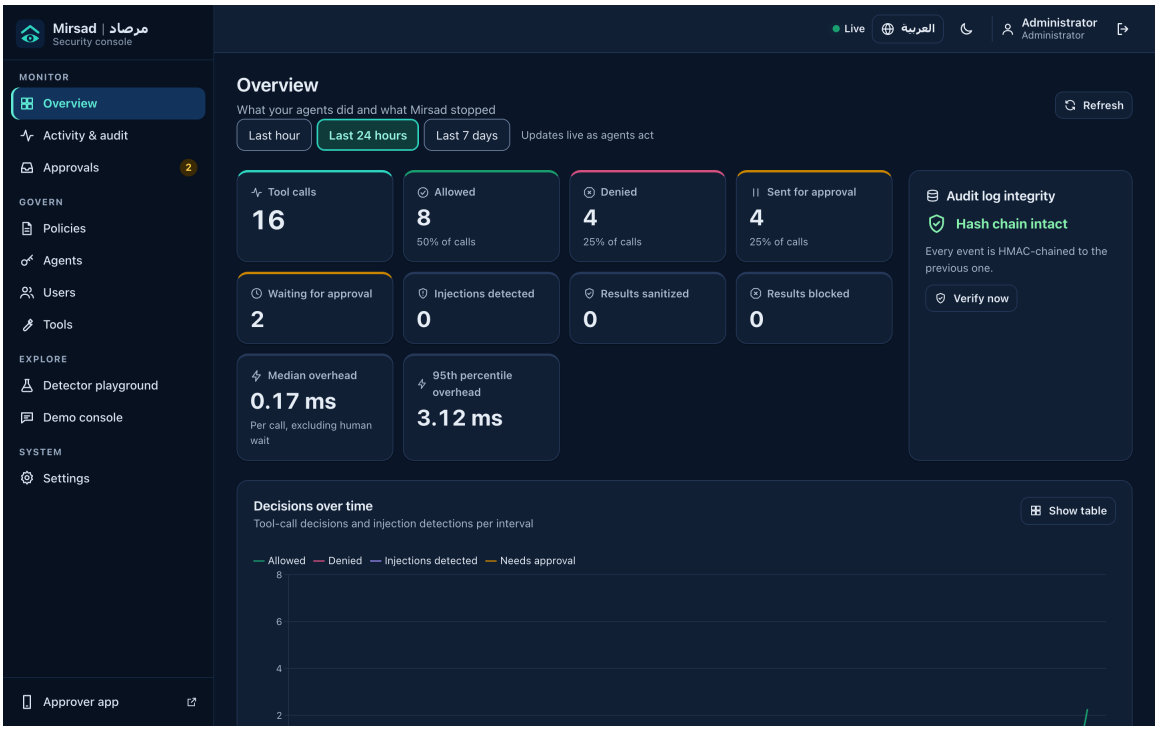


Figure C.16: Overview page in the dark theme, which can follow the operating-system setting or be selected in Settings

C.3 Approver App (English)

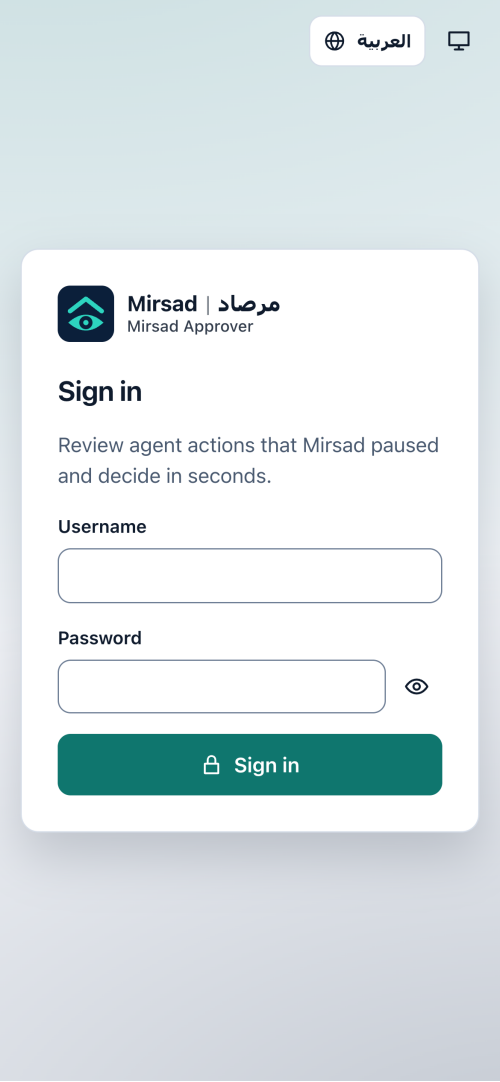


Figure C.17: Approver app sign-in on a phone-sized screen. The app is served by the gateway at `/approve` and can be installed to the home screen

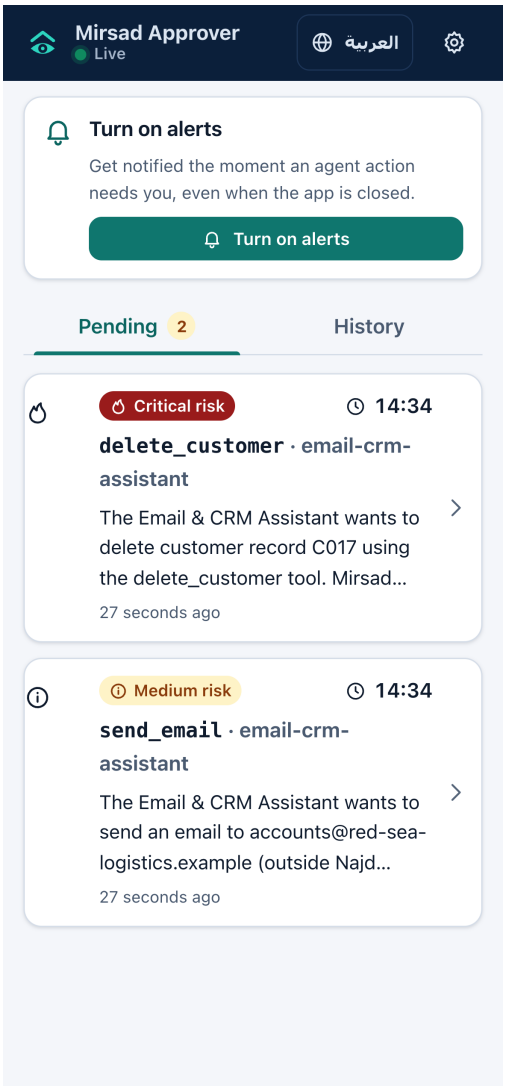


Figure C.18: Pending requests in the approver app: risk level, tool, requesting agent, the start of the plain-language summary, time since the request and countdown, a prompt to turn on push alerts, and Pending and History tabs

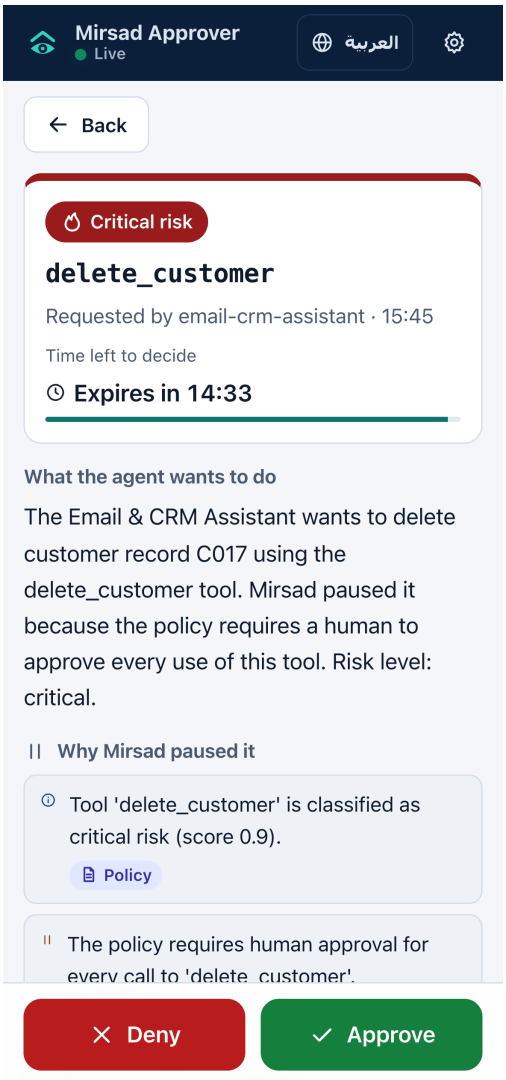


Figure C.19: Detail of a critical-risk request to delete a customer record: what the agent wants to do, why Mirsad paused it with the layer behind each reason, and Deny and Approve buttons fixed at the bottom of the screen

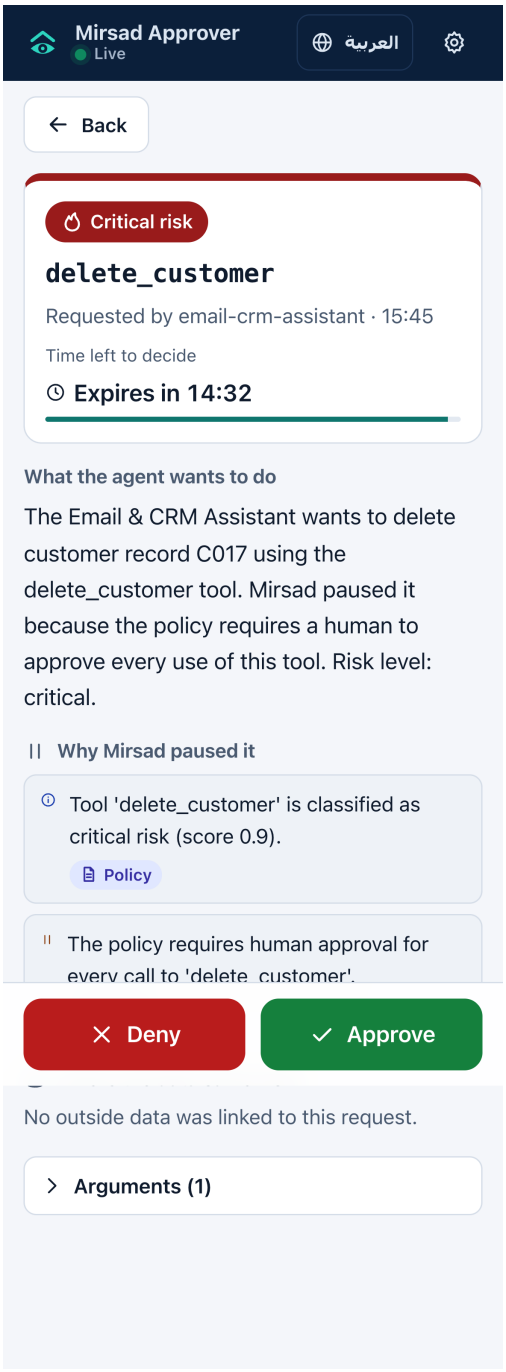


Figure C.20: Full-length capture of the same request, including the section on where the data came from (here no outside data was linked) and the collapsible list of arguments. The fixed Deny and Approve bar appears in the middle only because of the full-page capture method

C.4 Approver App (Arabic)

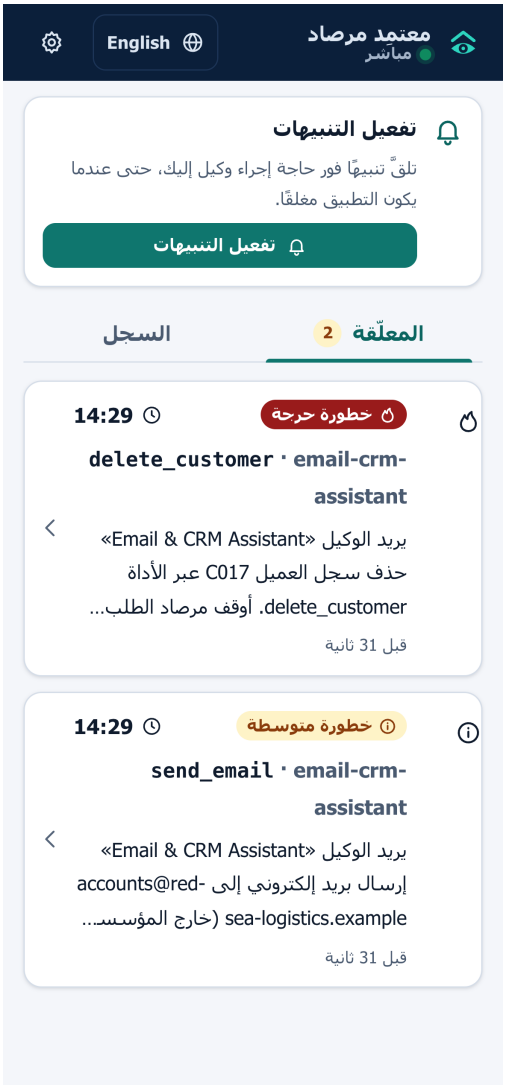


Figure C.21: Pending requests in the Arabic interface. The layout is mirrored, summaries are written in Arabic, and tool names and e-mail addresses keep their original form

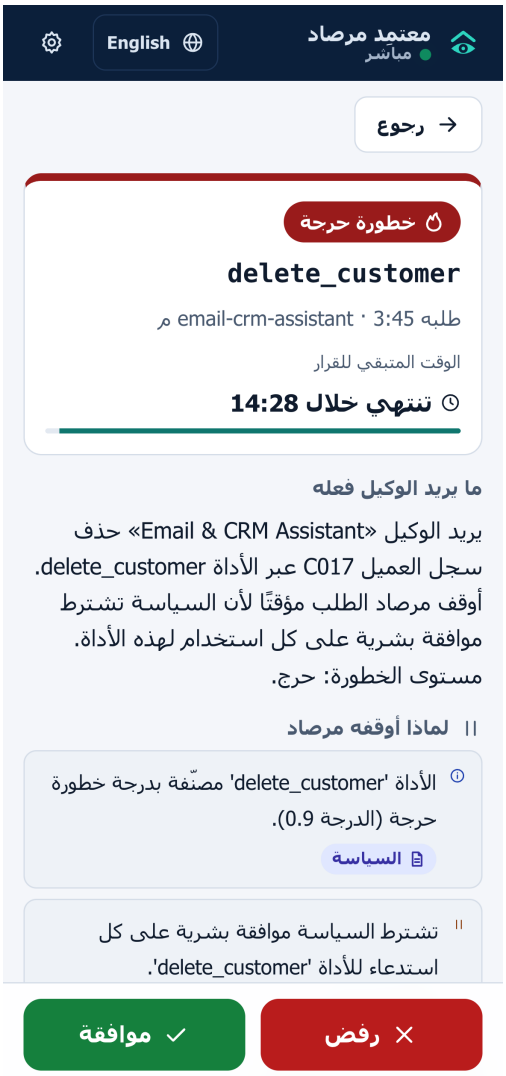


Figure C.22: Detail of the same deletion request in Arabic, with the summary, the reasons for the pause, and the Approve and Deny buttons in right-to-left order

Figure C.23: Full-length capture of the Arabic request detail, including the data-provenance section and the collapsible arguments; as in the English capture, the fixed button bar appears mid-page because of the capture method

Appendix D – Poster / Presentation Slides

The poster and the final presentation are generated from the project sources by `scripts/build_poster.py` and `scripts/build_slides.py`; both read every measured value from `reports/results_summary.json` at build time, so they always match this report. The editable files are `deliverables/Mirsad_Poster.pptx` (85 cm × 142 cm, the CCIS EXPO format) and `deliverables/Mirsad_Final_Presentation.pptx` (18 slides with speaker notes; the full speaker script is in `docs/presentation_script.md`).

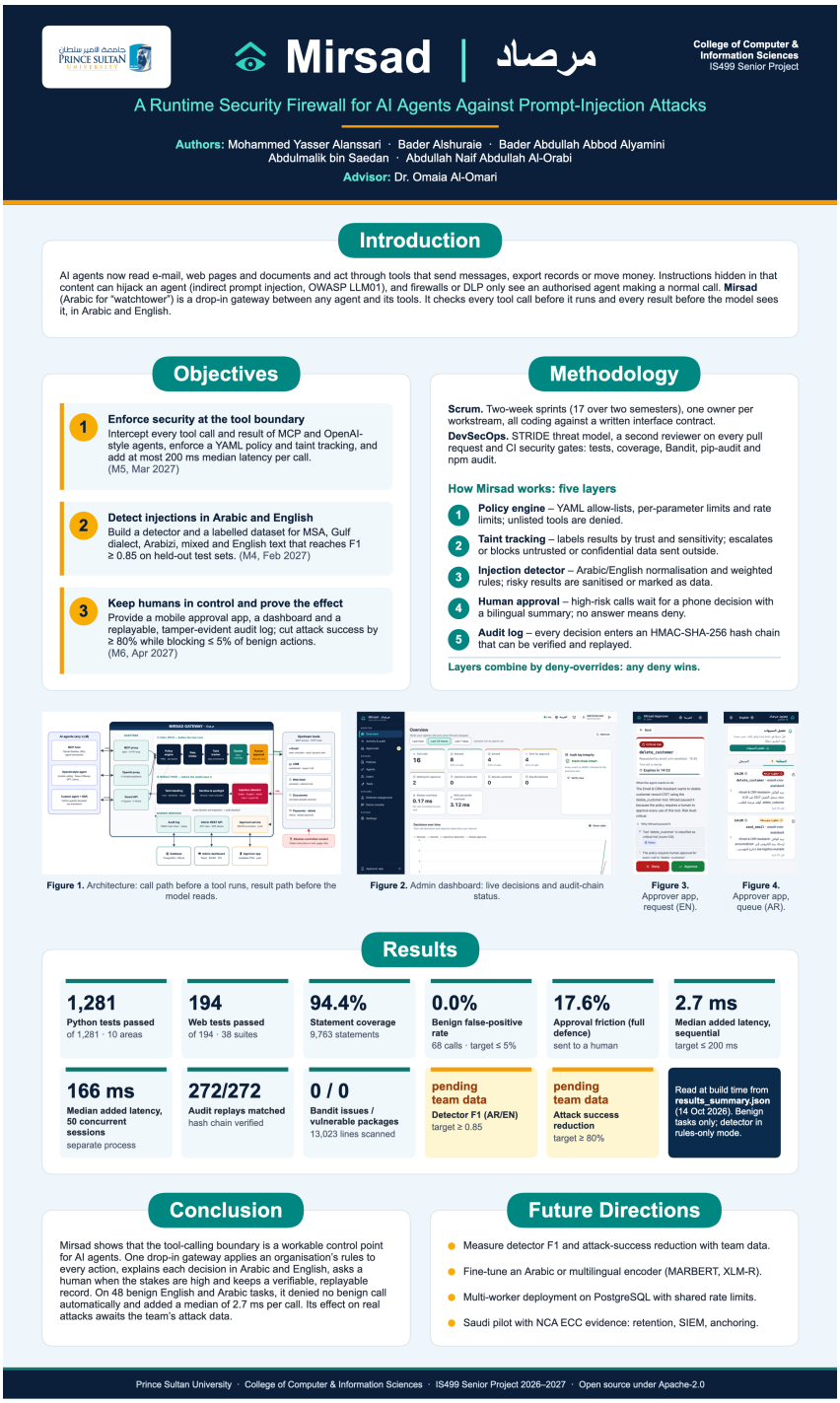


Figure D.1: CCIS EXPO poster (preview rendering)

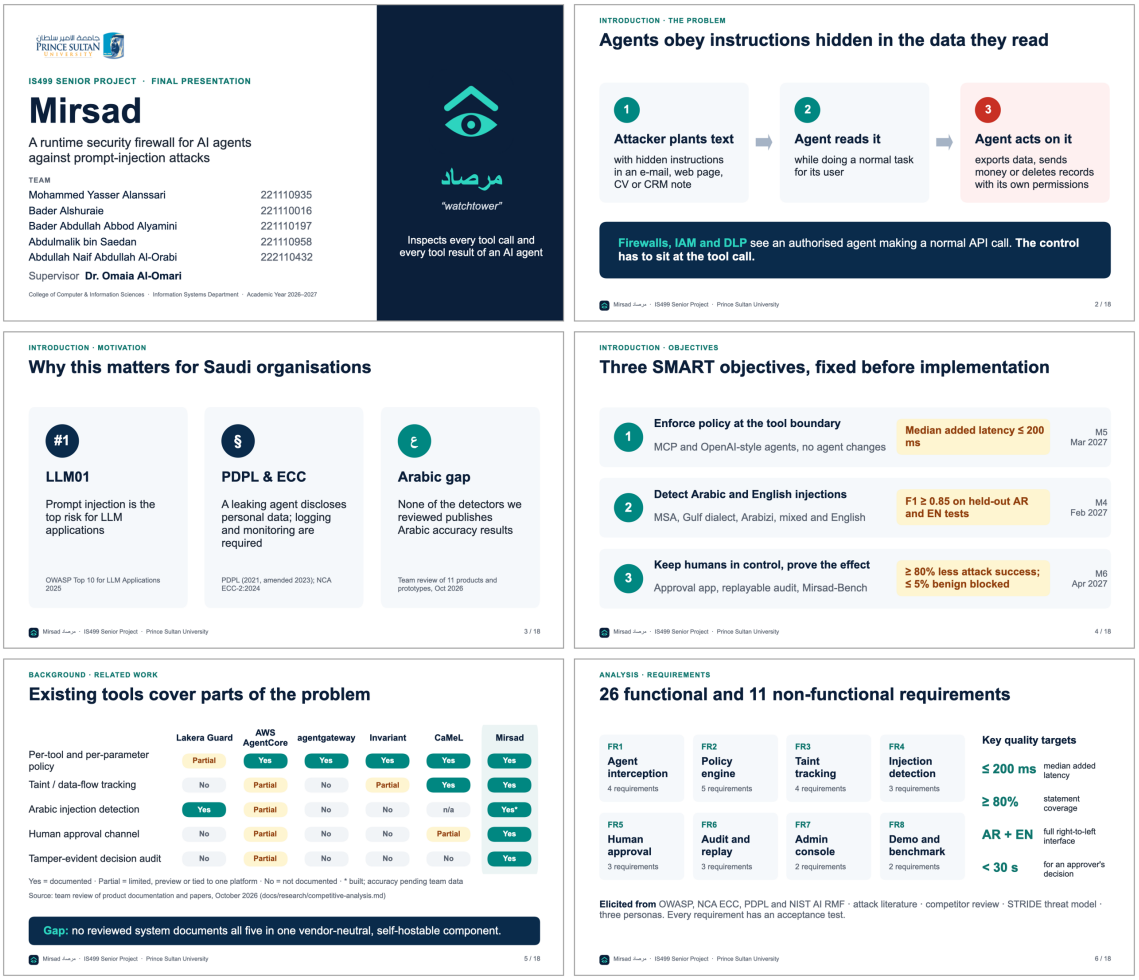
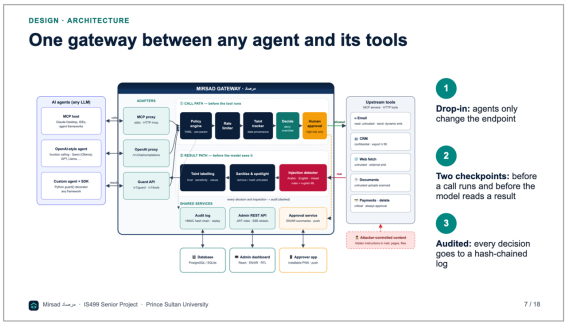
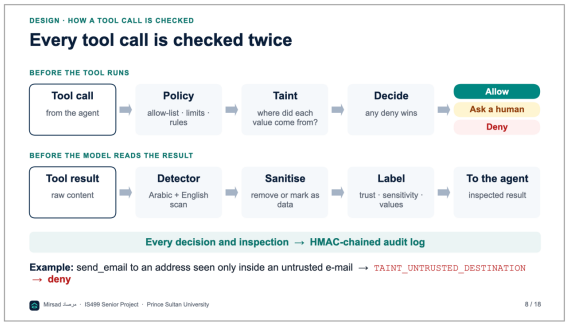
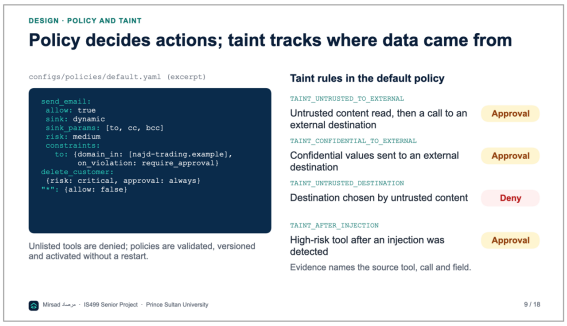


Figure D.2: Final presentation, slides 1–6







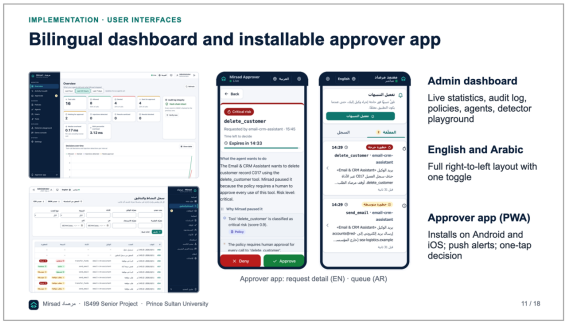


Figure D.3: Final presentation, slides 7–12

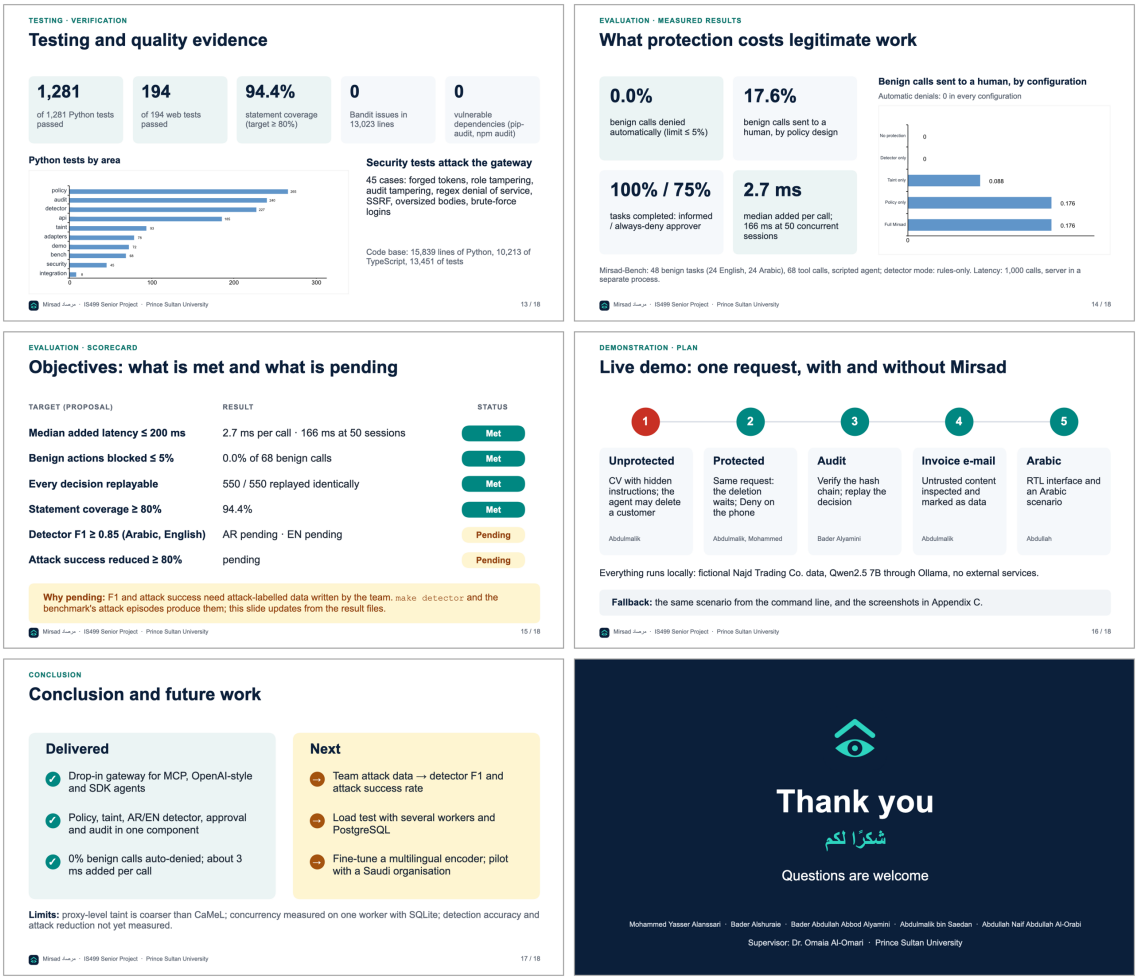


Figure D.4: Final presentation, slides 13–18

Appendix E – Software User Guide

This appendix is the Software User Guide of Mirsad 1.0.0 (IS499 deliverable D6). It explains how to install, configure, operate and integrate the system; Table E.1 shows which sections each reader needs. Commands are run from the project root, and `mirsad` is equivalent to `python -m mirsad.cli`. Measured values are read from `reports/results_summary.json` when the document is built; "[pending]" marks a value that has not been produced yet.

Table E.1: Reading guide by audience

Reader	Typical tasks	Sections
Administrators and IT staff	Install, configure, manage users and agents, write policies, harden the deployment	E.1–E.5, E.7, E.9, E.12, E.13
Approvers	Install the phone app, receive alerts, approve or deny paused actions	E.5.4, E.6
Auditors	Search activity, verify the hash chain, replay decisions, export evidence	E.5.3, E.10
Agent developers	Connect agents through MCP, the OpenAI-compatible proxy or the SDK; train the detector; run the benchmark	E.8, E.11, E.12

E.1 Overview and System Requirements

Mirsad is one Python service (FastAPI on Uvicorn) placed between AI agents and their tools. For each tool call it evaluates the active YAML policy, rate limits and the session's taint state, then allows, denies or pauses the call for a person. After a tool runs, Mirsad scans the result for instructions aimed at the agent, sanitises or blocks them, and marks the content as untrusted data. Every step is written to an audit log whose events are chained with an HMAC [67]. The same process serves the agent endpoints (`/mcp` for MCP [3], `/v1/chat/completions`, and the Guard API under `/v1/`), the dashboard at `/`, the approver app at `/approve` and API documentation at `/docs`.

Table E.2: System requirements

Item	Requirement	Notes
Operating system	macOS or Linux with Bash	Tests and measurements ran on macOS (Apple Silicon); Windows was not tested
Python	3.11 or later (3.12 recommended)	
Node.js	20.19 or later	Only to rebuild the web app; a build ships in `src/mirsad/static/`
Database	SQLite (default) or PostgreSQL	Compose uses PostgreSQL 16
Hardware	CPU only	No GPU needed

Item	Requirement	Notes
Demo model (optional)	Ollama serving `qwen2.5:7b`	Demo Console, demo agent, LLM benchmark
Approver phone	Android (Chrome, Edge, Samsung Internet) or iOS/iPadOS 16.4+ (Safari)	Server reachable over HTTPS

On the development machine the release passed 1,281 of 1,281 Python tests and 194 of 194 web tests (94.4% statement coverage). With the gateway in a separate process, Mirsad added a median of 2.7 ms (p95 6.0 ms) to a sequential tool call over HTTP, with the detector in `rules_only` mode.

E.2 Installation

E.2.1 Python virtual environment

```
cd mirsad                # repository clone or release archive
./scripts/setup.sh       # or: make setup
source .venv/bin/activate
mirsad --version         # mirsad, version 1.0.0
```

The script checks for Python 3.11 or later (`PYTHON_BIN` selects an interpreter), creates `.venv`, installs Mirsad in editable mode with the development extras, and runs `npm ci` in `web/` when `npm` exists. `make web` rebuilds the dashboard and approver app into `src/mirsad/static/`; without a build, `/` returns a short JSON note.

E.2.2 Docker Compose with PostgreSQL

The two-stage `Dockerfile` builds the web app with Node 22, then installs Mirsad with the PostgreSQL driver on `python:3.12-slim`, running as the unprivileged user `mirsad` with data in the `/data` volume, port 8700 and a `/healthz` health check. `docker-compose.yml` defines `mirsad`, `db` (`postgres:16-alpine`) and an optional `ollama` service in the `demo` profile.

> **Status.** The Dockerfile and Compose file were written but could not be executed on the development machine because Docker was not available there; treat them as reviewed but untested. PostgreSQL compatibility was verified separately: `tests/integration/test_postgres.py` starts an embedded PostgreSQL server (`pgserver` package) and runs the audit chain, tamper detection, authentication and approvals against it. These tests passed in the recorded run (`reports/tests/junit.xml`).

1. Copy `.env.example` to `.env` and replace every value: `MIRSAD_SECRET_KEY`, `MIRSAD_ADMIN_PASSWORD` (creates the first administrator at first start), `POSTGRES_PASSWORD` and `MIRSAD_ENABLE_DEMO` (`false` in production).
2. Add the two lines below to the `environment` of the `mirsad` service.
3. Run `docker compose up -d --build`; check with `docker compose ps` and `docker compose logs mirsad`.

4. Sign in at `http://<host>:8700/` as `admin` and create approvers on the Users page or with `docker compose exec mirsad mirsad user create <name> --role approver`.
5. For the demo: `MIRSAD_ENABLE_DEMO=true`, `docker compose --profile demo up -d`, `docker compose exec ollama ollama pull qwen2.5:7b`.

```
MIRSAD_POLICY_PATH: /app/configs/policies/default.yaml
MIRSAD_UPSTREAMS_PATH: /app/configs/upstreams.yaml
```

The defaults of these two settings are computed relative to the source tree. In the image, which installs Mirsad as a regular package, they point inside the Python installation, so the server would start with the built-in deny-all policy and no tools. This follows from `src/mirsad/config.py` and was not observed in a running container; it applies to any non-editable installation.

E.3 Configuration

Settings come from environment variables prefixed `MIRSAD_` or from a `.env` file in the start directory (environment variables win) and are read at start-up. Policies are stored in the database instead (E.7).

Table E.3: Environment variables and defaults

Variable	Default	Purpose
`MIRSAD_HOST`, `MIRSAD_PORT`	`127.0.0.1`, `8700`	Bind address and port
`MIRSAD_DATA_DIR`	`.mirsad` (Docker: `/data`)	SQLite file, `secret.key`, `credentials.txt`
`MIRSAD_DATABASE_URL`	SQLite in the data directory	SQLAlchemy URL, e.g. `postgresql+psycopg://user:pw@host/mirsad`
`MIRSAD_SECRET_KEY`	generated once into `secret.key`	Signs sign-in tokens; the audit HMAC key is derived from it
`MIRSAD_POLICY_PATH`	`configs/policies/default.yaml`	Initial policy when the database has none
`MIRSAD_UPSTREAMS_PATH`	`configs/upstreams.yaml`	Tool servers that Mirsad protects
`MIRSAD_DETECTOR_MODEL_DIR`	`src/mirsad/detector/model`	Trained detector; rules-only mode when absent
`MIRSAD_TOKEN_TTL_HOURS`	`12`	Sign-in lifetime
`MIRSAD_APPROVAL_TIMEOUT_S`	`120`	Time to decide; an expired request is denied
`MIRSAD_MAX_RESULT_CHARS`	`200000`	Longer tool results are truncated before scanning
`MIRSAD_AUDIT_PAYLOAD_MAX_CHARS`	`20000`	Size limit of one audit payload
`MIRSAD_ENABLE_DEMO`	`false`	Enables the Demo Console endpoints
`MIRSAD_LLM_BASE_URL`	`http://127.0.0.1:11434/v1`	LLM for the OpenAI-compatible proxy and demo agent

Variable	Default	Purpose
MIRSAD_LLM_MODEL, MIRSAD_LLM_API_KEY	qwen2.5:7b, ollama	Default model and upstream credential
MIRSAD_CORS_ORIGINS	the Vite development server (port 5173)	Allowed browser origins (JSON list)
MIRSAD_VAPID_PUBLIC_KEY, MIRSAD_VAPID_PRIVATE_KEY	empty	Enable Web Push (E.6.3)
MIRSAD_VAPID_SUBJECT	mailto:security@najd-trading.example	Contact sent to push services
MIRSAD_LOG_LEVEL	INFO	Log level (standard error)
MIRSAD_ADMIN_PASSWORD, MIRSAD_ADMIN_USERNAME	unset, admin	Creates the first administrator when no user exists
MIRSAD_NEW_USER_PASSWORD	unset	Password for mirsad user create without a prompt
MIRSAD_AGENT_KEY, MIRSAD_URL	unset, http://127.0.0.1:8700	Agent side: mirsad mcp-proxy and MirsadClient.from_env()

POSTGRES_PASSWORD is used by the Compose file and MIRSAD_API by the web development server.

E.4 First Run and Demonstration

E.4.1 Initialising and starting the server

```
mirsad init --demo     # database, first policy version, users, demo agent
mirsad serve --demo    # dashboard /, approver app /approve, API docs /docs
```

init stores the default policy as the first version, creates the users admin, approver and auditor and, with --demo, the agent demo-agent (policy agent email-crm-assistant). It prints only the path of .mirsad/credentials.txt (mode 600), which holds the generated passwords and the agent key; move them to a password manager and delete the file. Re-running init creates only what is missing. make demo runs both steps through scripts/run_demo.sh. Then sign in at http://127.0.0.1:8700/ as admin, and open /approve as approver in a second window.

E.4.2 Local model for the demo agent

The demo agent uses Ollama's OpenAI-compatible endpoint [102] with Qwen2.5-7B-Instruct [114]. Run ollama pull qwen2.5:7b; curl -s http://127.0.0.1:11434/v1/models should list it. Without the model the Demo Console returns error 502; nothing else depends on it.

E.4.3 Running a demonstration

1. In **Demo console**, choose a scenario, for example the invoice e-mail whose body hides an instruction to send the customer list outside. **Load scenario** resets the

simulated Najd Trading Co. workspace, plants the content and fills in a harmless user request.

2. Turn **Mirsad protection** off and send. Whether the agent follows the planted instruction depends on the model; anything that leaves the organisation appears in the **Attacker inbox**.
3. **Reset demo**, load the scenario again with protection on and send. The tool-call timeline shows each decision and its reason; paused calls reach the Approvals page and the approver app.

The command line runs the same scenarios and prints an outcome computed from the workspace side effects: `python -m demo.agent --list`, then `python -m demo.agent --protected --scenario invoice-exfil --lang en` (or `--unprotected`; `--approve` auto-approves escalations).

E.5 Administration Dashboard

The sidebar shows only the pages the signed-in role may use (Table E.4); the API enforces the same roles. The screenshots were captured by `scripts/capture_screenshots.py`, which starts a demo server, generates activity through the real API and photographs each page in Chrome. That server used a 900-second approval timeout, so countdowns show about 14–15 minutes instead of the default 120 seconds.

Table E.4: Dashboard pages and roles

Page	Roles	Main actions
Overview	all	Key figures, charts, audit chain status
Activity & audit	admin, auditor	Search, inspect, replay, verify, export
Approvals	admin, approver	Live queue, decisions with notes, history
Policies	admin (auditor read-only)	Edit, validate, compare, activate, roll back
Agents, Users	admin	Identities, keys, accounts and roles
Tools	admin, auditor	Discovered tools and their policy treatment
Detector playground	all	Scan Arabic, English or mixed text
Demo console	all (demo mode)	Chat with the demo agent
Settings	all	Theme, language, digits, desktop alerts

E.5.1 Signing in and navigation

After 5 failed sign-ins for one username and address (or 30 from one address) within 5 minutes the page reports "too many attempts" until the window passes. The sign-in lasts while the browser tab is open. The top bar holds the live indicator, the English/العربية toggle, the theme, the account and sign-out.

E.5.2 Overview

The Overview (Figure E.1) summarises the last hour, day or week: tool calls, allowed, denied, sent for approval, waiting, injections detected, results sanitised or blocked, and the median and 95th-percentile gateway overhead (excluding human waiting). Charts show decisions over time, per tool and per defence layer, each with a **Show table** view. **Verify now** re-checks the audit chain.

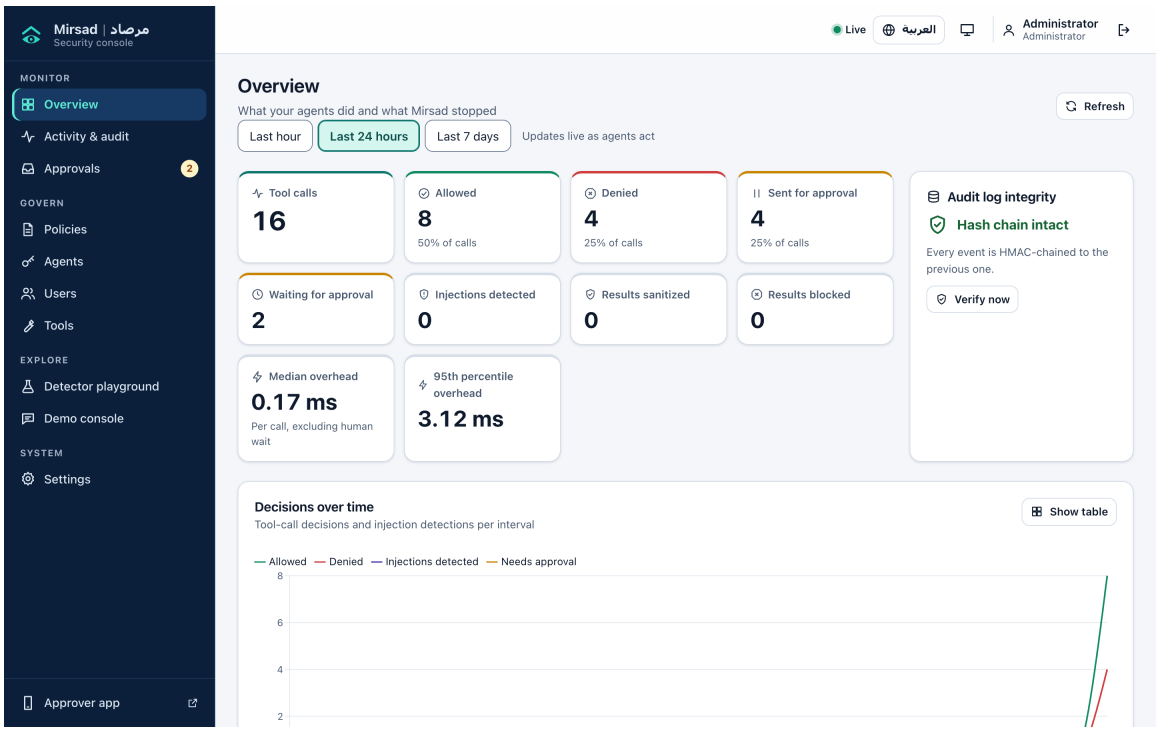


Figure E.1: Overview page of the dashboard

E.5.3 Activity and audit log

A paged list of all audit events, filtered by text, agent, tool, outcome, event type, session, call or time; filters stay in the address so a view can be shared. A row opens the detail drawer (Figure E.2): summary, reasons with bilingual explanations and evidence, detector spans for inspections, taint provenance (which earlier call and field a value came from), arguments, hashes and the raw event, with **Verify chain**, **Export event**, **All events of this call/session** and, for decisions and inspections, **Replay** (E.10).

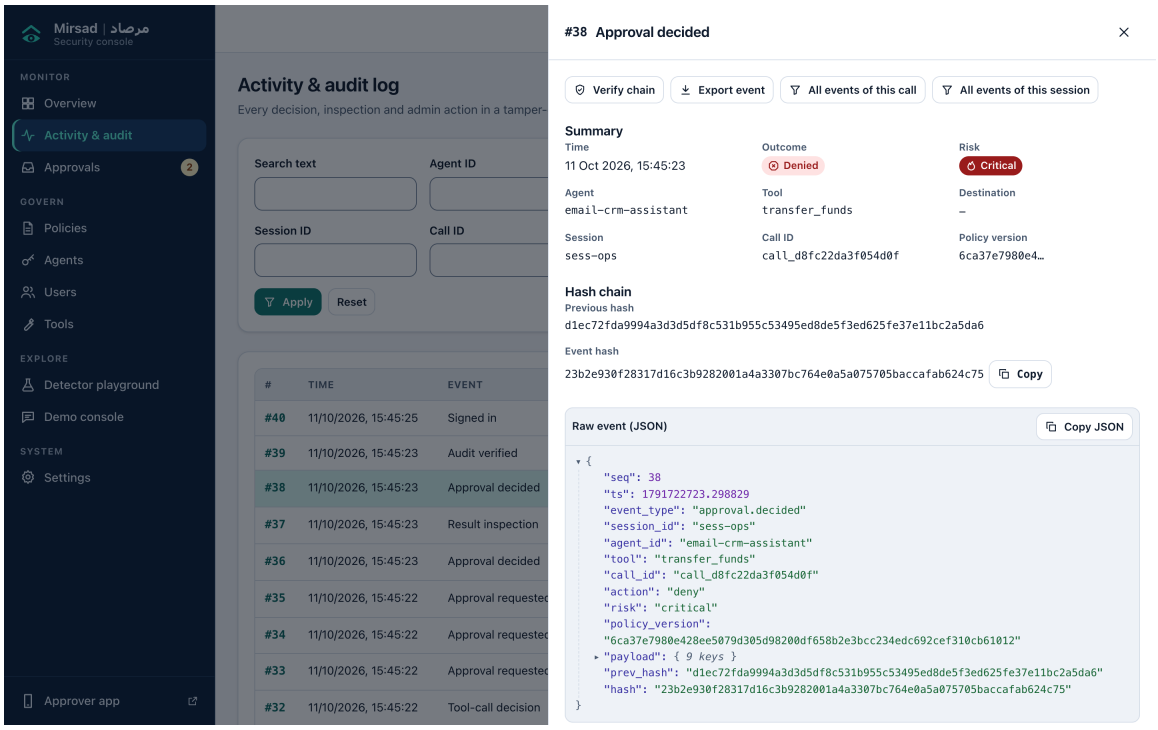


Figure E.2: Activity and audit log with the detail drawer of an approval decision

E.5.4 Approvals

The pending queue (Figure E.3) shows paused calls, soonest expiry first, with risk, tool, agent, countdown and a plain-language summary. **Review** opens the full explanation (E.6.2); **Approve** and **Deny** ask for confirmation and an optional note. If another approver decided first, the page says so. The history lists past requests with the decision maker and note.

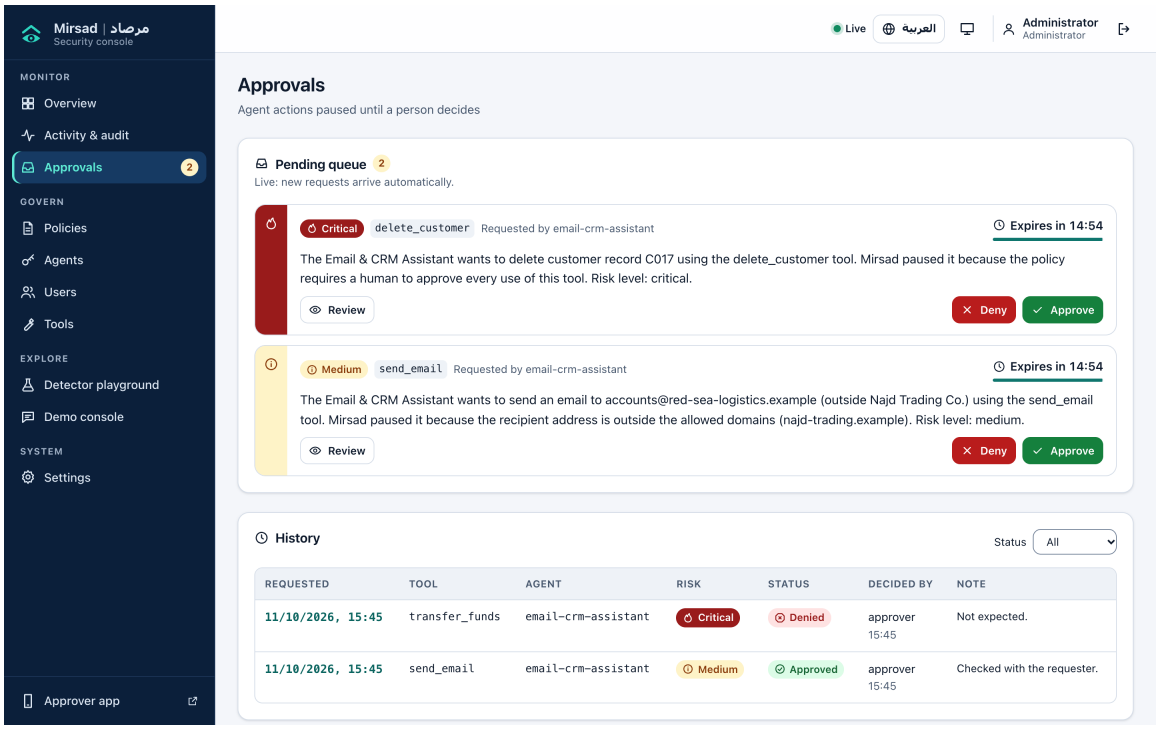


Figure E.3: Approvals page with two pending requests and the decision history

E.5.5 Policies, agents and users

Policies (Figure E.4) shows the active YAML in a line-numbered editor with **Validate** (errors link to their line), **Compare with active**, **Activate** (diff and change note) and a version history from which administrators can re-activate an older version. Auditors get a read-only view; E.7 explains policy writing.

Agents (Figure E.5) lists agent identities with policy agent, key prefix, last use and status. A new or rotated key appears once, in a dialog that closes only after "I have stored this key" is ticked, together with any warning, for example a policy agent id missing from the active policy. Rotation invalidates the old key immediately; **Disable** blocks an agent but keeps its history. **Users** creates accounts with the roles `admin`, `approver` or `auditor`, by default with a server-generated password shown once. The dashboard does not let you change your own role or status, and the last active administrator cannot be demoted, disabled or deleted.

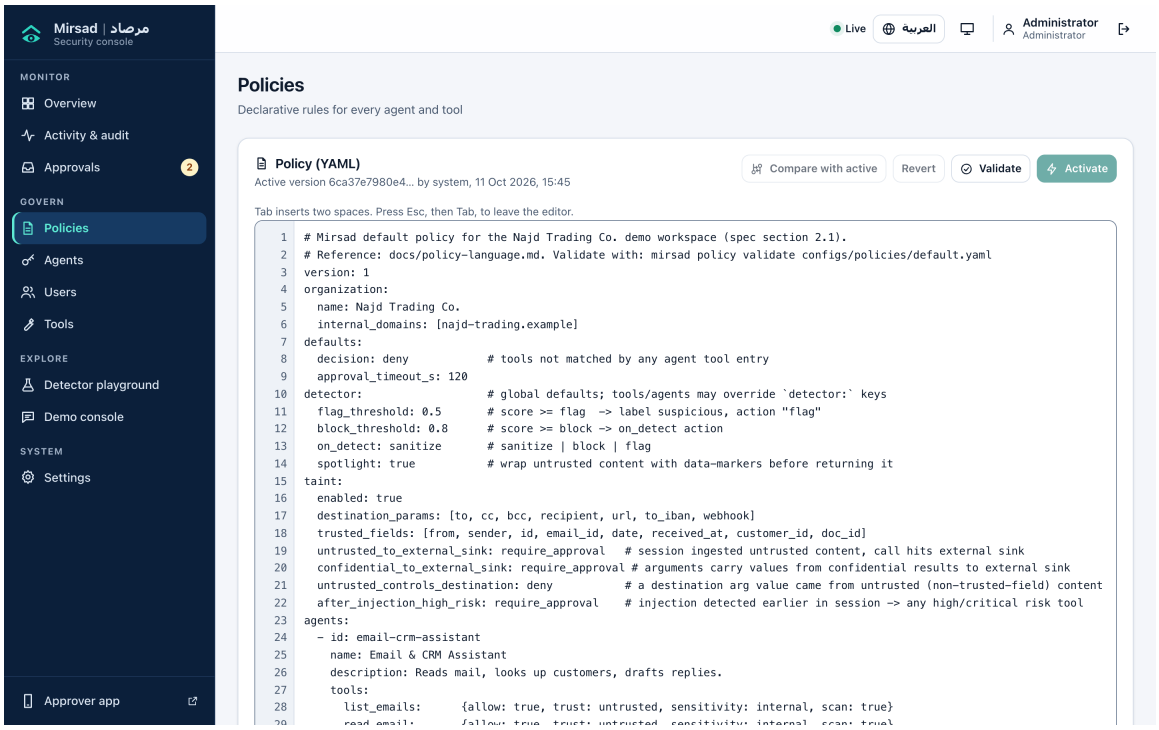


Figure E.4: Policies page with the YAML editor

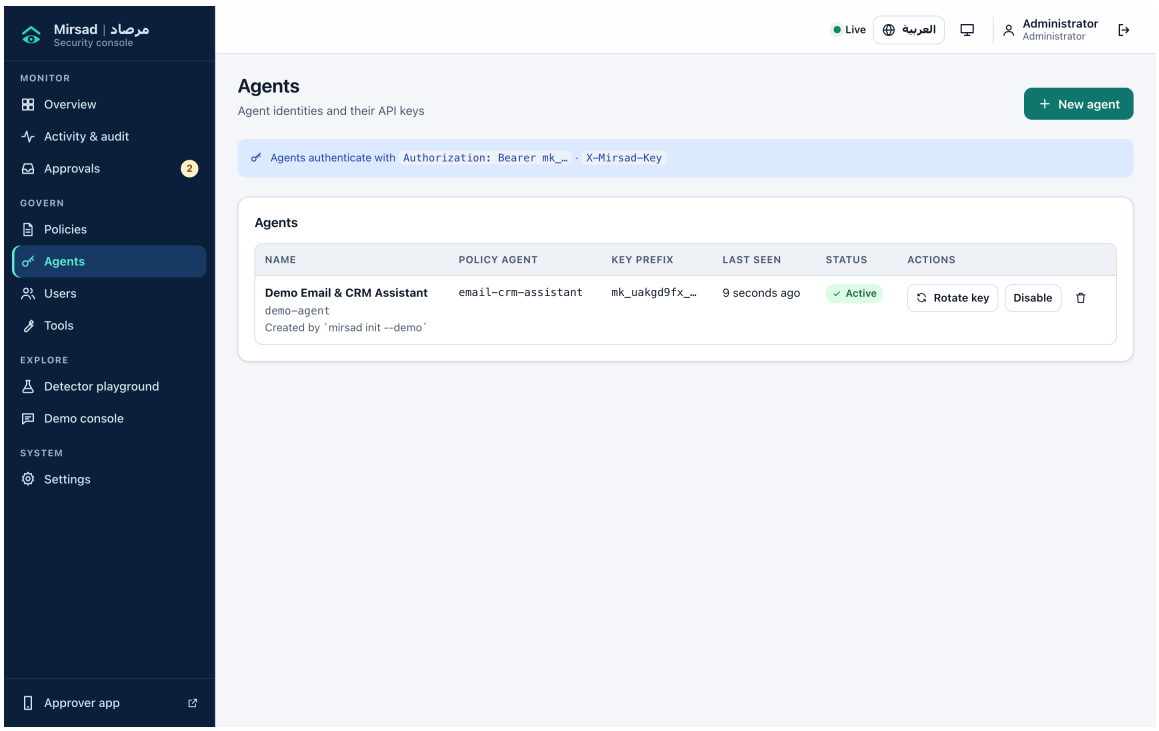


Figure E.5: Agents page with the demo agent created by mirsad init --demo

E.5.6 Tools, detector playground, demo console and settings

Tools shows each discovered upstream tool and how the active policy treats it per agent (allowed, trust, sensitivity, sink, risk, approval, scanning). The **Detector playground**

(Figure E.6) scans pasted text and shows score, thresholds, verdict, language, model and rule scores, latency, highlighted evidence and obfuscation signals; teams can add labelled samples in `src/mirsad/static/assets/playground-samples.json`. The **Demo console** (Figure E.7) holds the protection switch, scenario selector, conversation, tool-call timeline, attacker inbox and a count of workspace changes. The attacker inbox lists every message that left the organisation, including legitimate ones; the item in Figure E.7 is an approved reply to a supplier. The page never displays the planted content itself. **Settings** covers theme, language, Western or Arabic-Indic digits, desktop alerts and the server version.

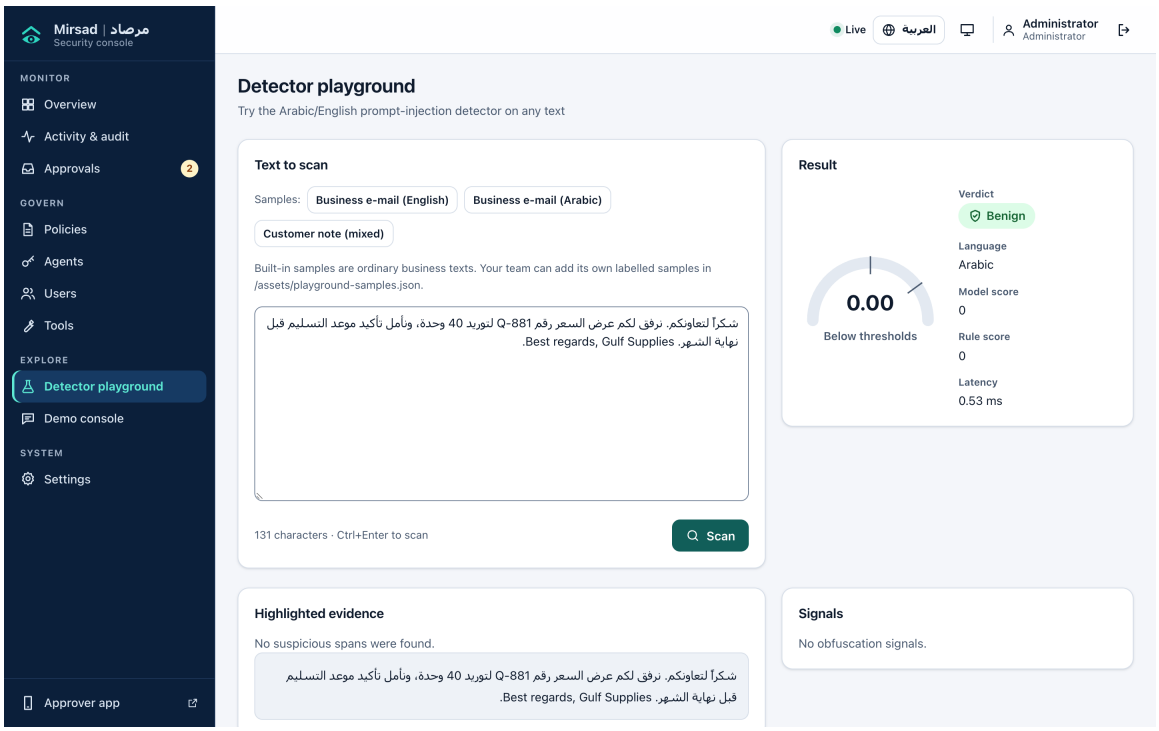


Figure E.6: Detector playground scanning an Arabic business e-mail

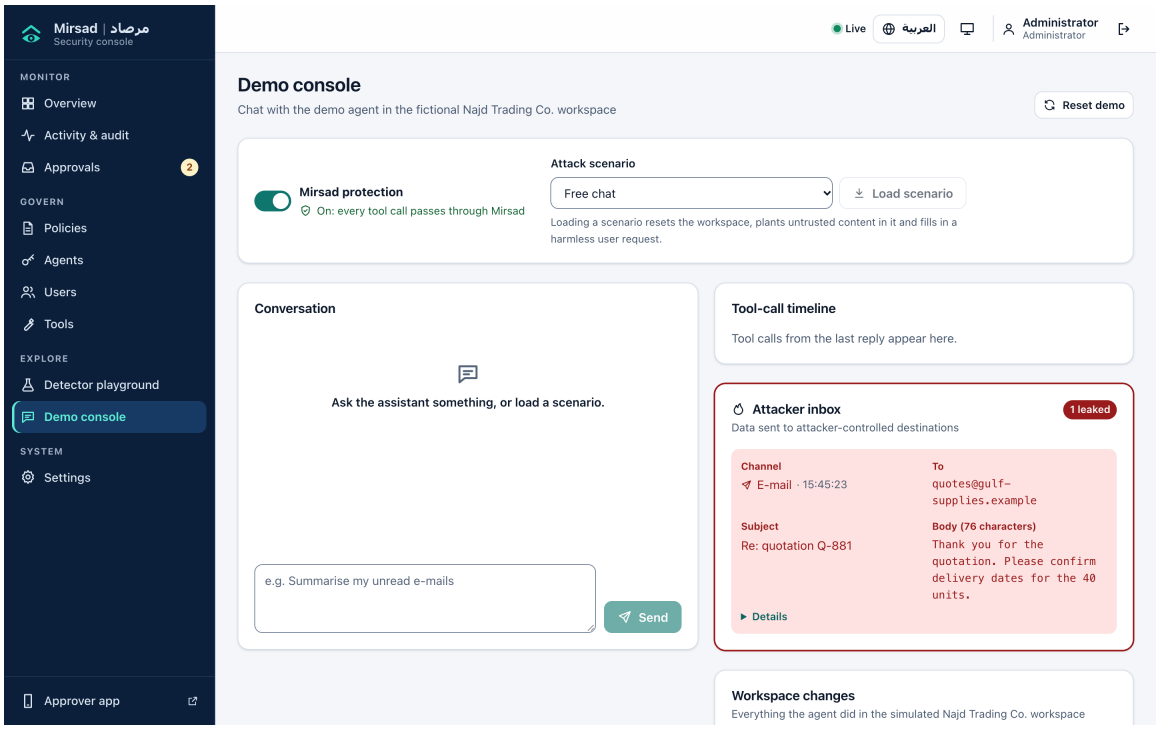


Figure E.7: Demo console with the protection switch, scenario selector and attacker inbox

E.6 Approver App

The approver app is an installable progressive web app [93], [94] at `/approve`. Browsers allow installation, service workers and push only on secure origins, so the server must use HTTPS (E.13); `http://localhost` works only on the same machine.

E.6.1 Installing on Android and iOS

Android (Chrome, Edge or Samsung Internet):

1. Open `https://<your-mirsad-host>/approve` and sign in with an approver account.
2. Tap **Install** on the "Install Mirsad Approver" card, or choose **Install app** / **Add to Home screen** in the browser menu.
3. Open Mirsad from the home screen, tap **Turn on alerts** and allow notifications.

iPhone or iPad (Safari; alerts need iOS/iPadOS 16.4 or later):

1. Open the same address in Safari and sign in.
2. Tap **Share**, **Add to Home Screen**, **Add**.
3. Open Mirsad from the home screen (not Safari), tap **Turn on alerts** and allow. iOS delivers web notifications only to home-screen apps [95].

The sign-in ends when the app is fully closed, by design. The phone screenshots were taken in Chrome emulating a 390 × 844 screen; installation and push delivery on physical phones were not part of the recorded tests.

E.6.2 Reviewing and deciding

Pending (Figure E.8) shows one card per request with risk, countdown, tool, agent and summary. The list updates live, or every 20 s while the live connection is down. The detail view (Figure E.9) has three parts:

- **What the agent wants to do**: a plain-language summary.
- **Why Mirsad paused it**: only the reasons that caused the pause, each with its layer.
- **Where the data came from**: the source tool, call and field of each value that came from outside content.

The arguments follow, collapsed. **Approve** and **Deny** open a sheet that restates the consequence and accepts a note, which is audited. Both are disabled when the request has expired or the phone is offline. A request not decided in time is denied, and the waiting agent receives the denial. **History** lists past requests.

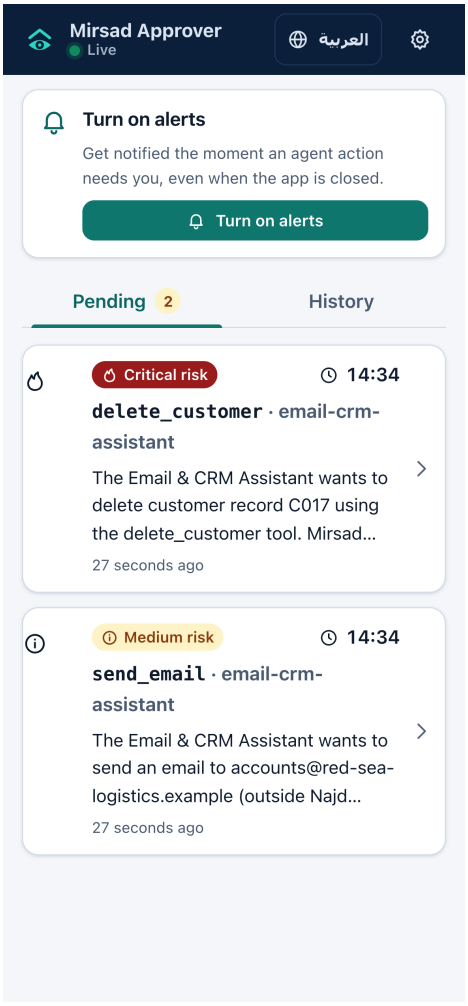


Figure E.8: Approver app: pending requests

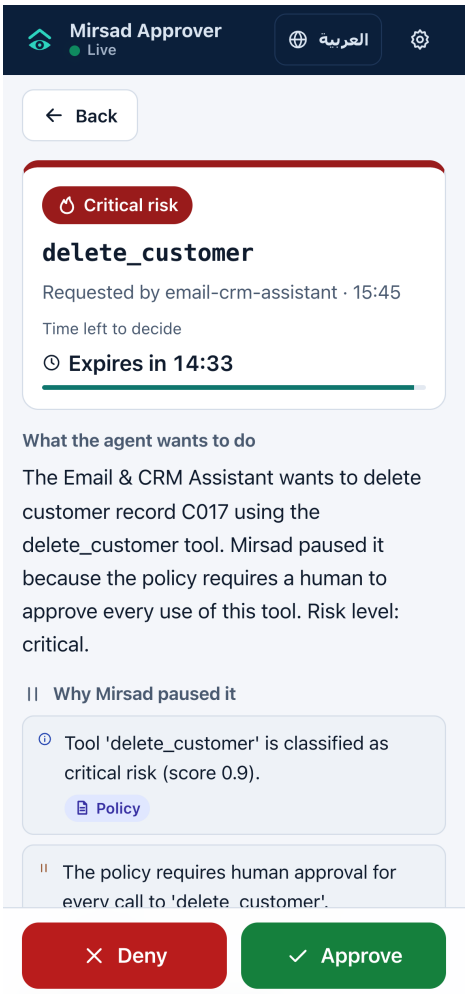


Figure E.9: Approver app: request detail with the Approve and Deny buttons

E.6.3 Notifications

While the app runs in the background it shows a system notification for each new request. For alerts while it is closed, the administrator enables Web Push with a VAPID key pair and restarts the server:

```
python -c "from mirsad.approvals import generate_vapid_keys; print(*generate_vapid_keys(), sep='\n')"
# line 1 -> MIRSAD_VAPID_PUBLIC_KEY, line 2 -> MIRSAD_VAPID_PRIVATE_KEY
```

Push messages pass, encrypted, through the browser vendor's push service; leave the keys unset if all data must stay on premises. Signing out removes the device's subscription. Push delivery was tested only against a simulated push library, so test it on your own phones first.

E.6.4 Arabic interface

The globe button switches to Arabic, which mirrors the layout right-to-left (Figure E.10); summaries and reasons arrive from the server in Arabic, while tool names, addresses and arguments stay left-to-right. A short instruction for Arabic-speaking approvers:

افتح تطبيق «معتمِد مرصاد» من الشاشة الرئيسية، واضغط على الطلب لتقرأ ما يريد الوكيل فعله، ولماذا أوقفه مرصاد، ومن أين جاءت البيانات. ثم اختر «موافقة» أو «رفض» مع ملاحظة عند الحاجة. إذا انتهت المهلة دون قرار يُرفض الطلب تلقائياً.

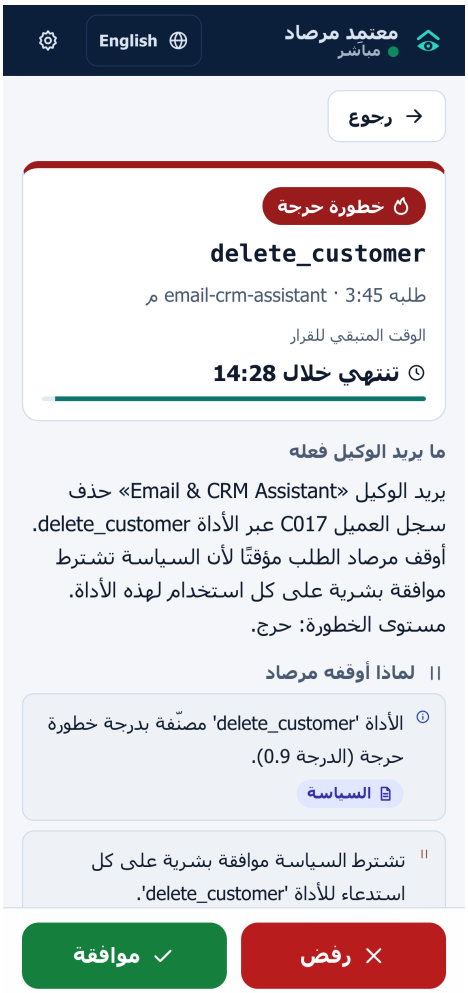


Figure E.10: Approver app: request detail in Arabic

E.7 Writing and Activating Policies

E.7.1 Policy structure and an example

A policy is one YAML document with `version`, `mode` (`enforce` or `monitor`), `organization.internal_domains`, `defaults`, `detector`, `taint`, `agents` and `rules`. Each agent lists its tools with `allow`, result trust and sensitivity, sink, risk, approval mode, argument constraints and rate limits. The agent `id` must equal the policy agent of the agent's key.

The full reference is `docs/policy-language.md`; samples are in `configs/policies/`. This example passed `mirsad policy validate`:

```
version: 1
mode: monitor                       # shadow mode first; change to enforce after tuning
organization:
  name: Najd Trading Co.
  internal_domains: [najd-trading.example]
defaults: {decision: deny, approval_timeout_s: 120}
agents:
  - id: support-assistant
    name: Customer Support Assistant
    tools:
      read_email: {allow: true, trust: untrusted, scan: true}
      get_customer: {allow: true, sensitivity: confidential, scan: true}
      send_email:
        allow: true
        sink: dynamic
        sink_params: [to, cc, bcc]
        risk: medium
        constraints:
          to: {domain_in: [najd-trading.example], on_violation: require_approval}
        rate_limit: {max_calls: 20, per_seconds: 3600}
      export_customers:
        allow: true
        sensitivity: confidential
        risk: high
        approval: on_risk
        constraints:
          count: {max: 50, on_violation: deny}
      "*": {allow: false}
```

Evaluating sample calls with the policy engine gave these decisions, which the gateway enforces once `mode` is `enforce`: an internal e-mail is allowed; an e-mail to an outside domain is paused (constraint violation); an export of 200 records is denied; an unlisted tool is denied (`TOOL_NOT_ALLOWED`). A smaller export is paused, because a `high` risk tool under `approval: on_risk` always reaches the approval threshold. Reading e-mail marks the session as tainted, so the default taint rules also apply: a later outside e-mail adds `TAINT_UNTRUSTED_TO_EXTERNAL`, and a recipient copied from an e-mail body is denied.

E.7.2 Validating

Run `mirsad policy validate FILE` (exit code 1 when invalid) locally or in continuous integration. Errors give line, column, path and a suggestion; two typing mistakes in the example produce:

```
error: line 16, column 9: agents[0].tools.send_email.sink_param: unknown key 'sink_param'
(did you mean 'sink_params'?); ...
error: line 24, column 15: agents[0].tools.export_customers.risk: invalid value 'hgih': ...
(did you mean 'high'?)
```

The validator also rejects duplicate keys, regular expressions prone to catastrophic backtracking, ambiguous tool globs and inconsistent thresholds.

E.7.3 Activating, monitor mode and rollback

Activate a policy with **Activate** on the Policies page, with `mirsad policy activate FILE --comment "..."` (a running server picks it up within about 5 s) or with `PUT /api/policies`. Activation needs no restart, keeps rate-limit counters and is audited as `policy.activated`. A version is the SHA-256 hash of the normalised document, so comment-only edits keep the version.

In **monitor mode** the gateway audits what it would have decided but always allows, never pauses and never alters results; such events are tagged "monitor mode: not enforced". A safe rollout is: activate with `mode: monitor`, review the would-be denials and approvals during normal work, adjust, then activate with `mode: enforce`. To **roll back**, re-activate an earlier version from the history, call `POST /api/policies/{version}/activate`, or activate the old file again (same content, same version).

E.8 Connecting Agents

Mirsad protects only traffic that passes through it. All paths in Table E.5 use the same pipeline and audit log.

Table E.5: Ways to connect an agent

Agent	Mirsad entry point	Agent-side change
MCP host launching local servers (Claude Desktop, IDEs)	`mirsad mcp-proxy` (stdio)	Replace tool servers with the proxy
MCP client with remote servers	`/mcp` (streamable HTTP)	URL and authorisation header
OpenAI-compatible client	`/v1/chat/completions`	`base_url` and `api_key`
Python code running its own tools	SDK `@guard` decorator	Decorate tool functions
Any language	`/v1/guard/tool-call`, `/v1/guard/tool-result`	One call before and after each tool

E.8.1 Before connecting

1. Create an agent in the dashboard or with `mirsad agent create "Support Assistant" --policy-agent support-assistant --key-file support.key`. The key (`mk_<prefix>_<secret>`) is shown or written once.
2. Make sure the active policy defines that policy agent; otherwise every call is denied with `UNKNOWN_AGENT`.
3. On the MCP paths Mirsad runs the tools itself: declare the tool servers in `configs/upstreams.yaml` (`inprocess`, `stdio` or `http`; see `docs/integration.md`). The shipped file connects only the demo workspace.
4. Remove the agent's direct access to the original tool servers, or it can bypass Mirsad.

5. Use one session id per conversation (`X-Mirsad-Session`, `--session-id` or the SDK's `session_id`); taint tracking works per session.

E.8.2 MCP hosts: stdio proxy and HTTP

`mirsad mcp-proxy` is a stdio MCP server that forwards each call to a running Mirsad server. For Claude Desktop, in `claude_desktop_config.json` (macOS: `~/Library/Application Support/Claude/`):

```
{
  "mcpServers": {
    "mirsad": {
      "command": "/opt/mirsad/.venv/bin/mirsad",
      "args": ["mcp-proxy", "--url", "https://mirsad.najd-trading.example"],
      "env": { "MIRSAD_AGENT_KEY": "mk_xxxxxxxx_yyyyyyyyyyyyyyyy" }
    }
  }
}
```

Hosts that support remote servers connect instead to `https://<host>/mcp` with the header `Authorization: Bearer mk_...`; the configuration format depends on the host. Either way the host sees only the tools its policy allows, and a denied call returns an MCP tool error explaining the block in English and Arabic.

E.8.3 OpenAI-compatible agents

```
from openai import OpenAI

client = OpenAI(base_url="https://mirsad.najd-trading.example/v1",
                api_key="mk_xxxxxxxx_yyyyyyyyyyyyyyyy",
                default_headers={"X-Mirsad-Session": "conversation-42"},
                timeout=180)   # longer than the approval timeout
```

Mirsad inspects every `role: tool` message before forwarding the request to `MIRSAD_LLM_BASE_URL` (never with the agent key), removes tools the policy forbids, and checks each tool call in the reply, removing denied calls and appending a bilingual explanation. Streaming is not supported (HTTP 400). Here the agent's own code runs the tools, so Mirsad assumes that code is honest; prefer the MCP paths when it is not.

E.8.4 Python SDK

```
from mirsad.sdk import MirsadClient, MirsadBlocked

mirsad = MirsadClient("https://mirsad.najd-trading.example", api_key="mk_...",
                      session_id="conversation-42")

@mirsad.guard(name="send_email")          # policy tool name
def send(to: str, subject: str, body: str) -> str:
    return smtp_send(to, subject, body)    # existing tool code

try:
    text_for_model = send("ops@najd-trading.example", "Stock report", "...")
except MirsadBlocked as exc:
    text_for_model = str(exc)              # bilingual explanation for the model
```

The decorator checks the call, runs the function only if allowed, sends the result for inspection and returns the inspected text; always give the model that text, not the raw value. `MirsadClient.from_env()` reads `MIRSAD_URL` and `MIRSAD_AGENT_KEY`; the default timeout of 150 s covers an approval wait.

E.9 Command-Line Reference

Usage is `mirsad [--data-dir DIR] [--log-level LEVEL] COMMAND`; results go to standard output and logs to standard error. Table E.6 was compiled from the `--help` output of version 1.0.0.

Table E.6: Commands of the mirsad command line

Command	Purpose
`serve [--host] [--port] [--demo/--no-demo]`	Run the gateway, API and web apps
`init [--demo] [--admin-password PW]`	Create the database, first policy version, users (and demo agent)
`policy validate FILE`	Check a policy; exit code 1 when invalid
`policy activate FILE [--comment TEXT]`	Store and activate a policy version
`audit verify`	Recompute the hash chain; exit code 1 if broken
`audit export [--format json|csv] [--out FILE] [--agent-id] [--tool] [--action] [--event-type] [--since] [--until]`	Export events (times in epoch seconds)
`audit replay SEQ`	Re-evaluate a stored decision or inspection
`scan TEXT` or `scan --file FILE`	Run the injection detector
`agent create NAME --policy-agent ID [--id] [--description] [--key-file FILE]`	Register an agent; key printed once or written to a mode-600 file
`user create USERNAME --role admin|approver|auditor [--password] [--display-name] [--lang en|ar]`	Create a user (password prompted if omitted)
`mcp-proxy --agent-key KEY [--url URL] [--session-id ID]`	Stdio MCP server forwarding to a running gateway
`detector train [ARGS]`	Train the detector (`python -m mirsad.detector.train`)

The Makefile wraps common tasks: `make setup`, `serve`, `demo`, `web`, `test` (without model-dependent tests), `test-all`, `lint`, `security` (Bandit, pip-audit, npm audit), `bench`, `latency`, `detector` and `docs`.

E.10 Audit Verification, Replay and Export

Decisions, inspections, approvals, policy activations, failed sign-ins and user or agent changes are audit events. Each event's hash is an HMAC-SHA-256 over its content and the previous hash, keyed from `MIRSAD_SECRET_KEY` [67], [87]; secrets are removed and identifiers such as IBANs and national IDs masked before hashing.

Verify with `mirsad audit verify`, **Verify chain** in the dashboard or `POST /api/audit/verify`. The report gives `ok`, `checked`, `head_seq`, `head_hash` and, on failure, `first_bad_seq` with the reason (modified event, missing sequence number, broken link or wrong key). The command exits with code 1 on failure, so it can be scheduled.

Replay (`mirsad audit replay SEQ` or **Replay**) re-evaluates a `tool_call.decision` or `tool_result.inspection` event with the policy version it recorded and reports whether the action matches. Masked or truncated arguments can cause differences, which the report notes; inspections are replayed with the current detector model. In the scripted benchmark 272 of 272 audited decisions replayed to the same action.

Export with `mirsad audit export --format json --out audit.json`. JSON keeps the chain fields and is the evidence format; CSV is for reading, with formula-like cells escaped. An unfiltered JSON export verifies offline with the key (filtered exports have gaps):

```
import json
from mirsad.audit import audit_key_from_secret, verify_events

export = json.load(open("audit.json", encoding="utf-8"))
key = audit_key_from_secret(open(".mirsad/secret.key").read().strip())  # or the
MIRSAD_SECRET_KEY value
anchor = {"seq": 4, "hash": "<head_hash recorded earlier>"}             # optional, see E.13
print(verify_events(export["events"], key, anchor=anchor))
```

A valid chain shows that nobody without the key changed, removed or reordered events inside it. Deleting the newest events, or a rewrite by a key holder, is detected only against a head recorded earlier outside the database (an anchor); version 1.0 checks anchors in Python, not from the command line.

E.11 Training the Detector and Running the Benchmark

E.11.1 Detector

Mirsad ships without a trained model, so the detector runs in rules-only mode. The attack-labelled data is written and reviewed by the project team (format in `docs/detector.md`, sections 11–12), not by the AI assistant used during development; `data/detector/raw/` currently holds only benign batches. To train a model:

1. Add training batches as `data/detector/raw/<batch>.jsonl` and separately written test batches as `raw/test-<name>.jsonl`.
2. Run `make detector`: it builds the splits (deduplication, split by batch, training-only augmentation, leakage check), trains, selects thresholds on the validation split and writes `src/mirsad/detector/model/`, `reports/detector_metrics.json` and figures.
3. Check the test metrics, run `pytest tests/detector`, and restart the server to load the model.

E.11.2 Benchmark

`make bench` runs 48 benign English and Arabic user tasks under every defence configuration, with an approver that denies every escalation and one that accepts only what the user asked for, and writes `reports/bench/comparison.md` (other runs: `docs/benchmark.md`). `scripts/run_evidence.sh` re-runs tests, scans and benchmarks, `scripts/collect_results.py` refreshes `reports/results_summary.json`, and `make docs` rebuilds this document.

Attack files are supplied by the team in `bench/scenarios/attacks/*.yaml` (format in the README of that folder) and checked with `python -m bench.loader`. Once they exist, `make bench` (`scripts/run_bench.sh`) also runs the attack episodes, and the attack success rates fill in.

Table E.7: Headline results and how they are produced

Result	Value	Produced by
Benign false-positive rate, full protection	0.0%	`make bench`
Approval friction on benign calls, full protection	17.6%	`make bench`
Utility with the plan-aware approver, full protection	100.0%	`make bench`
Added latency, sequential median	2.7 ms	`python -m bench.latency --server-process`
Detector F1, English test set	[pending]	`make detector` with team data
Detector F1, Arabic test set	[pending]	`make detector` with team data
Attack success rate reduction, full vs none	[pending]	Attack episodes with team attack files

E.12 Troubleshooting and FAQ

Table E.8: Common problems and solutions

Symptom	Likely cause	Action
Agent gets 401	Wrong, rotated or disabled key	Check the key prefix on the Agents page
All calls denied with `UNKNOWN_AGENT`	Policy agent not in the active policy	Add the agent to the policy
Tool missing or `TOOL_NOT_ALLOWED`	No `allow: true` entry for the tool	Add it to the agent's tools
503 on `/v1/tools/.../invoke`	No upstream tool servers	Check `MIRSAD_UPSTREAMS_PATH` and the log
Call hangs about two minutes, then denied	Approval expired	Enable approver alerts; raise client timeouts
Demo Console error 502	Ollama or model missing	`ollama pull qwen2.5:7b`
"Too many attempts" at sign-in	Login throttling	Wait 5 minutes

Symptom	Likely cause	Action
`/` shows a JSON note	Web app not built	`make web`
No push alerts	No HTTPS, VAPID keys or permission, or iOS app not on home screen	See E.6.1 and E.6.3
Nothing is blocked	Policy in `mode: monitor`	Activate with `mode: enforce`
`audit verify` fails at sequence 1	`MIRSAD_SECRET_KEY` differs from the one that wrote the log	Restore the original key
OpenAI client gets 400	`stream: true`	Use non-streaming requests
`make detector` stops: "train split needs both benign (0) and injection (1) examples"	Only benign batches in `data/detector/raw/` (the current state)	Add the team's labelled batches (E.11.1)

Can I run several Mirsad processes behind a load balancer? Not in version 1.0: taint sessions, rate-limit counters and the login throttle live in memory per process.

Does Mirsad send data elsewhere? Only to what you configure: upstream tool servers, the upstream LLM and, with VAPID keys, browser push services.

Does Mirsad stop a user who types a jailbreak? No. It governs tool calls and results; a harmful tool call is still subject to the policy whatever caused it.

E.13 Security Recommendations for Operators

Secret key. Supply `MIRSAD_SECRET_KEY` from a secrets manager (for example `python -c "import secrets; print(secrets.token_urlsafe(48))"`) rather than the generated `secret.key`, which sits beside the database and its backups. Restrict the data directory with `chmod 700`: on the development machine the SQLite file was created with mode 644, while `secret.key` and `credentials.txt` were mode 600.

Changing the secret key. It signs sign-in tokens and is the source of the audit HMAC key, so changing it signs everyone out and makes `audit verify` fail at sequence 1. Version 1.0 cannot re-key the chain. Change it only after a suspected compromise, and first verify, export the log as JSON, record the head and archive the old key separately so the old history stays verifiable offline (E.10).

Agent keys and passwords. Give each agent its own key and rotate keys on a schedule, when staff leave and after any suspected leak. Keep passwords and keys in a password manager, delete `credentials.txt`, and assign the least powerful role that fits.

TLS reverse proxy. Keep Mirsad on `127.0.0.1` behind a proxy that terminates TLS with a valid certificate, for example from Let's Encrypt [115]. Configure the proxy to send `Strict-Transport-Security` (Mirsad adds it only when it receives HTTPS itself), to pass `/api/stream` unbuffered, to allow read timeouts longer than the approval timeout on `/v1/*` and `/mcp`, and to restrict those paths to agent networks. Mirsad does not trust forwarded-

address headers, so behind a proxy the per-address login throttle is shared by all clients. With Compose, publish port 8700 only to the proxy.

Configuration. Set `MIRSAD_ENABLE_DEMO=false`, set `MIRSAD_CORS_ORIGINS` to `[]` or the dashboard origin, keep `defaults.decision: deny`, introduce policies in monitor mode, and remove direct routes from agents to tool servers.

Anchoring audit heads. At least daily and after each verification, record `head_seq` and `head_hash` from `mirsad audit verify` somewhere database administrators cannot change, such as an e-mail to auditors or write-once storage [90], and verify exports against these anchors (E.10).

Backups and updates. Back up the database and keep the key elsewhere. Run `make security` before upgrades; for this release Bandit reported 0 issues, pip-audit 0 vulnerable packages among 124, and npm audit 0 vulnerabilities.

Appendix F – Stakeholder Questionnaire

This questionnaire validates Mirsad's requirement priorities with security and IT practitioners in the target sectors (banking, government services, telecom, e-commerce). It takes about ten minutes, collects no personal data beyond role and sector, and participation is voluntary and anonymous. It is available in English and Arabic; responses are to be summarised in aggregate only.

Consent. "This survey is part of a Prince Sultan University senior project. Your answers are anonymous and used only in aggregate for academic purposes. You may stop at any time. By continuing you agree to take part."

Table F.1: Section A – Respondent profile

#	Question	Answer format
A1	Which best describes your role?	SOC / security analyst · security manager / CISO · IT / platform engineer · developer · compliance / audit · other
A2	Sector of your organisation	Banking / finance · government · telecom · healthcare · e-commerce / retail · education · other
A3	Does your organisation use, pilot or plan to use AI agents that can call tools (e-mail, CRM, APIs)?	In production · piloting · planned within 12 months · no plans
A4	In which languages do your users and customers communicate with these systems?	Arabic · English · both · other

Table F.2: Section B – Importance of capabilities (1 = not important, 5 = critical)

#	Capability	1–5
B1	Blocking an AI agent from sending company data to external addresses	
B2	Rules per tool and per parameter (e.g. "never export more than 50 records")	
B3	Detecting hidden instructions in e-mails, web pages and documents in **Arabic**	
B4	Detecting hidden instructions in **English**	
B5	Asking a human to approve high-risk actions (payments, deletions, bulk exports)	
B6	Approving from a mobile phone with a short explanation	
B7	A tamper-evident audit log usable as compliance evidence (NCA ECC, PDPL)	
B8	Working with any LLM provider (no vendor lock-in)	
B9	Low added delay (below 0.2 s per tool call)	
B10	Deployment on-premises / in a Saudi data centre	

Table F.3: Section C – Operational preferences

#	Question	Answer format
C1	Within how many seconds should an approver normally decide on a paused action?	< 30 s · 30–60 s · 1–5 min · > 5 min
C2	If nobody responds to an approval request, what should happen?	Deny (fail-safe) · allow · escalate to another approver
C3	What false-alarm rate (benign actions paused) would be acceptable?	< 1% · 1–5% · 5–10% · > 10%
C4	Preferred language for alerts	Arabic · English · both
C5	Which actions should always need human approval in your context?	Free text
C6	What would stop your organisation from adopting a gateway like Mirsad?	Free text

Table F.4: Section D – After a demonstration (optional, 1 = strongly disagree, 5 = strongly agree)

#	Statement	1–5
D1	I understood why the action was paused from the approval summary alone.	
D2	I could decide on the request in under 30 seconds.	
D3	The dashboard makes it clear which attacks were blocked and why.	
D4	I would trust the audit log as evidence in an audit.	
D5	The Arabic interface is clear and natural.	

Analysis plan. Section B priorities are reported as medians and the share of respondents rating 4–5; items with a median below 3 are candidates to lower in priority, and free-text answers in C5–C6 are coded thematically into new backlog items. Section D gives a small usability check of NFR4 to complement the Keystroke-Level Model estimate in Chapter 4.